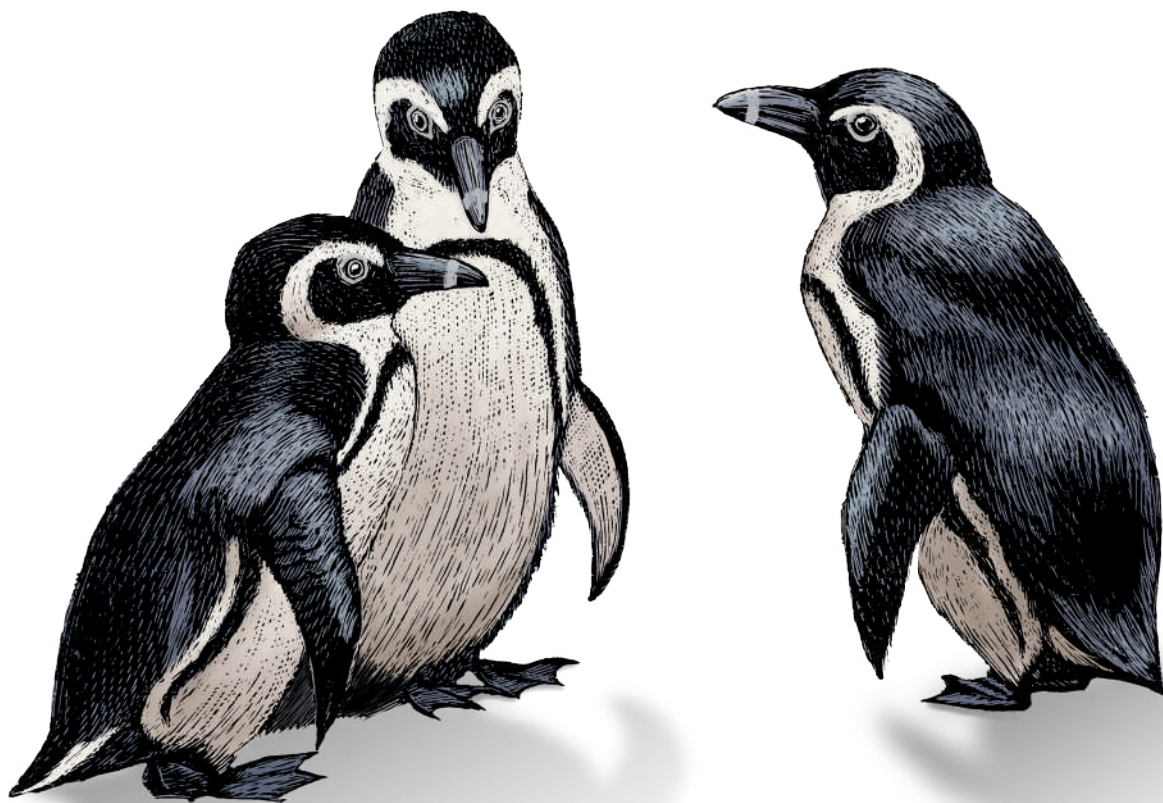


O'REILLY®

Reinforcement Learning

Industrial Applications of Intelligent Agents



Phil Winder, Ph.D.



Figure 6-9. The final resting place of an overworked servo.

Because of this it makes sense to build a simulator to do your initial tuning. It is likely to be far quicker and cheaper to evaluate than the real thing. So I added a basic simulation of the servo and did some hyperparameter tuning.

Here is a list of tips when tuning:

- The learning rate is inversely proportional to the required training time. Smaller, slower learning rates take longer to train.
- Smaller sized minibatches generalize better.
- Try to have multiple episodes in each update, to break the sample correlation.
- Tune the value estimation and policy networks independently.
- Don't update too quickly, or the training will oscillate.
- There is a trade-off between training speed and training stability. If you want more stable training, learn slower.

But to be honest it doesn't matter what hyperparameters you use for this. It can be solved by anything. All we're doing here is attempting to control the stability of the learning and improve the speed. Even so it still takes around 1,000 steps to get a reasonably stable policy. The main factor is the learning rate. You can decrease it and get faster training, but the policy tends to oscillate.

Resulting Policies

Figure 6-10 shows a trained policy over four different stages of training on the simulated environment: 250, 500, 750, and 1,000 environment steps, which correspond to simulated moves of the servo. You can generate a plot like this by asking the policy to predict the action (value denoted by shading) for a given goal (x-axis) and position (y-axis) observation. A perfect policy should look like smooth vertical stripes, smooth because this is a continuous action and vertical because we want an action that maps directly with the requested goal; the position observation is irrelevant.

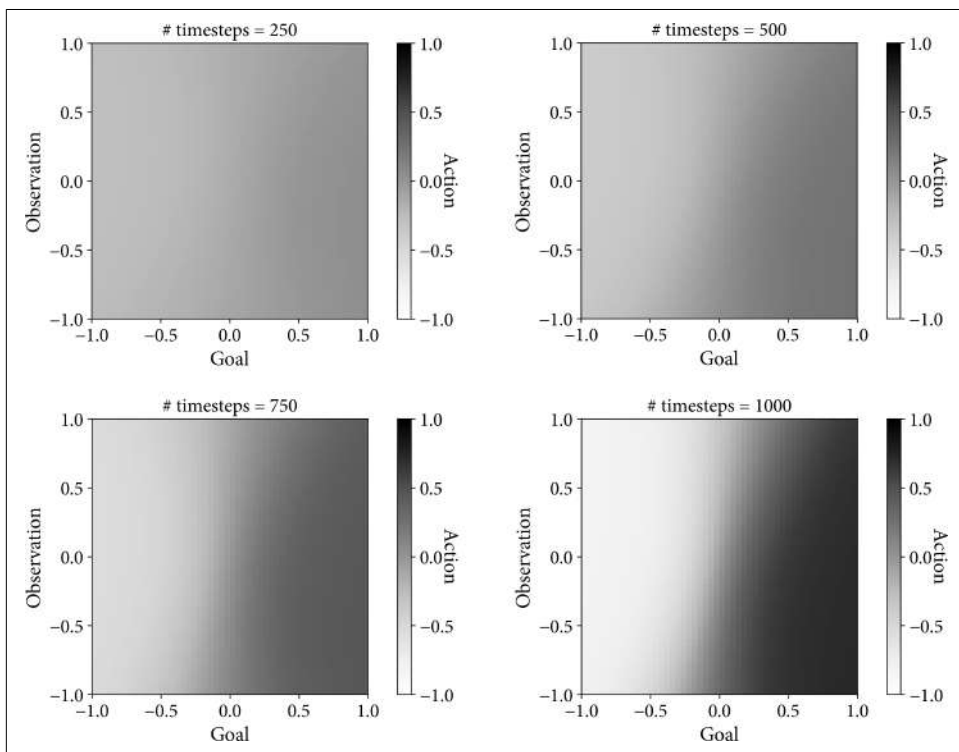


Figure 6-10. A trained policy on the simulated environment at different numbers of environment steps. This image shows the policy predicting the action (shading) for a given goal (x-axis) and observed position (y-axis).

At the start of the training all predictions are zero. Over time the agent is learning to move higher when the goal is higher and lower when the goal is lower. There is a subtle diagonal in the resulting policy because the model has enough freedom to make that choice and the various random samples have conspired to result in this policy over a perfect one. Given more training time and averaging this would eventually be perfect.

Figure 6-11 shows a very similar result when asking the agent to train upon a real servo. The diagonal is more pronounced, but this could just be due to the particular random samples chosen. But the actions aren't quite as pronounced either. The blacks are not as black, and the whites are not as white. It's as if in real-life there is more uncertainty in the correct action, which I account to the fact that the requested movement is not perfect. If I request a position of 0.5, it will actually move to 0.52, for example. I believe this positional error is manifesting as a slightly unsure policy.

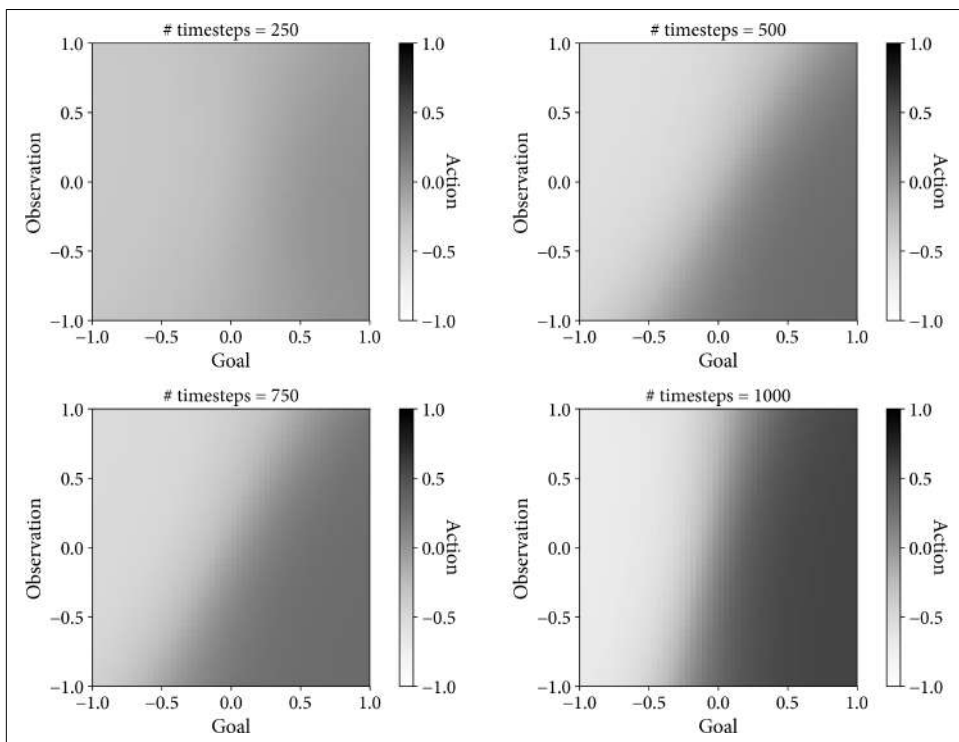


Figure 6-11. A trained policy using a real servo motor at different numbers of environment steps. Axes are the same as Figure 6-10.

Of course, like before, given more training time and averaging it would result in a perfectly viable and useful policy.

Other Policy Gradient Algorithms

Policy gradients, especially off-policy PG algorithms, are an area of enthusiastic research, because of the inherent attractiveness of directly learning a policy from experience. But in some environments, especially environments with complex state spaces, performance lags behind that of value methods. In this section I present introductions to two main streams of research: other ways of stabilizing off-policy

updates, which you have already seen in the form of TRPO and PPO, and emphatic methods, which attempt to use the gradient of the Bellman error, rather than advantages.

Retrace(λ)

Retrace(λ) is an early off-policy PG algorithm that uses weighted and clipped importance sampling to help control how much of the off-policy is included in an update.²⁵

Munos et al. truncate the importance sampling ratio to a maximum of 1, which is similar to the pessimistic bound in TRPO/PPO, and constrains improvements to the policy. At best the difference is minimal so the update is a standard A2C. At worst the update is negligible because the two policies are so different. The reason for clipping is to prevent variance explosion (repeated high values cause dramatic changes in the policy). The downside is that this adds bias; the result is more stable, but less able to explore.

The returns are also weighted by λ , eligibility traces from “[Eligibility Traces](#)” on page 80. This updates the policy based upon previous visitations, which provides a hyperparameter to control the bias–variance trade-off.

Actor-Critic with Experience Replay (ACER)

ACER extends Retrace(λ) by including a combination of experience replay, trust region policy optimization, and a dueling network architecture.²⁶

First, Wang et al. reformulate Retrace(λ) to compensate for the bias introduced by truncating the importance sampling. To put it another way, Retrace(λ) prevents large changes in policy, which makes it more stable (more biased), but less able to explore. The compensating factor compensates for this loss of exploration in proportion to the amount of truncation in Retrace(λ).

They also include a more efficient TRPO implementation, which maintains an average policy, predicted by the neural network. If you code the the KL divergence as a penalty, rather than a constraint, then you can use the auto-differentiation of the DL framework.

Finally, they improve sample efficiency using experience replay (see “[Experience Replay](#)” on page 92). For continuous action spaces, they include a dueling architecture (see “[Dueling Networks](#)” on page 102) to reduce variance. This leads to two separate algorithms: one for discrete actions and one for continuous.

The result is an algorithm that approaches the performance of DQN with prioritized experience replay (see “[Prioritized Experience Replay](#)” on page 102), but can also work with continuous action spaces. However, all the corrections lead to a rather complicated algorithm and therefore a complex neural network architecture.

Actor-Critic Using Kronecker-Factored Trust Regions (ACKTR)

In “[Natural Policy Gradients and Trust Region Policy Optimization](#)” on page 167 you saw that using the natural policy gradient and constraining by the KL divergence is beneficial because the gradient is independent of the parameterization of the approximating function and the KL constraint improves stability. However, calculating the gradient requires a costly computation to calculate the Fisher information matrix. TRPO’s linear projection is another source of inefficiency.

In 2017 Wu et al. proposed using approximations to speed up computation.²⁷ They use Kronecker-factored approximation (K-FAC), which is an approximation of the Fisher information matrix, to improve computational performance, and a rolling average of this approximation instead of a linear projection, to reduce the number of times the agent has to interact with the environment.

Furthermore, because you can use K-FAC with convolutional neural networks (like PPO), this means that ACKTR can handle discrete, visual state spaces, like the Atari benchmarks.

Since 2017, independent reviews have shown that the performance of ACKTR versus TRPO is highly dependent on environment. For example, in the highly unstable *Swimmer-v1* MuJoCo environment, TRPO outperforms most algorithms. But in the *HalfCheetah-v1* environment, TRPO performs significantly worse.²⁸ In the original paper, Henderson et al. present results from *Swimmer-v1* that show A2C, TRPO, and ACKTR performing similarly (see “[Which Algorithm Should I Use?](#)” on page 185). Anecdotally, practitioners prefer ACKTR over TRPO because the idea and resulting performance is similar but computational efficiency is dramatically improved.

Emphatic Methods

Gradient temporal-difference methods (see “[Gradient Temporal-Difference Learning](#)” on page 149) introduced the idea using a function to approximate the state-value function and computing the gradient of the projected Bellman error directly to avoid the approximation from diverging. However, they need two sets of parameters and two learning rates, which make them hard to use in practice. To solve this, Sutton et al. proposed emphatic temporal-difference methods.^{29,30}

In its most general form, emphatic temporal-difference learning uses a function to quantify interest in particular states according to the current policy. This is presented alongside importance sampling to compensate for off-policy agents. However, deciding upon a function has proven difficult and performance issues relating to the projection prevent breakout success.³¹

One particularly interesting emphatic proposal called generalized off-policy actor-critic (Geoff-PAC) suggests using the gradient of a counterfactual objective.³²

Counterfactual reasoning is the process of questioning data that is not observed; for example, asking what the reward of a new policy would be given data collected in the past. Although the details are beyond the scope of this book, the idea is that you can learn an approximation to the emphatic function (using a neural network of course) by predicting the *density ratio*, which is like a continuous version of importance sampling. The results are promising but have not gained widespread popularity.

Extensions to Policy Gradient Algorithms

Throughout this book you have seen a range of enhancements to algorithms that you could, in principle, apply to any algorithm. Every few years researchers release a paper that combines these enhancements to produce state-of-the-art results in benchmark environments and gains media attention. But from a practitioner's point of view, it is the accompanying ablation study that tends to be more interesting, because it shows you which tweaks might be best for your problem. To demonstrate this approach, here is an example of how you can add distributional RL to PG algorithms.

Quantile Regression in Policy Gradient Algorithms

Richter et al. demonstrate how you can incorporate distributional RL into policy gradient algorithms in much the same way “[Improving Rewards](#)” on [page 109](#) did for value methods.³³ They use a neural network to predict the quantiles for each action. Like before, this is useful because the mapping from actions to rewards is often stochastic and many environments have distributions that are multimodal. The key difference in this paper is the adaption to policy gradient algorithms. They begin with the standard A2C loss function ([Equation 6-8](#)) and instead of learning the advantage function, they replace it with quantile estimates. Next they make a series of approximations to factor out the quantile estimate, which leads to a version that iteratively updates the quantile estimates according to a temperature hyperparameter. They implemented an on-policy version of this algorithm, which beat TRPO in some of the benchmarks (they didn't compare it against PPO) and suggest that it should be possible to use this off-policy in a similar way.

Summary

In this rather extensive chapter you have seen a variety of state-of-the-art policy gradient algorithms, with a focus on off-policy actor-critic methods. [Chapter 5](#) introduced the idea of abstracting policy evaluation and policy improvement into actor-critic architectures. The aim of this chapter was to explain and demonstrate the power of learning off-policy.

The promise of off-policy learning is great, whereby an agent learns a multitude of policies that are different from the behavior policy. Learning off-policy frees the agent

from the tyranny of the exploration–exploitation trade-off, allowing the behavior policy to freely explore. Since the optimal policy is separate, you can use offline logs of data or learn from human demonstration. But the primary advantage is that you retrain your approximation functions as much as you like, because there is no environmental cost. This leads to algorithms that are far more sample-efficient, albeit at the expense of computational complexity, especially if you involve neural networks.

Which Algorithm Should I Use?

Your confusion over which algorithm is best is not lost on researchers. There have been a number of studies that have attempted to provide an answer, but to paraphrase one summary, it is very tempting for researchers to present opportunistic results that might not be entirely robust.³⁴ To give one specific example, the original ACKTR paper presents results for a number of environments that others have struggled to reproduce. Of course, this is probably due to an implementation detail or a lucky set of hyperparameters. But it does highlight the issue. However, I can make some general recommendations:

- Prefer TD3 over DDPG
- Prefer a trust region method in environments that have unstable dynamics, which is the situation where a slight move in policy can have disastrous consequences
- Prefer deterministic methods in environments that have stable dynamics
- Consider simpler algorithms and smaller neural networks if performance or robustness is a concern, like PPO or TD3

A Note on Asynchronous Methods

Another important feature of off-policy algorithms is that it frees you from using a single critic. I haven't mentioned it yet (see “[Scaling RL](#)” on page 299) but because the critics are learning from a replay buffer, there is no reason why you can't have many duplicate critics all learning separate policies. In these *asynchronous methods*, you need to recombine that knowledge back into the behavior policy at some point, but this can dramatically improve the wall-clock performance of algorithms. I haven't discussed it yet because I believe this is an implementation detail and depends on having a problem that needs to learn faster to justify the extra expense. For industrial problems, I would first suggest that you consider simplifying the problem or the domain to remove the need for such methods, because it adds a significant amount of operational complexity and cost.

Further Reading

- Natural policy gradients and TRPO (papers in references):
 - [Brief, but nice lectures from John Schulman](#)
 - [Slides from CMU 10703](#)
 - [Lecture from Sergey Levine](#)
 - [Explanation video from Xander Steenbrugge](#)
- Numerical optimization:
 - Nocedal, Jorge, and S. Wright. 2006. *Numerical Optimization*. 2nd ed. Springer Series in Operations Research and Financial Engineering. New York: Springer-Verlag.

References

- [1] Sutton, Richard S., Hamid R. Maei, and Csaba Szepesvári. 2009. “A Convergent $O(n)$ Temporal-Difference Algorithm for Off-Policy Learning with Linear Function Approximation”. In *Advances in Neural Information Processing Systems 21*, edited by D. Koller, D. Schuurmans, Y. Bengio, and L. Bottou, 1609–1616. Curran Associates, Inc.
- [2] Sutton, Richard S., Hamid Reza Maei, Doina Precup, Shalabh Bhatnagar, David Silver, Csaba Szepesvári, and Eric Wiewiora. 2009. “Fast Gradient-Descent Methods for Temporal-Difference Learning with Linear Function Approximation”. In *Proceedings of the 26th Annual International Conference on Machine Learning*, 993–1000. ICML 09. Montreal, Quebec, Canada: Association for Computing Machinery.
- [3] Sutton, Richard S., Hamid Reza Maei, Doina Precup, Shalabh Bhatnagar, David Silver, Csaba Szepesvári, and Eric Wiewiora. 2009. “Fast Gradient-Descent Methods for Temporal-Difference Learning with Linear Function Approximation”. In *Proceedings of the 26th Annual International Conference on Machine Learning*, 993–1000. ICML 09. Montreal, Quebec, Canada: Association for Computing Machinery.
- [4] Maei, Hamid Reza, Csaba Szepesvári, Shalabh Bhatnagar, and Richard S. Sutton. 2010. “Toward Off-Policy Learning Control with Function Approximation.” In *Proceedings of the 27th International Conference on International Conference on Machine Learning*, 719–726. ICML’10. Haifa, Israel: Omnipress.
- [5] White, Adam, and Richard S. Sutton. 2017. $GQ(\lambda)$ “Quick Reference and Implementation Guide”. ArXiv:1705.03967, May.
- [6] Degris, Thomas, Martha White, and Richard S. Sutton. 2013. “Off-Policy Actor-Critic”. ArXiv:1205.4839, June.

- [7] Silver, David, Guy Lever, Nicolas Heess, Thomas Degris, Daan Wierstra, and Martin Riedmiller. 2014. “**Deterministic Policy Gradient Algorithms**”. In *International Conference on Machine Learning*, 387–95.
- [8] Lillicrap, Timothy P., Jonathan J. Hunt, Alexander Pritzel, Nicolas Heess, Tom Erez, Yuval Tassa, David Silver, and Daan Wierstra. 2019. “**Continuous Control with Deep Reinforcement Learning**”. ArXiv:1509.02971, July.
- [9] Plappert, Matthias, Rein Houthooft, Prafulla Dhariwal, Szymon Sidor, Richard Y. Chen, Xi Chen, Tamim Asfour, Pieter Abbeel, and Marcin Andrychowicz. 2018. “**Parameter Space Noise for Exploration**”. ArXiv:1706.01905, January.
- [10] Salimans, Tim, and Durk P. Kingma. 2016. “**Weight Normalization: A Simple Reparameterization to Accelerate Training of Deep Neural Networks**”. In *Advances in Neural Information Processing Systems 29*, edited by D. D. Lee, M. Sugiyama, U. V. Luxburg, I. Guyon, and R. Garnett, 901–909. Curran Associates, Inc.
- [11] Fujimoto, Scott, Herke van Hoof, and David Meger. 2018. “**Addressing Function Approximation Error in Actor-Critic Methods**”. ArXiv:1802.09477, October.
- [12] Wang, Chaoyang, Zhiqiang Guo, Jianjun Li, Peng Pan, and Guohui Li. 2020. “**A Text-Based Deep Reinforcement Learning Framework for Interactive Recommendation**”. ArXiv:2004.06651, July.
- [13] Ie, Eugene, Vihan Jain, Jing Wang, Sanmit Narvekar, Ritesh Agarwal, Rui Wu, Heng-Tze Cheng, et al. 2019. “**Reinforcement Learning for Slate-Based Recommender Systems: A Tractable Decomposition and Practical Methodology**”. ArXiv:1905.12767, May.
- [14] Ie, Eugene, Chih-wei Hsu, Martin Mladenov, Vihan Jain, Sanmit Narvekar, Jing Wang, Rui Wu, and Craig Boutilier. 2019. “**RecSim: A Configurable Simulation Platform for Recommender Systems**”. ArXiv:1909.04847, September.
- [15] Barth-Maron, Gabriel, Matthew W. Hoffman, David Budden, Will Dabney, Dan Horgan, Dhruva TB, Alistair Muldal, Nicolas Heess, and Timothy Lillicrap. 2018. “**Distributed Distributional Deterministic Policy Gradients**”. ArXiv:1804.08617, April.
- [16] Cai, Qingpeng, Ling Pan, and Pingzhong Tang. 2019. “**Deterministic Value-Policy Gradients**”. ArXiv:1909.03939, November.
- [17] Kakade, Sham M. 2002. “**A Natural Policy Gradient**”. In *Advances in Neural Information Processing Systems 14*, edited by T. G. Dietterich, S. Becker, and Z. Ghahramani, 1531–1538. MIT Press.
- [18] Schulman, John, Sergey Levine, Philipp Moritz, Michael I. Jordan, and Pieter Abbeel. 2017. “**Trust Region Policy Optimization**”. ArXiv:1502.05477, April.

- [19] Peters, Jan, Sethu Vijayakumar, and Stefan Schaal. 2005. “**Natural Actor-Critic**”. In *Machine Learning: ECML 2005*, edited by João Gama, Rui Camacho, Pavel B. Brazdil, Alípio Mário Jorge, and Luís Torgo, 280–91. Lecture Notes in Computer Science. Berlin, Heidelberg: Springer.
- [20] Schulman, John, Filip Wolski, Prafulla Dhariwal, Alec Radford, and Oleg Klimov. 2017. “**Proximal Policy Optimization Algorithms**”. ArXiv:1707.06347, August.
- [21] Mnih, Volodymyr, Adrià Puigdomènech Badia, Mehdi Mirza, Alex Graves, Timothy P. Lillicrap, Tim Harley, David Silver, and Koray Kavukcuoglu. 2016. “**Asynchronous Methods for Deep Reinforcement Learning**”. ArXiv:1602.01783, June.
- [22] Schulman, John, Filip Wolski, Prafulla Dhariwal, Alec Radford, and Oleg Klimov. 2017. “**Proximal Policy Optimization Algorithms**”. ArXiv:1707.06347, August.
- [23] Ahn, Michael, Henry Zhu, Kristian Hartikainen, Hugo Ponte, Abhishek Gupta, Sergey Levine, and Vikash Kumar. 2019. “**ROBEL: Robotics Benchmarks for Learning with Low-Cost Robots**”. ArXiv:1909.11639, December.
- [24] Song, Xingyou, Yilun Du, and Jacob Jackson. 2019. “**An Empirical Study on Hyperparameters and Their Interdependence for RL Generalization**”. ArXiv:1906.00431, June.
- [25] Munos, Rémi, Tom Stepleton, Anna Harutyunyan, and Marc G. Bellemare. 2016. “**Safe and Efficient Off-Policy Reinforcement Learning**”. ArXiv:1606.02647, November.
- [26] Wang, Ziyu, Victor Bapst, Nicolas Heess, Volodymyr Mnih, Remi Munos, Koray Kavukcuoglu, and Nando de Freitas. 2017. “**Sample Efficient Actor-Critic with Experience Replay**”. ArXiv:1611.01224, July.
- [27] Wu, Yuhuai, Elman Mansimov, Shun Liao, Roger Grosse, and Jimmy Ba. 2017. “**Scalable Trust-Region Method for Deep Reinforcement Learning Using Kronecker-Factored Approximation**”. ArXiv:1708.05144, August.
- [28] Henderson, Peter, Riashat Islam, Philip Bachman, Joelle Pineau, Doina Precup, and David Meger. 2019. “**Deep Reinforcement Learning That Matters**”. ArXiv:1709.06560, January.
- [29] Sutton, Richard S., A. Rupam Mahmood, and Martha White. 2015. “**An Emphatic Approach to the Problem of Off-Policy Temporal-Difference Learning**”. ArXiv:1503.04269, April.
- [30] Zhang, Shangdong, Wendelin Boehmer, and Shimon Whiteson. 2019. “**Generalized Off-Policy Actor-Critic**”. ArXiv:1903.11329, October.
- [31] Gelada, Carles, and Marc G. Bellemare. 2019. “**Off-Policy Deep Reinforcement Learning by Bootstrapping the Covariate Shift**”. ArXiv:1901.09455, January.

- [32] Zhang, Shangdong, Wendelin Boehmer, and Shimon Whiteson. 2019. “Generalized Off-Policy Actor-Critic”. ArXiv:1903.11329, October.
- [33] Richter, Oliver, and Roger Wattenhofer. 2019. “Learning Policies through Quantile Regression”. ArXiv:1906.11941, September.
- [34] Henderson, Peter, Riashat Islam, Philip Bachman, Joelle Pineau, Doina Precup, and David Meger. 2019. “Deep Reinforcement Learning That Matters”. ArXiv:1709.06560, January.

Learning All Possible Policies with Entropy Methods

Deep reinforcement learning (RL) is a standard tool due to its ability to process and approximate complex observations, which result in elaborate behaviors. However, many deep RL methods optimize for a deterministic policy, since if you had full observability, there is only one best policy. But it is often desirable to learn a stochastic policy or probabilistic behaviors to improve robustness and deal with stochastic environments.

What Is Entropy?

Shannon entropy (abbreviated to *entropy* from now on) is a measure of the amount of information contained within a stochastic variable, where information is calculated as the number of bits required to encode all possible states. **Equation 7-1** shows this as an equation where $X \doteq \{x_0, x_1, \dots, x_{n-1}\}$ is a stochastic variable, $\mathcal{H}$ is the entropy, I is the information content, and b is the base of the logarithm used (commonly bits for $b \doteq 2$, bans for $b \doteq 10$, and nats for $b \doteq e$). Bits are the most common base.

Equation 7-1. The information content of a random variable

$$\mathcal{H}(X) \doteq \mathbb{E}[I(X)] = - \sum_{x \in X} p(x) \log_b p(x)$$

For example, a coin has two states, assuming it doesn't land on its edge. These two states can be encoded by a zero and a one, therefore the amount of information contained within a coin, measured by entropy in bits, is one. A die has six possible states, so you would need three bits to describe all of those states (the real value is 2.5849...).

A probabilistic solution to optimal control is a stochastic policy. To have an accurate representation of the action-probability distribution, you must sample enough states and actions. You could measure the number of states and actions you have visited, like in UCB (see “[Improving the \$\epsilon\$ -greedy Algorithm](#)” on page 33). But this isn’t directly coupled to a policy; UCB is an exploration strategy, not part of the policy. Instead, you can use a proxy measure of how well distributed a policy is, like entropy, and include that as a penalty term in the objective function.

Maximum Entropy Reinforcement Learning

Maximizing the entropy of a policy forces the agent to visit all states and actions. Rather than aiming to find a deterministic behavior that maximizes the reward, the policy can learn *all* behaviors. In other words, instead of learning the best way of performing a task, the entropy term forces the agent to learn *all* the ways to solve a task. This leads to policies that can intuitively explore and provide more robust behavior in the face of adversarial attacks—pelting a robot with bricks, for example. See the [videos on the PPO blog post](#) for some fun. I talk more about this in [Chapter 10](#).

[Equation 7-2](#) shows the objective function for maximum entropy RL, where α is a hyperparameter (called the *temperature*, which links back to the thermodynamic definition of entropy) that determines the relative importance of the entropy term against the reward and controls the stochasticity of the optimal policy.

Equation 7-2. Maximum entropy objective function

$$\pi^* \doteq \operatorname{argmax}_{\pi} \sum_{s \in \mathcal{S}, a \in \mathcal{A}} \mathbb{E}[r + \alpha \mathcal{H}(\pi(a \mid s))]$$

[Equation 7-2](#) states that the optimal policy is defined as the policy that maximizes the return regularized by the entropy of the policy. This is similar to the standard definition of an optimal policy except that it is augmented with a bonus to promote nondeterministic policies, the exact opposite of deterministic policy gradients.

How well this works depends on the problem. If you have a discretized problem that only has one optimal trajectory, and you never expect that the agent will veer off that trajectory, then a deterministic policy is a good choice. But in continuous problems, where it is likely that the agent will veer away from the optimal policy, then this might be a better option.

The α hyperparameter allows you to control how much of a bonus is included. Values close to zero become deterministic and larger values promote nondeterminate actions and therefore more exploration.

Soft Actor-Critic

The soft actor-critic (SAC) algorithm is an off-policy policy gradient approximation of the maximum entropy objective function.¹ The term “soft” is used to emphasize the nondeterministic elements within the algorithm; the standard Bellman equation with no entropy term could be considered “hard,” so Q-learning and DDPG are corresponding “hard” algorithms.

Haarnoja et al. begin by defining the critic as a soft approximation of the action-value function presented in an earlier paper.² Recall that the goal is to generate a policy that maximizes the entropy, which means the agent should value high entropy states more than others. Following Equation 7-2, Equation 7-3 shows that the value of a state is the action value plus a bonus for the entropy of that new state. I have simplified the notation to match the rest of this book; Equation 7-3 assumes that you are following a policy, but I hide that information to make the mathematics easier to understand. V and Q represent state-value and action-value functions, respectively.

Equation 7-3. Maximum entropy state-value function

$$\begin{aligned} V(s) &= \mathbb{E}[Q(s, a) + \alpha \mathcal{H}(\pi(a | s))] \\ &= \mathbb{E}[Q(s, a) - \alpha \log \pi(a | s)] \end{aligned}$$

Using Equation 7-3, Equation 7-4 shows the corresponding action-value function from dynamic programming.

Equation 7-4. Maximum entropy action-value function

$$Q(s, a) \doteq \mathbb{E}[r + \gamma \mathbb{E}[V(s')]]$$

To handle complex state spaces you can train a function to approximate Equation 7-4, for which you need an objective function (Equation 7-5).

Equation 7-5. Maximum entropy action-value objective function

$$\begin{aligned} J_Q(\theta) &= \mathbb{E}\left[\frac{1}{2}(Q_\theta(s, a) - Q(s, a))^2\right] \\ &= \mathbb{E}\left[\frac{1}{2}(Q_\theta(s, a) - (r + \gamma \mathbb{E}[V_\theta(s')]))^2\right] \end{aligned}$$

You can derive the gradient of Equation 7-4 (you will see this shortly), but SAC implementations typically leverage the automatic differential frameworks included with your deep learning library. Next, you need a policy for the actor and a similar objective function. It is possible to derive a soft policy gradient analytically.³ But Haarnoja et al. chose to minimize the Kullback–Leibler (KL) divergence instead (see

“Kullback–Leibler Divergence” on page 165). The reason for this was because Liu and Wang developed an approach called *Stein variational gradient descent*, which is an analytical solution to the gradient of the KL divergence, the derivation of which is beyond the scope of the book.⁴ Haarnoja et al. applied this technique to the soft policy, which results in the policy objective shown in Equation 7-6, which you can solve using auto-differentiation.

Equation 7-6. SAC policy objective function

$$J_{\pi}(\phi) = \mathbb{E} \left[\alpha \log \left(\pi_{\phi}(a \mid s) \right) - Q_{\theta}(s, a) \right]$$

SAC Implementation Details and Discrete Action Spaces

There are details within the derivation that impact the implementation. First, performance is highly sensitive to the choice of temperature parameter. You need to perform a hyperparameter scan to establish the best temperature for each problem. Second, because of the sensitivity to the temperature hyperparameter, this also means that it is sensitive to the scale of the rewards. For example, large rewards would overwhelm small temperature values. Rescale your rewards to mitigate. These two issues are fixed in the next algorithm.

Finally, Haarnoja et al. assume Gaussian actions, which makes the problem tractable (called the *reparameterization trick*) and allows it to work with continuous action spaces.⁵ SAC does not work with discrete action spaces out of the box.

This capability is added in the SAC algorithm suggested by Christodoulou, which reformulates the maximum entropy problem with discrete actions—the differences are minimal.⁶ You might ask why use a policy gradient algorithm over something like DQN. The major benefit of using SAC with discrete action spaces is that it can compete with value methods in terms of learning performance but with zero hyperparameter tuning. SAC implements this via a subsequent algorithm that automatically adjusts the temperature.

Automatically Adjusting Temperature

A follow-up paper from Haarnoja et al. presented a way to automatically adjust the SAC temperature parameter by converting the problem into a constraint, rather than a penalty. They constrain the current policy to have a higher entropy than an expected minimum entropy of the solution. Then they find that they can iteratively update a model of the temperature parameter to minimize the difference between the entropy of the current policy and expected entropy, as shown in Equation 7-7, where α is the temperature parameter and $\mathcal{H} \doteq -\log |\mathcal{A}|$ is the expected entropy, which is the same as the negative of the number of actions.⁷

Equation 7-7. SAC temperature parameter objective function

$$J_{\alpha} = \mathbb{E} \left[-\alpha \log (\pi(a \mid s)) - \alpha \overline{\mathcal{H}} \right]$$

Haarnoja et al. add this line to the original SAC algorithm to turn it into a hyperparameter-free off-policy policy gradient algorithm. This is an enticing feature, because it mitigates against developer bias and speeds up development, with state-of-the-art performance. Of course, you still have tuning to do: neural network structure, state engineering, action class, and so on. But this is a step in the right direction.

Case Study: Automated Traffic Management to Reduce Queuing

There is nothing more universal than the synchronized groan from passengers when approaching a traffic jam. Bottlenecks generated by work zones and incidents are one of the most important contributors to nonrecurrent congestion and secondary accidents. “Smart” highways aim to improve average speed in a high-density flow by suggesting when users should merge into open lanes. Recent developments in automated driving like adaptive cruise control, which matches your speed with the vehicles in front of you, allow vehicles to drive at high speed while maintaining a small gap. But this doesn’t take advantage of knowledge of vehicles in other lanes or behind. In the future, vehicles will be able to access greater amounts of local information, which will potentially allow vehicles to collaborate. If vehicles can collaborate, then you can use RL to derive a global optimal policy to increase speed and density to reduce congestion and the number of secondary incidents.

Ren, Xie, and Jiang have developed an RL approach to solving this problem. They first split the road into zones, one of which is a merging zone where vehicles are forced to merge into an open lane. A previous zone allows for the vehicle to speed up or slow down so that they can align with traffic in an open lane.⁸

The state of the MDP is a velocity and acceleration matrix within this zone, and local information about the vehicle under control. This is fed into quite a complicated neural network that includes convolutional, fully connected, and recurrent elements.

The action is control over a single vehicle’s speed, in order to position it within a safe gap. They iterate over all vehicles currently within the control zone.

The reward is complex, but aims to promote successful merges (no crashes and with a safe distance) and increased speed.

They choose SAC as the algorithm of choice, because of the continuous actions, its off-policy sample efficiency, and its stability during exploration.

They simulate this formulation in a traffic simulation tool called **VISSIM** and compare the RL implementation to common road-sign-based merge strategies, which either tell vehicles to merge into the open lane early or late. The RL solution

significantly improves mobility over the basic algorithms, reducing delays by an order of magnitude. It is also much safer, with stable, low-valued densities throughout the queue, whereas the basic algorithms promoted a dangerous backward-forming shockwave.

Despite impressive results, there is quite a lot of room for improvement. First, this is a simulation. It would have been great to have some real-life results, although I know from experience that it is incredibly difficult to develop against potentially dangerous environments such as this. I also would like to see more realistic observations. For example, vehicle position and local information can be inferred from video cameras. There is no need to assume that the vehicles or the oracle have perfect knowledge of the vehicles around them. Finally, I am concerned about the complexity of the policy model for what is quite a simple state space. I don't believe that this much complexity is required, and that the state representation should be delegated. I'm confident the policy itself, for selecting the right speed to fit into a gap, is actually quite simple to learn.

Extensions to Maximum Entropy Methods

SAC provides you with a simple and performant off-policy algorithm out of the box but like other algorithms, researchers continue to offer improvements in the search for maximum performance. As usual, these improvements do not guarantee better performance for your specific problem; you have to test them.

Other Measures of Entropy (and Ensembles)

Shannon entropy is only one way of measuring entropy. There are other forms that alter the way SAC penalizes the policy to prefer or ignore unpromising actions. Chen and Peng try Tsallis and Rényi entropy and find that these improve performance on baseline algorithms.⁹ They also demonstrate the use of an ensemble of policies, whereby you train multiple policies at the same time. Upon each new episode, the agent uses a random policy from this ensemble to choose actions. This approach comes from bootstrapped DQN to improve exploration (see [“Improving Exploration” on page 108](#)).

Optimistic Exploration Using the Upper Bound of Double Q-Learning

Similar to how double Q-learning formulates a lower bound on the action-value estimate (see [“Double Q-Learning” on page 74](#)) Ciosek et al. propose using the pessimistic bound as the target policy (with an entropy term that makes it look similar to standard SAC), but the *upper bound*, the action value that overestimates the reward, for the behavior policy.¹⁰ They suggest that this helps prevent *pessimistic under-exploration* caused by relying on the lower bound. Empirically, the results on the standard MuJoCo environments are better, but not dramatically. You might want to

consider this approach if you are working in an environment where agents are struggling to explore fully.

Tinkering with Experience Replay

As described in “[Prioritized Experience Replay](#)” on page 102, which experiences you choose to learn from have an impact on learning performance. Wang and Ross demonstrate that adding prioritized experience replay can improve SAC on some environments, but oversampling more recent experience improves results consistently across all environments.¹¹ This is a simple addition and worth a try, but watch out for *catastrophic forgetting* (see “[Experience Replay](#)” on page 92), whereby the agent forgets previous experiences. A large replay buffer mitigates against this effect.

Soft Policy Gradient

Rather than penalizing by an entropy measure, why not derive an entropy-inspired soft policy gradient instead? Shi, Song, and Wu introduced the soft policy gradient method that is reportedly more principled and stable than SAC.¹²

Shi et al. start by taking the maximum entropy definition of RL and differentiate the policy to result in an gradient calculation that is surprisingly simple, as shown in [Equation 7-8](#). This is more effective than using the automatic differential frameworks because you can then observe and operate directly on the gradients, so they chose to clip the gradients directly rather than apply trust-region optimization, to prevent catastrophic updates. This results in a simpler implementation and better stability. I would not be surprised if this becomes the “default” maximum entropy interpretation in the future.

Equation 7-8. Deep soft policy gradient

$$\nabla J_{\pi}(\phi) = \mathbb{E}[(Q_{\theta}(s, a) - \log \pi(a | s) - 1) \nabla_{\phi} \log \pi(a | s)]$$

Soft Q-Learning (and Derivatives)

You can also apply maximum entropy RL to Q-learning. Haarnoja et al. first showed that you can derive entropy-based value and action estimates.¹³ Others have applied the same idea to calculate the advantage function and found it to work well on complex, potentially continuous environments that have discrete action spaces.¹⁴ I can imagine that integrating these estimates into other algorithms, like PPO, may bring positive results.

Path Consistency Learning

Path consistency learning (PCL) from Nachum et al. is related to SAC, because it appreciates that entropy encourages exploration and derives the same soft state and action value functions as soft Q-learning. However, PCL investigates using rollouts, in the style of n -step algorithms, and leads to an n -step off-policy algorithm that does not need any correction (importance sampling, for example).¹⁵ A subsequent paper implemented a KL-divergence-based trust method (Trust-PCL) that beats TRPO.¹⁶ This highlights that even with deep learning and off-policy sampling techniques, you can still incorporate improvements to the underlying temporal-difference algorithms. Despite this, SAC tends to dramatically outperform PCL methods.

Performance Comparison: SAC Versus PPO

There is an elephant in the room and it runs amok throughout RL: which algorithm is better for continuous control tasks, PPO or SAC? It is generally accepted that SAC and its derivatives perform better than PPO. But it's not as clear as it should be. As I mentioned before, researchers measure performance in different ways and implementation details severely alter performance.¹⁷ The main issue for this section was that researchers mostly don't publish the raw values for learning curves. And often they don't even report values in tables, just in plots. This makes it incredibly hard to compare results between papers.

Table 7-1 shows a sample of results from papers and implementations. All results were taken at 1 million steps, except for (†) at 1.5 million steps. (*) only published images of results, so I estimated performance from the images.

Table 7-1. Performance results from continuous control benchmarks for SAC and PPO

Environment	SAC ^a	SAC ^b	SAC ^{†c}	SAC-auto ^{*d}	PPO ^e	PPO ^{†f}	SAC-auto ^g	PPO ^h
MuJoCo Walker	3475	3941	3868	4800	3425	3292		
MuJoCo Hopper	2101	3020	2922	3200	2316	2513		
MuJoCo Half-cheetah	8896	6095	10994	11000	1669			
MuJoCo Humanoid	4831		5723	5000	4900	806		
MuJoCo Ant	3250	2989	3856	4100				
MuJoCo Swimmer			42		111			
AntBulletEnv-v0							3485	2170
BipedalWalker-v2							307	266
BipedalWalkerHardcore-v2							101	166
HalfCheetahBulletEnv-v0							3331	3195
HopperBulletEnv-v0							2438	1945
HumanoidBulletEnv-v0							2048	1286

Environment	SAC ^a	SAC ^b	SAC ^{†c}	SAC- auto ^{*d}	PPO ^e	PPO ^{†f}	SAC- auto ^g	PPO ^h
InvertedDoublePendulumBulletEnv-v0							9357	7703
InvertedPendulumSwingupBulletEnv-v0							892	867
LunarLanderContinuous-v2							270	128
MountainCarContinuous-v0							90	92
Pendulum-v0							-160	-168
ReacherBulletEnv-v0							18	18
Walker2DBulletEnv-v0							2053	2080

[a] Haarnoja, Tuomas, Aurick Zhou, Pieter Abbeel, and Sergey Levine. 2018. “Soft Actor-Critic: Off-Policy Maximum Entropy Deep Reinforcement Learning with a Stochastic Actor”. ArXiv:1801.01290, August.

[b] Wang, Tingwu, Xuchan Bao, Ignasi Clavera, Jerrick Hoang, Yeming Wen, Eric Langlois, Shunshi Zhang, Guodong Zhang, Pieter Abbeel, and Jimmy Ba. 2019. “Benchmarking Model-Based Reinforcement Learning”. ArXiv:1907.02057, July.

[c] Wang, Che, and Keith Ross. 2019. “Boosting Soft Actor-Critic: Emphasizing Recent Experience without Forgetting the Past”. ArXiv:1906.04009, June.

[d] Haarnoja, Tuomas, Aurick Zhou, Kristian Hartikainen, George Tucker, Sehoon Ha, Jie Tan, Vikash Kumar, et al. 2019. “Soft Actor-Critic Algorithms and Applications”. ArXiv:1812.05905, January.

[e] Dhariwal, Prafulla, Christopher Hesse, Oleg Klimov, Alex Nichol, Matthias Plappert, Alec Radford, John Schulman, Szymon Sidor, Yuhuai Wu, and Peter Zhokhov. 2017. *OpenAI Baselines*. GitHub Repository.

[f] Logan Engstrom et al. 2020. “Implementation Matters in Deep Policy Gradients: A Case Study on PPO and TRPO”. ArXiv: 2005.12729, May.

[g] Raffin, Antonin. (2018) 2020. Aaffin/RI-Baselines-Zoo. Python. <https://oreil.ly/H3Nc2>

[h] Raffin, Antonin. (2018) 2020. Aaffin/RI-Baselines-Zoo. Python. <https://oreil.ly/H3Nc2>

The first thing you will notice is the variance of the results. Some SAC implementations and experiments perform far worse than others. In these it is likely that worse hyperparameter tuning caused worse performance. This is why the automatic temperature variant of SAC is so important. This achieves good results without tuning. If you scan across all the numbers, then SAC mostly outperforms PPO in the MuJoCo environments. Looking at the open source PyBullet results also suggests that automatic temperature SAC is better on the whole, but there are many environments where performance is similar.

These numbers also don't tell the whole story, though. They don't tell you whether performance will continue to increase if you left them training. They also represent average performance; this doesn't tell you how stable training runs were. As I have recommended several times already, always retest algorithms and implementations yourself for your specific problems. Your results might not match published baselines.

How Does Entropy Encourage Exploration?

Maximum entropy reinforcement learning encourages application, but how? [Equation 7-2](#) added a bonus proportional to the entropy of the actions in a state, which means that the agent observes an artificial reward higher than the actual reward. The amount of extra reward is defined by the temperature value and the entropy of the actions.

Imagine a child in a candy store. You could measure candy anticipation using entropy; more choices mean more enjoyment. In the middle of the shop, the child could move in hundreds of different directions and get a new candy. But at the door, right at the edge, there's only a few to choose from. Entropy suggests that the child would have more choices if they move toward the center of the shop.

By combining the reward with an entropy bonus you are promoting actions that are not strictly optimal. This helps to ensure that all states are adequately sampled.

To demonstrate, I created a simple grid environment with the goal in the top right-hand corner and the starting point in the middle. The agent is allowed to move in any direction (including diagonals) and there is a sparse reward of one when the agent reaches the goal. The episode ends with zero reward if agents do not reach the goal within 100 steps.

I used an ϵ -greedy Q-learning agent as a baseline. [Figure 7-1](#) shows the result of running the agent with action values plotted as pixels; white colors represent high action values, black low. The arrow denotes the optimal trajectory (no exploration) for the policy. You can see that it has found a path to the goal, but it is suboptimal in terms of the number of steps (remember, I am not penalizing for steps and the exploration is dependent on the random seed) and the agent hasn't explored other regions of the state. If you changed the starting location of the agent, it would not be able to find its way to the goal. This is not a robust policy.

One simple enhancement that accounts for much of SAC's performance is the use of the action values as a distribution, rather than relying on the random sampling of ϵ -greedy for exploration. When this crude probabilistic approach is applied to the Q-learning algorithm by using a softmax function to convert the action values to probabilities, exploration improves dramatically, as you can see in [Figure 7-2](#). I also plot the most probable action as arrows for each state, of which the direction and magnitude is based upon the weighted combination of action values. Note how the direction arrows are all pointing toward the optimal trajectory, so no matter where the agent starts, it will always reach the goal. This policy is far more robust and has explored more of the state space than the previous agent.

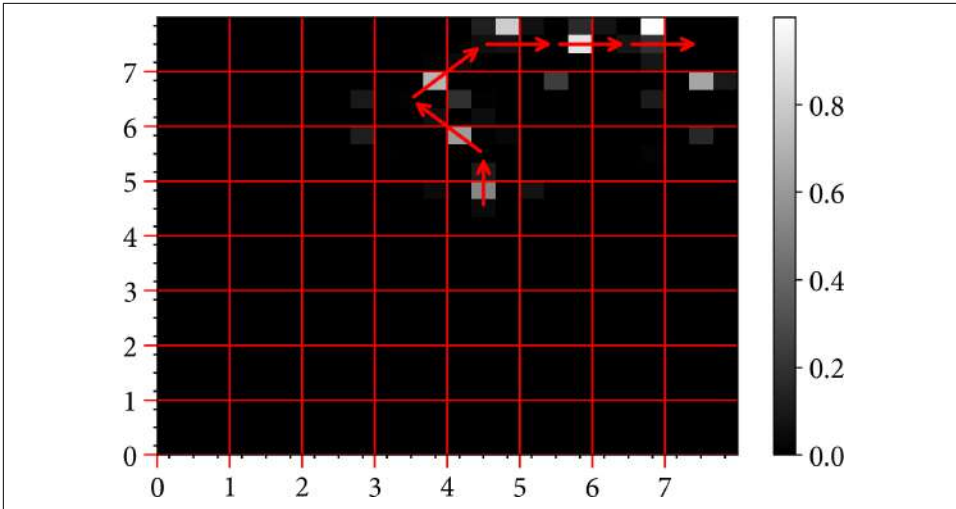


Figure 7-1. The results of using an ϵ -greedy Q-learning agent on a grid environment. Pixels represent action values and the grid represents the state. The arrows are the optimal trajectory according to the current policy.

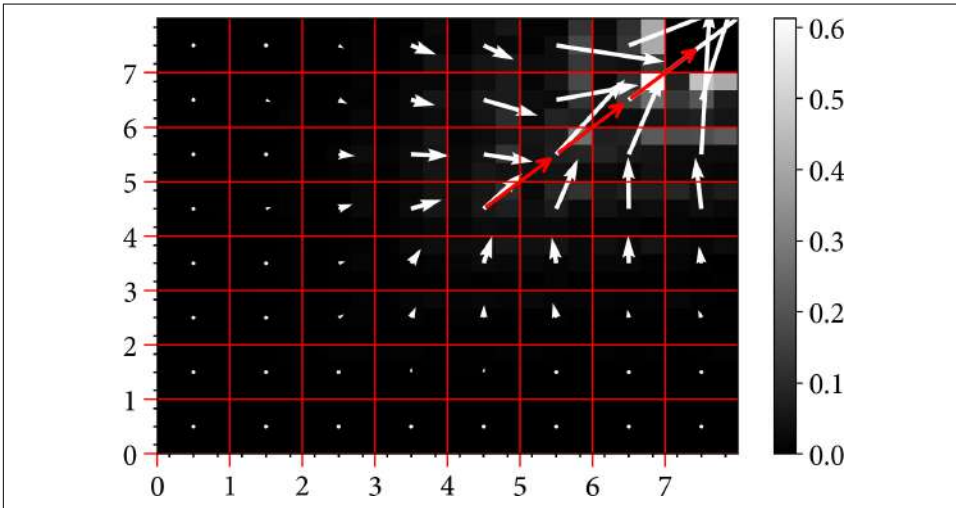


Figure 7-2. The results of using a crude probabilistic, softmax-based Q-learning agent on a grid environment. Decorations are the same as Figure 7-1 except state arrows represent the most probable action with a direction and magnitude that is weighted by the action values.



The performance of a robust policy should degrade gracefully in the presence of adversity. Policies that have uncharted regions near the optimal trajectory are not robust because the tiniest of deviations could lead to states with an undefined policy.

Next I implemented soft Q-learning (see “[Soft Q-Learning \(and Derivatives\)](#)” on page 197) and set the temperature parameter to 0.05. You can see the results in [Figure 7-3](#). The optimal policies are unremarkable, even though they generally point in the right direction. But if you look closely, you can see that the action values and the direction arrows are smaller or nonexistent on the Q-learning side of the plot. Soft Q-learning has visited states other than the optimal trajectory. You can also see that the white colors representing larger action values are shifted toward the center of the images for soft Q-learning. The reason is explained by the candy store example; entropy promotes states with the greatest amount of choice.

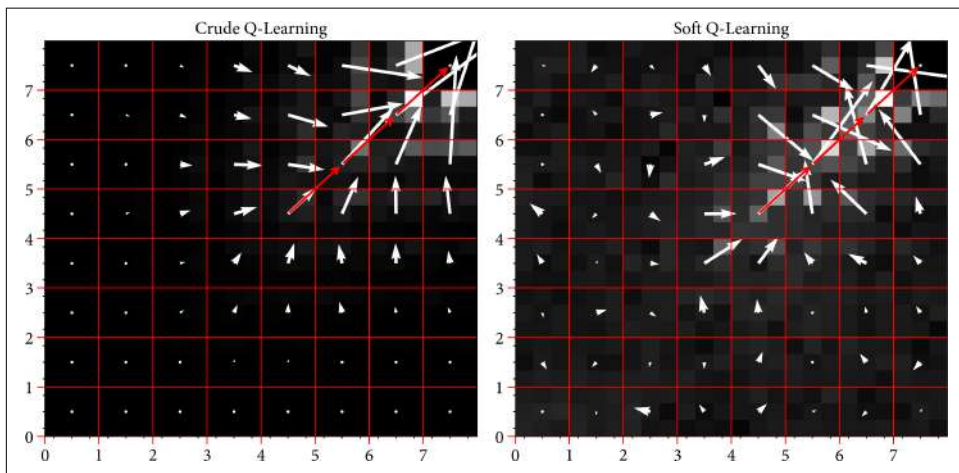


Figure 7-3. Comparing crude probabilistic Q-learning against soft Q-learning with the same setup as [Figure 7-2](#).

The lack of exploration in the Q-learning agent is even more pronounced if you keep track of the number of times each agent visits each action, as I did in [Figure 7-4](#). You can see that the soft Q-learning agent has sampled states more uniformly. This is why entropy helps exploration in more complex state spaces.

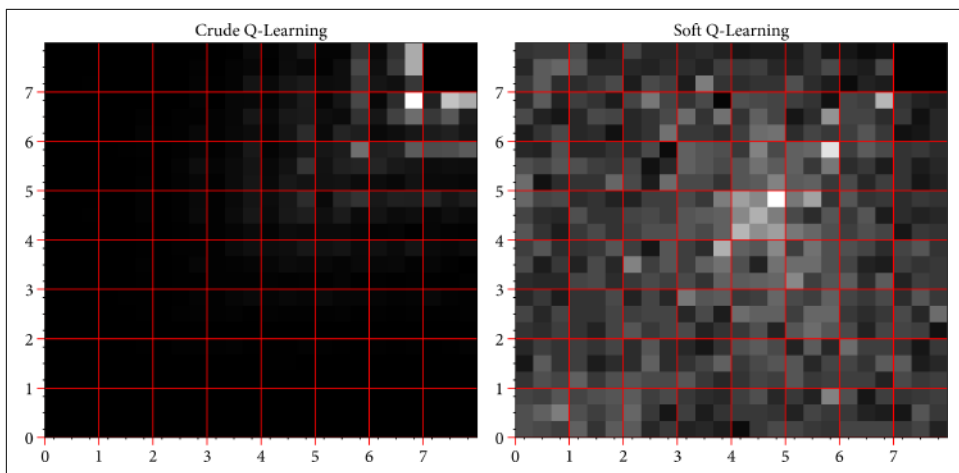


Figure 7-4. Number of action-value interactions for both agents. Colors represent the number of times the agent has visited each action.

How Does the Temperature Parameter Alter Exploration?

The soft Q-learning (SQL) or SAC algorithm is very dependent on the temperature parameter. It controls the amount of exploration, and if you weight entropy too highly then the agent receives higher rewards for exploration than it does for achieving the goal!

There are two solutions to this problem. The simplest is *annealing*. By reducing the temperature parameter over time, this forces the agent to explore early on, but become deterministic later. A second approach is to automatically learn the temperature parameter, which is used in most SAC implementations.

In the experiment in [Figure 7-5](#) I increase the temperature value to promote more exploration. As you can see from the magnitudes of the action values, the entropy term is now so large that it overpowers the reward (with a value of 1). So the optimal policy is to move toward the states with the highest entropy, which are the states in the middle, like in the candy store.

To combat this problem, you can reduce the temperature over time to guide the child, I mean agent, toward the goal. In the experiment shown in [Figure 7-6](#) I anneal the temperature parameter from 0.5 to 0.005. Note that the original SAC paper used annealing.

This experiment demonstrates how annealing helps the agent explore in early episodes. This is extremely useful in complex environments that require large amounts of exploration. The change in temperature causes the agent to become more Q-learning-like and homes in on the optimal policy.

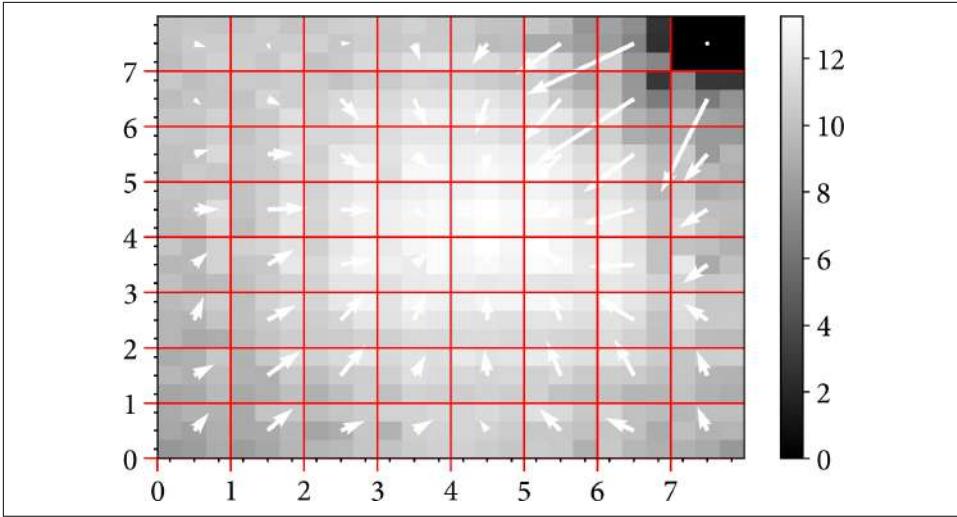


Figure 7-5. Setting the temperature parameter to 1 in the soft Q-learning agent. To emphasize the effect, I increased the number of episodes to 500. All other settings are as in Figure 7-3.

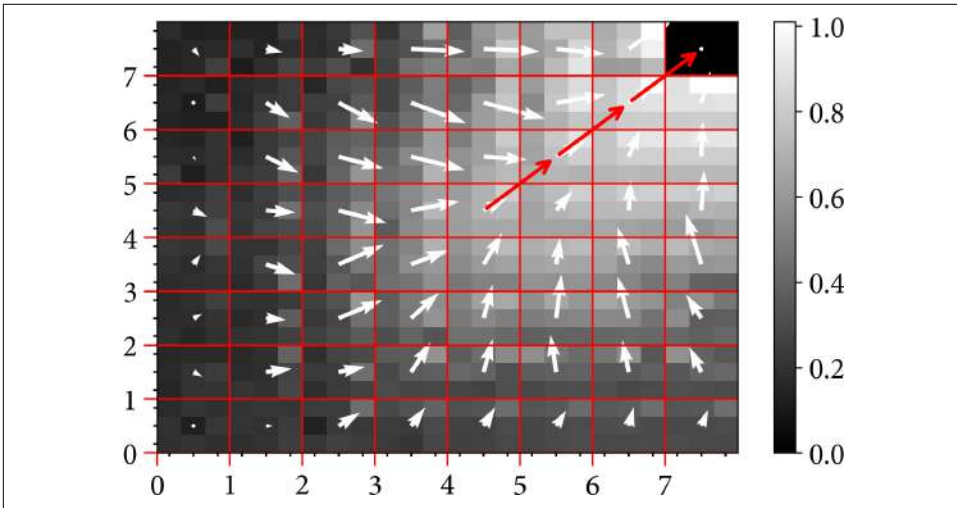


Figure 7-6. Annealing the temperature parameter to from 0.5 to 0.005 in the soft Q-learning agent. All other settings are as in Figure 7-5.

However, the starting, stopping, and annealing rate parameters are important. You need to choose these carefully according to your environment and trade off robustness against optimality.

Industrial Example: Learning to Drive with a Remote Control Car

Donkey Car began life as an open source Raspberry Pi-based radio control car and made the bold move to include a *single* sensor; a wide-angle video camera. All of the control is derived from a single image, which poses a significant challenge and opportunity for RL. The name Donkey was chosen to set low expectations. But don't let the name fool you; this straightforward piece of hardware is brought to life through advanced machine learning. Take a look at some of the impressive results and learn more on the [Donkey Car website](#).

Tawn Kramer, a Donkey Car enthusiast, developed a simulation environment based upon the Unity framework.¹⁸ You can use this as a rapid development platform to develop new self-driving algorithms. And after I was inspired by Antonin Raffin, one of the core contributors to the *Stable Baselines* range of libraries, I decided to have a go myself.¹⁹ In this example I demonstrate that a SAC-based RL approach generates a viable policy.

Description of the Problem

The state is represented by an observation through a video camera. The video has a resolution of 160×120 with 8-bit RGB color channels. The action space is deceptively simple, allowing a throttle amount and a steering angle as input. The environment rewards the agent for speed and staying in the center of a lane/track on every time step. If the car is too far outside its lane it receives a negative reward. The goal is to reach the end of the course in the fastest time possible.

There are several tracks. The simplest is a desert road with clearly defined and consistent lanes. The hardest is an urban race simulation, where the agent has to navigate obstacles and a track without clear boundaries.

The Unity simulation is delivered **as a binary**. The Gym environment connects to the simulation over a websocket and supports multiple connections to allow for RL-driven competitive races.

Minimizing Training Time

Whenever you work with data that is based upon pixels, the majority of your training time will be taken up by feature extraction. The key to rapid development of a project like this is to split the problem; the more you can break it down, the faster and easier it is to improve individual components. Yes, it is possible to learn a viable policy directly from pixels, but it slows down development iterations and is costly. Your wallet will thank you.

One obvious place to start is to separate the vision part of the process. The RL agent doesn't care if the state is a flattened image or a set of high-level features. But performance is impacted by how informative the features are; more informative features lead to faster training and more robust policies.

One computer vision approach could be to attempt to segment the image or detect the boundaries of the track. Using a perspective transformation could compensate for the forward view of the road. A segmentation algorithm or a trained segmentation model could help you pick out the road. You could use a Hough or Radon transform to find the lines in the image and use physical assumptions to find the nearest lanes. You can probably imagine more. The goal, remember, is to make the observation data more informative.

Another (or symbiotic) approach is to train an autoencoder (see “[Common Neural Network Architectures](#)” on page 89) to constrain the number of features to those that are the most informative. This consists of using a convolutional neural network that is shaped like an hourglass to decompose and reconstruct images from a dataset until the small number of neurons in the middle reliably encode all necessary information about the image. This is an offline process, independent of the RL. The pretrained low-dimensional representation is then used as the observation for RL.

Figures 7-7 and 7-8 show the original observation and reconstruction, respectively, after training a variational autoencoder for 1,000 training steps. Note how there are blurry similarities at this stage.



Figure 7-7. Raw observation of the DonkeyCar environment at 1,000 training steps.



Figure 7-8. Reconstruction of the DonkeyCar environment at 1,000 training steps by the variational autoencoder.

Figures 7-9 and 7-10 show another observation and reconstruction after 10,000 training steps. Note how much detail is now in the reconstruction, which indicates that the hidden neural network embedding has learned good latent features.



Figure 7-9. Raw observation of the DonkeyCar environment at 10,000 training steps.



Figure 7-10. Reconstruction of the DonkeyCar environment at 10,000 training steps by the VAE.

Figure 7-11 compares videos of the observation from the RGB camera on the Donkey Car against a reconstruction of the low-dimensional observation, which is the input to the SAC algorithm.

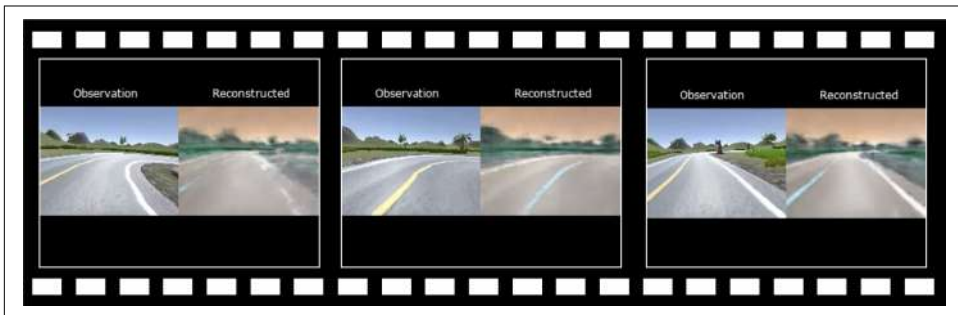


Figure 7-11. A reconstruction of the DonkeyCar simulation through the eyes of a variational autoencoder. The video on the left shows the raw video feed from the simulation. On the right is the reconstructed video from a low-dimensional state. This reconstructed video has been cropped to remove the sky region, which is unnecessary for this problem.

Reducing the state space, the input to the reinforcement learning algorithm, will reliably cut down RL training time to a matter of minutes, rather than days. I talk more

about this in “Transformation (dimensionality reduction, autoencoders, and world models)” on page 267.

Dramatic Actions

In some environments it is possible to apply actions that dramatically alter the environment. In the case of DonkeyCar, large values for the steering action cause the car to spin, skid, and otherwise veer off the optimal trajectory. But of course, the car needs to steer and sometimes it needs to navigate around sharp corners.

But these actions and the subsequent negative rewards can cause training to get stuck, because it constantly skids and never manages to capture enough good observations. In the DonkeyCar environment, this effect becomes worse when large jumps in actions are allowed. For example, when you ride a bike, you know it is technically possible to turn the steering sharply at speed, but I doubt that you have ever tried it. You’ve probably tried something much less vigorous and had a bad experience, so you will never try anything like turning the steering by 90 degrees. Figure 7-12 shows some example training footage with the original action space.



Figure 7-12. A video of DonkeyCar training when the action space is left unaltered.

You can influence the training of the agent by including this prior information in your RL model. You could alter the reward signal to promote (or demote) certain states. Or, as in this case, you can filter the action of the agent to make it behave in a safer way.

If you try training the agent using the full range of steering angles (try it!) then you will see that after a long time it learns to steer away from the edges, but it does it so forcefully it sends the car into a three-wheel drift. Despite this looking cool, it doesn’t help finish the course. You need to smooth out the steering.

One of the simplest ways of constraining how an agent behaves is to alter the action space. Like in machine learning, constraining the problem makes it easier to solve. In the training shown in Figure 7-13 I have limited the throttle to a maximum of 0.15 (originally 5.0) and steering angle to an absolute maximum of 0.5 (originally 1.0).

The slow speed prevents the car from going into a skid and the smaller steering angle helps to mitigate against the jerky steering. Note that training performance is greatly impacted by these parameters. Try altering them yourself.



Figure 7-13. A video of DonkeyCar training after simplifying the action space.

Hyperparameter Search

The maximum throttle and steering settings are hyperparameters of the problem; they are not directly tuned. Changes to the problem definition tend to impact performance the most, but there are a host of other hyperparameters from the SAC algorithm, like the architecture of the policy model. The objective is to find what combination of parameters leads to the best rewards. To do this you would perform a hyperparameter search, by writing some code that iterates through all permutations and reruns the training. You can read more about hyperparameter tuning in [“Hyperparameter tuning”](#) on page 324.

I ran an optimized grid search using the `optuna` library to tune the hyperparameters of the SAC algorithm and viewed all the results in TensorBoard. I found that I could increase the learning rates by an order of magnitude to speed up learning slightly. Tweaking the entropy coefficient also helped; by default the `stable-baselines` implementation starts with a value of 1.0 and learns the optimal value. But in this example the agent trains a small number of steps, so it doesn't have time to learn the optimal coefficient. Starting from 0.1 and learning the optimal coefficient worked better. But overall I was underwhelmed by how much the hyperparameters affected performance (which is a good thing!). Altering the hyperparameters of the variational autoencoder and the changing the action space impacted performance much more.

Final Policy

After all the engineering work, all the code, and searching for hyperparameters, I now have a viable policy that has been trained for between 5,000 and 50,000 environment steps, depending on the environment and how perfect you want to the policy to be. This took about 10–30 minutes on my old 2014 Macbook Pro. Figures 7-14 and 7-15

show example policies for the donkey-generated-track-v0 and donkey-generated-road-v0 environments, respectively.



Figure 7-14. A video of the final DonkeyCar policy for the donkey-generated-track-v0 environment.



Figure 7-15. A video of the final DonkeyCar policy for the donkey-generated-road-v0 environment.

But note throughout this I barely touched upon the RL algorithm. All the hard work went into feature engineering, hyperparameter searches, and constraining the problem to make it simpler. These are precisely the tasks that take all the time in industrial ML projects. Also like ML, RL algorithms will become standardized over time, which will lead to the situation where you won't need to touch the RL algorithms at all.

Further Improvements

You could make a multitude of improvements to this policy. By the time you read these words, other engineers will have crushed my feeble effort. You could spend your time improving the autoencoding or trying any of the other ideas suggested in “**Minimizing Training Time**” on page 205. You could alter the action space further, like increasing the maximum actions until it breaks. One thing I wanted to try was annealing the action space limit, whereby the action space starts small and increases up to the maximum values over training time. You could try altering the rewards, too;

maybe speed shouldn't be rewarded so highly. Attempting to penalize jerkiness makes sense, too.

You can probably imagine many more, but if you are working on an industrial problem then you will need to take more care: take baby steps, make sure you are evaluating correctly, make sure what you are doing is safe, split the problem up, constrain the problem, and test more. You can read more about this in [Chapter 9](#).

Summary

RL aims to find a policy that maximizes the expected return from sampled states, actions, and rewards. This definition implies maximization of the expected return irrespective of the consequences. Research has shown that bounding or constraining the policy updates leads to more stable learning and better rounded policies.

[Chapter 6](#) approached this problem by constraining updates so they don't dramatically alter the policy. But this can still lead to brittle, overly sparse policies where most actions have zero value except for the optimal trajectory. This chapter introduced entropy-based algorithms, which modify the original definition of MDP returns to rescale or *regularize* the returns. This led to proposals that other definitions of entropy might perform better, because Shannon entropy is not sparse enough. I would argue that it is possible to tweak the temperature parameter to produce a policy that suits your own problem. But it is reasonably likely that another entropy measure will perform better for your specific problem.

The logical conclusion of this argument is to use an arbitrary function. This was proposed by Li, Yang, and Zhang in *regularized MDPs* and mandates several theoretical rules that the function has to follow, like that it should be strictly positive, differentiable, concave, and fall to zero outside of the range of zero to one. They suggest that other exponential-based functions and trigonometric functions are interesting proposals.²⁰

Irrespective of the implementation, augmenting the value function with an exploration bonus is a good thing for production implementations. You want your agent to be robust to change. Be careful not to overfit to your environment.

Equivalence Between Policy Gradients and Soft Q-Learning

To round this chapter off I wanted to come back to a comparison of value methods to policy methods. Simplistically, Q-learning and policy gradient methods both attempt to reinforce behavior that leads to better rewards. Q-learning increases the value of an action whereas policy gradients increase the probability of selecting an action.

When you add entropy-based policy gradients to the mix (or equivalent) you boost the probability of an action proportional to some function of the action value (refer

to [Equation 7-2](#)). Or more succinctly, the policy is proportional to a function of the Q-values. Hence, policy gradient methods solve the same problem as Q-learning.²¹

Even empirically, comparisons of Q-learning and policy gradient approaches conclude that performance is similar (look at Figure 4 in the ACER paper, where policy gradient and Q-learning approaches are almost the same).²² Many of the differences can be accounted for when considering external improvements to things like sample efficiency or parallelism. Schulman and Abbeel went so far as to normalize Q-learning and policy gradient-based algorithms to be equivalent and prove they produce the same results.²³

What Does This Mean For the Future?

The conclusion you can draw from this is that given time there will be little difference between value methods and policy gradient methods. The two disciplines will merge (which they have already done to some extent with the actor-critic paradigm) since both policy improvement and value estimation are inextricably linked.

What Does This Mean Now?

But for now it is clear that no one single implementation is good for all problems, the classic interpretation of the *no free lunch* theorem. You have to consider several implementations to be sure that you have optimized the performance. But make sure you pick the simplest solution that solves your problem, especially if you are running this in production. The cumulative industrial knowledge and experience of these algorithms also helps. For example, you wouldn't pick some esoteric algorithm over SAC by default, because people, platforms, and companies have more experience with SAC.

References

- [1] Haarnoja, Tuomas, Aurick Zhou, Pieter Abbeel, and Sergey Levine. 2018. “[Soft Actor-Critic: Off-Policy Maximum Entropy Deep Reinforcement Learning with a Stochastic Actor](#)”. ArXiv:1801.01290, August.
- [2] Haarnoja, Tuomas, Haoran Tang, Pieter Abbeel, and Sergey Levine. 2017. “[Reinforcement Learning with Deep Energy-Based Policies](#)”. ArXiv:1702.08165, July.
- [3] Shi, Wenjie, Shiji Song, and Cheng Wu. 2019. “[Soft Policy Gradient Method for Maximum Entropy Deep Reinforcement Learning](#)”. ArXiv:1909.03198, September.
- [4] Liu, Qiang, and Dilin Wang. 2019. “[Stein Variational Gradient Descent: A General Purpose Bayesian Inference Algorithm](#)”. ArXiv:1608.04471, September.

- [5] Haarnoja, Tuomas, Aurick Zhou, Pieter Abbeel, and Sergey Levine. 2018. “Soft Actor-Critic: Off-Policy Maximum Entropy Deep Reinforcement Learning with a Stochastic Actor”. ArXiv:1801.01290, August.
- [6] Christodoulou, Petros. 2019. “Soft Actor-Critic for Discrete Action Settings”. ArXiv:1910.07207, October.
- [7] Haarnoja, Tuomas, Aurick Zhou, Kristian Hartikainen, George Tucker, Sehoon Ha, Jie Tan, Vikash Kumar, et al. 2019. “Soft Actor-Critic Algorithms and Applications”. ArXiv:1812.05905, January.
- [8] Ren, Tianzhu, Yuanchang Xie, and Liming Jiang. 2020. “Cooperative Highway Work Zone Merge Control Based on Reinforcement Learning in A Connected and Automated Environment”. ArXiv:2001.08581, January.
- [9] Chen, Gang, and Yiming Peng. 2019. “Off-Policy Actor-Critic in an Ensemble: Achieving Maximum General Entropy and Effective Environment Exploration in Deep Reinforcement Learning”. ArXiv:1902.05551, February.
- [10] Ciosek, Kamil, Quan Vuong, Robert Loftin, and Katja Hofmann. 2019. “Better Exploration with Optimistic Actor-Critic”. ArXiv:1910.12807, October.
- [11] Wang, Che, and Keith Ross. 2019. “Boosting Soft Actor-Critic: Emphasizing Recent Experience without Forgetting the Past”. ArXiv:1906.04009, June.
- [12] Shi, Wenjie, Shiji Song, and Cheng Wu. 2019. “Soft Policy Gradient Method for Maximum Entropy Deep Reinforcement Learning”. ArXiv:1909.03198, September.
- [13] Haarnoja, Tuomas, Haoran Tang, Pieter Abbeel, and Sergey Levine. 2017. “Reinforcement Learning with Deep Energy-Based Policies”. ArXiv:1702.08165, July.
- [14] “End to End Learning in Autonomous Driving Systems”. EECS at UC Berkeley. n.d. Accessed 5 May 2020.
- [15] Nachum, Ofir, Mohammad Norouzi, Kelvin Xu, and Dale Schuurmans. 2017. “Bridging the Gap Between Value and Policy Based Reinforcement Learning”. ArXiv:1702.08892, November.
- [16] Nachum, Ofir, Mohammad Norouzi, Kelvin Xu, and Dale Schuurmans. 2018. “Trust-PCL: An Off-Policy Trust Region Method for Continuous Control”. ArXiv:1707.01891, February.
- [17] Engstrom, Logan, Andrew Ilyas, Shibani Santurkar, Dimitris Tsipras, Firdaus Janoos, Larry Rudolph, and Aleksander Madry. 2020. “Implementation Matters in Deep Policy Gradients: A Case Study on PPO and TRPO”. ArXiv:2005.12729 Cs, Stat, May.
- [18] Kramer, Tawn. 2018. Tawnkramer/Gym-Donkeycar. Python.

- [19] Raffin, Antonin and Sokolov, Roma. 2019. *Learning to Drive Smoothly in Minutes*. GitHub.
- [20] Li, Xiang, Wenhao Yang, and Zhihua Zhang. 2019. “A Regularized Approach to Sparse Optimal Policy in Reinforcement Learning”. ArXiv:1903.00725, October.
- [21] Schulman, John, Xi Chen, and Pieter Abbeel. 2018. “Equivalence Between Policy Gradients and Soft Q-Learning”. ArXiv:1704.06440, October.
- [22] Mnih, Volodymyr, Adrià Puigdomènech Badia, Mehdi Mirza, Alex Graves, Timothy P. Lillicrap, Tim Harley, David Silver, and Koray Kavukcuoglu. 2016. “Asynchronous Methods for Deep Reinforcement Learning”. ArXiv:1602.01783, June.
- [23] Schulman, John, Xi Chen, and Pieter Abbeel. 2018. “Equivalence Between Policy Gradients and Soft Q-Learning”. ArXiv:1704.06440, October.

Improving How an Agent Learns

Complex industrial problems can often be decomposed into *directed acyclic graphs* (DAGs). This helps development productivity, by splitting one long project into many smaller projects that are easier to solve. It helps reinforcement learning (RL) because smaller components are often easier and quicker to train and can be more robust. “[Hierarchical Reinforcement Learning](#)” on page 220 shows one common formalism, which is to derive a hierarchy of policies, where low-level policies are responsible for fine-grained “skills” and high-level policies do long-term planning.

So far in this book I have only considered single-agent problems. Some problems need teams of agents, or at least there may be multiple agents operating within the same environment. “[Multi-Agent Reinforcement Learning](#)” on page 225 shows you how agents can cooperate or compete to solve multi-agent problems with global or local rewards.

Another rapidly evolving area of RL is redefining how you should think about rewards. Traditionally, it is the sole responsibility of the agent to use the reward signal to learn a policy. But you can augment this process by providing extra, potentially external information in the form of expert guidance. “[Expert Guidance](#)” on page 235 discusses how to incorporate expertise into policies that ultimately help to speed up learning by improving exploration. It’s even possible to *learn* the optimal reward, which aims to negate the need for reward engineering.

But first, RL is inextricably linked to Markov decision processes (MDPs) by the Bellman equation, and all RL algorithms rely on theoretical guarantees of optimality when making sequential decisions (you might want to review “[Markov Decision Processes](#)” on page 35). But a variety of real-life scenarios break the underlying assumptions of an MDP, which often leads to the silent failure of the assumptions made by the Bellman equation and manifest as policies that won’t converge, no matter how

much hyperparameter tinkering you do. The next section investigates variations on the basic MDP to accommodate these scenarios.



This chapter summarizes the state-of-the-art. I think that the four most important themes are how you define the MDP, hierarchical RL, multi-agent RL, and providing expert guidance. But all of these can be considered as “advanced” and if discussed in full would take another book. However, I do think it is useful to present an overview so that you know it is there and can dig deeper if you want to. As usual, the citations and recommendations in “[Further Reading](#)” on page 242 can provide a wealth of extra information.

Rethinking the MDP

RL algorithms that leverage the MDP framework and the resulting guarantees from the Bellman equation assume that the underlying environment is Markovian. Although there may be a great deal of uncertainty about the effects of an agent’s actions, the perception of the state of the environment must be perfect. The agent then uses this omnipotent knowledge to choose the next action according to the current policy, or $\pi(a \mid s)$.

But what if the state representation is not perfect? What if when you tell the servo to move to a position, it moves to a position that is randomly distributed about that position? Or what if there are latent, hidden features that represent how likely a person is to click a recommendation? In these situations the state is not fully observable and the Markovian assumptions are broken.

Partially Observable Markov Decision Process

One solution to this *partial observability* is to split the representation of the state. A partially observable Markov decision process (POMDP) includes extra parameters over the MDP to include a finite set of observations that the agent can experience, Ω , and an *observation function*, O , that maps a state and action to a probability distribution over actions. You can then convert the problem back into an MDP by approximating the inverse of the observation function and using it to convert observations back into states.

[Figure 8-1](#) shows that a POMDP is an extension of the MDP, in which the agent is unable to directly observe the current state of the environment. Instead it uses an observation and information about the previous action and state to generate a prediction of the current state, $\hat{s}$. The prediction of the state is often called the *belief state*, b , which you can describe as “this is the state that I believe the environment is in.”

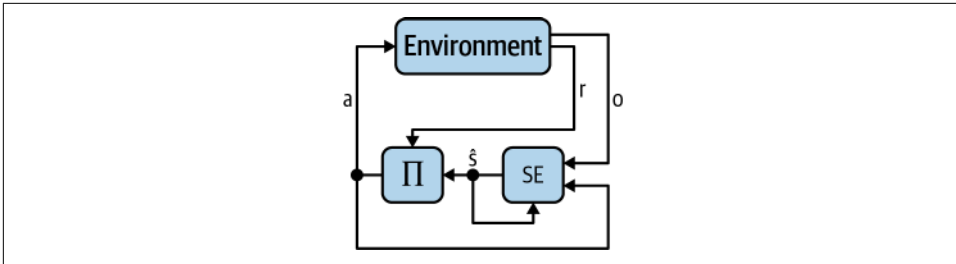


Figure 8-1. You can decompose a POMDP into an MDP (with states, actions, rewards, and a policy) and a state estimator (SE) that predicts states from observations (o).

Predicting the belief state

So what is the belief state and how can you predict it? Consider an example where you were trying to direct a robot through a simple maze. The agent doesn't know the layout of the maze, but it might be intelligent enough to be able to detect that there is a "T-junction" ahead and it needs to make a choice on which direction to take. A naive policy would overfit the training environments and converge to a point where it suggests a single decision every time.

A simple solution to this problem, as indicated by the recurrence in [Figure 8-1](#), is that the best action can be based upon past experience. I don't mean past experience in the training sense, not a replay buffer, but past experience of the current episode and environment. For example, if you know that the robot has already tried left and that led to a dead end, then the hidden state of the maze (the best route) is probably the option to the right.

Many sophisticated examples leverage this simple idea by including past actions and observations in the state representation. For example, the Atari benchmarks always stack four frames of pixel observations in order to retain some "history" of the environment. This allows the agent to predict trajectories of Pong balls or the movement of enemies. And many robotics examples include previous actions and states to help account for inaccuracies in positioning and smooth out movement.

Although this idea helps in many cases, it is not sufficient in general. In order to act optimally, the agent needs to actively consider how uncertain it is about the current situation. A more sophisticated solution is to model the state estimator using an approximator that is capable of learning time-dependent features.

There are multiple options for solving this problem. You can attempt to track the belief state; a simple version could utilize a Kalman filter although more recent approaches attempt to build neural networks to derive the optimal Bayes filter.¹ You could incorporate an approximation of the belief model directly into the policy model using memory-enabled predictors or belief trackers.^{2,3}

But the simplest (the previous reference had 24 authors!) and most common approach is to slap on a recurrent neural network (RNN) at the end of the state representation function. Depending on the problem, the RNNs don't need to be complicated and they allow the agent to make reasoned decisions based upon past observations. This allows you to train the RNN using standard techniques and tooling.

As a conceptual example, consider the experiment from “[Industrial Example: Learning to Drive with a Remote Control Car](#)” on page 205, where I used a variational autoencoder to represent the state. I could add an RNN to the output of the state representation and train it by predicting future observations. Eventually it would be able to provide predictions of what would happen given certain actions and therefore break the partial observability problem. The policy would have full observability of all possible future observations so you can optimize the MDP using standard RL methods.

Case Study: Using POMDPs in Autonomous Vehicles

You might suggest that the previous example is too contrived to apply to real life. But a surprising number of real-life scenarios fit perfectly into the previous model. One such example is that of autonomous vehicles.

A common autonomous task is to decide when it is safe for the vehicle to move; for example, traversing an intersection with stop signs or a roundabout is a common driving situation. Upon arrival, an agent has to time its actions to perfection to avoid hitting other vehicles. Yet if the vehicle waits too long, valuable time is lost and the efficiency of the intersection decreases.

The vehicle has a limited view of the road. The onboard sensors, such as a camera or a LIDAR module, are line of site and cannot see around obstacles. This makes the problem only partially observable, because the agent cannot sense important features outside of this space, like a speeding vehicle hurtling toward the intersection.

Traditional algorithms attempt to quantify the time to collision, but estimating the time accurately and dealing with unexpected situations is hard and not reliable enough for autonomous operation.

By modeling the problem as a POMDP, you can learn an optimal policy using RL. Qiao et al. demonstrate (in a simulation) that an agent with an LSTM-enabled policy is capable of learning when to set off from a four-way intersection to turn left, right, or move straight ahead.⁴ The LSTM policy is able to learn which partially observed situations lead to a collision, via an appropriately engineered reward, and avoid them. You can [see the results of this experiment on YouTube](#).

Qiao et al. went on to add hierarchical RL to the previous model to better model the high-level actions (or low-level subtasks, depending on how you think of it) of go,

stop, follow vehicle in front, and so on.⁵ You read more about hierarchical RL in “[Hierarchical Reinforcement Learning](#)” on page 220.

Contextual Markov Decision Processes

There are many problems in which the underlying MDP is the same, but the contextual situation is different. For example, imagine you were training an agent to administer a drug to a diabetes patient based upon their sugar levels. The underlying MDP is the same for every patient, sugar levels are the observation, and administering the drug is the action. But the observation is affected by external variables, like the gender or age of the patient.⁶

This sounds like a situation similar to POMDPs, but there is a crucial difference. These contextual variables do not change with each measurement. By modeling the context as a set of fixed parameters you constrain the space the agent has to search, which can lead to more robust models and quicker learning.⁷

This extension is crucial when presented with a problem that has a small number of steps in an episode (short Markov chains) but massive numbers of independent episodes. This is precisely the situation that recommendation algorithms find themselves in, which is why most of the contextual MDP (CMDP) research is driven by recommendation or ranking applications.⁸

MDPs with Changing Actions

Some applications have changing action sets and the research has followed two paths that lead to similar results. The first set of research concentrates on situations when actions are not always available. For example, in path planning applications like vehicle or packet routing, paths may be closed due to damage or maintenance, or in advertising, ad selection may change depending on the time of day.⁹

The premise is a stochastic action set (SAS), which includes a set of actions available at a certain time, modeled as a random variable. A common solution to this problem is to learn a policy derived from only the actions observed in that particular state. Boutilier et al. showed that you can then utilize standard Q-learning variants.¹⁰ Chandak et al. extended the idea to incorporate policy gradient methods.¹¹

A second, but more general strand of research focuses more on lifelong learning, or continuous learning. Here, action sets change because problems and priorities change. For example, you might want to add a new capability to your robotic arm or introduce a new set of ads without having to retrain your model and lose that useful experience. The major problem here is the curse of dimensionality, with ever-growing potential action sets and a lack of data to sample those actions. In theory, adding more actions is as simple as adding them to the MDP. But creating an architecture that can handle such a change is challenging.

Chandak et al. demonstrated that this was possible by splitting the policy in two. The first is responsible for making decisions based upon the state, but the output of this is in a meta-space, not the action space. The second component then maps this meta-space to the action space. Together they form a complete policy.¹²

The benefit is that you can alter the meta-action model parameters to account for new actions without touching the state-representation part. Then you can use standard supervised learning on a replay buffer to train the new meta-action model. This limits the amount of retraining required from the live environment to only the parts of the model that impact the new action.

Regularized MDPs

It is worth mentioning again that the class of algorithms that include regularizers are also a reformulation of the MDP. If you recall (see “[Summary](#)” on page 211), SAC and other similar algorithms attempt to find an optimal policy that maximizes the discounted return, like any algorithm based upon an MDP, but they also include a regularization term. The idea here is that you can use this term to enforce robustness, similar to how regularizers impact linear regression. But the addition fundamentally alters the Bellman equality, which means there is an impact on the underlying MDP. Thankfully, Li et al. proved that it doesn’t matter what regularization you apply, the algorithm will still converge, so long as you follow some rules defining the regularizing function.¹³

Hierarchical Reinforcement Learning

A range of problems utilize low-level behaviors that are atomic and repetitive; they involve mastering a “simple” skill like picking up an object or recommending a single product. Problems that involve multilevel reasoning or combine several skills together are much more complex.

In engineering, splitting challenges into smaller components can help to simplify a complex problem. *Hierarchical reinforcement learning* (HRL) uses this idea by decomposing a problem into subtasks, where solving each subtask is easier or better than solving it in entirety. This can lead to computational improvements, greater robustness, more flexibility, and sometimes better performance. It is also one way to mitigate the “curse of dimensionality,” by reducing the search space. For example, when learning to walk, there’s little point searching the action space that includes walking on your hands, unless you’re a gymnast.

With a hierarchy of policies, where only the low-level policies interact with the environment, you can train high-level policies to plan or make strategic decisions over a longer timescale. Another appealing idea is that of encapsulating policies to perform one task well, which you can reuse in other problems; a library of policies.

However, there are a range of challenges that you must consider. How do you define and produce well-encapsulated, low-level policies? How do you train higher-level policies and how do you design the hierarchy? And how do you train all of this in a reasonable amount of time?

Naive HRL

You should consider designing hierarchies manually because in many problems there is a clear separation of concerns. For example, teaching a complex robot to move is clearly distinct from teaching it to make coffee, a rewarding conversation is dependent on the words and utterances used in the conversation, and successful completion of an educational course depends on the content within it.^{14,15} Optimizing these high-level goals through low-level, bespoke skills leads to policies that are flexible, robust, and performant.

In simpler situations it may be preferable to design the abstractions, and therefore the hierarchy, manually. You can then design a bespoke RL implementation that takes advantage of this hand-designed, problem-specific model. Figure 8-2 shows an example of decomposing a notional coffee-making task into a hierarchy of behaviors.

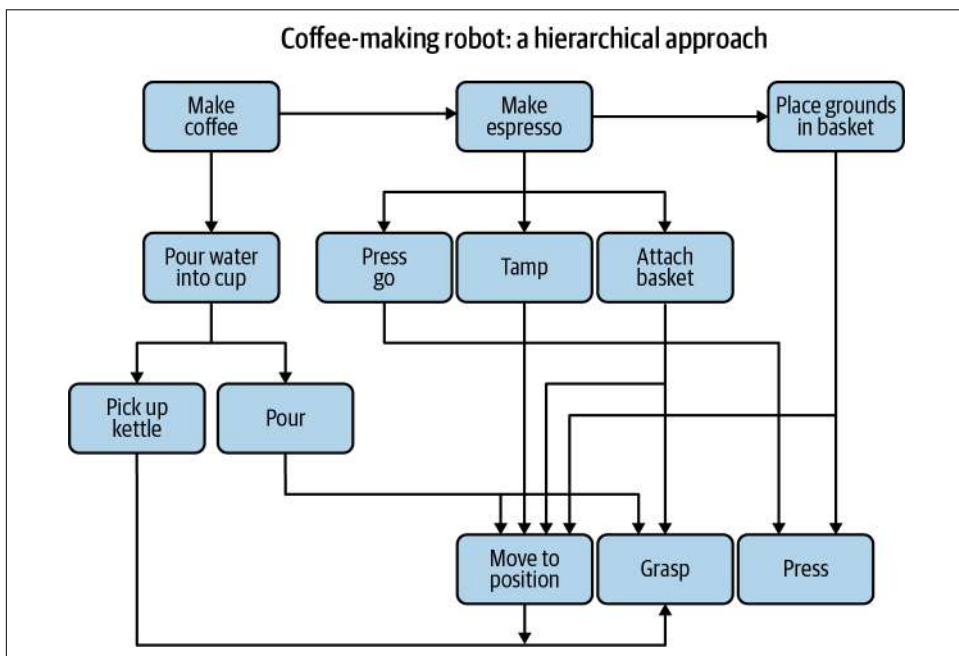


Figure 8-2. You can manually decompose complex tasks into a hierarchy of atomic behaviors and treat each challenge separately. This is an example of decomposing a notional coffee-making task into dependent subskills. Follow the arrows to find the “base” skill.

The major benefit of this approach is that it is likely to be more robust and simpler than a more sophisticated, general approach. It's a bit like asking a neural network to predict the price of a car from a photo; you will be far more successful building sub-networks that are capable of classifying the make and model, locating the number plate for automated number plate recognition, and so on, and feeding those into a high-level model that predicts price. From a practical standpoint, it also helps you focus on neat, pint-sized problems, rather than one big make-or-break unicorn.

But this approach can only take you so far. In many problems the demarkation of boundaries in the hierarchy is a challenge in itself and an arbitrary line is likely to be suboptimal. When riding a bike, for example, is “pedal” a base skill, or should it be “push down on right foot”? You need a way to automatically find these skills.

High-Low Hierarchies with Intrinsic Rewards (HIRO)

The first step toward an automated HRL solution is to split the policy. A low-level policy is responsible for interacting with the environment through states and actions, as usual, but it does not receive the reward. A high-level policy suggests a high-level action and rewards the low-level policy with a reward that is intrinsic to the requested goal. When the environment produces a reward, it is given the high-level policy. **Figure 8-3** presents this idea and is explained in full by Nachum et al.¹⁶

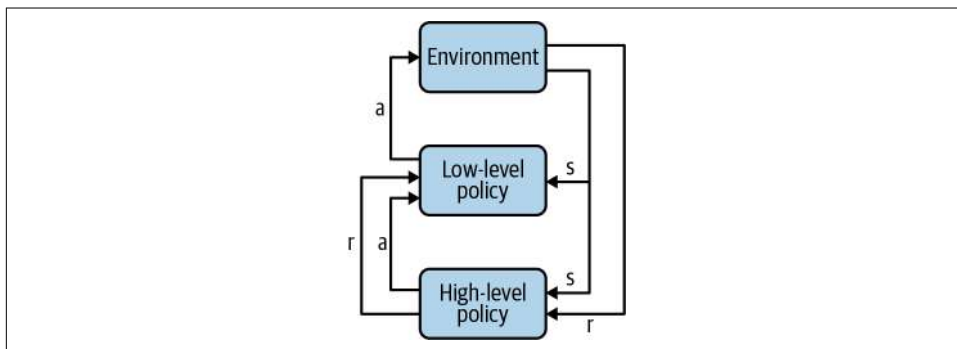


Figure 8-3. Hierarchical RL through intrinsic rewards.

For example, the high-level policy would tell the low-level policy to attempt to move to a specific state, and reward the low-level policy according to how close it gets. The high-level policy samples at a slower rate than the low-level policy to provide a temporal abstraction.

You can parameterize the low-level reward function in any way you like. But researchers propose that a distance-to-state reward helps speed up learning.

The next challenge is that although you can use on-policy algorithms like PPO out of the box, you'd likely want to use a replay buffer to improve sample efficiency. The

problem is that the high-level replay buffer will contain observations from previous low-level policies, like right at the start when the low-level policy is random. This means the replay buffer isn't representative and will lead to convergence issues. A simple solution could be to weight the random sample to prefer more recent observations, a form of prioritized experience replay.

But Nachum et al. chose to dynamically *relabel* the high-level action/goals to match the actions that occurred. For example, imagine the coffee example again and the agent has a replay buffer full of the skill “grasp coffee,” but the current goal being trained is “push button.” In their implementation, the high-level agent sampling from this replay buffer will dynamically change the goal to suit the requested skill. In this example it would alter the goal to be “grasp coffee,” to match what is in the buffer.

This might feel like you are fudging the results after the fact to make the current high-level policy look better. But the reward from the environment tends to counteract some of the bias added by the relabeling. In the worst case it is like duplicating observations that are already in the replay buffer. Ideally you want a large selection of uncorrelated, randomly distributed transitions. But if you don't, random sampling helps to make it appear more random.

The **results show** that a complex legged simulated robot is capable of both learning to walk and learning high-level tasks such as block pushing, hidden goal finding, and resource gathering.

Learning Skills and Unsupervised RL

A major problem with the previous interpretation of HRL is that this is still a closed-loop system that optimizes for the specific problem at hand. There is no generalization. You couldn't use this in any other situation. It is overfitting the environment.

But you know that a robot arm, for example, moves in a similar way for every task. A command of “move to position x” is specific to the robot and the way it is built, not the environment. All this time I have been describing an agent as if it is implicit to the problem, but actually it is an independent entity. So how can you teach policies, or *skills*, that relate to the agent, not the environment?

Researchers propose that agents should be trained to perform a wide variety of skills, even those that don't produce positive rewards. Another higher-level policy can then choose which skills to use to solve a problem. Training an unmanned aerial vehicle (UAV) to do a flip might not be a good thing for a stable communication network, but it might be useful at a **Metallica concert**.

Diversity is all you need (DIAYN) is a novel idea that promotes the unsupervised discovery of low-level policies. Eysenbach et al. propose that skills should be distinguishable, so they should aim to produce policies that visit different states, and the resulting skills should be as diverse as possible. To encourage randomness you can

maximize an entropy measure conditioned on states, actions, or both. **Figure 8-4** shows a simplified depiction of this process.¹⁷

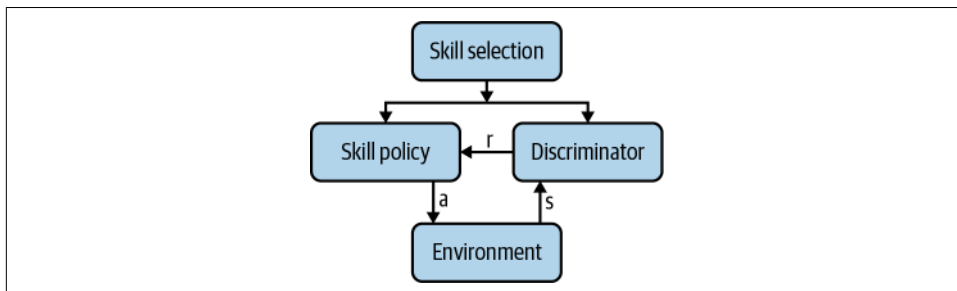


Figure 8-4. Schematic diagram of unsupervised skill learning framework (DIAYN). A skill is chosen at random and a policy acts upon the environment. A discriminator calculates the entropy of the policy, states, and actions that is used as a reward. Over time the policy leans distinct skills.

The **result is a collection of diverse policies** that may or may not be useful for a problem. But the diversity acts like a stepping stone to help solve more complicated problems. I particularly like the results from the **HalfCheetah** environment that **show running, crawling, and sitting**.

Using Skills in HRL

Given a library of skills, how can you train an HRL algorithm to solve a specific problem? I can imagine that pretrained skills could help nearly all RL algorithms by allowing them to concentrate on environment exploration rather than exploring how to explore. This is especially true in environments where there is a significant amount of complexity in the agent itself.

HRL with advantage-based rewards (HAAR) is one algorithm that takes advantage of pretrained skills to dramatically improve initial exploration and therefore speed up learning. Li et al. also introduce the idea of using advantages as the reward signal, rather than a Euclidean distance to state like in HIRO. The reason for this is that the state space isn't necessarily mapped to the real distance to goal. For example, if you are using a video camera, pixel distance isn't the same as physical distance. So instead they propose using the advantage estimate from the high-level policy to retrain and reward the low-level skills. This way, skills that produce a positive advantage, the ones that do well in the high-level task, are promoted over those that don't.¹⁸

Another interesting idea is to anneal the amount of time that each skill is repeated to further encourage long-range exploration.

HRL Conclusions

Building a hierarchy of policies is useful when your problem is sufficiently complex to benefit from splitting. From an engineering point of view, if you can imagine that splitting your policy will help simplify the problem or build more robust abstractions, then HRL might be a good fit. The potential for hierarchical policies is hard to spot. Look for situations where agents have to manipulate complex domains. Robotics is an obvious example, but other domains are just as complex, like agents that deal with text.¹⁹ Conversely, in many real-life problems it is the environment that is complex, not the agent, so HRL may not be beneficial.

In the extreme, HRL is akin to a graph problem, where connections between mini-subpolicies form a large graph. Recent research attempts to treat state as a graph and use topological features with graph convolutional neural networks to help solve grid-based problems.²⁰



Even if you do not work with complex agents, I think that HRL shows that unsupervised learning has an important role to play in RL. And yet it is only just gaining attention from researchers.²¹ Take the time to consider if your problem could benefit from an initial library of skills or pretraining; it could significantly improve learning speed.

Another interesting question is how much influence do you need over your policies? The skills generated by HRL are not determined by an expert, so they may produce action sequences that are unfamiliar, unexpected, or even unwanted. Of course they are directed by the definition of the unsupervised or supervised reward, so you can influence the skills that way. But an open question, which is probably problem specific, is whether you should interfere to produce human “imagineered” policies or allow the agent to discover its own. I suspect that in most real-life problems it is some combination of both. You don’t want to be overly prescriptive, because that is the whole point of RL—you can’t handcode an optimal policy in complex problems. But you also need to constrain the problem and restrain the agent so that it doesn’t overfit and become brittle.

Multi-Agent Reinforcement Learning

Throughout this text I have considered lone agents. But many problems involve more than one player/agent and it is likely that you can improve performance if you systematically model the problem with multiple agents in mind.

Multi-agent reinforcement learning (MARL) addresses the sequential decision-making problem of multiple autonomous agents that operate in a common

environment. Each agent attempts to optimize the long-term return by interacting with the environment and other agents.

MARL applications can fall into three main groups: cooperative, competitive, or a mixture of both. In the cooperative setting, agents attempt to collaborate to optimize a common long-term goal. For example, a cooperative MARL can simulate and optimize a taxation policy.²² In competitive settings, the global reward often sums to zero, since agents can only improve their policy at another agent's expense. This situation is common in domains that include price bidding, like trading and advertising. Finally, the environment may contain both of the previous two situations. For example, an advertisement marketplace wants to maximize both global revenue and long-term user engagement, which means that a cooperative model would be optimal, but advertisers want to pay the least amount of money for the highest impact, which adds a competition into the environment.

The literature generally discusses MARL in the context of games, often traditional games like chess or poker, and increasingly computer games. Like in standard RL, the reason for using games is that they are easy to simulate and verify. Real-life or industrial examples are less common, but are emerging. For example, applications include using MARL to formally define governmental policies, low-level TCP packet congestion mitigation, and multi-robot human-assistance collaboration.^{23,24,25}

MARL Frameworks

The literature tends to model MARL problems in one of two ways, which you can see in [Figure 8-5](#).

Looking at [Figure 8-5\(a\)](#), first recall that an MDP is modeled as an agent that acts upon an environment, which in return produces a new state and a reward. If the environment is episodic then the environment also produces a termination signal.

[Figure 8-5\(b\)](#) illustrates how the first MARL framework is an extension of an MDP called a *Markov game* (MG), where multiple agents operate in parallel. Agents submit unique actions and receive unique rewards, but the state of the environment is global. This is the most obvious extension to an MDP, in the context of RL.

[Figure 8-5\(c\)](#) presents the second and more common model, an *extensive-form game* (EFG), where chance, state, and actions are represented as a tree, much like the roll-out of an MDP. An EFG still alters the state of an environment through actions, even though it is not explicitly shown. For example, if you choose to bet in a poker game then your opponent is forced to match or fold. Your action is impacting the state of the game. But you can see that sequential actions between agents are far more explicit. In [Figure 8-5\(c\)](#), it is clear that agent 2 cannot have a turn until agent 1 has completed theirs. This scenario is not possible with an MG. Note that EFGs are always considered from the point of view of one of the agents.

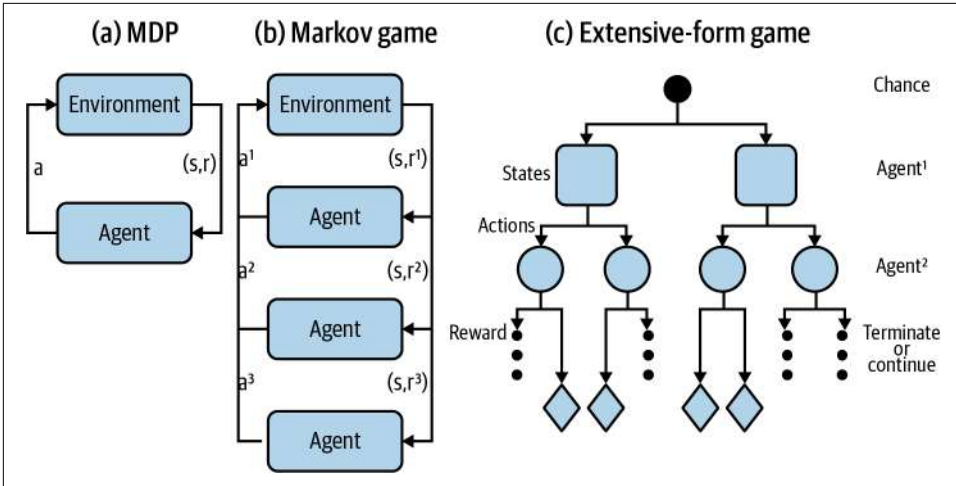


Figure 8-5. Schematic diagrams for different MARL frameworks. (a) represents an MDP, where agents act upon environments that produce states and rewards. (b) is a Markov game where multiple agents have independent actions and rewards. (c) is an extensive-form game, where dots represent chance, squares and circles represent two agents, and diamonds represent terminal states.

Simultaneous actions (like an MG) or imperfect information (like a POMDP) can be represented by *information sets*, which are groups of nodes belonging to a single agent that does not know which node has been reached. These are usually represented by a dotted line or an enclosing circle. You can see an example in [Figure 8-6](#).

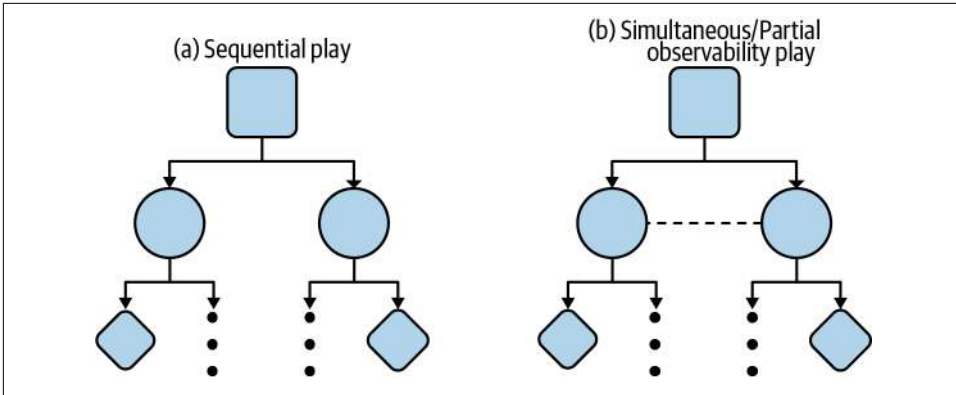


Figure 8-6. Extensive-form game schematics for (a) sequential and (b) simultaneous or imperfect/partially observable games. The only difference is whether other agents know what state they are going to land in.

As an example, consider a game of Rock, Paper, Scissors. This is a classic simultaneous game where two people reveal one of these three actions at the same time to receive the reward. To map this into an EFG you could ask the people to write the actions on a piece of paper, then reveal the choices simultaneously. The agents acted at different times and the second agent could not observe the action of the first player. This means, from the perspective of agent 2, the game is in one of three potential states, but it doesn't know which one.

EFGs are more general and are preferred by theorists. But intuitively, I still like to use MGs as my mental model because the vast majority of industrial situations can be designed to use simultaneous actions and I find the simplicity of a recursive diagram less burdensome than a rolled-out tree. Many researchers agree and use MGs in their applications.

Centralized or Decentralized

Other than the cooperative/competitive distinction, the most important decision to make is whether your problem would benefit from centralized or decentralized learning. [Figure 8-7](#) depicts four scenarios with differing amounts of agent independence.

In centralized learning, or more typically centralized learning and distributed execution, experiences are fed back to a single learner and policies are returned for local execution. The primary benefit of this method is that you can create policies from the experience of multiple actors. For example, if a child could learn centrally, through the experiences of others, they would benefit from the large number of experiences and their diversity. Similarly, using the experience of a disparate set of agents allows them to search the state-action space much more quickly. In essence, independent experiences help exploration and centralized learning ensures all agents have similar, viable policies.

In a decentralized model (often called a *DEC-MDP*), both the execution *and* the learning happens locally. This is more scalable because it doesn't matter how many agents you introduce. For example, imagine if everyone “drove” an autonomous vehicle. A single centralized learning process is unlikely to be optimal for all driving conditions. Local learning allows agents to adapt to their local perception of the environment. For example, a policy for driving on the left side of the road is likely to be subtly different than a policy for driving on the right.

Another problem is that agents may have different capabilities. For example, the experience of a cyclist is just as valid as the experience of a car, but combining these experiences in a centralized way is difficult. Robots may have different configurations. Decentralized learning allows agents to learn to use the actions at their disposal.

A further benefit is that the decentralized model potentially allows you to use standard single-agent RL algorithms, because you can treat each agent independently.

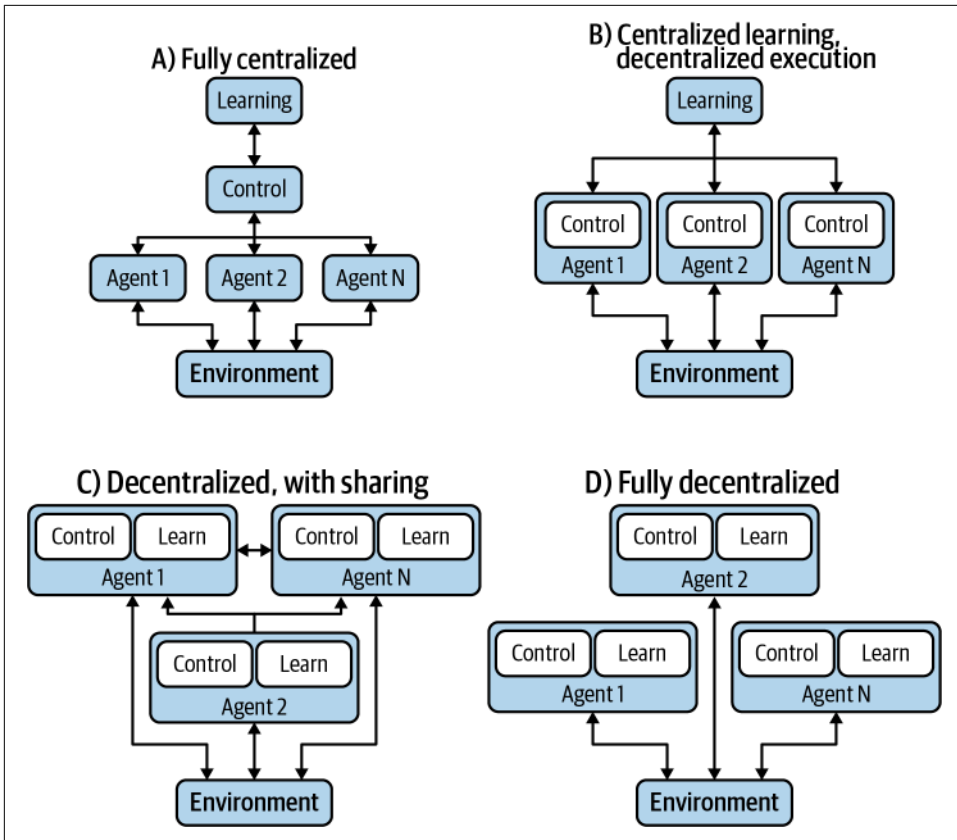


Figure 8-7. Different forms of control and learning in MARL. (a) In fully centralized settings, agents have no control over execution or learning. (b) In centralized learning, agents have full execution control but policies are distributed by a central source. (c) In a decentralized setting, if communication is possible, then agents can share experience and policies. (d) In a fully decentralized model, agents are independent.

Single-Agent Algorithms

The first and most obvious route to solving MARL problems is to assume each agent is independent and apply any of the single-agent RL approaches discussed in this book. Unfortunately, traditional RL algorithms such as Q-learning or policy gradient are poorly suited to multi-agent environments. The primary concern is that during training other agents are interacting and altering the environment state. This makes the environment nonstationary from the perspective of any single agent. When the environment is constantly changing, this impedes the agent's ability to create a stable policy. This also means that common RL add-ons, like basic experience replay, do not help. This does not mean that they fail completely, however. The agent can learn

stable policies in simple environments, but they are usually suboptimal and tend not to converge when environments increase in complexity.

It is technically possible to use model-based policy gradient algorithms, because the model provides you with a perfect description of the dynamics of the environment, but for many problems a model is not available.

Case Study: Using Single-Agent Decentralized Learning in UAVs

Using UAVs, often called drones, is becoming increasingly common in applications where geospatial coverage is important but hard to achieve. UAVs can fly into an unknown environment to monitor wildfires, perform search and rescue, and establish communication networks.^{26,27,28} Their popularity is due to the low operational cost, the array of sensors, and their flexibility of application.

It is impractical to attempt to handcode controllers to perform tasks due to the lack of an accurate model. RL provides a natural model-free solution that is capable of navigating through difficult environments. There are a variety of examples of using RL to learn low-level UAV primitives like flying, moving, and avoiding obstacles. But UAVs shine when they are used in collaboration to solve a problem.

Cui, Liu, and Nallanathan use a simulated fleet of UAVs to provide wireless infrastructure to regions with limited coverage. UAVs are particularly useful in this situation because it is typically easy to establish line of sight. The researchers use MARL to solve a complex optimization problem that maximizes network performance.

Their work assumed agent independence and leveraged a simple single-agent solution. They used Q-learning to learn the optimal choice of user to connect to, sub-channel, and power level. Note that positions are fixed in this work. They develop a reward function based upon quality metrics and solve in a simulation of three UAVs. Note that there is no correction for the nonstationarity of the environment and agents must compete for reward.

So there is a lot of room for improvement. A better decentralized technique could account for the instability of the environment. Or since they have a network, a centralized technique would make development easier. And a more sophisticated agent could handle a much more complex state space, like spatial positioning, which would help the solution scale. With these fixes it should be feasible to run this in real life. The problem with the way that research is celebrated today is that it only promotes success, not failure. It's likely that Cui, Liu, and Nallanathan tried some of these ideas but ran out of time.

Despite this I am optimistic and can imagine a more stable set of algorithms that are able to learn more complex behaviors (like optimal multi-agent positioning).

Centralized Learning, Decentralized Execution

Recent successes have arisen by combining improvements in RL with the sample efficiency of centralized learning. One such algorithm that gained notoriety combines DDPG (“Deep Deterministic Policy Gradients” on page 154) with a centralized learning, decentralized execution approach called *multi-agent DDPG* (MADDPG).²⁹

DDPG (or TD3/PPO/others) provides the backbone that allows agents to learn policies without a model of the environment. Policy gradient algorithms provide a theoretical basis for combining the gradients of multiple agents, as opposed to value methods in which the global action-value function is not representative of local situations. Using a centralized learning approach during training time improves sample efficiency and speeds up learning. During evaluation, agents use independent policies and local observations.

The primary motivation for this approach is that if you know the actions of all agents, then it is possible to predict or track the changes to the environment. Like in quantum mechanics, if you know the state and actions of all atoms in the universe, then life is stationary, and you can predict the future (or past).

To calculate the value of a policy for an individual agent, you need to know the outcome of the agent’s policy and the value predicted in the environment. But the environment is affected by all other agents, so you need to provide information about the actions of all other agents. Looking back at the different policy gradient implementations (Equation 5-11) it is clear that the actor-critic paradigm naturally considers the actions of an agent, so all you need to do is extend it to consider the actions of *all* agents, which is shown from the perspective of a single agent in Equation 8-1, with N other agents in the environment.

Equation 8-1. Multi-agent actor-critic

$$\begin{aligned}\nabla_{\theta} J(\theta) &= \mathbb{E}_{\pi_{\theta}} [\nabla_{\theta} \ln \pi_{\theta}(a \mid s) Q_w(s, a)] && \text{(Actor-Critic)} \\ &= \mathbb{E}_{\pi_{\theta}} [\nabla_{\theta} \ln \pi_{\theta}(a \mid s) Q_w(s, a_1, a_2, \dots, a_N)] && \text{(Multi-Agent Actor-Critic)}\end{aligned}$$

This leads to the idea that we can have a local, independent actor but a centralized critic. Interestingly, because there is an actor and critic learned for every agent, agents can have different policies and rewards, which means that this works in both cooperative and competitive settings. Next, and I leave this as a homework exercise, you can apply the various improvements to vanilla policy gradient algorithms to end up with multi-agent versions of standard actor-critic RL algorithms.

Most actor-critic algorithms need the policy to generate the next action for the training of the action-value function. See any of the actor-critic algorithms, for example. One simple solution is to ask every agent to pass the policy, or all of the next actions

(depending on the algorithm), back to the centralized critic. This causes synchronization and latency issues, which hampers efficiency, but is quite simple.

The approach that MADDPG took was to *learn* the policies of the other agents, while they themselves are learning. This is an obvious source of nonstationarity. To limit the effects of nonstationarity, MADDPG slows down the learning rate by updating a limited subset of random policies. Not ideal, but it works. See the paper for more details.

Decentralized Learning

If you can't use a centralized model, then you need ways in which to stabilize decentralized learning. Temporal-difference hysteresis is a technique that uses two different learning rates depending on the size of the temporal-difference error.³⁰ If the temporal-difference error is below a threshold usually set at zero, which means that the agent obtained less reward than expected, a slower (smaller) update rate is applied. It assumes that the bad return was because the agent was exploring—you wouldn't want your child to copy you if you do something stupid. When the error is positive, the agent uses the normal learning rate. In essence it is biasing the algorithm to be more optimistic, which itself can lead to issues, but that can often be controlled by the learning rate hyperparameters. The key, however, is that it helps with the nonstationarity of the environment; the agent doesn't revert a potentially good policy just because of a few lousy updates that were due to a changing environment.

The second challenge in decentralized learning is partial observability. The classic and simplest solution is to include a recurrent layer to record histories, although other approaches are available. I anticipate that more formal approaches are just around the corner.

The final challenge is the inability to use standard replay buffers. The problem is that experiences are local and nonstationary when sampled independently, despite agents learning concurrently. The local view at that point in time is unlikely to represent the environment at a future point in time. A related issue is that randomly sampling from different parts of the replay buffer may have entirely different environmental states, which in the worst case could cause two agents with exactly the same local conditions resulting in completely different policies. There are many potential ways to improve upon this, but Omidshafiei et al. implemented a replay buffer that was synchronized over all agents. Concurrent experience replay tends to lead to local correlations, which is typically a bad thing and resolved by random sampling, but in this situation helps because it forces all agents to focus on the current replay, which should all have the same environmental conditions. For example, if two agents had similar local conditions, then sampling from the buffer at the same point should yield similar experiences and therefore lead to similar policies. In this particular instance the experiences

are trajectories in order to train the recurrent neural networks. This combination of DQN, hysteresis, and synchronized replay buffer is called Dec-HDRQN.³¹

Note that this is just one of many potential ways to solve the problem of nonstationarity during updates, different local optima during experience replay, and partial observability. You could swap out any of the previous components with other implementations, although they will have their own idiosyncrasies.

Other Combinations

The centralized and decentralized paradigms can be mixed together in a huge variety of ways to suit the application. For example, you could use a centralized critic, with a centralized policy that is shared with agents for remote execution. Or you could use agents with decentralized critics and policies that are shared with their immediate neighbors. Many permutations have been investigated and there is likely to be an algorithm that suits your use case. A detailed investigation of this varied field is outside of the scope of this book, but there are some great resources in [“Further Reading” on page 242](#).

For me, MARL highlights the importance of the problem design. The sheer number of permutations, combined with the number of RL algorithms, combined with the number of neural network possibilities, means that it is computationally difficult to automate or perform a large-scale hyperparameter scan. You have to choose a small subset of algorithms and architectures to make the feedback cycle small enough to produce anything in a reasonable amount of time. This means that it is vitally important to have a correct and robust problem definition; otherwise, if you make a small change to the problem, this can have huge impacts on the implementation. In the worst case you might have to throw away all your work and all your research and start again.

The Difficulties of Making RL Work in Real Life

Imagine you had a problem where you decided that centralized training was possible. You spent many weeks doing your research, building simulators and testing lots of algorithms. Your initial tests were performing well and you told your stakeholders that you are ready to move to the next step. Then you start testing in the real world on a much greater number of agents. You leave for the day happy and sleep well.

In the morning you turn on your laptop, excited to see what new policies have been found. But you realize that (a) all policies are no better than random and (b) there have only been four training steps. After denial, anger, “quick fixes,” and depression, you realize that most of the time the agents weren’t able to communicate with the server and your code waits until every agent has checked in before it runs a training step. It quickly dawns on you that in real life agents do not have reliable

communication and even if they did, you will eventually have too many agents, which will slow down the whole system. You need a decentralized model.

The only option is to report a failure to your stakeholders and go back to the start. I wouldn't call it a complete waste of time, because this experience allowed you to learn a lot of transferable knowledge and skills, but in my experience, stakeholders generally don't like moving backward on a product roadmap.

This pattern of failure, an incorrect problem definition, is common in data science, too. But in RL there tends to be a much higher barrier to experimentation because of the agent-environment interaction, so the amount of time wasted can be much greater.



One mitigation, among others discussed throughout this book, is to take your time when defining the problem and subsequent modeling. Your model needs to be simple enough to solve and reason about, but it must also be representative of the real problem. It's like software: the best code is simple and obvious, yet seems to handle the majority of situations. But don't think that simple means easy; simplicity arises out of effort, trial, and many errors.

Challenges of MARL

Despite significant progress and a daunting amount of research, MARL is hard. This doesn't mean that it is impossible and I don't want to dissuade you from going down this path; the research shows that solutions are entirely doable. But I want to make you aware of the challenges to watch out for, which include:

Diversity of problems

You need to accurately model your environment, making decisions about cooperative or competitive, sharing policies or not, sequential or simultaneous, and you also need to consider all of the normal MDP questions like value or policy or fully observable or partial. Once you have a solid model, then you need to dig to find an appropriate algorithm that works within the assumptions of your model. MARL is a much more complex problem, so there are far more potential solutions.

Multidimensional and multimodal goals

The goal of one agent may be significantly different than another, due to the local conditions of the environment. This can make it challenging to share policies.

Scalability

In many tasks it is important to synchronize information, whether that be external, like the current state of an agent, or internal, like the current policy, which leads to communication efficiency issues in larger deployments.

Instability

Since agents are actively altering the environment, from the perspective of another agent, the environment is nonstationary. This breaks or invalidates some of the basic assumptions provided by the MDP and leads to lack of stability.

Optimality

It's hard to precisely define optimality for all but the simplest of examples, which means there are no convergence guarantees for nontrivial environments.

These challenges are unique to MARL, but they sit on top of all the standard challenges in RL like partial observability and sample efficiency. It is likely that given time these challenges will reduce in severity as researchers find different ways of tackling RL.

MARL Conclusions

MARL is an exciting and rapidly developing extension of RL. I anticipate that significant improvements are just around the corner. In the month of June 2020, for example, there were more than 50 new MARL research papers on arXiv. But even now it is possible to leverage MARL to solve a variety of commercially viable use cases, some of which you can see on the [accompanying website](#).

In this brief overview of MARL I have ignored most of the game-related literature. It is still very important from a research and theoretical perspective, but out of the scope of this book.

Expert Guidance

In many of the examples or situations presented throughout this book I have talked about teaching an agent a policy from scratch. But humans possess domain knowledge that can direct, encourage, and improve how agents learn. For example, the age at which children start school varies by country. In the US and United Kingdom, kids start around the age of four or five, but in many countries like Serbia, Singapore, South Africa, and Sweden, children don't start until they are seven. The argument for starting later is that children are given more time to randomly explore, to fine-tune low-level emotional and motor skills before they are forced to learn arbitrary high-level abstractions. Whereas the counterargument is that children waste time with random exploration and that it's better to subtly guide them using a curriculum. The same is true in RL; expert guidance is likely to provide valuable information, but the agent also needs to explore by itself. There isn't a correct amount of either.

But I am certain when saying that problems can benefit from some amount of expert tutelage. This section introduces how to incorporate this expertise into your policy.

Behavior Cloning

A traditional approach to incorporating expert policies is to use supervised learning to maximize the likelihood of an agent choosing actions that are the same as the expert for encountered states. This is called *behavior cloning* but isn't popular because it overfits the expert demonstrations. If an agent were to find itself in a state that wasn't demonstrated, then the agent wouldn't know what to do. This situation, because of high-dimensionality in complex problems, is very likely to happen, which makes it important to use algorithms that continue to learn from their actions, even if they aren't demonstrated.³²

Imitation RL

In 2017, Hester et al. proposed that in many real-life situations an accurate simulation is not available or feasible; the only possibility is to train the agent in situ. But there are external sources that already use viable policies, like a previous controller or human guidance. Developing solutions to transfer knowledge from these experts to agents enables a range of solutions for problems where demonstrations are common but accurate simulators are not.³³

First, an expert (human or machine) generates trajectories in the environment. Then a supervised loss is incorporated into the Bellman update that measures the difference between the actions of the expert and the agent. Over time the agent learns to generate trajectories similar to the expert.

The problem with this method is that you don't want the agent to mimic the expert, since the expert's policy is likely to be suboptimal. *Deep Q-learning from demonstrations (DQfD)* accounts for this by attempting to prevent overfitting to the expert trajectory through a series of regularizations. *Generative adversarial imitation learning (GAIL)* suggests a similar approach that adversarially perturbs action distributions toward an expert. But this doesn't answer or have an intuitive knob to tweak that controls *how* much expert advice to use.³⁴

Soft Q imitation learning (SQIL) provides one idea and at the same time simplifies DQfD and GAIL by changing the way standard algorithms use experience replay. Reddy et al. found that (a) carefully controlling the trajectories that reside in the replay buffer, (b) reconstructing the reward signal for those trajectories, and (c) using a loss function that allows exploration but stays close to expert trajectories allows the agent to learn from experience even when there is no extrinsic reward signal. The resulting algorithm is broadly similar to behavior cloning, except that it allows for deviations into states that were not demonstrated. It penalizes those states through regularization and nudges the agent back toward expert behavior. You could even control the amount of expert advice to use by weighting the regularization. So I can imagine an annealing version that initially uses expert demonstrations and tends

toward pure RL over time. But the main selling point is the simplicity, and this can be adapted to any off-policy algorithm, discrete or continuous.³⁵

Inverse RL

The problem with imitation RL is that there is no guarantee that the expert is demonstrating an optimal policy. For example, imagine you were training your car to “drive well” and to help your algorithm you recorded your driving and submitted that as an expert demonstration. Are you really an expert driver? Even if you are, should the agent be learning from your biased view of driving?

Another approach is ignoring the trajectories and concentrating on the goal. Producing a reward for “driving well,” however, is hard to quantify. You could suggest that you want to get to your destination in the shortest time possible and use total elapsed time as a reward, but this ignores getting there safely. You could iteratively establish all of the main concerns but then how do you weight them? In general, it is difficult to design rewards for complex domains so instead, why not learn the reward?

Inverse RL (IRL) attempts to predict what a reward would look like, given demonstrations from an environment.³⁶ The general aim is to find an arbitrary reward function, based upon demonstrated trajectories, that explain expert behavior. Designing reward functions is a major challenge in RL, and IRL promises to replace the manual specification with demonstrations, which are arguably less biased and easier to specify. It is also argued that learning from demonstrations is more robust because they don’t make any assumptions about the underlying environment.

Most IRL methods fit the following pattern:

1. Assume the expert’s demonstrations as a solution to an MDP
2. Use a parameterized model as the reward function
3. Solve the MDP with the current reward function
4. Update the reward function to minimize the difference between the expert’s trajectory and the learned trajectory (or policies if those are available)
5. Repeat until stable

Figure 8-8 depicts this process for a simple 3×3 discrete grid. An RL agent is trained to solve MDP with a parameterized reward function. Each time the MDP has learned an optimal policy it is compared to the expert trajectories. If the learned policy visits a state that is also visited by the expert, the reward for that state is increased (or equivalent reward parameters are changed). Over time the reward function tends toward the expert trajectory.

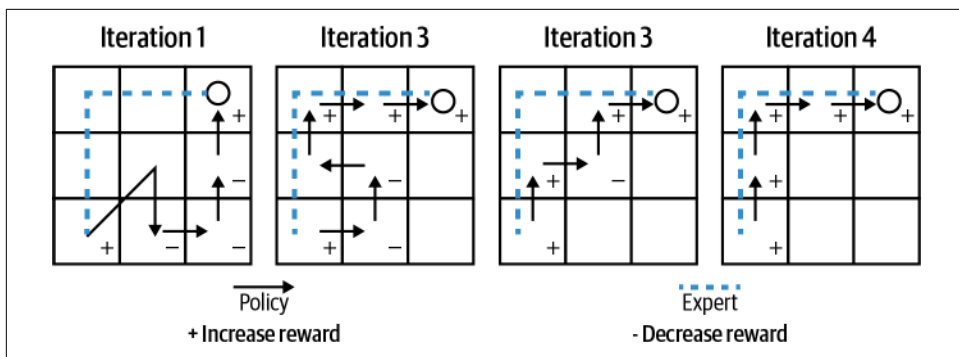


Figure 8-8. Example iterations of a notional IRL problem. The environment is a 3×3 grid. A single expert trajectory is provided (dashed lines). The policy (solid lines) uses a parameterized reward function in the MDP. Over time the reward function is altered to be more likely to produce expert trajectories. Eventually you end up with a reward function that mimics expert behavior.

Most of the problems with IRL stem from the fact that the expert trajectories are likely to be noisy and suboptimal. Researchers have developed a wide range of algorithms that adapt this basic process to improve robustness against noise (see the entropy-based or probabilistic IRL algorithms) or to loosely direct learning toward optimality, rather than matching the expert trajectory, which tends to be more problem specific.

Another major problem is that these methods expect you to repeatedly retrain RL algorithms, which as you know can take a long time. This means IRL doesn't scale to anything but the simplest of state spaces. All is not lost, though; you can use a dimensionality reduction technique like autoencoding or hashing to reduce the high-dimensional state into something that IRL can handle.^{37,38}

One final remark I want to make is that when researchers talk about expert guidance in their papers, they tend to assume that the experts are successful at what they do. Failure is far more common than success, and there is no reason why you can't include disastrous trajectories in IRL; just demote the rewards for these trajectories instead. Shiarlis et al. suggest that generating failed demonstrations is even easier than generating good ones.³⁹

Curriculum Learning

When you learn science, sailing, or snowboarding, there is a well-defined series of steps or tests that you have to achieve before you can move onto something more advanced. In snowboarding you first train your leg muscles how to control the edge, then you train your core to shift your weight to turn, and finally you learn how to twist in order to carve and spin. In science you first learn about atoms before fusion,

Newton's laws of motion before fluid dynamics, and cell structures before DNA. These are curricula, high-level meta-learning guides that show you common strategies to learning advanced policies.

Curricula can be used in RL to help reduce the amount of exploration an agent needs to perform. You can specify a series of simpler subgoals that lead to learning the main goal. This sounds very similar to [“Imitation RL” on page 236](#) and several techniques under the curriculum banner use imitation techniques to derive curriculum subtasks.

Markov games (or other MARL frameworks) are important use cases for curriculum learning. You can often alter the formulation of a problem to play agents against each other in *self-play*. The most obvious form of self-play is in the highly publicized board game examples, but researchers have applied the same idea to slightly more realistic environments like mazes or robotic movement.^{40,41} There are many options for how to design the interaction in self-play but one common form is to use one instantiation of an agent as a teacher and another as a student. The teacher suggests “tasks” by performing them in the environment (and therefore generating a trajectory) and the student solves that task by trying to perform the same “task.” I say “task” because in practice they are just moves to a particular state, which may or may not translate to something humans would consider a task. Rewards are shaped to encourage the student to tackle more complex tasks over time by increasing the path length of the teacher. Another common form is to incorporate imitation-like techniques to automatically classify skills and repeatedly ask the agent being trained to practice skills.⁴²

Another interesting idea that is similar to both imitation and curriculum learning is the combination of policies. *Residual RL* defines a way of combining two policies to solve a single task. It's not presented as hierarchical RL but could also be considered as such. The idea is that if you have an accurate model for part of the task, which is common in robotics, for example, then you can combine that policy with another to achieve tasks that are much harder to model like grasping or touching. This is introduced through the addition of policies. I like this idea because it is so simple. You have seen many examples of altering the reward function or the loss function by arbitrarily adding bonuses or penalties, but this is the first example of combining actions. Intuitively, I think there is more to this than first meets the eye. I can imagine a more general action-altering framework that is applicable to other areas of RL (safe-RL, for example).⁴³

Curriculum learning is another hot topic because it is one avenue that could potentially help RL and ML to scale to really complex problems. The idea that you can engineer your problem so that there is an unsupervised solution to learn a range of important skills or possibly even solutions is enticing. For more information see the recent reviews in [“Further Reading” on page 242](#).

Other Paradigms

Researchers like to coin a term. This can lead to a deluge of other paradigms that can be hard to interpret. Often some of them will break out into popular parlance, like Q-learning, whereas others could be big in the future but might not be, like meta-learning. I introduce this here along with transfer learning because they didn't quite fit anywhere else. They are big topics in their own right, but I consider them to be quite advanced and therefore a detailed discussion is out of the scope of this book.

Meta-Learning

Meta-learning, or learning-to-learn, aims to develop a general framework that optimizes the performance of learning. In other words, if you knew the best way to learn, you derive an algorithm that is optimal for your task.

One of the most important applications of meta-learning is quantifying or improving learning performance when there are few observations or rare events. Humans are capable of learning patterns and policies from small numbers of observations by transferring knowledge from past experience or other domains.

The problem with *meta-RL* is that it is so abstract there are many ways in which you could imagine improving learning performance. For example, some implementations implement artificial curiosity and boredom.⁴⁴ Others use genetic, evolutionary, or adversarial approaches.⁴⁵ Some even combine imitation and inverse RL to the mix.⁴⁶

All this noise makes it hard to define precisely what meta-RL is or where it is heading. If the previous research is anything to go by, then meta-RL will continue to be a mash-up of RL and ML techniques with the goal of solving problems with less data, faster.

Transfer Learning

Transferring the experience of one agent to another is of obvious interest. If you can train a robot in simulation and then optimally transfer that knowledge to the real world, you can artificially increase the sample efficiency of your RL algorithm. Historically, transfer learning was considered separately, but today it tends to be absorbed into meta-learning. It is still important for industrial applications, however.

A common problem when converting research-oriented RL algorithms to industrial applications is that nearly all research papers train from scratch. But most of the time similar problems have already been solved, as demonstrated in the literature or by your previous work. It makes no sense to throw away all that experience and the resulting policy.

Like meta-RL, however, there are many ways in which you can reincorporate past experience into new models. You could reuse raw experiences via replay buffers,

copy/paste autoencoders, use previous policies to help guide the new one via curriculum learning or concoct some kind of cooperative MARL solution. The possibilities are endless and there is no right way.

Fundamentally there are three types of transfer. The first, and most common, is to attempt to transfer policies that are used for the same task in the same domain. Pre-training with a simulation is an example of this. The second is transferring a policy in the same domain to achieve a slightly different task. This is the goal of curriculum learning and common in applications where a single agent is capable of solving multiple tasks, like robotics. The third is transferring policies across domain boundaries, where there may be different states and actions. This happens often accidentally due to partial observability, but a classic example would be attempting to train an RL algorithm to play the Atari game *Pong*, then get the same policy to play *Breakout*.

I think this is still an important subject for industrial applications, because it allows you to build solutions and train algorithms in a more sample-efficient way, which is often the bottleneck in real applications. But it is beyond the scope of this book, so if you are interested, I provide some starting points in “[Further Reading](#)” on page 242.

Summary

The purpose of RL is to learn optimal strategies, so it makes perfect sense to include as much extra information as you can to help solve your problem better and quicker. The first step on this journey is to ensure that your basic assumptions are valid and to check if your problem definition can be improved. This often starts by questioning the validity of the MDP and leads down new avenues of potential solutions. The classic example is that many real-life domains are partially observable, but are rarely modeled as such. Accepting the partial observability can lead you to solutions that happily handle uncertainty.

Hierarchical RL (HRL), especially skill-aware RL, has the potential to significantly scale industrial solutions. If original equipment manufacturers or vendors can derive and reuse “skills,” this might help them provide more solutions using the same processes, hardware, and software. This will help open up new use cases and help diversify revenue streams. For these reasons I believe that HRL has a very important part to play in industry.

Multi-agent RL (MARL) is a very popular research topic because it presents a good way to solve distributed problems. For example, bush fires are increasing in both quantity and intensity, so having intelligent solutions for geospatial problems like this can help in that fight.⁴⁷ On a technical level, too, it makes sense to tackle the exploration problem by using the experience of multiple agents in the same environment, perhaps cooperatively. And competitive scenarios arise in a number of domains, especially in those that involve money.

Utilizing expert advice to guide strategies is another pattern that doesn't need explaining. One of the main issues in complex industrial problems is constraining the problem definition and therefore the exploration space. Industry expects and demands more performance and improved efficiencies and one of the best ways to get there is to bootstrap, just like n -step methods did way back in [Chapter 3](#). On a strategic level, bootstrapping is essentially a process of copying successful strategies, whether that be from expert guidance or another model. Once you have extracted that expertise then you can work on fine-tuning it with standard RL techniques.

And finally, RL is a rapidly expanding topic and as abstractions become higher and higher they tend to veer off in weird and wonderful directions. Some of these will become the next big thing, others will sit quietly in a corner. I think that the use of meta-learning in RL, for example, will become more important as engineers gain more practical experience with RL. I think we're in the *what* phase now and soon engineers will be asking *why*. With adoption that will turn toward *how* and meta-RL presents a set of ideas to answer that question. But as the name suggests, the topic is very philosophical and can be hard to qualify. I'm sure that will improve with time, if indeed that is the right direction to head in.

Further Reading

- Hierarchical reinforcement learning:
 - [Great overview video of the technical difficulties by Doina Precup](#).
- Multi-agent reinforcement learning:
 - Schwartz, H. M. 2014. *Multi-Agent Machine Learning: A Reinforcement Approach*. John Wiley & Sons.
 - A great RL-focused recent review from Oroojlooy and Hajinezhad.⁴⁸
 - Another review that spans games and EFGs from Zhang et al.⁴⁹
 - Decentralized MARL:
 - A great [introduction to decentralized MARL from Christopher Amato](#), and accompanying book by Oliehoek and Amato.⁵⁰
 - Expert guidance:
 - A recent survey of inverse RL.⁵¹
 - Curriculum learning:
 - Two recent surveys.^{52,53}
 - [Lillian Weng's blog](#) is always a great resource and has a nice overview on curriculum learning.

- Meta-RL:
 - Huimin Peng has bravely created a good review on meta-RL.⁵⁴
- Transfer learning:
 - There's a great chapter by Alessandro Lazaric that develops transfer learning at a high level.⁵⁵
 - An interesting recent review of transfer learning in general (not RL-specific).⁵⁶

References

- [1] Karl, Maximilian, Maximilian Soelch, Justin Bayer, and Patrick van der Smagt. 2017. “Deep Variational Bayes Filters: Unsupervised Learning of State Space Models from Raw Data”. ArXiv:1605.06432, March.
- [2] Hefny, Ahmed, Zita Marinho, Wen Sun, Siddhartha Srinivasa, and Geoffrey Gordon. 2018. “Recurrent Predictive State Policy Networks”. ArXiv:1803.01489, March.
- [3] Wayne, Greg, Chia-Chun Hung, David Amos, Mehdi Mirza, Arun Ahuja, Agnieszka Grabska-Barwinska, Jack Rae, et al. 2018. “Unsupervised Predictive Memory in a Goal-Directed Agent”. ArXiv:1803.10760, March.
- [4] Qiao, Zhiqian, Katharina Muelling, John Dolan, Praveen Palanisamy, and Priyantha Mudalige. 2018. “POMDP and Hierarchical Options MDP with Continuous Actions for Autonomous Driving at Intersections”. 2018 21st International Conference on Intelligent Transportation Systems (ITSC), 2377–82.
- [5] Qiao, Zhiqian, Zachariah Tyree, Priyantha Mudalige, Jeff Schneider, and John M. Dolan. 2019. “Hierarchical Reinforcement Learning Method for Autonomous Vehicle Behavior Planning”. ArXiv:1911.03799, November.
- [6] Hochberg, Irit, Guy Feraru, Mark Kozdoba, Shie Mannor, Moshe Tennenholtz, and Elad Yom-Tov. 2017. “A Reinforcement Learning System to Encourage Physical Activity in Diabetes Patients”. *Journal of Medical Internet Research* 19 (10): e338.
- [7] Hallak, Assaf, Dotan Di Castro, and Shie Mannor. 2015. “Contextual Markov Decision Processes”. ArXiv:1502.02259, February.
- [8] Ie, Eugene, Chih-wei Hsu, Martin Mladenov, Vihan Jain, Sanmit Narvekar, Jing Wang, Rui Wu, and Craig Boutilier. 2019. “RecSim: A Configurable Simulation Platform for Recommender Systems”. ArXiv:1909.04847, September.
- [9] Chandak, Yash, Georgios Theocharous, Blossom Metevier, and Philip S. Thomas. 2020. “Reinforcement Learning When All Actions Are Not Always Available”. ArXiv:1906.01772, January.

- [10] Boutilier, Craig, Alon Cohen, Amit Daniely, Avinatan Hassidim, Yishay Mansour, Ofer Meshi, Martin Mladenov, and Dale Schuurmans. 2018. “**Planning and Learning with Stochastic Action Sets**”. ArXiv:1805.02363, May.
- [11] Chandak, Yash, Georgios Theodorou, Blossom Metevier, and Philip S. Thomas. 2020. “**Reinforcement Learning When All Actions Are Not Always Available**”. ArXiv:1906.01772, January.
- [12] Chandak, Yash, Georgios Theodorou, Chris Nota, and Philip S. Thomas. 2020. “**Lifelong Learning with a Changing Action Set**”. ArXiv:1906.01770, May.
- [13] Li, Xiang, Wenhao Yang, and Zhihua Zhang. 2019. “**A Regularized Approach to Sparse Optimal Policy in Reinforcement Learning**”. ArXiv:1903.00725, October.
- [14] Saleh, Abdelrhman, Natasha Jaques, Asma Ghandeharioun, Judy Hanwen Shen, and Rosalind Picard. 2019. “**Hierarchical Reinforcement Learning for Open-Domain Dialog**”. ArXiv:1909.07547, December.
- [15] Zhou, Guojing, Hamoon Azizsoltani, Markel Sanz Ausin, Tiffany Barnes, and Min Chi. 2019. “**Hierarchical Reinforcement Learning for Pedagogical Policy Induction**”. *Artificial Intelligence in Education*, edited by Seiji Isotani, Eva Millán, Amy Ogan, Peter Hastings, Bruce McLaren, and Rose Luckin, 544–56. Lecture Notes in Computer Science. Cham: Springer International Publishing.
- [16] Nachum, Ofir, Shixiang Gu, Honglak Lee, and Sergey Levine. 2018. “**Data-Efficient Hierarchical Reinforcement Learning**”. ArXiv:1805.08296, October.
- [17] Eysenbach, Benjamin, Abhishek Gupta, Julian Ibarz, and Sergey Levine. 2018. “**Diversity Is All You Need: Learning Skills without a Reward Function**”. ArXiv:1802.06070, October.
- [18] Li, Siyuan, Rui Wang, Minxue Tang, and Chongjie Zhang. 2019. “**Hierarchical Reinforcement Learning with Advantage-Based Auxiliary Rewards**”. ArXiv:1910.04450, October.
- [19] Saleh, Abdelrhman, Natasha Jaques, Asma Ghandeharioun, Judy Hanwen Shen, and Rosalind Picard. 2019. “**Hierarchical Reinforcement Learning for Open-Domain Dialog**”. ArXiv:1909.07547, December.
- [20] Waradpande, Vikram, Daniel Kudenko, and Megha Khosla. 2020. “**Deep Reinforcement Learning with Graph-Based State Representations**”. ArXiv:2004.13965, April.
- [21] Sharma, Archit, Shixiang Gu, Sergey Levine, Vikash Kumar, and Karol Hausman. 2020. “**Dynamics-Aware Unsupervised Discovery of Skills**”. ArXiv:1907.01657, February.

- [22] Zheng, Stephan, Alexander Trott, Sunil Srinivasa, Nikhil Naik, Melvin Gruesbeck, David C. Parkes, and Richard Socher. 2020. “The AI Economist: Improving Equality and Productivity with AI-Driven Tax Policies”. ArXiv:2004.13332, April.
- [23] Cox, Tony. 2019. “Muddling-Through and Deep Learning for Managing Large-Scale Uncertain Risks”. *Journal of Benefit-Cost Analysis* 10(2): 226–50.
- [24] Xiao, Kefan, Shiwen Mao, and Jitendra K. Tugnait. 2019. “TCP-Drinc: Smart Congestion Control Based on Deep Reinforcement Learning”. *IEEE Access* 7:(11) 892–904.
- [25] Xiao, Yuchen, Joshua Hoffman, Tian Xia, and Christopher Amato. 2020. “Learning Multi-Robot Decentralized Macro-Action-Based Policies via a Centralized Q-Net”. ArXiv:1909.08776, March.
- [26] Pham, Huy X., Hung M. La, David Feil-Seifer, and Matthew Deans. 2017. “A Distributed Control Framework for a Team of Unmanned Aerial Vehicles for Dynamic Wildfire Tracking”. ArXiv:1704.02630, April.
- [27] Tomic, Teodor, Korbinian Schmid, Philipp Lutz, Andreas Domel, Michael Kas-secker, Elmar Mair, Iris Lynne Grixia, Felix Ruess, Michael Suppa, and Darius Burschka. 2012. “Toward a Fully Autonomous UAV: Research Platform for Indoor and Outdoor Urban Search and Rescue”. *IEEE Robotics Automation Magazine* 19(3): 46–56.
- [28] Cui, Jingjing, Yuanwei Liu, and Arumugam Nallanathan. 2018. “Multi-Agent Reinforcement Learning Based Resource Allocation for UAV Networks”. ArXiv: 1810.10408, October.
- [29] Lowe, Ryan, Yi Wu, Aviv Tamar, Jean Harb, Pieter Abbeel, and Igor Mordatch. 2020. “Multi-Agent Actor-Critic for Mixed Cooperative-Competitive Environments”. ArXiv:1706.02275, March.
- [30] Matignon, Laetitia, Guillaume J. Laurent, and Nadine Le Fort-Piat. 2007. “Hysteric Q-Learning: An Algorithm for Decentralized Reinforcement Learning in Cooperative Multi-Agent Teams”. *2007 IEEE/RSJ International Conference on Intelligent Robots and Systems*, 64–69.
- [31] Omidshafiei, Shayegan, Jason Pazis, Christopher Amato, Jonathan P. How, and John Vian. 2017. “Deep Decentralized Multi-Task Multi-Agent Reinforcement Learning under Partial Observability”. ArXiv:1703.06182, July.
- [32] Ross, Stephane, Geoffrey J. Gordon, and J. Andrew Bagnell. 2011. “A Reduction of Imitation Learning and Structured Prediction to No-Regret Online Learning”. ArXiv:1011.0686, March.

- [33] Hester, Todd, Matej Vecerik, Olivier Pietquin, Marc Lanctot, Tom Schaul, Bilal Piot, Dan Horgan, et al. 2017. “Deep Q-Learning from Demonstrations”. ArXiv:1704.03732, November.
- [34] Ho, Jonathan, and Stefano Ermon. 2016. “Generative Adversarial Imitation Learning”. ArXiv:1606.03476, June.
- [35] Reddy, Siddharth, Anca D. Dragan, and Sergey Levine. 2019. “SQIL: Imitation Learning via Reinforcement Learning with Sparse Rewards”. ArXiv:1905.11108, September.
- [36] Ng, Andrew Y., and Stuart J. Russell. 2000. “Algorithms for Inverse Reinforcement Learning.” *Proceedings of the Seventeenth International Conference on Machine Learning*, 663–670. ICML 2000. San Francisco, CA: Morgan Kaufmann Publishers Inc.
- [37] Tucker, Aaron, Adam Gleave, and Stuart Russell. 2018. “Inverse Reinforcement Learning for Video Games”. ArXiv:1810.10593, October.
- [38] Cai, Xin-Qiang, Yao-Xiang Ding, Yuan Jiang, and Zhi-Hua Zhou. 2020. “Imitation Learning from Pixel-Level Demonstrations by HashReward”. ArXiv:1909.03773, June.
- [39] Shiarlis, Kyriacos, Joao Messias, and Shimon Whiteson. 2016. “Inverse Reinforcement Learning from Failure.” *Proceedings of the 2016 International Conference on Autonomous Agents & Multiagent Systems*, 1060–1068. AAMAS 2016. Singapore: International Foundation for Autonomous Agents and Multiagent Systems.
- [40] Sukhbaatar, Sainbayar, Zeming Lin, Ilya Kostrikov, Gabriel Synnaeve, Arthur Szlam, and Rob Fergus. 2018. “Intrinsic Motivation and Automatic Curricula via Asymmetric Self-Play”. ArXiv:1703.05407, April.
- [41] Laux, Melvin, Oleg Arenz, Jan Peters, and Joni Pajarinen. 2020. “Deep Adversarial Reinforcement Learning for Object Disentangling”. ArXiv:2003.03779, March.
- [42] Jabri, Allan, Kyle Hsu, Ben Eysenbach, Abhishek Gupta, Sergey Levine, and Chelsea Finn. 2019. “Unsupervised Curricula for Visual Meta-Reinforcement Learning”. ArXiv:1912.04226, December.
- [43] Johannink, Tobias, Shikhar Bahl, Ashvin Nair, Jianlan Luo, Avinash Kumar, Matthias Loskyll, Juan Aparicio Ojea, Eugen Solowjow, and Sergey Levine. 2018. “Residual Reinforcement Learning for Robot Control”. ArXiv:1812.03201, December.
- [44] Alet, Ferran, Martin F. Schneider, Tomas Lozano-Perez, and Leslie Pack Kaelbling. 2020. “Meta-Learning Curiosity Algorithms”. ArXiv:2003.05325, March.
- [45] Wang, Rui, Joel Lehman, Jeff Clune, and Kenneth O. Stanley. 2019. “Paired Open-Ended Trailblazer (POET): Endlessly Generating Increasingly Complex and Diverse Learning Environments and Their Solutions”. ArXiv:1901.01753, February.

- [46] Paine, Tom Le, Sergio Gómez Colmenarejo, Ziyu Wang, Scott Reed, Yusuf Aytar, Tobias Pfaff, Matt W. Hoffman, et al. 2018. “One-Shot High-Fidelity Imitation: Training Large-Scale Deep Nets with RL”. ArXiv:1810.05017, October.
- [47] Pham, Huy X., Hung M. La, David Feil-Seifer, and Matthew Deans. 2017. “A Distributed Control Framework for a Team of Unmanned Aerial Vehicles for Dynamic Wildfire Tracking”. ArXiv:1704.02630, April.
- [48] OroojlooyJadid, Afshin, and Davood Hajinezhad. 2020. “A Review of Cooperative Multi-Agent Deep Reinforcement Learning”. ArXiv:1908.03963, June.
- [49] Zhang, Kaiqing, Zhuoran Yang, and Tamer Başar. 2019. “Multi-Agent Reinforcement Learning: A Selective Overview of Theories and Algorithms”. ArXiv:1911.10635, November.
- [50] Oliehoek, Frans A., and Christopher Amato. 2016. *A Concise Introduction to Decentralized POMDPs*. 1st ed. Springer Publishing Company.
- [51] Arora, Saurabh, and Prashant Doshi. 2019. “A Survey of Inverse Reinforcement Learning: Challenges, Methods and Progress”. ArXiv:1806.06877, August.
- [52] Narvekar, Sanmit, Bei Peng, Matteo Leonetti, Jivko Sinapov, Matthew E. Taylor, and Peter Stone. 2020. “Curriculum Learning for Reinforcement Learning Domains: A Framework and Survey”. ArXiv:2003.04960, March.
- [53] Portelas, Rémy, Cédric Colas, Lilian Weng, Katja Hofmann, and Pierre-Yves Oudeyer. 2020. “Automatic Curriculum Learning For Deep RL: A Short Survey”. ArXiv:2003.04664, May.
- [54] Peng, Huimin. 2020. “A Comprehensive Overview and Survey of Recent Advances in Meta-Learning”. ArXiv:2004.11149, June.
- [55] Lazaric, Alessandro. 2012. “Transfer in Reinforcement Learning: A Framework and a Survey”. *Reinforcement Learning: State-of-the-Art*, edited by Marco Wiering and Martijn van Otterlo, 143–73. Adaptation, Learning, and Optimization. Berlin, Heidelberg: Springer.
- [56] Zhuang, Fuzhen, Zhiyuan Qi, Keyu Duan, Dongbo Xi, Yongchun Zhu, Hengshu Zhu, Hui Xiong, and Qing He. 2020. “A Comprehensive Survey on Transfer Learning”. ArXiv:1911.02685, June.

Practical Reinforcement Learning

Reinforcement learning (RL) is an old subject; it's decades old. Only recently has it gained enough prominence to raise its head outside of academia. I think that this is partly because there isn't enough disseminated industrial knowledge yet. The vast majority of the literature talks about algorithms and contrived simulations, until now.

Researchers and industrialists are beginning to realize the potential of RL. This brings a wealth of experience that wasn't available in 2015. Frameworks and libraries are following suit, which is increasing awareness and lowering the barrier to entry.

In this chapter I want to talk less about the gory algorithmic details and more about the process. I want to answer the question, "What does a real RL project involve?" First I will go over what an RL project looks like and propose a new model for building industrial RL products. Along the way I will teach you how to spot an RL problem and how to map it to a learning paradigm. Finally, I'll describe how to design, architect, and develop an RL project from simple beginnings, pointing out all of the areas that you need to watch out for.

The RL Project Life Cycle

Typical RL projects are designed to be solved by RL from the outset, typically because of prior work, but sometimes because designers appreciate the sequential nature of the problem. RL projects can also emerge from a machine learning (ML) project where engineers are looking for better ways to model the problem or improve performance. Either way, the life cycle of an RL project is quite different from ML and very different to software engineering. Software engineering is to RL what bricklaying is to bridge building.

RL development tends to follow a series of feelings that are probably familiar to you. This rollercoaster is what makes engineering such a fulfilling career. It goes something like this:

1. *Optimism*: “RL is incredible, it can control robots so surely it can solve this?”
2. *Stress and depression*: “Why doesn’t this work? It can control robots so why can’t it do this? Is it me?”
3. *Realization*: “Oh right. This problem is much harder than controlling robots. But I’m using deep learning so it will work eventually (?). I’m going to pay for more GPUs to speed it up.”
4. *Fear*: “Why is this still not working? This is impossible!”
5. *Simplification*: “What happens if I remove/replace/alter/augment the real data?”
6. *Surprise*: “Wow. It converged in only 5 minutes on my laptop. Note to self, don’t waste money on GPUs again...”
7. *Happiness*: “Great, now that’s working I’ll tell my project manager that I’m getting close, but I need another sprint...”
8. *GOTO 1*

In a more general form, this is true for any project that I’ve worked on. I’ve found one simple trick that helps to manage stress levels and improve project velocity. And if you only take one idea away from this book, then make it this:

Start Simple

That’s it. Starting simple forces you to think about the most important aspects of the problem, the MDP. It helps you create really big abstractions; you can fill in the details later. Abstractions help because they free up mental resources that would otherwise be ruminating on the details; they allow you to communicate concepts to other people without the gory technicalities.

I know nothing about neuroscience or psychology, but I find that stress during development, which I think is different than normal stress, is proportional to the amount of mental effort expended on a problem and inversely proportional to the probability of success. If I had a long day of intense concentration, which was required because the darn thing wasn’t working properly, I’m much more stressed and grumpy when I see my family. Having an easier problem, which leads to more successes, seems to help reduce my stress levels.

Project velocity is the speed at which development is progressing. It is a fictional measure made up by process coaches. When comparing you or your team’s velocity over a period of time (not between teams or people) then it becomes easier for an outsider to spot temporary issues. On a personal level, velocity is important for the

reasons in the previous paragraph. If you get stuck, you have to force yourself to work harder on the problem and without successes you get stuck in a spiral of despair.

One problem with the typical points-based velocity measures is that it doesn't take task complexity or risk into consideration. One week you can be doing some boilerplate software and burn through a load of single-point tasks (5×1 points, for example), the next you can be working on a model and banging your head against the wall (1×5 points). They might have exactly the same number of estimated points, but the second week is far more stressful.

To alleviate this problem, here is my second tip:

Keep Development Cycles as Small as Possible

I think of a development cycle as the time between my first keystrokes and the point at which things work as I intended. One representational metric is the time between git commits, if you use commits to represent working code. The longer these cycles get, the greater the chance of nonworking code and more stress. Furthermore, long cycles mean that you have to wait a long time for feedback. This is especially troublesome when training big models on lots of data. If you have to wait for two days to find out whether your code works, you're doing it wrong. Your time is far too valuable to sit doing nothing for that length of time. And undoubtedly you will find a mistake and have to cancel it anyway. Reduce the amount of time between cycles or runs to improve your efficiency and if you have to, run those long training sessions overnight; new models fresh out of the oven in the morning is like a being a kid at Christmas.

Neither of these ideas is new, but I find they are incredibly important when doing RL—for my own sanity, at least. It is so easy to get sucked into solving a problem in one attempt, a whirlpool of intent. The likely result is failure due to a mismatch between problem and MDP or model and data, all of which are due to too much complexity and not enough understanding. Follow the two preceding tips and you have a fighting chance.

Life Cycle Definition

Other than the proceeds of the MDP paradigm, many of the tools and techniques used in machine learning are directly transferable to RL. Even the process is similar.

Data science life cycle

Figure 9-1 presents a diagram of the four main phases of a data science project that I see when working with my colleagues at Winder Research. It largely follows the classic cross-industry standard process for data mining from the '90s, but I have simplified it and accentuated the iterative nature.

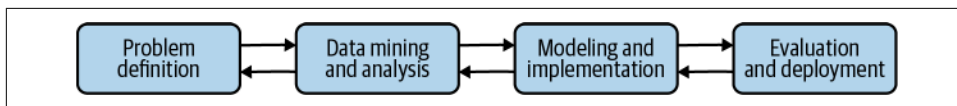


Figure 9-1. A diagram of the four main phases of a data science project, a simplified version of the CRISP-DM model.

All projects start with a problem definition and it is often wrong. When the definition changes it can massively impact engineering work, so you must make sure that it both solves a business challenge and is technically viable. This is one of the most common reasons for perceived failure—*perceived* because technically projects can work as intended, but they don't solve real business challenges. Often it takes several iterations to refine the definition.

Next is data mining and analysis. Around two-thirds of the time of a data scientist is spent in this phase, where they attempt to discover and expose new information that might help solve the problem and then analyze and clean the data ready for modeling.

In the modeling and implementation phase, you build and train models that attempt to solve your problem.

Finally, you evaluate how well you are solving the problem using quantitative and qualitative measures of performance and deploy your solution to production.

At any one of these phases you may find it necessary to go back a step to reengineer. For example, you may develop a proof of concept in a Jupyter notebook to quickly prove that there is a viable technical solution, but then go back to build production-ready software. Or you might find that during initial evaluation you don't have enough of the right data to solve the problem to a satisfactory standard. Even worse, you have a solution and report back to the stakeholders only to find that you solved the wrong problem.

Reinforcement learning life cycle

Figure 9-2 shows a depiction of a typical process for an RL project. The first thing you will notice is how similar the phases are. The goals are the same (to solve a business challenge using data) so it is reasonable to expect the process to be similar, too. But the similarities exist only on the surface. When you dig into each phase the tasks are quite different.

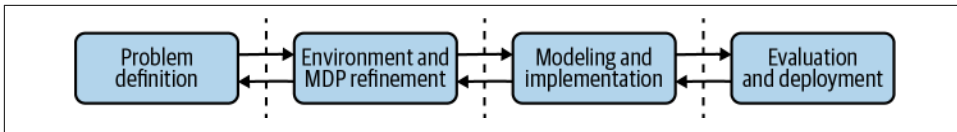


Figure 9-2. A diagram of the four main phases of an RL project.

In an ML project, it is unlikely that you have the ability to dramatically alter the problem definition. You can probably tinker with the performance specification, for example, but fundamentally the problem you are trying to solve is defined by the business. Then you would search for and locate any data that already exists within the business, get permission to use it, and put the technology in place to access it. If you don't have any data, then you can try to gather new data or use an external source. Once you have the data at your disposal you would spend time analyzing, understanding, cleaning, and augmenting to help solve the problem. This represents the first two boxes of the data science process.

In RL the situation is quite different, even perverse at times. First you are given a business challenge. The problem is strategic, at a higher level of abstraction compared to “normal” data science problems, and it has a real-life definition of success like profit, number of subscribers, or clicks. Then you, yes you, take that definition of success and carefully engineer the problem to fit into an MDP and design the reward function. Until there is an inverse RL (see [“Inverse RL” on page 237](#)) algorithm that works in every situation, you will need to design the reward yourself. This is a big burden to place on your shoulders. You have the power to decide how to quantify performance, in real terms. In one sense this is great, because it directly exposes how the project affects the business in human-readable terms like lives saved (not accuracy or F1-score, for example). But this also places more burden on the engineer. Imagine a strategic-level agent that chooses the direction of the business and it leads to a bad result. Historically, this risk is carried by the executives in your company—hence the girth of the paycheck—but in the future it will be delegated to algorithms like RL, which are designed by engineers. I haven't seen any evidence of how this is going to affect the hierarchical control structures present in so many companies but I anticipate that there will be a shakeup. Shareholders pay for productivity and when productivity is driven by the engineers that design the algorithms, does that also mean they run the company?

The next phase in the process is to decide how best to learn. In machine learning you only need to iterate on the data you have. But in RL, because it relies on interaction and sequential decisions, you will never have all the data you need to generate realistic rollouts. In the vast majority of cases it makes sense to build simulations to speed up the development feedback cycle, rather than use a live environment. Once you have a viable solution then you can roll out to real life.

Notice the difference here. Either you are generating artificial observations or you don't have real data until you actively use your algorithm. If someone said to me that I had to deploy my fancy deep learning model into production before collecting *any* data, I'd probably cry a little inside.

But this is precisely the situation in RL and what makes state, action, and reward engineering so important. You can't hope to build a robust solution unless you fully understand your state and actions and how they affect the reward. Trying to throw deep learning at a problem you don't understand is like buying a \$1,000 golf club and expecting to become a golf pro.

There's more, too, which I discuss throughout this chapter. But I want to emphasize that although there are similarities and transferable skills, RL is a totally different beast. The first two boxes in [Figure 9-2](#), problem definition and environment/MDP refinement, I discuss in this chapter. I will leave the final two boxes until the next chapter.

Problem Definition: What Is an RL Project?

I think by now you have a fairly good idea about the definition of RL, but I want to spend this section talking about applying RL to everyday challenges. To recap, you should base your model on a Markov decision process (MDP, see [Chapter 2](#) and [“Rethinking the MDP” on page 216](#)). The problem should have unique states, which may be discrete or continuous—I find that it often helps to think of discrete states, even if it is continuous. In each state there should be some probability of transitioning to another state (or itself). And an action should alter the state of the environment.

Although this definition captures nearly everything, I find it intuitively difficult to think in terms of states and transition probabilities. So instead I will try to paint a different picture.

RL Problems Are Sequential

RL problems are sequential; an RL agent optimizes trajectories over several steps. This alone separates RL from ML; the paradigm of choice for single-step decision making. ML cannot optimize decisions for a long-term reward. This is crucial in many business challenges. I don't think any CEO with an inkling of humanity would squeeze a prospect for every penny *right now*. Instead, they know that managing the relationship and managing how and when the sales that are made to optimize for the long-term support of the customer is the best route toward a sustainable business.

This philosophy maps directly to how you would solve industrial, customer-centric problems. You shouldn't be using ML models to optimize your ad placement or recommend which product to buy because they optimize for the wrong thing. Like a

voracious toddler, they can't see the bigger picture. ML models are trained to make the best possible decision at the time, irrespective of the future consequences. For example, ML models are very capable of optimizing product recommendations, but they might include the product equivalent of click-bait: products which get lots of clicks but are ultimately disappointing. Over time, a myopic view will likely annoy users and could ultimately hurt profit, retention, or other important business metrics.

RL algorithms are perfectly suited to situations where you can see that there are multiple decisions to be made, or where the environment doesn't reset once you've made a decision. This opens up a massive range of possibilities, some of which you saw in "RL Applications" on page 7. These may be new problems or problems that already have a partial solution with ML. In a sense, a problem that has already been partially solved is a better one than a new project, because it's likely that engineers are already very familiar with the domain and the data. But be cognizant that initial RL results may produce worse results in the short term while algorithms are improved. Visit the [accompanying website](#) to get more application inspiration.

RL Problems Are Strategic

I like to think of software engineering as a way of automating processes and machine learning can automate decisions. RL can automate strategies.

Figure 9-3 hammers this idea home. Modern businesses are made of three core functions. Businesses have a plethora of processes, from performance reviews to how to use the printer. Enacting of any one of these processes is rarely valuable, but they tend to occur often. Because of the frequency and the fact it takes time and therefore money for a human to perform this process, it makes sense to automate the process in software.

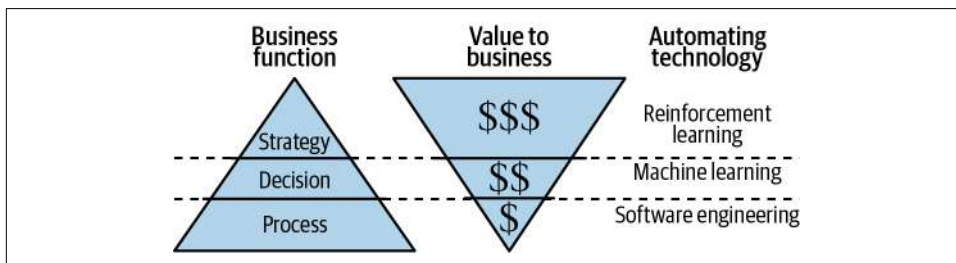


Figure 9-3. A depiction of the different functions of a modern business, the value of each action in that function, and the technology that maps to the function.

Business decisions are rarer than business processes, but they still happen fairly often. Making the decision to perform CPR, phone a warm prospect, or block a suspicious looking transaction can save lives/be profitable/prevent fraud. All of these challenges can be solved using machine learning. These actions are quantifiably valuable, but

like I said previously, they have a limited life span. They intentionally focus on the single transitive event. Decisions need processes in place to make them possible.

Business strategies form the heart of the company. They happen rarely, primarily because there are limited numbers of people (or one) designing and implementing the strategy. They are closely held secrets and direct a multitude of future people, decisions, and processes. They have wide-reaching consequences, like the healthcare for a nation or the annual profits of the company. This is why optimal strategies are so valuable and why RL is so important.

Imagine being able to entrust the running of your company/healthcare/country to RL. How does that sound? What do you think?

You might suggest that this is a pipe dream, but I assure you it is not. Taxation is being tackled with MARL.¹ Healthcare use cases are obvious and there are thousands of references.² And companies must automate to become more competitive, which is nothing new—companies have been automating for over a century now.

The future, therefore, is in the hands of the engineers, the people who know how to build, adapt, and constrain algorithms to be robust, useful, and above all, safe.

Low-Level RL Indicators

The previous two suggestions, that RL problems are sequential and strategic, are high level. On a lower level you should be looking for individual components of the problem.

An entity

Look for entities, which are concrete things or people, that interact with the environment. Depending on the problem, the entity could sit on both sides of the environment interface. For example, a person viewing a recommendation is an entity but they are part of the environment. Other times people are simulated agents, like in the taxation example.

An environment

The environment should be, in the language of domain-driven design, a bounded context. It should be an interface that encapsulates all of the complexities of the situation. You should not care whether behind the interface there is a simulation or real life; the data at the interface should be the same. Beware of making an environment too big or too complex, because this impacts the solvability of the problem. Splitting or simplifying the environment makes it easier to work with.

A state

The easiest way to spot state is to observe or imagine what happens when the state changes. You might not think of it as a change, but look carefully. The price of a stock changes because the underlying state of all investors changes. The state of a robot changes because it has moved from position A to position B. After showing a recommendation your target's state may have changed from "ignore recommendation" to "click on the recommendation."

What can you observe about the environment (which may include entities)? What are the most pertinent and informative features? Like in ML, you want to be careful to only include clean, informative, uncorrelated features. You can add, remove, or augment these features as you wish but tend toward minimizing the number of features if possible, to reduce the exploration space, improve computational efficiency, and improve robustness. Don't forget that you can merge contextual information from other sources, like the time of day or the weather.

The domain of the state can vary dramatically depending on the application. For example, it could be images, matrices representing geospatial grids, scalars representing sensor readings, snippets of text, items in a shopping cart, browser history, and more. Bear in mind that from the perspective of an agent, the state may be uncertain or hidden. Remind yourself often that the agent is not seeing what you can see, because you have the benefit of an oracle's view of the problem.

An action

What happens to the environment when you apply an action? The internal state of an environment should have the potential to change (it may not, because the action was to stay put or it was a bad one) when you apply an action. Ideally you want to observe that state change and if you don't, consider looking for observations that expose this information. Again, try to limit the number of actions to reduce the exploration space.

Quantify success or failure

In this environment, how would you define success or failure? Can you think of a way to quantify that? The value should map to the problem definition and use units that are understandable to both you and your stakeholders. If you are working to improve profits, then quantify in terms of monetary value. If you are trying to improve the click rate of something, go back and try to find a cash amount for that click. Think about the real reason why your client/boss/stakeholder is interested in you doing this project. How can you quantify the results in terminology they understand and appreciate?

Types of Learning

The goal of an agent is to learn an optimal policy. The rest of this book investigates *how* an agent learns, through a variety of value-based and policy gradient-based algorithms. But what about *when* it learns?

Online learning

The overwhelming majority of examples, literature, blog posts, and frameworks all assume that your agent is learning *online*. This is where your agent interacts with a real or simulated environment and learns at the same time. So far in this book I have only considered this regime. One of the major problems with this approach is the sample efficiency, the number of interactions with an environment required to learn an optimal policy. In the previous couple of decades researchers have been chasing sample efficiency by improving exploration and learning guarantees. But fundamentally there is always going to be an overhead for exploration, sampling delays due to stochastic approximation, and the all important stability issues. Only recently have they begun to consider learning from logs.

Offline or batch learning

Until quite recently, it was thought that all learning had to be performed online because of the inherent coupling between the action of an agent and the reaction of an environment.³ But researchers found that it was possible to learn from a *batch* of stored data, like data in a replay buffer. Because this data has no further interaction with the environment it is also known as learning *offline* or in *batches*. **Figure 9-4** presents the idea. First, data is generated online using a policy (possibly random) and saved in a buffer. Offline, a new policy is trained upon the data in the buffer. The new policy is then deployed for use.

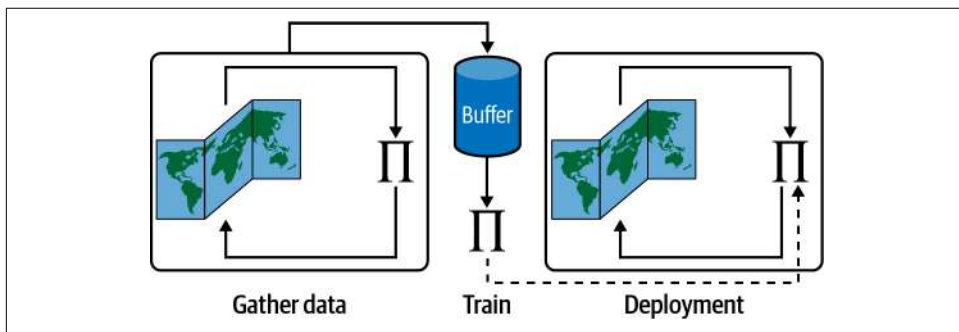


Figure 9-4. Batch reinforcement learning.

The primary benefit of learning offline is the increase in sample efficiency. You can capture or log a set of data and use it to train models as many times as you like with zero impact on the environment. This is an incredibly important feature in many domains where collecting new samples is expensive or unsafe.

But hang on. If I collect and train upon a static snapshot of data, then isn't this pure supervised learning? The answer is yes, kind of. It is supervised in the sense that you have a batch of data with labeled transitions and rewards. But this doesn't mean you can use any old regression or classification algorithm. The data is (assumed to be) generated by an MDP and therefore the algorithms designed to find optimal policies for MDPs are still useful. The major difference is that they can no longer explore.

If an agent can't explore, then it can't attempt to improve the policy by "filling in the gaps," which was the primary reason for rapid advances in RL over the past decade. A similar argument is that it prevents the agent from asking counterfactual questions about the action. Again, this is a theoretical method to provide more robust policies by allowing them to ask "what if" questions. Fundamentally, the distribution of the training data (for state, actions, and rewards) is different than that observed at deployment. Imagine how that affects a typical supervised learning algorithm.

In principle, *any* off-policy RL algorithm could be used as an offline RL algorithm, by removing the exploration part from the algorithm. However, this is problematic in practice for a number of reasons. Value methods struggle because they rely on accurate value estimates. But many of the states or actions will not be present in the buffer and therefore estimates will be inaccurate at best. Policy gradient methods struggle because of the distribution shift, which at best produces gradients that point in the wrong direction and at worst blow up. These problems get worse when you use overly specified models like deep learning.⁴

At the time of this writing, researchers are working hard to solve these problems to open up the next large gains in sample efficiency. One avenue of research is to further refine the stability guarantees of basic RL algorithms so that they don't misbehave when encountering unstable distributions or undersampled states or actions. Better theoretical guarantees when using deep learning is an obvious avenue that would dramatically improve the situation. Other approaches are attempting to use techniques like imitation learning to learn policies that generate positive rewards.⁵ An even simpler approach is to train an autoencoder upon the buffer to create a simulated environment that agents *can* explore in.⁶ In practice these techniques are already being applied and are showing impressive results, especially in **robotics**.⁷ From a practical perspective, then, you might find the **DeepMind RL Unplugged benchmark** a useful tool to find or prototype ideas.⁸

Concurrent learning

The literature contains a mix of learning techniques that fall somewhere between multi-agent RL (MARL) and multiprocess frameworks, but are interesting enough to mention here.

The length of your feedback cycles are dictated by the complexity of the problem; the harder the challenge, the longer it takes to solve. This is partially caused by increasing model complexities, but primarily it is an exploration problem. Large state and action spaces need more exploration to “fill in the gaps.”

Many researchers have concentrated on improving exploration in the single-agent case, which has worked wonders. But that only scales to a point. Ultimately an agent still needs to visit all of the state-action space. One option is to run multiple agents at the same time (see “[Scaling RL](#)” on page 299), but if they are using the same exploration algorithm then they can tend to explore the same space, again negating the efficiency as the state-action space is scaled up.

Instead, why not explore *concurrently*, where teams of agents have the *joint* task of exploring the state-action space. Beware that this term is overloaded, though; researchers have used the word *concurrent* to mean anything from cooperative MARL to concurrent programming for RL.^{9,10} I use the term to denote the goal of performing coordinated exploration of the same environment, first suggested in 2013.¹¹

The implementation of this idea is somewhat domain dependent. As one example, many companies are interacting with thousands or millions of customers in parallel and are trying to maximize a reward local to that customer, like lifetime value or loyalty. But these customers reside within the confines of the same environment; a company only has a limited set of in-stock products and you have roughly the same demographic information for each customer. The learnings from one customer should be transferred to others, so this implies either centralized learning or at least parameter sharing.

I don’t think there is one right answer because there are many small challenges that need unique adaptations. Like what would happen if you introduced the ability to retarget customers with emails or ads? This would improve the performance metrics but the rewards may be delayed. So you would have to build in the capacity to “skip” actions and deal with the partial observability.

Dimakopoulou and Van Roy formalized this idea in 2018 and proved that coordinated exploration could both increase learning speed and scale to large numbers of agents.¹² Their solution is surprisingly simple: make sure each agent has a random seed that maximizes exploration. The benefit of this approach is you can use standard RL algorithms and each agent is independently committed to seeking out optimal performance. It also adapts well to new data because starting conditions are

repeatedly randomized. The challenge, however, is that the optimal seed sampling technique is different for every problem.

In concurrent learning then, agents are independent and do not interact. Agents may have different views of the environment, like different customers have different views of your products. Despite the different name, this is very similar to MARL with centralized learning. So if you are working on a problem that fits this description, then make sure you research MARL as well as concurrent RL.

Reset-free learning

Included in the vast majority of examples and algorithms is the implicit use of “episodes,” which imply that environments can be “reset.” This makes sense for some environments where the starting position is known, fixed, or easily identifiable. But real life doesn’t allow “do-overs,” even though sometimes I wish it would, and would require omnipotent oversight. To avoid an Orwellian version of the film *Groundhog Day*, many applications need some way of not specifying a reset.

Early solutions suggested that agents could learn how to reset.¹³ Agents are in control of their actions so you can store previous actions in a buffer. To get back to the starting point you can “undo” those actions. This works in many domains because action spaces have a natural opposite. But action uncertainties and unnatural “undos” prevent this approach being generally applicable.

Another idea is to predict when terminations or resets or unsafe behavior is going to occur and stop it before it happens.¹⁴ But again this requires oversight, so another suggestion is to learn a function to predict when resets will occur as part of the learning itself.¹⁵ As you can see from the vocabulary this has hints of meta-learning. But none of these approaches is truly reset free; why not eliminate resets entirely?

You could, technically, never reset and use a discounting factor of less than one to prevent value estimates tending toward infinity. But the reset is more important than you realize. It allows agents to have a fresh start, like when you quit your boring job and start an exciting career in engineering. It is very easy for exploring agents to get stuck and spend significant time in similar states. The only way to get out of a rut is for someone or something to give it a kick. A reset is a perfect solution to this problem because it gives the agent another chance to try a different path.

A related problem is when an agent is able to reach a goal or a global optimum, then the best next action is to stay put, gorging on the goal. Again, you need a reset or a kick to explore more of the suboptimal state to make the policy more robust.

So rather than learning how to reset, maybe it is better to learn how to encourage the agent away from the goal and away from regions it can get stuck. Or provide a simple controller to shuffle the environment like Zhu et al., which results in **robust robotic policies** with no resets.¹⁶

To conclude, reset-free learning is possible in some simple applications but it's quite likely that your agents will get stuck. At some point you will need to consider whether you are able to manipulate your environment to move the agent to a different state. You might also be able to get the agent to move itself to a different state. Again, different problem domains will have different solutions.

RL Engineering and Refinement

RL problems, like any engineering problem, have high-level and low-level aspects. Changing the high-level design impacts the low-level implementation. For lack of a better term, I denote the high-level analysis and design as *architectural* concerns. These include refining the model of the Markov decision process, defining the interfaces, defining interactions between components, and how the agent will learn. These decisions impact the implementation, or *engineering* details.

I've talked a lot already about high-level architectural aspects, so in this section I want to discuss the implementation details. I endeavor to cover as many of the main concerns as possible, but many implementation details are domain and problem specific. In general, I recommend that you search the literature to find relevant examples and then scour the implementation details for interesting tidbits. These details, often glossed over in the researcher's account, offer vital insight into what works and what doesn't.

Process

Before I dive into the details I think it is worth explaining how I perform this engineering work in general terms. I find that most tasks in engineering, irrespective of the abstraction level, tend to follow some variant of the the observe, orient, decide, act (OODA) loop. What is ostensibly the scientific method, the OODA loop is a simple, four-step approach to tackling any problem, popularized by the American military. The idea is that you can iterate on that loop to utilize context and make a quick decision, while appreciating that decisions may change as more information becomes available. **Figure 9-5** shows this loop in engineering terms, where I suggest the words *analysis*, *design*, *implementation*, and *evaluation* are more appropriate, but they amount to the same thing.

The most important aspect is that this is a never-ending process. On each iteration of the loop you evaluate where you are and if that is not good enough, you analyze where improvements could be made. You then pick the next best improvement and implement that.

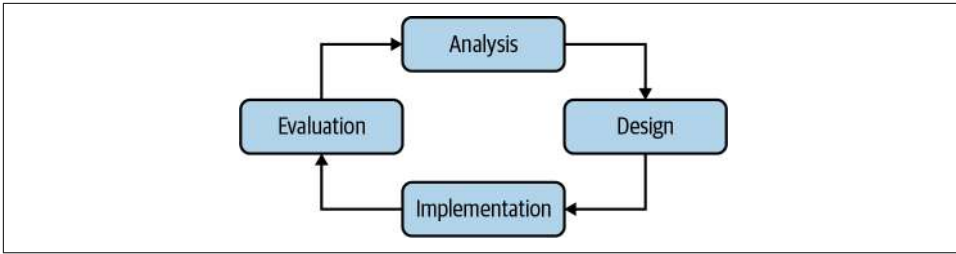


Figure 9-5. The process of RL refinement.

No one single improvement will ever be enough and there is always one more improvement that you could make. You have to be careful not to get stuck in this loop, and you can do that by remembering the **Pareto distribution**: 80% of the solution will take 20% of the time and vice versa. Consider if implementing that last 20% is really worth it, given all the other value you could be providing with that time.

Environment Engineering

I find that the most productive first step of an RL project is to create an *environment*. Throughout this book I use this term to denote an abstraction of the world that the agent operates within. It is the interface between what actually happens and what the agent wants (see “**Reinforcement Learning**” on page 4).

But now I am specifically talking about the implementation of this interface. Whether you are working in real life submitting real trades or you are creating a simulation of a wind turbine, you need an implementation of the environment interface.

The benefit of concentrating on this first is that you will become intimately acquainted with the data. This maps to the “data understanding” step in the CRISP-DM model. Understanding your environment is crucial for efficiently solving industrial challenges. If you don’t, what tends to happen is what I call the “improvement shotgun.” By throwing a bunch of different ideas at the problem you will probably find one that hits the target, but most will miss. This type of work is not productive, efficient, or intelligent.

Instead, understand the environment fully, pose scientifically and engineeringly—yes, apparently that is a word—robust questions, and implement functionality that answers those questions. Over time you will gain insight into the problem and you will be able to explain *why* improvements are important, not just state that they are.

Implementation

OpenAI's Gym has organically become the de facto environment interface in the RL community.¹⁷ In fact, it's one of the most cited RL papers of all time. You've probably come across it before, but just in case you haven't, a Gym is a Python framework for expressing environments. It exposes useful primitives like definitions for action and state space representation and enforces a simple set of functionality via an interface that should suit most applications. It also comes with a big set of example environments that are used throughout the internet as toy examples.

From an industrial perspective, the interface is a great way to model your problem and help refine your MDP. And because it is so popular, it is likely that you can rapidly test different algorithms because the majority expect the Gym interface.



I'm not going to go through a detailed example of an implementation, because it is out of the scope of this book, but you can find many great examples from a quick search or visiting the [Gym GitHub website](#).

Simulation

A simulation is supposed to capture the most important aspects of a real-life problem. Simulations are not intended to be perfect representations, although many people, researchers, and businesses invest vast amounts of time making simulations more realistic. The benefit of having a simulation is that it is easier, cheaper, and quicker to operate within the confines of a simulation than it is in real life. This reduces the length of the developmental feedback cycles and increases productivity. But the precise benefits depend on the problem; sometimes learning in real life is easier and simpler than building a simulation.

You should first search to see if someone has already attempted to build a simulation. Understanding the fundamental aspects of the problem, the physics, biology, or chemistry, is often more difficult than solving the problem, although building a simulation will certainly improve your knowledge about the subject, so it might be worthwhile for that reason alone. Many commercial simulators like MuJoCo or Unity are available, but come with a commercial price tag. You will have to decide whether the cost is worth the money for your problem. In general I recommend evaluating whether the simulation is a competitive differentiator for your business or product. If it is, it may be worth investing in development; if not, save yourself a lot of time and buy off-the-shelf. And I have found that these vendors are immensely helpful and professional, so ask them for help. It is in their interest for your idea or business to succeed, because that means future revenue for them.

If you do decide to implement the simulator yourself then you will need to build models to simulate the problem and decide which aspects to expose. This is where

MDP design plays a crucial role. In general, you should try to expose the same aspects of the problem that can be observed in real life. Otherwise, when you try to transfer your learnings to real life you will find that your agent doesn't behave as you expect.

Similarly, many researchers attempt to take models trained in simulation and apply them to real-life tasks. This is certainly possible, but is entirely defined by how representative the simulation is. If this is your intention, then you should implicitly couple the implementations of the simulation and real life. The vast majority of algorithms and models don't like it when you change the underlying state or action space.

Simulations are perfect for proof-of-concept projects, to validate that an idea is technically viable. But be careful to not place too much hope on the simulation because there is no guarantee that the policy your agent has derived is possible in real life. It is eminently possible that real-life projects can fail spectacularly even though simulations work perfectly. More often than not the simulation misses some crucial aspect or information.

Interacting with real life

If your experience with the simulation is encouraging, or you have decided that it is easier to skip the simulation, then the next step is to build another (Gym) interface to interact with real life. The development of such an implementation is often much easier, because you don't need to understand and implement the underlying mechanics; nature does that for you. But adapting your algorithms to suit the problem is utterly dependent on understanding and controlling the data.

The primary concerns of the real-life implementation consist of the operational aspects, things like scaling, monitoring, availability, and so on. But architecture and design still plays an important part. For example, in real life there are obvious safety concerns that are not present in a simulation. In some domains a significant amount of time must be invested in ensuring safe and reliable operation. Other common problems include partial observability, which doesn't exist in an omnipotent simulation, and excessive amounts of stochasticity; you would have thought that motors move to the position you tell them to, but often this is not the case.

I don't think this small section sufficiently expresses the amount of engineering time that can go into real-world implementations. Of course, implementations are wildly domain specific, but the main problem is that this work is pretty much pure research. It is hard to accurately estimate how long the work will take, because neither you nor anyone else in the world has tackled this exact problem before. You are a digital explorer. Remember to remind your project managers of this fact. And remember the process from the start of this chapter. *Start simple*. Life is hard enough without you making it harder.

Learning How to Simulate

One technical adjunct I want to throw into the mix is the idea of making the creation of a simulation an optimization problem. If you can formulate your simulation so that the main components are parameterized or even better, have formal mathematical definitions, then it is quite possible to optimize your simulation to behave more like real life.

All you need is a source of ground truth: log data from the real environment, past experiments, or possibly a more sophisticated simulation. Then you can automatically tweak the parameters of your simulation to generate data that looks more like this. You could treat this as a brute-force hyperparameter search problem, using an appropriate “distance from real-life” measure, or in the spirit of this book, use RL to learn optimal parameters!¹⁸ Again, this is very meta, but I like the idea of unifying the value chain through the MDP abstraction and RL.

State Engineering or State Representation Learning

In data science, *feature engineering* is the art/act of improving raw data to make it more informative and representative of the task at hand. In general, more informative features make the problem easier to solve. A significant amount of time after a proof of concept is spent cleaning, removing, and adding features to improve performance and make the solution more robust.

I like to use similar terminology to refer to the engineering of the state, although I haven’t seen it used much elsewhere. *State engineering*, often called *state representation learning*, is a collection of techniques that aim to improve how the observations represent the state. In other words, the goal is to engineer better state representations. There is no right way of doing this, however, so you are primarily guided by metrics like performance and robustness.

Although there are parallels, this isn’t feature engineering. The goal is not to solve the problem by providing simpler, more informative features. The goal is to create the best representation of the state of the environment as you can, then let the policy solve the problem. The reason for this is that in most industrial challenges, it is very unlikely that you are capable of crafting features that can compete with an optimal multistep policy. Of course, if you can, then you should consider using standard supervised learning techniques instead.

Like in feature engineering, domain expertise is important. The addition of a seemingly simple feature can dramatically improve performance, far more than trying different algorithms. For example, Kalashnikov et al. found that in their robotics experiments that used images as the state representation, adding a simple “height of gripper” measurement improved the robot’s ability to pick up unseen objects by a

whopping 12%.¹⁹ This goes to show that even though researchers are always trying to learn from pixels, simple, reliable measurements of the ground truth are far better state representations.

Note that many papers and articles on the internet bundle both the state engineering and policy/action engineering into the same bucket. But I do consider them separate tasks because the goal of state engineering is to concentrate on better representations of the state, not improve how the policy generates actions.

Learning forward models

In some cases it may be possible to access ground truth temporarily, in a highly instrumented laboratory experiment, for example, but not in general. You can build a supervised model to predict the state from the observations and ship that out to the in-field agents. This is also called learning a *forward model*.

Constraints

Constraining the state space helps to limit the amount of exploration required by the agent. In general, policies improve when states have repeated visits, so limiting the number of states speeds up learning.

Apply constraints by using *priors*, information that is known to be true, formally defined by a probability distribution. For example, if you know that features should be strictly positive, then ensure your policy knows that and doesn't attempt to reach those states.

Transformation (dimensionality reduction, autoencoders, and world models)

Dimensionality reduction is a task in data science where you try to reduce the number of features without removing information. Traditional techniques are useful in RL too. Using principal component analysis, for example, reduces the state space and therefore reduces learning time.²⁰

But the crown jewels of state engineering are the various incarnations of autoencoders to reconstruct the observation. The idea is simple: given a set of observations, train a neural network to form an internal representation and regenerate the input. Then you tap into that internal representation and use it as your new state.

Typically, implementations attempt to reduce the dimensionality of the input by using a neural architecture that progressively compresses the state space. This constraint tends to force the network to learn higher-level abstractions and hopefully, an important state representation. However, there are suggestions that *increasing* the dimensionality can improve performance.²¹ I remain somewhat skeptical of this claim as it actively encourages overfitting.

Learning representations of individual observations is one thing, but MDPs are sequential, therefore there are important temporal features locked away. A new class of MDP-inspired autoencoders is beginning to treat this more like a prediction problem. Given an ordered set of observations, these autoencoders attempt to predict what happens in the future. This forces the autoencoder to learn temporal features as well as stateful features. The particular implementation differs between domains, but the idea remains the same.^{22,23}

The term *world models*, after the paper with the same name, refers to the goal of attempting to build a model of an environment from offline data, so that the agent can interact as if it was online. This idea is obviously important for practical implementations where it is hard to learn on live data and so the term has become popular to describe any method that attempts to model an environment.²⁴ Ha and Schmidhuber deserve extra kudos because of their excellent [interactive version](#) of their paper. I consider this work to be a specific implementation of state engineering, which may or may not be useful depending on your situation, but it is a very useful tool to help segment the RL process.

These techniques can be complicated by MDP issues like partial observability, but in general autoencoder research can be directly applied to RL problems. If this is important for your application then I recommend you dig into these papers. Frameworks are also beginning to see the value in state representation, like [SRL-zoo](#), a collection of representation learning methods targeting PyTorch, which you might find useful.

Policy Engineering

An RL policy is responsible for mapping a representation of the state to an action. Although this sounds simple, in practice you need to make careful decisions about the function that implements this mapping. Like in machine learning, the choice of function defines the limits of performance and the robustness of the policy. It also impacts learning performance, too; using nonlinear policies can cause divergence and generally take much longer to train.

I generalize this work under the banner of *policy engineering*, where the goal is to design a policy that best solves a problem given a set of constraints. The vast majority of the RL literature is dedicated to finding improved ways of defining policies or improving their theoretical guarantees.

The policy is fundamentally limited by the Bellman update, so unless there is a major theoretical shift, all policies have a similar aim. Researchers introduce novelty by affixing new ways to represent state, improve exploration, or penalize some other constraint. Underneath, all RL algorithms are value-based or policy-based.

Given this similarity between all algorithms I visualize a policy as having three distinct tasks, like in [Figure 9-6](#). First it has to convert the representation of state into a

format that can be used by an internal model. The internal model is the decision engine, learning whatever it needs to learn to be able to convert state to possible actions. A policy has to explore and you have a lot of control over that process. Finally, the policy has to convert the hidden representation into an action that is suitable for the environment.

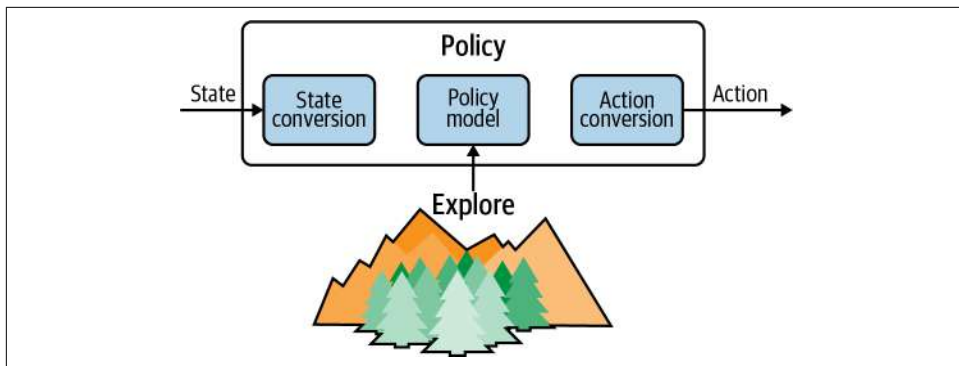


Figure 9-6. A depiction of the three phases of a policy.

The engineering challenge is to design functions to perform these three phases in an efficient and robust way. The definition of the policy model has been the subject of most of this book. In the following subsections I describe how to use discrete or continuous states and actions.

Discrete states

A *discrete* feature is permitted to only take certain values, but can have an infinite range. The vast majority of data is received in a form quantized by analog-to-digital converters, so you can consider most data to be discrete. But in practice when people say “discrete space” they typically mean an integer-based state space, where state can take only integer values. Most of the time quantized data is converted to a float and is considered continuous.

Discrete state inputs are useful because of their simplicity. There are fewer states that an agent has to explore and therefore training performance and time can be reduced, even predicted. Problems with smaller state spaces will converge faster than larger ones.

A discrete state space is any GridWorld-like environment. Geospatial problems are a good example. Even though positions could be encoded by continuous measures like latitude and longitude, it is often computationally and intuitively simpler to convert a map into a grid, like Chaudhari et al. did when optimizing ride-share routing policies in the city of New York.²⁵

Problems with discrete state spaces are considered to be easier to solve, mainly because it is so easy to debug and visualize. Discrete domains are often easy to reason about, which makes it possible to make accurate theoretical priors and baselines. You can directly compare policies to these assumptions, which makes it easy to monitor training progress and fix problems as they arise. Another benefit is that value functions can be stored in a dictionary or lookup table, so you can leverage industrially robust technology like databases.

But the most important benefit is that discrete spaces are fixed; there is no chance of any “in-between” state, so there is no need for any approximation of any kind. This means you can use basic Q-learning. Using a simpler, more theoretically robust algorithm like Q-learning leads to simpler and more robust implementations, a very important factor when using these models in production.

The problem, however, is that naturally occurring discrete states are uncommon. In most implementations the engineer has forced the discrete state upon the problem to obtain the aforementioned benefits. This arbitrary discretization is likely to be sub-optimal at best and causes a loss of information detail. Even though the arguments for discrete state spaces are enticing, it is unlikely that they yield the best performance. In the previous example, for instance, the diameter of each cell in New York’s grid was about 2 miles. Even after optimizing the positioning of ride-share vehicles, they could end up being 2 miles from their customer.

Continuous states

A *continuous* feature is an infinite set of real values. They may be bounded, but there is no “step” between neighboring values. An example could be the speed of an autonomous vehicle or the temperature of a forge. Continuous state spaces are considered to be slightly more difficult to work with than discrete spaces because this leads to an infinite number of potential states, which makes it impossible to use a simple lookup table, for example.

Policies of continuous states have to be able to predict the best next action or value for a given state, which means that the policies must approximate, rather than expect known states. You can provide an approximation using any standard machine learning technique, but functional approximators are most common. Neural networks are increasingly popular due to the unified approach and tooling.

The main worry about continuous state spaces is that they are only provably convergent when using linear approximators. As of 2020, no convergence proofs exist for nonlinear approximators, which means that any “deep” algorithm isn’t guaranteed to converge, even if it is a good model, although empirical evidence suggests they do. Linear approximations can model a significant amount of complexity via domain transformations. Simplicity, stability, and configurable amounts of complexity mean

that like in machine learning, linear methods are very useful. In general, there are three ways to provide an approximation:

Linear approximations

Learning a set of linear weights directly based upon the input features is possibly one of the simplest approaches to deal with continuous states. This is effectively the same as linear regression. You can also improve the quality of the regression through any standard linear regression extension, like those that reduce the influence of outliers, for example. A linear policy is provably convergent.

Nonparametric approximations

You can use a classification-like model to predict values, if that makes sense. For example, nearest neighbor and tree-based algorithms are capable of providing robust, scalable predictions. Again, their simplicity and popularity has led to a wide range of off-the-shelf, robust industrial implementations, which you can leverage to make your implementation more scalable and robust. There is no guarantee that nonparametric policies will converge.

Nonlinear approximation

Nonlinear approximations provide a flexible amount of complexity in a uniform package: neural networks. You can control the amount of complexity through the architecture of the network. Neural networks have produced state-of-the-art results in many domains so it would be wise to evaluate them in your problem. There is no guarantee that nonlinear policies will converge.

The provable convergence guarantees are a strong feature of linear approximations. And you can obtain a surprising amount of model complexity by transforming the data before passing to the linear approximation. The following below is a selection of popular techniques to increase model complexity while maintaining the simplicity and guarantees of linear methods:

Polynomial basis

A polynomial expansion is the generation of all possible combinations of features up to a specified power. Polynomials represent curves in value space, which enables slightly more complexity over linear methods.

Fourier basis

Fourier expansions generate oscillations in value space. These oscillations can be combined in arbitrary ways to generate complex shapes, similar to how sounds are comprised of tonal overlays. Fourier bases are complete, in the sense that they can approximate any (well-behaved) function for a given level of complexity, controlled by the order of the basis function. They work well in a range of domains.²⁶ The Fourier basis struggles in environments with discontinuities, like the `Cliffworld` environment, however.

Radial basis

Radial basis functions are often Gaussian-shaped transformations (other radial bases exist) that measure the distance between the state and the center of the basis, relative to the basis's width. In other words, these are parameterized Gaussian-like shapes that can be moved and resized to approximate the value function. The main problem with radial basis functions (and other tiling schemes) is that you have to choose the width of the basis, which effectively fixes the amount of “smoothing.” Optimal trajectories tend to produce “spikes” in the value function so you end up having to use quite narrow widths, which can lead to problematic local optima. Despite this radial basis functions are useful because the width hyperparameter provides a simple and effective way to control the “amount of complexity.”

Other transforms

Signal processing provides a wide range of other transformations that could be applied to generate different feature characteristics, like the Hilbert or Wavelet transforms, for example. But they are not so popular, typically because most RL researchers have a mathematical or machine learning background. Those with an electronics background, for example, would be more than happy to use these “traditional” techniques.

For more details about linear transformations, see the extra resources in [“Further Reading” on page 288](#).

Converting to discrete states

You may want to consider discretizing your domain. Converting continuous states is especially common where there are natural grid-like representations of a domain, like on a map. The main benefit of discretizing is that it might remove the need for a function approximator, because of the fixed size of the state space. This means that simple, fast, and robust Q-learning-based algorithms can be used. Even if you do still need a function approximator, discretization decreases the apparent resolution of the data, making it much easier for agents to explore and easier for models to fit. This has proven to result in an increase in raw performance for difficult continuous problems like in the *Humanoid* environment.²⁷ The following list presents an overview of discretization possibilities (more information can be found in [“Further Reading” on page 288](#)):

Binning

Fix the range of a feature and quantize it into bins, like a histogram. This is simple and easy to comprehend but you lose accuracy. You can place bin boundaries in a range of nonuniform ways, like constant width, constant frequency, or logarithmically, for example.

Tile coding

Map overlapping tiles to a continuous space and set a tile to 1 if the observation is present in that tile's range. This is similar to binning, but provides increased accuracy due to the overlapping tiles. Tiles are often square, but can be any shape and nonuniformly positioned.

Hashing

When features are unbounded, like IP or email addresses, you can forcibly constrain the features into a limited set of bins without loss of information by hashing (there will still be a loss in accuracy due to binning).

Supervised methods

In some domains you may have labeled data that is important to the RL problem. You might want to consider building a classification model and pass the result into the RL algorithm. There are a wide variety of algorithms suitable for this purpose.²⁸

Unsupervised methods

If you don't have labels you could use unsupervised machine learning methods, like k-means, to position bins in better locations.²⁹

Mixed state spaces

If you have a mixed state space then you have three options:

1. Unify the state space through conversion
2. Cast the discrete space to a continuous one and continue using continuous techniques
3. Split the state space into separate problems

Options 1 and 2 are self explanatory. The majority of functional approximators will happily work with discretized values, even though it may not be optimal or efficient.

Option 3, however, is interesting. If you can split the state into different components that match bounded contexts in the problem domain, that might indicate that it is more efficient to use submodels or subpolicies. For example, a crude approach could be to separate the continuous and discrete states, feed them into two separate RL algorithms, then combine them into an ensemble.³⁰ As you can see this is touching upon hierarchical RL, so be sure to leverage ideas from there.

Mapping Policies to Action Spaces

Before you attempt to define how a policy specifies actions, take a step back and consider what type of actions would be most suitable for your problem. Ask yourself the following questions:

- *What value types can your actions take?* Are they binary? Are they discrete? Are they bounded or unbounded? Are they continuous? Are they mixed?
- *Are simultaneous actions possible?* Is it possible to select multiple actions at the same time? Can you iterate faster and interleave single actions to approach selecting multiple actions?
- *When do actions need to be made?* Is there a temporal element to action selection?
- *Are there any meta-actions?* Can you think of any high-level actions that might help learning or handle the temporal elements?
- *Can you split the actions into subproblems?* Are there obvious bounded contexts in your actions? Can these be split into separate problems? Is this a hierarchical problem?
- *Can you view your actions in another way?* For example, rather than moving to a continuous-valued position, could you consider a “move forward” action? Or vice versa?

All of these decisions have grave impacts on your work, so be careful when making them. For example, switching an action from continuous to discrete might force you to change algorithms. Or realizing that your agent requires simultaneous actions might make your problem much harder to solve.

Binary actions

Binary actions are possibly the simplest to work with, because they are easy to optimize, visualize, debug, and implement. Policies are expected to output a binary value mapping to the action that they want to choose. In problems where only a single action is allowed, picking the maximum expected return (for value methods) or the action with the highest probability (for policy gradient methods) is a simple implementation.

As seen in [“How Does the Temperature Parameter Alter Exploration?” on page 203](#), you can encourage broad exploration by making the policy output probabilities, or values that can be interpreted as probabilities. The agent can then randomly sample from this action space according to the assigned probabilities. For value methods, you can assign the highest probabilities to the highest expected values. I would recommend using advantages, rather than values directly, to avoid bias. An exponential softmax function is recommended to convert values to probabilities. In policy gradient methods you can train the models to predict probabilities directly.

Continuous actions

Continuous actions are very common, like determining the position of a servo or specifying the amount to bid on an ad. In these cases it is most natural to use some

kind of approximation function to output a continuous prediction for a given state (just like regression); deterministic policy gradients do this. But researchers have found that modeling the output as a random variable can help exploration. So rather than directly predicting the output value, they predict the mean of a distribution, like a Gaussian. Then a value is sampled from distribution. Other parameters of the distribution, like the standard deviation for the Gaussian, can also be predicted. Then you can set the initial standard deviation at the beginning high, to encourage exploration, and allow the algorithm to shrink it as it learns. I like to think of this as a crude attempt to incorporate probabilistic modeling. Of course this assumes that your actions are normally distributed, which may not be accurate, so think carefully about what distribution you should choose for your actions.

An alternative is to leverage newer methods that are based upon Q-learning, but have been adapted to approximate continuous functions: continuous action Q-learning.³¹ This is by no means as common as using policy gradient methods but claims to be more efficient.

One final trick that I want to mention, which is helpful in simple examples, is called the *dimension scaling trick*. When using linear approximators that need to predict multiple outputs, like the mean and standard deviation, you can have distinct parameters for each action and update the parameters for the chosen action individually. This is effectively a dictionary of parameters, where the key is the action and the values are the parameters for the continuous mapping, like the mean and standard deviation.

Hybrid action spaces

Spaces with both discrete and continuous actions are relatively common, like deciding to buy and how much to spend, and can be tricky to deal with. First consider if you can split the problem. If it's a hierarchical problem, go down that route first. Next consider if you can get away with only continuous or only discrete. You'll probably lose a little bit of granularity, but it might make it easier to get something up and running.

New algorithms are emerging that learn when to select either a continuous algorithm or a discrete one, depending on the required action space. Parameterized DQN is one such example, but the idea could be applied to any algorithm.³² Another approach called *action branching* simply creates more heads with different activations and training goals on a neural network, which seems like a nice engineering solution to the problem.³³ This multihead architecture also enables simultaneous actions. In a sense this just a handcrafted form of hierarchical RL.

When to perform actions

Here's a curveball: what about when you want to output no action? That's actually a lot harder than it sounds, because everything from the Bellman equation to neural network training is set up to expect an action on every step. The Atari games implement frame skipping without really considering how many frames to skip; 4 are just taken as given. But researchers have found that you can optimize the the size of number of skipped frames to improve performance.³⁴

A more generalized approach is to use the *options* framework, which include a termination condition. These methods learn not only *how* to act optimally, but also when.^{35,36} A potentially simpler solution is instead to add an action that causes the agent to repeat previous actions; I have seen this used in robotics to reduce wear and tear on motors. It is also possible to learn to skip as many actions as possible, to reduce costs associated with an action.³⁷

As usual, the best solution is problem dependent. Generally I would recommend staying within the realms of “normal” RL as much as possible, to simplify development.

Massive action spaces

When state spaces become too large you can use state engineering to bring them back down to a reasonable size. But what about when you have massive action spaces, like if you are recommending one of millions of products to a customer? YouTube has 10^8 items in its corpus, for example.³⁸

Value-based methods like Q-learning struggle because you have to maximize of a set of possible action on every step. The obvious challenge is that when you have continuous actions, the number of actions to maximize over is infinite. But even in discrete settings there can be so many actions that it might take an infeasibly long time to perform that maximization. Policy gradient methods overcome this problem because they sample from a distribution; there is no maximization.

Solutions to this problem include using embeddings to reduce the total number of actions to a space of actions.³⁹ This has instinctive appeal due to the successes of embeddings in other ML challenges, like in natural language processing. If you have action spaces that are similar to problems where embeddings have been successful, then this might be a good approach. Another approach is to treat the maximization as an optimization problem and learn to predict maximal actions.⁴⁰

A related issue is where you need to select a fixed number of items from within the action space. This presents a combinatorial explosion and is a common enough issue to have attracted research time. There are a wide range approximate solutions like hierarchical models and autoencoders.^{41,42} A more general solution is to move the complexity in the action space into the state space by inserting fictitious dummy

states and using group actions predicted over multiple steps to produce a selection of recommendations.⁴³ A simpler and more robust solution is to assume that users can select a single action. Then you only need to rank and select the top- k actions, rather than attempting to decide on what combination of actions is best. This solution has been shown to increase user engagement on YouTube.⁴⁴



Consider this result in the context that Google (which owns YouTube) has access to one of the most advanced pools of data scientists in the world. This algorithm is competitive with its highly tuned recommendations model that (probably) took teams of highly experienced data scientists many years to perfect. Incredible.

Exploration

Children provide personal and professional inspiration in a range of sometimes surprising ways, like when I'm working on a presentation or a new abstraction I routinely try to explain this to my kids. If I can't explain it in a way that sounds at least coherent to them, then that's a signal to me that I need to work harder at improving my message. The book *The Illustrated Children's Guide to Kubernetes* is a cute testament to this idea.

Children also provide direction for improvements in exploration. This is necessary because I have spent most of this book explaining that most problems are an exploration problem: complex states, large action spaces, convergence, optima, and so on. All these problems can be improved with better exploration. Recent research has shown that children have an innate ability, at a very early age, to actively explore environments for the sake of knowledge. They can perform simple experiments and test hypotheses. They can weave together multiple sources of information to provide a rich mental model of the world and project this knowledge into novel situations.⁴⁵

RL algorithms, on the other hand, are primarily driven by error, and learning is almost random. Without a goal, agents would stumble around an environment like an annoying fly that is seemingly incapable of flying back through the open window it came through.

It seems that children tend to operate using an algorithm that approaches *depth-first* search. They like to follow through with their actions until there is a perceived state that blocks the current path, like a dead end in a maze. Incredibly, this works without any goal, which means they are transferring knowledge about the states from previous experience. When the same children are then presented with a goal, they are able to use the mental map produced by their exploration to dramatically improve the initial search for a goal. Contrast this with current RL algorithms that generally rely on finding a goal by accident.

To this end, there is a large body of work that investigates how to improve exploration, especially in problems that have sparse rewards. But first consider whether you can provide expert guidance in the form of improved algorithms (like replacing ϵ -greedy methods with something else) or intermediary rewards (like distance-to-goal measures), or using something like imitation RL to actively guide your agent. Also consider altering your problem definition to make the rewards more frequent. All of these will improve learning speed and/or exploration without resorting to the following methods.

Is intrinsic motivation exploration?

Researchers are trying to solve the problem of exploration by developing human-inspired alternatives to random movement, which leads to a glut of proposals with anthropomorphic names like surprise, curiosity, or empowerment. Despite the cutesy names, exploration methods in humans are far too complex to encode in a simple RL algorithm, so researchers attempt to generate similar behavior through mathematical alterations to the optimization function.

This highlights the problem with all of these methods; there is no right answer. I'm sure you can imagine several mathematical solutions to encourage exploration, like the entropy bonus in the soft actor-critic, but they will never come close to true human-like exploration without prior experience. This is why I think that this field is still young and why words like "curiosity" or "intrinsic motivation" are bad monikers, because they imply that exploration is all about trying new actions. It's not. Exploration *is* RL. You use prior knowledge to explore new states, based upon the anticipation of future rewards.

Imagine a classic RL game where your agent is looking at a wall. If it knows anything about walls, it should never explore the states close to the wall, because there is no benefit from doing so. All RL algorithms neglect the simple physical rule that you can't walk through walls, so they waste time literally banging their head against the wall. But it's also interesting to note that these assumptions can prevent us from finding unexpected states, like those annoying invisible blocks in the Super Mario series.

So if you want to perform optimal exploration then you need prior experience. I think that transferring knowledge between problems is the key to this problem and unsupervised RL has a potential solution. And counterfactual reasoning will become more important as algorithms begin to start using "libraries" of prior knowledge.

Intrinsic motivation and the methods discussed next are not really about exploration at all. They attempt to improve the *cold start* problem. When you have zero knowledge with sparse rewards, how can you encourage deep exploration?

Visitation counts (sampling)

One of the simplest ways to improve the efficiency of exploration is to make sure that the agent doesn't visit a state too many times. Or conversely, encourage the agent to visit infrequently visited states more often. Upper-confidence-bound methods attempt to prevent the agent from visiting states too often (see [“Improving the \$\epsilon\$ -greedy Algorithm” on page 33](#)). Thompson sampling is an optimal sampling technique for this kind of problem and probabilistic random sampling, based upon the value function or policy predictions, can serve as a useful proxy.

Information gain (surprise)

Information gain is used throughout machine learning as a measure of the reduction in entropy. For example, tree-based algorithms use information gain to find splits in feature space that reduce the amount of entropy between classes; a good split makes pure or clean classes, so the information gain is high.⁴⁶

Many researchers have attempted to use information gain to direct the agent toward regions of greater “surprise.” The major benefit over raw sampling techniques is that they can incorporate internal representations of the policy, rather than relying on external observations of the movement of an agent.

One example of this is *variational information maximizing exploration* (VIME), which uses the divergence (see [“Kullback–Leibler Divergence” on page 165](#)) between observed trajectories and those expected by a parameterized model as an addition to the reward function.⁴⁷ Note the similarity between this idea and those discussed in [“Trust Region Methods” on page 163](#). This results in exploration that appears to “sweep” through the state space.

State prediction (curiosity or self-reflection)

Curiosity is a human trait that rewards seeking out and finding information that surprises. The previous approach focused upon encouraging surprising trajectories, but you might not want this, because interesting paths are quite likely to lead to a result that is not very surprising at all. Consider if this is important in your problem. For example, you might be searching for new ways of achieving the same result, in which case this would be very useful. In many problems, however, it is the result that is important, not how you got there.

So rather than encouraging different paths, like different routes to school, another solution is to encourage new situations, like going to a different school. One example of this is to create a model that attempts to predict the next state based upon the current state and action, then rewarding according to how wrong the prediction is.⁴⁸ The problem with this method is that predicting future states is notoriously difficult due to the stochastic and dynamic nature of the environment. If it was that easy, you could use machine learning to solve your problem. So they attempt to model only the

parts of the state that affect the agent. They do this by training a neural network that predicts an action from two consecutive states, which creates an internal representation of the policy, and then rewarding based upon the difference between a model that predicts what the internal representation *should be* and what it actually is after visiting that state. A simpler, similar approach would be to use a delayed autoencoder that compares the hidden representation against that which is observed.

Curious challenges

In the real world, novel states are encountered all the time. Like the previous method suggests, we build long-term models to predict what is novel. That implies that we have also learned how to ignore the noise. Think of when you are driving a car. You have somehow learned how to automatically ignore 99% of the irrelevant information around you and concentrate on the hazards. But put a TV screen or a mobile phone in view and your eyes will be drawn toward it. This promise of seeing something new, at the expense of spotting hazards, is what makes driving with phones or tablets so dangerous.

Researchers have found that a similar problem exists in RL. **Figure 9-7** shows a hilarious scenario where a TV was placed upon a wall.⁴⁹ The state-curiosity-driven agent is perpetually rewarded because it cannot predict what is going to be shown next on TV. Obviously this result is a little contrived; the whole point of these models is to filter out such noise, which should be fairly easy to do if you understand the domain. But in general, researchers need to focus more on improving global exploration, not local.



Figure 9-7. Using observations as a source of surprise can encourage agents to get stuck in noisy states. Adapted from a video by © Deepak Pathak, used under CC BY 2.0 license.

Random embeddings (random distillation networks)

One idea to overcome the “noisy-TV” problem, where agents become fixated with stochastic states, is to use random embeddings. *Random distillation networks* are a pair of neural networks, one of which is randomly initialized and never touched again. The random network is used to generate random embeddings. A second

network is trained to predict the random embeddings. In the work of Burda et al., the error between the random and predicted embedding is used as a “novelty” measure; a greater error means that this type of observation is not well represented in the buffer and therefore should be visited again in the future. This error value is appended to the reward to promote visiting new states.⁵⁰

The major benefit of this approach is that it is not predicting the next observation; it is just comparing the next observation to its current model of the environment. This means that the stationary view of a stochastic states, like a single image on the TV, can be modeled quite well and therefore the agent is not rewarded for being a “couch potato.”

I find the idea of using a randomly initialized neural network as an embedding hard to fathom. But imagine if your vision was afflicted by some random transformation. Given time your brain would learn to compensate for this and would develop new abstractions; survivors of a stroke have to go through this ordeal. Another benefit is that this architecture is simpler, in the sense that there is only a single network to train. But the exploration is still purely local; it does not promote exploration in depth. Another problem is that it is crucial to have an architecture that produces an effective embedding in the domain. Convolutional neural networks clearly work well in image-based domains, but what about others? These cases need your domain and modeling expertise.⁵¹

Distance to novelty (episodic curiosity)

One interesting avenue of research to promote global exploration is to use distance between states as a better measure of novelty. Savinov et al. propose using an experience buffer to record novel states, which are encoded by an embedding. The buffer is cleared at the start of every episode. When an agent happens upon a new state, the distance between the states in the buffer and the current state is estimated. If the new state is within a few environment steps of those in the buffer, it is not novel and is punished. If the new state is further than a few steps away then the agent is rewarded for finding novel states. This encourages agents to seek out new states on an episodic basis.⁵²

Figure 9-8 is a representation of the trajectory followed during one episode. It shows the sign of the reward and when embedded observations were placed in the buffer. You can see that the algorithm has the effect of forcing the agent to keep moving, which promotes depth-first exploration and prevents the agent from becoming a “couch-potato.”

The implementation is quite complex. First, images are converted to hidden embeddings by a convolutional neural network using one-half of a *siamese* network, which is a neural network architecture used in image matching tasks.⁵³ I don’t like this name but that’s what the literature calls it. Next, the current embedding is compared to all

of the embeddings in the buffer, creating a measure of distance to all other observations. Finally, an aggregation is performed to find the distance to next closest state, which represents the novelty of this observation, and is used to calculate the reward.

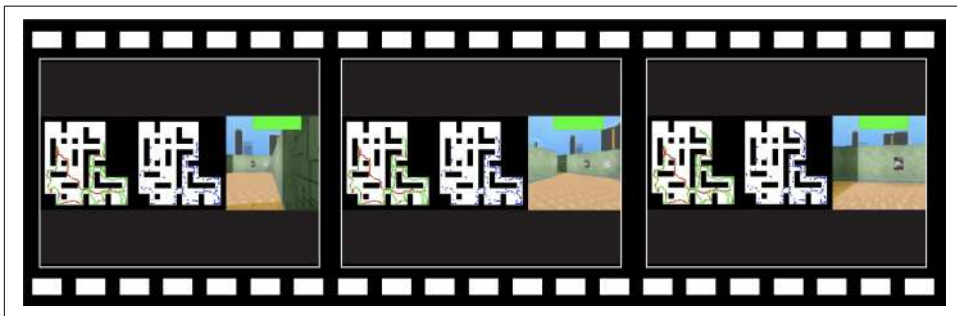


Figure 9-8. Trajectories from an episode of the DMLab environment with episodic curiosity. The lefthand box shows how the agent was rewarded for that state (green means positive reward, red means negative). The middle box represents when “memories” were added to the episodic buffer. The right box is the first-person view of the agent. Video adapted from Savinov et al.

Exploration conclusions

We know that people explore in depth, using prior knowledge. Transferring that idea to RL is incredibly difficult, because there is no obvious way to direct exploration without breaking the assumptions of an MDP. It feels like there is a gap, one that an MDP cannot fill, where part of the problem is purely supervised, like revising for an exam, but another is entirely opportunistic, like spending time in a different culture.

This is an important problem to solve, because sparse states occur often in RL, due to the nature of goal-driven engineering. Sparse rewards are difficult to work with because they are hard to find. Creating an alternative, pseudoreward that encourages an arbitrary exploration strategy is one solution, but in many problems it is relatively straightforward to manually design rewards to suit your problem (see “[Reward Engineering](#)” on page 283), or alter the problem definition to make it easier.

Current attempts to automate exploration take a local, episodic view and do not consider global or long-term exploration goals. In fact, researchers have observed that promoting novelty actively encourages dangerous states, like walking along the edge of the cliff, because they are difficult to get close to and are rarely represented in the agent’s past experience.⁵⁴

All efforts encourage exploration by augmenting the reward function, rather than by guiding action selection. There are clear parallels between this, curriculum learning, and inverse RL that could improve this situation. To be sure, the future of exploration research will be directed toward either altering the fundamental definition of the

MDP, by allowing some high-level coordination (thus breaking the Markovian assumption), or incorporating the work of other fields in RL to encourage long-time horizon planning.

Reward Engineering

Designing a reward to solve your problem is arguably the most important task in RL. This is where, as an engineer, it is your job to convert the problem definition into a metric that guides the agent toward a solution. This is incredibly important. Your definition dictates the solution and it is easy to accidentally cause the agent to solve the wrong problem. **Figure 9-9** is a great visual example of this happening. An agent in the Unity framework is encouraged to reach a goal and it can use a box to jump over a wall. Instead, because of the slight edge around the environment, the agent has learned to sneak around the side. Despite having a reasonable reward, the agent has learned to exploit the environment in ways that you cannot imagine.

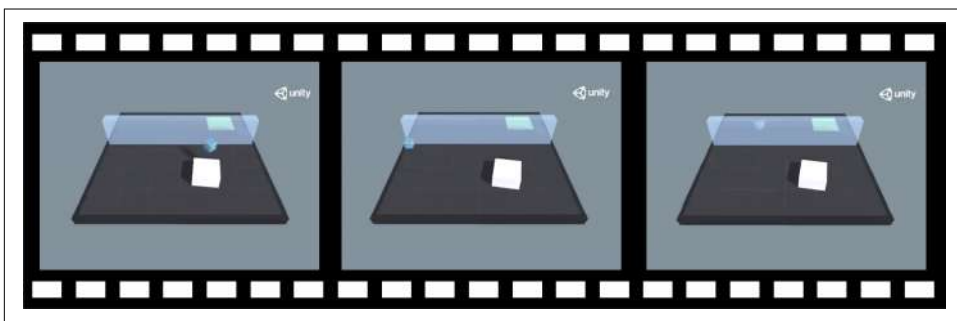


Figure 9-9. In this example from Unity, the goal was to train an agent to use a box to jump over a wall by encouraging the agent to get closer to the goal. Instead, it learned to cheat and sneak around the edge. Adapted from a presentation by Danny Lange, © Unity Technologies, with permission.

In machine learning, you directly define the algorithm to solve a problem. So as long as you have a reasonable grasp of the problem, which is a problem in itself, it is easy to observe when projects go awry. But in RL, you have to wait until training is complete to see how an agent solves a problem. This dramatically increases the length of the feedback loop and demonstrates why understanding the resultant policy is so important.

To make matters worse, the reward also defines the ultimate performance of an algorithm and how fast it gets there. Rewards should help guide the agent, and the more guidance you provide the faster it should learn. However, many problems are sparse, in the sense that there is only a reward when a task is complete. Researchers have proposed many ways in which to mitigate the sparsity, but the simplest thing you can do is design a reward that is a proxy of the task you want to complete. But remember,

there is no optimal policy without a reward and your reward changes the optimal policy.

Reward engineering guidelines

Each domain has its own specific problems, which makes it difficult to create hard and fast rules about how to specify a reward. But in this section I want to outline a few high-level objectives that apply to any situation. A reward should do the following:

Be fast

Reward functions should be fast to compute, because they are called millions of times in complex environments.

Correlate with the “true” reward

Proxy rewards should correlate with true rewards, otherwise you run the risk of solving the wrong problem. This means they should equate or approximate the terminology used in the problem domain; for example, monetary value, numbers of customers, lives saved.

De-noise

Proxy rewards can be noisy, which means that they don't perfectly correlate to the true reward. Consider combining different proxy rewards to reduce reward variance.

Encode nonfunctional requirements

Consider adding reward proxies that map to nonfunctional requirements like safety, efficiency, or speed.

Consider time

The discounting factor forces the agent to become more or less *myopic*; it controls how much emphasis is placed on optimizing short- or long-term rewards. Consider this when shaping rewards.

Prevent plateaus

Your reward should provide strong discrimination between states and the difference should be greater than the noise of the Q-values. If states have very similar rewards, agents tend to go around in circles.

Smooth

To produce “smooth” behavior, policies need to learn from smooth rewards. Discontinuities in the reward space can manifest as “sharp,” “jolting” actions in the policy.

Be simple

Like everything else in RL, complexity makes it much harder to understand. If you have a simple reward, the policy is easy to understand. If it is complex, you will see unexpected policies.

Reward shaping

This may be an obvious statement to make, but it is still worth saying. Policies are affected by the magnitude of the reward, since agents set out to maximize the total reward. Since you are designing the reward, you also have to specify the size of the reward when the goal is achieved. When there are subgoals, multiple goals, or situations that must be avoided, balancing the rewards of each type can be excruciating. In circumstances such as these you might want to attempt to split the problem or look at hierarchical RL.

In general, you can arbitrarily alter the *shape* of the reward to encourage certain behavior or improve learning speed, for example. When designing a reward you should:

Normalize rewards

Be very careful with the scale of the reward. In general, all algorithms benefit from normalized rewards because they reduce the variance of the gradients in the policy or value function optimization.

Clip rewards

Clipping rewards is a common form of normalization but the truncation can throw away information. For example, DQN recommends clipping rewards to $+/-1$, but in the game *Pac-Man* this treats both dots and fruit as having the same reward, which is not true; fruit produces a much higher score (this proxy reward does not correlate to the true reward). If you instead normalize the rewards in *Pac-Man*, the DQN agent becomes a hunter-gatherer and learns to actively hunt ghosts and search for fruit, not just collect dots.⁵⁵

Transform rewards

Altering the gradient of the reward can help smooth policies and provide greater encouragement when the agent is far away from the goal. For example, a linear reward provides the same amount of urgency irrespective of where the agent currently resides in the state space. You could pass your reward through a function where rewards change rapidly when the agent is far away from the goal. This is a slippery slope, because transformations in the reward can have unexpected manifestations in the policy.

Model rewards

Many domains have environments where some modeling is possible, at least locally, so it might be possible to derive a proxy reward that effectively encodes

domain knowledge. For example, in a robotics problem you might be able to include some prior knowledge, like the fact that a robot has to stand up before it can run; you could include a sensor that measures how far off the ground the robot is. In general, including ground truth such as this improves performance.

Common rewards

Check to see if there are any published problems that are similar to yours. Common domains tend to stick to tried and tested reward definitions. Following is a list of common types of reward:

Sparse

Many tasks are only considered complete when a particular state is reached, sometimes multiple states in a specific order. It is common to see a big reward when this task is complete, but they are hard to solve because rewards don't guide the agent toward the goal.

Distance to goal

Distance-to-goal metrics are very strong reward proxies because they not only correlate strongly to the true reward, but they also guide the agent toward the goal, no matter where the agent is in the state space. This improves exploration and speeds up learning time.

Punishing steps

Punishing an agent for every action it takes encourages it to take as few steps as possible.

Punishing damaging or dangerous behaviors

A strong negative reward for destructive behaviors is often used, but be careful that this doesn't adversely impact optimal reward paths, like veering a mile away from a cliff when 10 meters would be fine.

Image-based rewards

Rewards are usually based upon a low-dimensional state. But increasingly researchers are using high-dimensional observations like images to define the goal state. For example, Zhu et al. use a set of images as a definition of a goal state, and use both random network distillation and a latent model produced by an autoencoder. This is incredibly powerful in many real-world applications, because you only need to collect a batch of images representing your goal state and then leave RL to do the rest.⁵⁶

Reward conclusions

Specifying a reward that solves the problem definition, promotes swift learning, and results in optimal policies is a tremendously difficult thing to achieve. I would suggest that in many applications this list of issues is also in priority order, to the point where

so long as the problem definition is correct, engineers tend not to worry about optimality or speedy learning, because you can use other techniques to address these problems. But if you can find a balance, then the result will be simpler, more efficient algorithms.

Again, you might have noticed that some of the suggestions are linked to other topics. Indeed, automated reward generation is the whole goal of intrinsic motivation, unsupervised RL, and even meta-learning. You must take a holistic view of these techniques and choose to use them at the right times, to solve the right problems. Otherwise you might be overcomplicating the problem, which often leads to wasted time or even complete failure.

There is more research that attempts to combine some of these ideas into a unified algorithm. For example, one interesting idea suggests that reward shaping can lead to local optima that can be hard to get out of. Trott et al. use two independent counterfactual rollouts to check if there is another optima greater than the current one.⁵⁷

Summary

Despite RL being decades old, it has only recently gained prominence as an industrially viable and practical tool. I believe the reason for this delay is not a technical one. I think the main problem is that abstracting the technicalities in a way that makes it understandable is difficult. It's hard enough to define what a “strategy” is, or how it can help a business, without sounding all “robots will rule the world.” The idea is powerful, but as an engineer, it is your responsibility to translate the potential (and the caveats) in a way that resonates with your stakeholders.

In this chapter I focused less on the gory details and more on the process. But remember that this is a new field, with new opportunities. I have presented what I think is reasonable, but you might have better ideas. Don't be afraid to customize your approach to suit your problem. This also means that these best practices will continue over time. When you do use RL in an industrial project, I would love to hear about your experiences!

Hopefully you now have enough knowledge to at least start an RL project. If you don't, get in touch (see [About the Author](#) for contact details) and I'll do my best to help out. Now you know what the process looks like and what to look for in an RL project. You also know about the amount engineering effort that goes into making a project viable and then optimal. There's also a wealth of research freely available on the internet that may map directly to your problem and I also include some important materials in [“Further Reading” on page 288](#) that I found useful during my research.

Further Reading

- Offline reinforcement learning:
 - Slightly old, but still relevant chapter.⁵⁸
 - Recent review of challenges.⁵⁹
- Reset-free learning:
 - A nice review of lifelong learning focusing on robotics.⁶⁰
- State engineering:
 - A good review on state representation.⁶¹
- Policy engineering:
 - There is a concise but robust section on linear approximation and transformation in the book by Sutton and Barto. Also good for details on continuous action parameterization.⁶²
 - An old but relevant review of discretization techniques in ML.⁶³
 - I found this [video about Gaussian policies](#) very helpful.
- Exploration:
 - Interesting comparison of psychological intrinsic motivation against computational approaches.⁶⁴
 - Comprehensive tutorial on Thompson sampling.⁶⁵

References

- [1] Zheng, Stephan, Alexander Trott, Sunil Srinivasa, Nikhil Naik, Melvin Gruesbeck, David C. Parkes, and Richard Socher. 2020. “[The AI Economist: Improving Equality and Productivity with AI-Driven Tax Policies](#)”. ArXiv:2004.13332, April.
- [2] Håkansson, Samuel, Viktor Lindblom, Omer Gottesman, and Fredrik D. Johansson. 2020. “[Learning to Search Efficiently for Causally Near-Optimal Treatments](#)”. ArXiv:2007.00973, July.
- [3] Lange, Sascha, Thomas Gabel, and Martin Riedmiller. 2012. “[Batch Reinforcement Learning](#)”. *Reinforcement Learning: State-of-the-Art*, edited by Marco Wiering and Martijn van Otterlo, 45–73. Adaptation, Learning, and Optimization. Berlin, Heidelberg: Springer.
- [4] Levine, Sergey, Aviral Kumar, George Tucker, and Justin Fu. 2020. “[Offline Reinforcement Learning: Tutorial, Review, and Perspectives on Open Problems](#)”. ArXiv:2005.01643, May.

- [5] Chen, Xinyue, Zijian Zhou, Zheng Wang, Che Wang, Yanqiu Wu, and Keith Ross. 2020. “**BAIL: Best-Action Imitation Learning for Batch Deep Reinforcement Learning**”. ArXiv:1910.12179, February.
- [6] Dargazany, Aras. 2020. “**Model-Based Actor-Critic: GAN + DRL (Actor-Critic) $\Rightarrow$ AGI**”. ArXiv:2004.04574, April.
- [7] Mandlekar, Ajay, Fabio Ramos, Byron Boots, Silvio Savarese, Li Fei-Fei, Animesh Garg, and Dieter Fox. 2020. “**IRIS: Implicit Reinforcement without Interaction at Scale for Learning Control from Offline Robot Manipulation Data**”. ArXiv:1911.05321, February.
- [8] Gulcehre, Caglar, Ziyu Wang, Alexander Novikov, Tom Le Paine, Sergio Gomez Colmenarejo, Konrad Zolna, Rishabh Agarwal, et al. 2020. “**RL Unplugged: Benchmarks for Offline Reinforcement Learning**”. ArXiv:2006.13888, July.
- [9] Banerjee, Bikramjit, Sandip Sen, and Jing Peng. 2004. “**On-Policy Concurrent Reinforcement Learning**”. *Journal of Experimental & Theoretical Artificial Intelligence* 16(4): 245–60.
- [10] Marthi, Bhaskara, Stuart Russell, David Latham, and Carlos Guestrin. 2005. “Concurrent Hierarchical Reinforcement Learning.” *Proceedings of the 19th International Joint Conference on Artificial Intelligence*, 779–785. IJCAI ’05. Edinburgh, Scotland: Morgan Kaufmann Publishers Inc.
- [11] Silver, David, Leonard Newnham, David Barker, Suzanne Weller, and Jason McFall. 2013. “**Concurrent Reinforcement Learning from Customer Interactions**”. *International Conference on Machine Learning*, 924–32.
- [12] Dimakopoulou, Maria, and Benjamin Van Roy. 2018. “**Coordinated Exploration in Concurrent Reinforcement Learning**”. ArXiv:1802.01282, February.
- [13] Han, Weiqiao, Sergey Levine, and Pieter Abbeel. 2015. “**Learning Compound Multi-Step Controllers under Unknown Dynamics**”. *2015 IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*, 6435–6442.
- [14] Richter, Charles, and Nicholas Roy. 2017. “**Safe Visual Navigation via Deep Learning and Novelty Detection**”. MIT Web Domain, July.
- [15] Eysenbach, Benjamin, Shixiang Gu, Julian Ibarz, and Sergey Levine. 2017. “**Leave No Trace: Learning to Reset for Safe and Autonomous Reinforcement Learning**”. ArXiv:1711.06782, November.
- [16] Zhu, Henry, Justin Yu, Abhishek Gupta, Dhruv Shah, Kristian Hartikainen, Avi Singh, Vikash Kumar, and Sergey Levine. 2020. “**The Ingredients of Real-World Robotic Reinforcement Learning**”. ArXiv:2004.12570, April.

- [17] Brockman, Greg, Vicki Cheung, Ludwig Pettersson, Jonas Schneider, John Schulman, Jie Tang, and Wojciech Zaremba. 2016. “OpenAI Gym”. ArXiv:1606.01540, June.
- [18] Ruiz, Nataniel, Samuel Schuster, and Manmohan Chandraker. 2019. “Learning to Simulate”. ArXiv:1810.02513, May.
- [19] Kalashnikov, Dmitry, Alex Irpan, Peter Pastor, Julian Ibarz, Alexander Herzog, Eric Jang, Deirdre Quillen, et al. 2018. “QT-Opt: Scalable Deep Reinforcement Learning for Vision-Based Robotic Manipulation”. ArXiv:1806.10293, November.
- [20] Karakovskiy, Sergey, and Julian Togelius. 2012. “The Mario AI Benchmark and Competitions”. *IEEE Transactions on Computational Intelligence and AI in Games* 4(1): 55–67.
- [21] Ota, Kei, Tomoaki Oiki, Devesh K. Jha, Toshisada Mariyama, and Daniel Nikovski. 2020. “Can Increasing Input Dimensionality Improve Deep Reinforcement Learning?”. ArXiv:2003.01629, June.
- [22] Hafner, Danijar, Timothy Lillicrap, Jimmy Ba, and Mohammad Norouzi. 2020. “Dream to Control: Learning Behaviors by Latent Imagination”. ArXiv:1912.01603, March.
- [23] Hafner, Danijar, Timothy Lillicrap, Ian Fischer, Ruben Villegas, David Ha, Honglak Lee, and James Davidson. 2019. “Learning Latent Dynamics for Planning from Pixels”. ArXiv:1811.04551, June.
- [24] Ha, David, and Jürgen Schmidhuber. 2018. “World Models”. ArXiv:1803.10122, March.
- [25] Chaudhari, Harshal A., John W. Byers, and Evimaria Terzi. 2020. “Learn to Earn: Enabling Coordination within a Ride Hailing Fleet”. ArXiv:2006.10904, July.
- [26] “Value Function Approximation in Reinforcement Learning Using the Fourier Basis”. *Proceedings of the Twenty-Fifth AAAI Conference on Artificial Intelligence*. n.d. Accessed 18 July 2020.
- [27] Tang, Yunhao, and Shipra Agrawal. 2020. “Discretizing Continuous Action Space for On-Policy Optimization”. ArXiv:1901.10500, March.
- [28] Gonzalez-Abril, L., F. J. Cuberos, F. Velasco, and J. A. Ortega. 2009. “Ameva: An Autonomous Discretization Algorithm”. *Expert Systems with Applications: An International Journal* 36 (3): 5327–5332.
- [29] Dougherty, James, Ron Kohavi, and Mehran Sahami. 1995. “Supervised and Unsupervised Discretization of Continuous Features.” *Proceedings of the Twelfth International Conference on International Conference on Machine Learning*, 194–202. ICML’95. Tahoe City, California, USA: Morgan Kaufmann Publishers Inc.

- [30] Seijen, Harm van, Bram Bakker, and L. J. H. M. Kester. 2017. “Switching between Different State Representations in Reinforcement Learning”. November.
- [31] Ryu, Moonkyung, Yinlam Chow, Ross Anderson, Christian Tjandraatmadja, and Craig Boutilier. 2020. “CAQL: Continuous Action Q-Learning”. ArXiv:1909.12397, February.
- [32] Xiong, Jiechao, Qing Wang, Zhuoran Yang, Peng Sun, Lei Han, Yang Zheng, Haobo Fu, Tong Zhang, Ji Liu, and Han Liu. 2018. “Parametrized Deep Q-Networks Learning: Reinforcement Learning with Discrete-Continuous Hybrid Action Space”. ArXiv:1810.06394, October.
- [33] Tavakoli, Arash, Fabio Pardo, and Petar Kormushev. 2019. “Action Branching Architectures for Deep Reinforcement Learning”. ArXiv:1711.08946, January.
- [34] Khan, Adil, Jiang Feng, Shaohui Liu, Muhammad Zubair Asghar, and Ling-Ling Li. 2019. “Optimal Skipping Rates: Training Agents with Fine-Grained Control Using Deep Reinforcement Learning”. *Journal of Robotics 2019* (January).
- [35] Sutton, Richard S., Doina Precup, and Satinder Singh. 1999. “Between MDPs and Semi-MDPs: A Framework for Temporal Abstraction in Reinforcement Learning”. *Artificial Intelligence* 112(1): 181–211.
- [36] Khetarpal, Khimya, Martin Klissarov, Maxime Chevalier-Boisvert, Pierre-Luc Bacon, and Doina Precup. 2020. “Options of Interest: Temporal Abstraction with Interest Functions”. ArXiv:2001.00271, January.
- [37] Biedenkapp, A., R. Rajan, F. Hutter, and M. Lindauer. 2020. “Towards TempoRL: Learning When to Act.” In *Workshop on Inductive Biases, Invariances and Generalization in RL* (BIG@ICML ’20).
- [38] Ie, Eugene, Vihan Jain, Jing Wang, Sanmit Narvekar, Ritesh Agarwal, Rui Wu, Heng-Tze Cheng, et al. 2019. “Reinforcement Learning for Slate-Based Recommender Systems: A Tractable Decomposition and Practical Methodology”. ArXiv:1905.12767, May.
- [39] Dulac-Arnold, Gabriel, Richard Evans, Hado van Hasselt, Peter Sunehag, Timothy Lillicrap, Jonathan Hunt, Timothy Mann, Theophane Weber, Thomas Degris, and Ben Coppin. 2016. “Deep Reinforcement Learning in Large Discrete Action Spaces”. ArXiv:1512.07679, April.
- [40] Van de Wiele, Tom, David Warde-Farley, Andriy Mnih, and Volodymyr Mnih. 2020. “Q-Learning in Enormous Action Spaces via Amortized Approximate Maximization”. ArXiv:2001.08116, January.
- [41] Mehrotra, Rishabh, Mounia Lalmas, Doug Kenney, Thomas Lim-Meng, and Golli Hashemian. 2019. “Jointly Leveraging Intent and Interaction Signals to Predict User Satisfaction with Slate Recommendations”. *The World Wide Web Conference*,

1256–1267. WWW 2019. San Francisco, CA, USA: Association for Computing Machinery.

[42] Jiang, Ray, Sven Gowal, Timothy A. Mann, and Danilo J. Rezende. 2019. “Beyond Greedy Ranking: Slate Optimization via List-CVAE”. ArXiv:1803.01682, February.

[43] Metz, Luke, Julian Ibarz, Navdeep Jaitly, and James Davidson. 2019. “Discrete Sequential Prediction of Continuous Actions for Deep RL”. ArXiv:1705.05035, June.

[44] Ie, Eugene, Vihan Jain, Jing Wang, Sanmit Narvekar, Ritesh Agarwal, Rui Wu, Heng-Tze Cheng, et al. 2019. “Reinforcement Learning for Slate-Based Recommender Systems: A Tractable Decomposition and Practical Methodology”. ArXiv:1905.12767, May.

[45] Kosoy, Eliza, Jasmine Collins, David M. Chan, Sandy Huang, Deepak Pathak, Pulkit Agrawal, John Canny, Alison Gopnik, and Jessica B. Hamrick. 2020. “Exploring Exploration: Comparing Children with RL Agents in Unified Environments”. ArXiv:2005.02880, July.

[46] Murphy, Kevin P. 2012. *Machine Learning: A Probabilistic Perspective*. The MIT Press.

[47] Houthoofd, Rein, Xi Chen, Yan Duan, John Schulman, Filip De Turck, and Pieter Abbeel. 2017. “VIME: Variational Information Maximizing Exploration”. ArXiv:1605.09674, January.

[48] Pathak, Deepak, Pulkit Agrawal, Alexei A. Efros, and Trevor Darrell. 2017. “Curiosity-Driven Exploration by Self-Supervised Prediction”. ArXiv:1705.05363, May.

[49] Burday, Yuri, Harry Edwards, Deepak Pathak, Amos Storkey, Trevor Darrell, Alexei A. Efros. “Large-Scale Study of Curiosity-Driven Learning”. n.d. Accessed 22 July 2020.

[50] Burda, Yuri, Harrison Edwards, Amos Storkey, and Oleg Klimov. 2018. “Exploration by Random Network Distillation”. ArXiv:1810.12894, October.

[51] Saxe, Andrew M., Pang Wei Koh, Zhenghao Chen, Maneesh Bhand, Bipin Suresh, and Andrew Y. Ng. 2011. “On Random Weights and Unsupervised Feature Learning.” *Proceedings of the 28th International Conference on International Conference on Machine Learning*, 1089–1096. ICML '11. Bellevue, Washington, USA: Omnipress.

[52] Savinov, Nikolay, Anton Raichuk, Raphaël Marinier, Damien Vincent, Marc Pollefeys, Timothy Lillicrap, and Sylvain Gelly. 2019. “Episodic Curiosity through Reachability”. ArXiv:1810.02274, August.

[53] Zagoruyko, Sergey, and Nikos Komodakis. 2015. “Learning to Compare Image Patches via Convolutional Neural Networks”. ArXiv:1504.03641, April.

- [54] Burda, Yuri, Harrison Edwards, Amos Storkey, and Oleg Klimov. 2018. “Exploration by Random Network Distillation”. ArXiv:1810.12894, October.
- [55] Hasselt, Hado van, Arthur Guez, Matteo Hessel, Volodymyr Mnih, and David Silver. 2016. “Learning Values across Many Orders of Magnitude”. ArXiv:1602.07714, August.
- [56] Zhu, Henry, Justin Yu, Abhishek Gupta, Dhruv Shah, Kristian Hartikainen, Avi Singh, Vikash Kumar, and Sergey Levine. 2020. “The Ingredients of Real-World Robotic Reinforcement Learning”. ArXiv:2004.12570, April.
- [57] Trott, Alexander, Stephan Zheng, Caiming Xiong, and Richard Socher. 2019. “Keeping Your Distance: Solving Sparse Reward Tasks Using Self-Balancing Shaped Rewards”. ArXiv:1911.01417, November.
- [58] Lange, Sascha, Thomas Gabel, and Martin Riedmiller. 2012. “Batch Reinforcement Learning”. *Reinforcement Learning: State-of-the-Art*, edited by Marco Wiering and Martijn van Otterlo, 45–73. Adaptation, Learning, and Optimization. Berlin, Heidelberg: Springer.
- [59] Levine, Sergey, Aviral Kumar, George Tucker, and Justin Fu. 2020. “Offline Reinforcement Learning: Tutorial, Review, and Perspectives on Open Problems”. ArXiv:2005.01643, May.
- [60] Wong, Jay M. 2016. “Towards Lifelong Self-Supervision: A Deep Learning Direction for Robotics”. ArXiv:1611.00201, November.
- [61] Lesort, Timothée, Natalia Díaz-Rodríguez, Jean-François Goudou, and David Filliat. 2018. “State Representation Learning for Control: An Overview”. *Neural Networks* 108 (December): 379–92.
- [62] Sutton, Richard S., and Andrew G. Barto. 2018. *Reinforcement Learning: An Introduction*. MIT Press.
- [63] Dougherty, James, Ron Kohavi, and Mehran Sahami. 1995. “Supervised and Unsupervised Discretization of Continuous Features.” *Proceedings of the Twelfth International Conference on International Conference on Machine Learning*, 194–202. ICML’95. Tahoe City, California, USA: Morgan Kaufmann Publishers Inc.
- [64] Oudeyer, Pierre-Yves, and Frederic Kaplan. 2007. “What Is Intrinsic Motivation? A Typology of Computational Approaches”. *Frontiers in Neurorobotics*. 1 November.
- [65] Russo, Daniel, Benjamin Van Roy, Abbas Kazerouni, Ian Osband, and Zheng Wen. 2020. “A Tutorial on Thompson Sampling”. ArXiv:1707.02038, July.

Operational Reinforcement Learning

The closer you get to deployment, the closer you are to the edge. They call it the cutting edge for good reason. It's hard enough getting your reinforcement learning (RL) project to this point, but production implementations and operational deployment add a whole new set of challenges.

To the best of my knowledge, this is the first time a book has attempted to collate operational RL knowledge in one place. You can find this information scattered throughout the amazing work of the researchers and books from many of the industry's brightest minds, but never in one place.

In this chapter I will walk you through the process of taking your proof of concept into production, by detailing the implementation and deployment phases of an RL project. By the end I hope that these ideas will resonate and you will have a broad enough knowledge to at least get started, and understand what you need to dig in further. With this chapter, and of course the whole book, my goal is to bring RL to industry and demonstrate that production-grade industrial RL is not only possible, but lucrative.

In the first half I review the implementation phase of an RL project, looking deeper into the available frameworks, the abstractions, and how to scale. How to evaluate agents is also prominent here, because it is not acceptable to rely on ad hoc or statistically unsound measures of performance.

The second half is about operational RL and deploying your agent into production. I talk about robust architectures and the necessary tooling you will need to build robust agents. And finally, but possibly most importantly, there is a section about making RL safe, secure, and ethical; make sure you read this.

Implementation

Personally, I find the implementation phase of a machine learning/artificial intelligence/big data/{insert buzzword here} project one of the most fascinating and interesting parts of the whole life cycle. This is the phase where you need to take care in the code that you write because this code might survive for decades, maybe more. GitHub has just created an archive in the Arctic that has stored all active projects on microfilm and is predicted to last at least 1,000 years.¹ Some people are privileged enough to talk about a legacy, but for the vast majority of engineers around the world this is about as good as it gets. If you have ever committed to a reasonably popular GitHub repo, congratulations, people in the year 3000 might read your code.

As humbling as this sounds, the primary reason for quality code is to reduce the maintenance burden. Every line of code adds another source of potential error. You can mitigate the probability of error via simplicity, testing, and more eyes on the code. Another reason for quality code is that it makes it easier to use. Projects can increase in scope quite rapidly, so following fundamental software quality guidelines can help make that transition easier.

The previous topics are not specific to RL; there are a host of excellent software engineering books that I recommend every engineer reads, some of which I suggest in “Further Reading” on page 332. This section concentrates on the aspects that are specific to RL.

Frameworks

One of the fastest ways to develop quality RL applications is to use a framework. Engineers have invested significant amounts of time to make it easier for you to develop your project.

RL frameworks

Open source RL frameworks benefit from the number of people using them. Greater popularity means more testing and more impetus to help refine the project. The ability to inspect the public source code is immensely valuable, especially when you come up against a tricky bug.

The problem with popularity is that it tends to be proportional to the size of the marketing budget and the perceived expertise. For example, any repository that has some relationship with Google (look for the staple “not an official Google product,” which generally means it was developed during work time, but is not officially supported) tends to be immensely popular, even when there are few actual users. A similar thing can be suggested for OpenAI, too.

Consider whether the aims of an open source framework align with your goals. For example, many of the frameworks out there are strongly biased toward research, since they were developed by individuals or organizations that are paid to perform research. They are not designed to be used in products or operationally. **OpenAI's baselines project** is a classic example of this, which remains one of the most popular RL codebases available, but the open source project **Stable Baselines**, which is far less visible from a marketing perspective, is much easier to use and more robust, in my experience.^{2,3}

So what should you look for? This depends on what you are trying to do. If you are looking to rapidly test different algorithms on your problem then you would have different requirements over someone who needs to run a framework in production. You should prioritize and weight factors according to your circumstances and needs, but the following list presents some of the things that I look for:

Abstractions

Does the framework have good abstractions that make it easier to swap out RL components? Like policy models, exploration techniques, replay buffer types, and so on. How easily can you develop new ones that suit your problem? This tends to be the most difficult to evaluate, because you really don't know until you try it.

Simplicity

Is the codebase simple enough to navigate around? Does it make sense and is it intuitive? Or is it so large or have so many abstractions that you need weeks just to figure out how to install it?

Observability

Does it include logging and monitoring? If not, is it easy to add when you need it?

Scalability

Can it scale when you need it to?

Minimal dependencies

How dependent is it on other software? Is it locked into a particular framework, like a specific deep learning library? If so, do you approve of the underlying libraries?

Documentation and examples

Is the project well documented? Not just API documentation: are there examples? Are they up-to-date and tested? Is there architecture documentation? Use case documentation? Algorithm documentation?

Included algorithms

Does it come with algorithm implementations baked in? Are these well covered by tests and examples? How easy is it to switch out algorithms?

Ancillary algorithms

Does it come with code for a wide variety of other situations, like inverse RL, hierarchical learning, different replay buffer implementations, or exploration algorithms, and so on?

Support

Is there support? Is there a community? Is it well represented on crowd-sourced self-help websites? Is the support pleasant or aggressive?

Commercial offerings

Is it backed by a company (not necessarily a good thing, but at least you should be able to talk to someone)? Is there an opportunity to receive commercial support?

There are a variety of RL frameworks available, all placing emphasis on different elements in this list. Software evolves quickly and any comments about specific frameworks would be quickly out of date. Instead, please visit the [accompanying website](#) to find an up-to-date review of many RL frameworks.

Other frameworks

Alone, an RL framework is not enough. You will need a variety of other frameworks to build a solution. Many parts of an RL algorithm have been abstracted away, like the various deep learning frameworks used in policy models and value approximations. [PyTorch](#) and [TensorFlow](#) tend to be the most common, of which I tend to prefer PyTorch because of its simpler and intuitive API, but the [Keras](#) API can help overcome that. Most deep learning frameworks are mature and have similar feature sets, but their APIs are quite different. It is a significant source of lock-in, so choose carefully.

There are also an increasing number of libraries that abstract other common components of RL. For example, experience replay is important for many algorithms. [Reverb](#) is a queuing library that was specifically designed to handle implementations like prioritized experience replay.⁴ [Apache Bookkeeper](#) (a distributed ledger) is an example of a production-grade tool that could be leveraged as part of an operational architecture as a log.

I do want to stress that you need to think carefully about an operational architecture and evaluate what tools are available to help solve that problem (see “[Architecture](#)” on [page 319](#)). You can find more up-to-date information about tooling on [the website](#) that accompanies this book.

Scaling RL

Complex domains are harder to solve so they need more complex models and sophisticated exploration. This translates to increased computational complexity, to the point where it becomes infeasible to train an agent on a single machine.

During development, the feedback cycle time is the most important metric to monitor, because all the time you are waiting for a result is time not spent improving the agent. You can run experiments in parallel but this has a limited impact at this point (see “[Experiment tracking](#)” on page 323). In the early phases of RL research the largest improvements come from rapid, repeated improvement cycles. So I recommend that you initially constrain the scope of your work by oversimplifying the problem, using simulations, and massively reducing the complexity of your models. When you have working code, you can scale up.

But how do you scale up? Unlike other machine learning techniques, providing more horsepower doesn’t necessarily improve training speed or performance. The following is a list of reasons why you need dedicated algorithms to scale RL:

Sequential

An agent’s actions have to be ordered, which makes it difficult to scale an individual agent across machines.

Combining experience

If you use multiple agents, then they need synchronizing, otherwise you’d end up with many independent copies that aren’t capable of sharing experience. *When* you synchronize can have a big impact on training speed and aggregate performance. Network bandwidth and latency can also become a problem.

Independence

Agent independence means that some agents may get stuck or never end, which can cause blocking if the synchronization is waiting for agents to finish.

Environment complexity

Some environments are so complex, like 3D simulators, that they themselves need acceleration, which can cause GPU contention if not intelligently orchestrated.

Cost efficiency

Scaling can be expensive, so you want to be sure you fully utilize the hardware. Creating architectures and algorithms that better leverage the benefits of both GPU and CPU computation is necessary.

The last item, cost, is a big problem in machine learning in general. For example, OpenAI's recent **GPT-3 language model** would cost in the range of \$10 million to train from scratch at the current cloud vendor's list prices. And that's assuming a single run. Imagine the horror when you find a bug!

This is obviously an extreme example. A recent paper by Espeholt et al. measured the cost of running 1 billion frames in the DeepMind Lab and Google Research Football environments. Both environments provide 3D simulation for a range of tasks. They tested two algorithms called IMPALA and SEED (discussed shortly) and found that the total cost for a single run (one billion frames) was \$236 and \$54 for DeepMind Lab and \$899 and \$369 for Google Research Football, respectively. These aren't astronomical figures, in the context of an enterprise IT project, but they are worth considering.⁵

Distributed training (Gorila)

Early attempts at scaling RL concentrated on creating parallel actors to better leverage multicore machines or multimachine clusters. One of the earliest was the *general reinforcement learning architecture* (Gorila), which was demonstrated with a distributed version of deep Q-networks (DQN), based upon a distributed system for training neural networks called *DistBelief*.^{6,7} DistBelief consists of two main components: a centralized parameter server and model replicas. The centralized parameter server maintains and serves a master copy of the model and is responsible for updating the model (with new gradients) from the replicas. The replicas, spread out across cores or machines, are responsible for calculating the gradients for a given batch of data, sending them to the parameter server, and periodically obtaining a new model. To use this architecture in an RL setting, Gorila makes the following changes. The centralized parameter server maintains and serves the model parameters (which are dependent on the chosen RL algorithm). Actor processes obtain the current model and generate rollouts in the environment. Their experience is stored in a centralized replay buffer (if appropriate for the algorithm). Learner processes use the sampled trajectories to learn new gradients and hand them back to the parameter server. The benefit of this architecture is that it provides the flexibility to scale the parts of the learning process that pose a bottleneck. For example, if you have an environment that is computationally expensive, then you can scale the number of actor processes. Or if you have a large model and training is the bottleneck, then you can scale the number of learners. **Figure 10-1** depicts these two architectures in a component diagram.

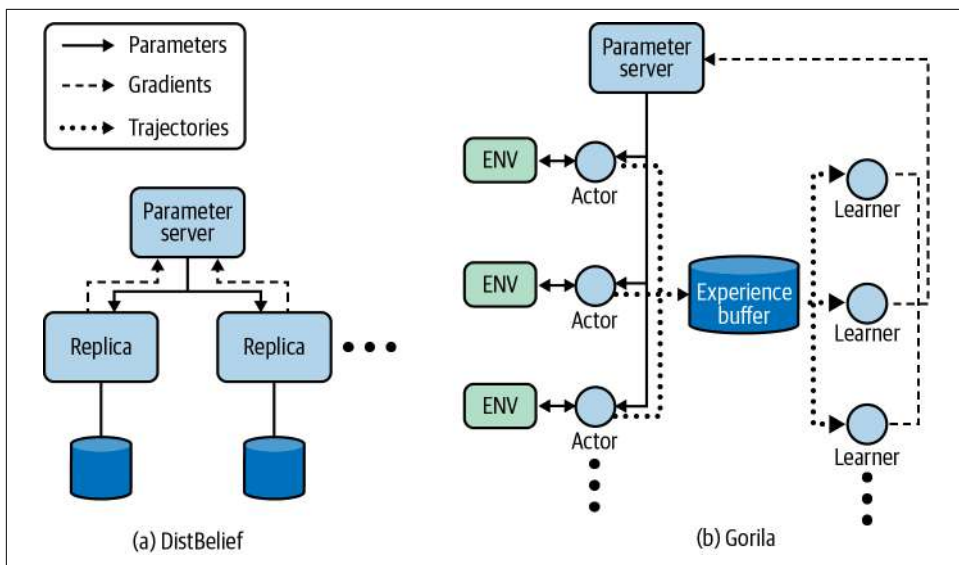


Figure 10-1. A component diagram summarizing the architecture for (a) DistBelief and (b) Gorila. Gorila enables scaling by separating the tasks of acting and learning and then replicating.

Single-machine training (A3C, PAAC)

Using Gorila in the Atari environments led to superior performance over 20 times faster (wall-clock time) than DQN alone, but researchers found that a distributed architecture, although flexible, required fast networks; transferring trajectories and synchronizing parameters started to become a bottleneck. To limit the communication cost and simplify the architecture, researchers demonstrated that parallelizing using a single multicore machine is a viable alternative in an algorithm called *asynchronous advantage actor-critic* (A3C).⁸ This is a parallel version of the advantage actor-critic algorithm (A2C). Since updates reside within the same machine, the learners can share memory, which greatly reduces the communication overhead due to speedy RAM read/writes. Interestingly, they don't use a lock when writing to shared memory across threads, which is typically an incredibly dangerous thing to do, using a technique called *Hogwild*.⁹ They find that the updates are sparse and stochastic, so overwrites and contention isn't a problem. Figure 10-2 shows the differences between A2C and A3C algorithms, which both run on a single machine.

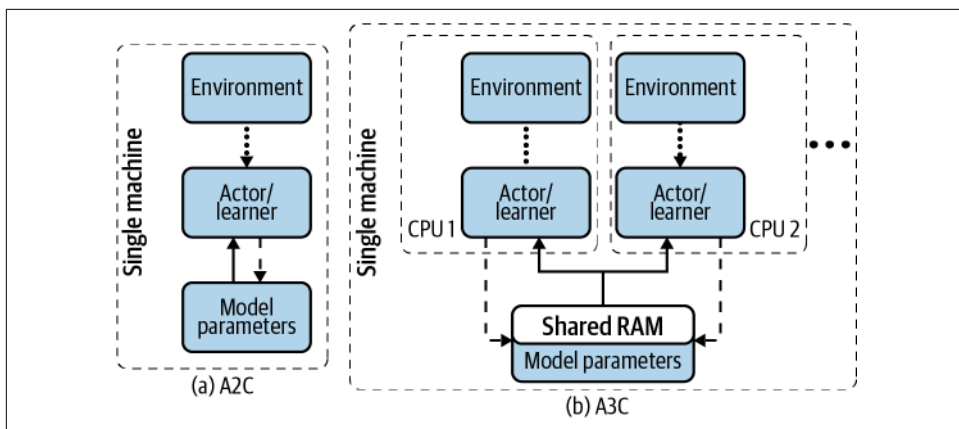


Figure 10-2. A component diagram summarizing the architecture for (a) A2C and (b) A3C. Both algorithms run on a single machine, but A3C uses multiple threads with replicated agents that write to a shared memory.

Surprisingly, using a single machine with 16 CPU cores with no GPU, A3C obtained comparable results to Gorila in one-eighth of the time (1 day) and better results in about half the time (4 days). This clearly demonstrates that synchronization and communication overheads are a big problem.

The simplicity of not using GPUs holds A3C back. It is very likely that a GPU will speed up training of anything “deep,” for greater cost, of course. The *parallel advantage actor critic* (PAAC) implementation, which isn’t a great name because you can use this architecture with any RL algorithm, brings back the ability to use a GPU. It uses a single learner thread, which is able to use a single attached GPU, and many actors in as many environments. In a sense, this is an intentionally less scalable version of both Gorila and A3C, but it produces comparable results to A3C in one-eighth of the time (again!) creating efficient policies for many Atari games in about 12 hours, depending on the neural network architecture. This implementation shows that increasing exploration through independent environments is a very good way of decreasing the wall-clock training time; training time is proportional to the amount of the state-action space you can explore. Similar to A3C, this implementation demonstrates that data transfer and synchronization is a bottleneck and if your problem allows, you will benefit from keeping the components as close to each other as possible, preferably inside the same physical machine. This is simple, too, in the sense that there are no orchestration or networking concerns. But obviously this architecture might not fit all problems. Figure 10-3 presents this architecture.¹⁰

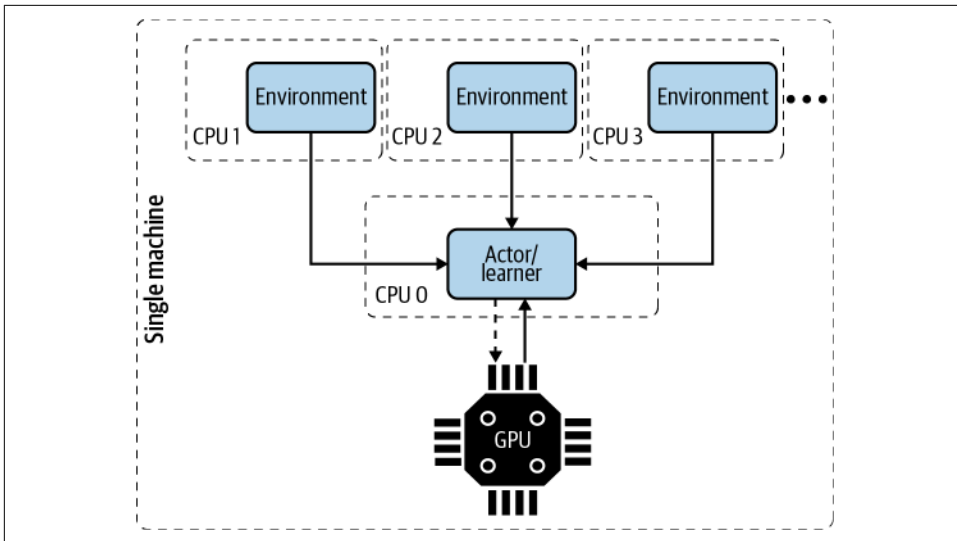


Figure 10-3. A component diagram summarizing the architecture for PAAC. A single machine with a GPU and many CPUs is used to scale the number of environments.

Distributed replay (Ape-X)

Using other types of replay buffer in a distributed setting is as simple as augmenting Gorila’s (or other’s) architecture. *Ape-X* is one such example, in which greater exploration via distributed actors with environments led to better results, because more diverse experiences were collected. Performance measures on Atari environments were approximately three times greater than AC3 and four times greater than Gorila. But these scores were trained on 16 times the number of CPUs (256 versus 16 for A3C), which led to two orders of magnitude more interaction with the environment. It is unclear whether there was a difference in performance when using the same number of environments/CPU’s.¹¹

Synchronous distribution (DD-PPO)

Attempting to use a single machine with many CPUs for training an RL algorithm, like in PAAC or AC3, is enticing due to the simplicity of the architecture. But the lack of GPUs becomes a limiting factor when your problem uses 3D simulators that need GPUs for performance reasons. Similarly, a single parameter server with a single GPU, like in Gorila or PAAC/AC3, becomes a limiting factor when you attempt to train massive deep learning architectures, like *ResNet50*, for example. These factors call for a distributed architecture that is capable of leveraging multiple GPUs.

The goal of *decentralized distributed proximal policy optimization* (DD-PPO) is to decentralize learning, much like decentralized, cooperative multi-agent RL. This architecture alternates between collecting experience and optimizing the model and

then synchronously communicating with other workers to distribute updates to the model (the gradients). The workers use a distributed parallel gradient update from PyTorch and combine the gradients through a simple aggregation of all other workers. The results show that DD-PPO is able to scale GPU performance by increasing the number of workers almost linearly, tested up to 256 workers/GPUs. The primary issue with this implementation is that the update cannot occur until all agents have collected their batch. To alleviate this blocking, Wijmans et al. implemented a preemptive cutoff, which forces a proportion of stragglers to quit early. **Figure 10-4** summarizes this architecture and Wijmans et al. suggest that it is feasible to implement any RL algorithm.¹² Other researchers have suggested a similar architecture.¹³

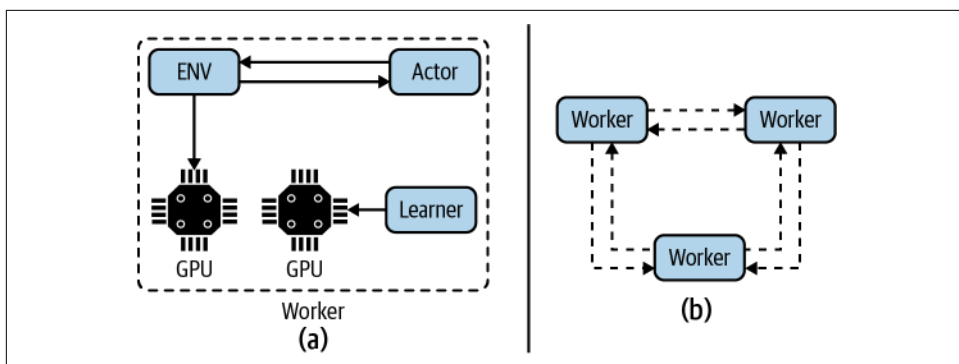


Figure 10-4. A component diagram summarizing the architecture for DD-PPO. A worker (a) comprises an environment, actor, and learner that are able to leverage GPUs. (b) After the agent collects a batch of data each worker performs a distributed update to share weights.

Improving utilization (IMPALA, SEED)

Importance weighted actor-learner architecture (IMPALA) is like a combination of Gorila and DD-PPO. The key advancement is that it uses a queue to remove the need for synchronizing updates. The actor repeatedly rolls out experience and pushes the data into a queue. A separate machine (or cluster of machines) reads off the queue and retrains the model. This means that the update is off-policy; the learner is using data from the past. So IMPALA uses a modified version of the retrace algorithm (see **“Retrace(λ)” on page 182**) to compensate for the lag in policy updates. This also enables the use of a replay buffer, which improves performance.

Espeholt et al. demonstrate that the basic off-policy actor-critic (see **“Off-Policy Actor-Critics” on page 151**) works almost as well, which implies that more sophisticated versions like TD3 might work better. Indeed, OpenAI used an IMPALA-inspired framework with PPO for its Dota 2 video game competition.¹⁴

Single learners with multiple actors typically run on a single machine. You can duplicate the architecture to increase the number of learners and use distributed training like DD-PPO. This architecture, depicted in [Figure 10-5](#), improves training speed by an order of magnitude compared to A3C, but is proportional to the amount of hardware in the cluster (they used 8 GPUs in their experiments). Espeholt et al. also claim that this architecture can be scaled to thousands of machines.¹⁵

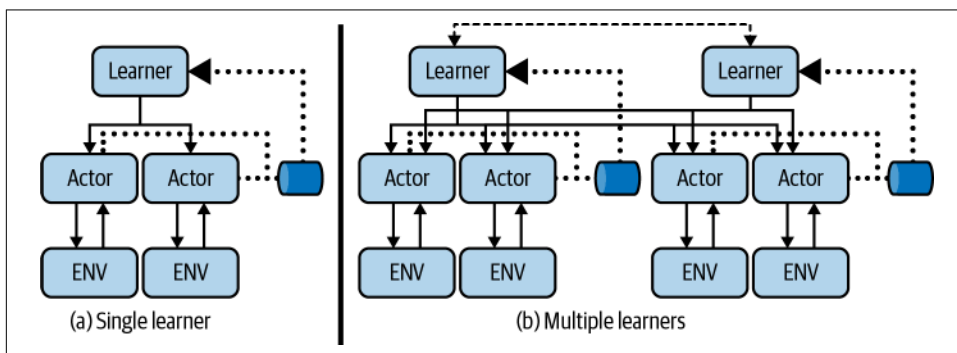


Figure 10-5. A component diagram summarizing the architecture for IMPALA. A single learner (a) has multiple actors (typically on the same machine). Actors fill a queue and training is performed independently of the actors. When using multiple learners (b) gradients are distributed and trained synchronously (like DD-PPO).

Although IMPALA is efficient and is capable of state-of-the-art results, there are still parts of the architecture that can be optimized. One problem is the communicating of the neural network parameters. In large networks these are represented by up to billions of floats, which is a large amount of data to transfer. Observations, on the other hand, are typically much smaller, so it might be more efficient to communicate observations and actions. Also, environments and actors have different requirements, so it could be inefficient to colocate them on the same CPU or GPU. For example, the actor might need a GPU for quick prediction, but the environment only needs a CPU (or vice versa). This means that for much of the time the GPU won't be utilized.

Scalable and efficient deep RL with accelerated central inference (SEED) aims for an architecture where environments are distributed, but request actions from a central, distributed model. This keeps the model parameters close to the model and allows you to choose hardware that maps directly to either environment simulation or model training and prediction. This leads to increased utilization and slightly faster training times (approximately half the time under similar conditions). [Figure 10-6](#) depicts a simplified version of the architecture.¹⁶

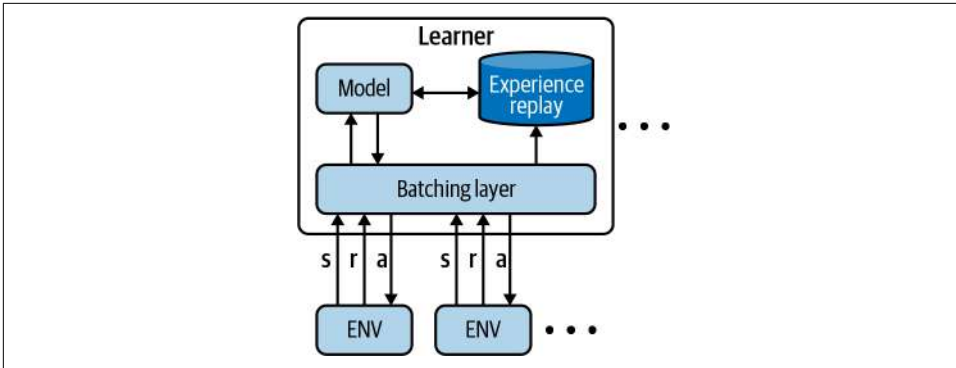


Figure 10-6. A component diagram summarizing the architecture for SEED RL. Hardware is allocated according to the need of the model or environment.

One interesting aspect of the SEED paper is that it highlights how coupled the architecture is to performance. In order to generate their results the researchers had to treat the number of environments per CPU *and* the number of CPUs as hyperparameters. In general more replicas produced higher scores in less time, but precise configurations are probably environment dependent and therefore hard to pinpoint. Another point to make is that SEED goes to great lengths to optimize on a software level, too, using a binary communication protocol (gRPC) and compiled C++.

Both IMPALA and SEED are capable of running on a single machine, like A3C and PAAC, and are generally more performant due to higher utilization.

Scaling conclusions

The previous algorithms achieve state-of-the-art results in a short period of time. But these comparisons are made on toy environments—the Atari environment, mostly, but sometimes more continuous environments like Google Research Football. You can extrapolate results from these environments to real-life scenarios, but the reality is you are going to have to do your own testing to be sure.

Researchers have a lot of experience with these environments, so they don't have to waste too much time tuning them and they already know the pitfalls to look out for. But they also aim for headline results, like state-of-the-art performance or speed. There is a hyper-focus on utilization and uber-scale and that sometimes comes at a cost. For example, having a fully distributed deployment like DD-PPO might actually be better for operational use because it would be easy to make it self-healing (just form a cluster of some number of distributed components), which increases robustness and high-availability. This might not be of interest for your application, but my point is that fastest isn't always best. There are other important aspects, too.

Similarly, you can start to see diminishing returns, too. Early improvements yielded a reduction of an order of magnitude in training speed. The difference between IMPALA and SEED was about half. Researchers are having to go into increasing depth to extract more performance, like statically compiling code and using binary protocols. Again, this may not be appropriate for your problem. If you had to write code in C++ rather than Python, for example, the cost of your time or the opportunity loss may be greater than the gains obtained through code-level optimization.

Finally, some of these architectures tend to blur the abstractions that have emerged in the frameworks. It is common to have abstractions for agents, environments, replay buffers, and the like, but these architectures enforce locations for these abstractions, which might not be ideal from a design perspective. For example, bundling the model, the training, the replay buffer, and some custom batching logic into the learner in the SEED architecture could get messy. It would have been great if they were physically distinct components, like distributed queues and ledgers. Obviously there is nothing stopping you from implementing something like that, but it is generally not a priority for this kind of performance-driven research.

Evaluation

To add value to a product, service, or strategy, you and your stakeholders need to carefully consider what you are trying to achieve. This sounds abstract and high level, but it is too easy to work hard on a problem only to find that the thing that you are optimizing for doesn't fit the goal. The reason is that engineers can forget, or even ignore, to connect their work back to the objective.

Psychologists and neurologists have studied multitasking at great depth. They found that humans have a limited amount of mental energy to spend on tasks and that switching between several severely impacts performance. But when working on engineering problems, switching between abstraction levels is just as difficult. Your brain has to unload the current stack of thought, save it to short-term memory, load up a new mental model for a different abstraction level, and start the thought process. This switching, along with the effort of maintaining your short-term memory, is mentally taxing. This is one reason why it is easier to ignore high-level objectives when working on low-level day-to-day challenges.

Another reason, even after the dawn of DevOps, is that engineering is deemed to be successful when forward progress is made: new products, new features, better performance, and so on. Moving backward is viewed as a lack of progress; I want to make it clear that it is not. "Going back to the drawing board" is not a sign of failure, it is a sign that you have learned important lessons that you are now exploiting to design a better solution.

Even though it can be hard, try to take time to move around the development process, and up and down through abstraction layers to make sure they are all aligned toward the goal you are trying to achieve.



Here's one trick that I use to force myself to switch abstractions or contexts: ask the question "Why?" to move up an abstraction layer and ask the question "How?" to move down.

Coming back to the task at hand, when evaluating your work, you should initially ask *why* you want to measure performance. Is this to explain results to your stakeholders? Or is it to make it more efficient? Answers to this question will help direct *how* you want to implement.

Policy performance measures

The efficacy of an RL agent can be described by two abstract metrics: *policy performance* and *learning performance*. Policy performance is a measure of how well the policy solves your problem. Ideally, you should use units that map to the problem domain. Learning performance measures how fast you can train an agent to yield an optimal policy.

In a problem with a *finite horizon*, which means there is a terminating state, the policy performance is most naturally represented by the sum of the reward. This is the classic description of an optimal policy. In *infinite-horizon* problems you can include discounting to prevent rewards tending to infinity. Discounting is often used in finite problems, too, which might lead to a subtly different policy, so you should consider removing it when you compute the final performance measure. For example, your stakeholders are probably not interested in discounted profit; they want to talk about profit.

Another option is the maximization and measurement of average rewards. In the *average-reward* model, you have to be careful to ensure that the mean is a suitable summary statistic for your distribution of rewards. For example, if your reward is sparse, then an average reward is a poor summary statistic. Instead, you could choose other summary statistics that are more stable, like the median or some percentile. If your rewards conform to a specific distribution, then you might want to use parameters of that distribution.

Measuring how fast an agent can learn an optimal policy is also an important metric. It defines your feedback cycle time, so if you use an algorithm that can learn quicker you can iterate and improve faster. In problems where agents learn online, for example if you are fine-tuning policies for specific individuals, learning quickly is important to mitigate the cost of missing an opportunity due to suboptimal policies.

The simplest measure of training performance is the amount of time it takes for the agent to reach optimality. Optimality is usually an asymptotic result, however, so the *speed of convergence to near-optimality* is a better description. You can arbitrarily define what constitutes “near” for your problem, but you should set it according to the amount of noise in your reward score; you may need to perform averaging to reduce the noise.

You can define time in a few different ways, depending on what is important to your problem. For simulations, using the number of steps in the environment is a good temporal measure because it is independent of confounders and environment interactions tend to be expensive. Wall-clock time could be used if you are concerned about feedback cycle time and can limit the number of environment interactions. The number of model training steps is an interesting temporal dimension to analyze model training performance. And computational time can be important if you want to improve the efficiency of an algorithm.

Your choice of temporal dimension depends on what you are trying to achieve. For example, you might initially use environment steps to improve the sample efficiency. Then you might start looking at wall-clock time when you are ready to speed up training. Finally, you might squeeze out any remaining performance by looking at process time. Typically you would use all of these dimensions to gather a broad picture of performance.

Regret is used in online machine learning to measure of the total difference between the reward if an agent was behaving optimally in hindsight and the actual reward obtained over all time. It is a positive value that represents the area between the optimal episodic reward and the actual, as shown in [Figure 10-7](#). Imagine you run a solar power plant. The total amount of power you generate in an episode, which might be one day, has a fixed maximum that is dependent on the amount of sun and the conversion efficiency of the solar panels. But you are training a policy to alter the pitch of the panels to maximize power output. The regret, in this case, would be the difference between the theoretical maximum power output and the result of the actions of your policy summed over a period of time.

Regret has been used by researchers as a mathematical tool during the formulation of policy algorithms, where the policy optimizes to limit the regret. This often takes the form of a regularization term or some adaptation of the rewards to promote statistically efficient exploration (see [“Delayed Q-Learning” on page 74](#), for example). Regret is important conceptually, too, since humans reuse experience to infer causation and question the utility of long-term decisions.¹⁷

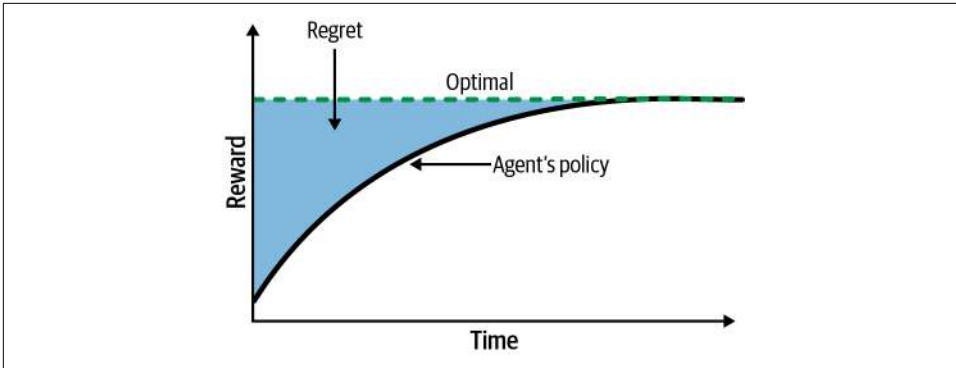


Figure 10-7. A depiction of how to calculate regret while training a policy over time. Regret is the shaded area.

You can use regret as a measurement of performance, too, but how you decide what the optimal policy is, even in hindsight, is dependent on the training circumstances. You might not be able to obtain a theoretically optimal policy or it may not be stationary, for example. If the policies are operating in a nonstationary environment, then approaches include attempting to calculate the regret within a small interval. In simpler problems you might be able to, in hindsight, establish what the best action was and use that as the optimal reward. In more complex problems you should be able to use another algorithm as a baseline and make all comparisons to that. If possible, you could use self-play to record the opponent's positions and then after the game has finished see if there were better actions you could have played. There isn't yet a simple unified method of calculating regret for any problem.

Statistical policy comparisons

Whenever you are making comparisons I always recommend that you visualize first. Statistics is full of caveats and assumptions and it's very easy to think that you are doing the right thing, only to have someone more mathematically inclined say that what you did only makes sense in a very specific circumstances. Visualizing performance is much easier because it is intuitive. You can leverage visual approximations and models that you have learned throughout your life. For example, fitting a linear regression model by eye, even with noise and outliers, is much easier for humans than the mathematics suggest. Similarly, comparing the performance of two algorithms is as simple as plotting two metrics against each other and deciding which is consistently better than the other.

However, there are times when you do need statistical evidence that one algorithm is better than another. If you want to automate the search for a better model, for example, then you don't want to have to look at thousands of plots. Or if you want a yes/no

answer (given caveats!). Here is a list of possible statistical tests with brief notes of their assumptions when comparing two distributions:

Student's t-test

Assumes normal distributions and equal variances.

Welch's t-test

Same as Student's but relaxes equal variance assumption.

Wilcoxon Mann-Whitney rank sum test

Compares medians, is nonparametric, therefore no assumption on distribution shapes. But does assume they are continuous and have the same shape and spread.

Ranked t-test

Compares medians, are ranked and then fed into t-test. Results are essentially the same as the Wilcoxon Mann-Whitney rank sum test.

Bootstrap confidence interval test

Randomly samples from distributions, measures difference in percentiles, and repeats many times. Requires very large sample size, approximately greater than 10^3 . No other assumptions.

Permutation test

Randomly samples from distributions, randomly assigns them to hypotheses, and measures the difference in percentiles, and repeats many times. Similar to the bootstrap confidence interval test but instead tests to see if they are the same, to some confidence level. Requires very large sample sizes again.

You can see that there are a lot of varied assumptions, which make it really easy to accidentally use the wrong test. And when you do, they fail silently. The results won't show you that you have done something wrong. So it's really important to have strong expectations when performing these tests. If the results don't make sense, you've probably made a mistake.

What are the problems with these statistical assumptions?

- RL often produces results that are not normal. For example, an agent can get stuck in a maze 1 in 100 times and produce an outlier.
- Rewards may be truncated. For example, there may be a hard maximum reward that results in discontinuity in the distribution of results.
- The standard deviations of the runs are not equal between algorithms and may not even be stationary. For example, different algorithms might use different exploration techniques that lead to different variations in results. And within the

evaluation of a single algorithm, there may be some fluke occurrence like a bird flying into a robotic arm.

Colas et al. wrote an excellent paper on statistical testing specifically for comparing RL algorithms. They compare the performance of TD3 and SAC on the Half-Cheetah-v2 environment and find that they need to rerun each training at least 15 times to get statistically significant evidence that SAC is better than TD3 (on that environment). They suggest that for this environment, rewards tend to be somewhat normal so Welch's t-test tends to be the most robust for small numbers of samples. The nonparametric tests have the least assumptions but require large sample sizes to approach the power of assuming a distribution.¹⁸ They also have a useful [repository](#) that you can leverage in your experiments.

You might be asking, "What does he mean by rerun?" That's a good question. If you read through enough papers on RL algorithms you will eventually see a common evaluation theme. Researchers repeat an experiment 5 or 10 times then plot the learning curves, where one experiment represents a full retrain of the algorithm on a single environment. Upon each repeat they alter the *random seed* of the various random number generators.

This sounds simple but I've found it to be quite tricky in practice. If you use the same seed on the same algorithm it should, in theory, produce the same results. But often it doesn't. Usually I find that there is yet another random generator somewhere buried within the codebase that I haven't set. If you train an algorithm using a typical set of RL libraries then you have to set the seed on the random number generators for Python, Numpy, Gym, your deep learning framework (potentially in several places), in the RL framework, and in any other framework you might be using, like a replay buffer. So first check that you can get exactly the same result on two runs. Then you can repeat the training multiple times with different values for the random seed. This produces a range of performance curves that are (hopefully) normally distributed.

The problem is that researchers tend to use a small, fixed number of repeats because training is expensive. If there is a large separation between the two distributions then a small sample, just enough to be confident of the mean and standard deviation, is adequate. But when the difference in performance is much smaller, you need a much larger sample size to be confident that performance is different. Another paper by Colas et al. suggests that you should run a pilot study to estimate the standard deviation of an algorithm, then use that to estimate the final number of trials required to prove that it outperforms the other algorithm, to a specified significance level.¹⁹

Finally, statistical performance measures are one way of quantifying how robust a policy is. The standard deviation of an obtained reward is a good indication of whether your policy will survive the natural drift of observations that occur so often in real life. But I would recommend taking this a step further, actively perturbing

states, models, and actions to see how robust the policy really is. Adversarial techniques are useful here, because if you can't break your model, this makes it less likely that real life will break your model, too—life breaks everything eventually.²⁰

Algorithm performance measures

I would suggest that in most industrial problems you have a single or a small number of problems to target and therefore a small number of environments. However, if you are more involved in pure research or you are trying to adapt your implementation to generalize over different problems, then you should consider more abstract performance measures that explain how well it performs at RL-specific challenges.

One of the most useful solutions so far is the *behavior suite* by Osband et al. They describe seven fundamental RL challenges: credit assignment, basic (a competency test, like bandit problems), scalability, noise, memory, generalization, and exploration. An agent is tested against a handpicked set of environments that correspond to the challenge. The agent then receives a final aggregated score for each challenge that can be represented in a radar plot, like in Figure 10-8.²¹

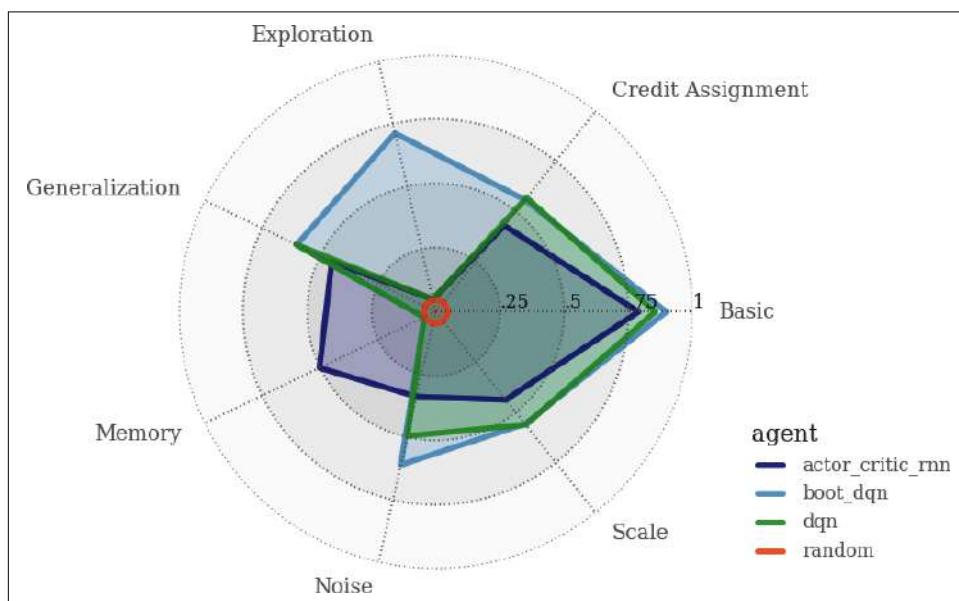


Figure 10-8. Comparing several algorithms using the behavior suite. Each axis represents a different RL challenge. Note how the random agent performs poorly on all challenges and standard DQN performs well on many except for exploration and memory. Used under license, Apache 2.0.

Problem-specific performance measures

There are also a range of benchmarks available for domain-specific tasks, or for tasks that have a particularly tricky problem. For example, there are benchmarks for **offline RL**, **robotics**, and **safety**.^{22,23,24} Many more excellent benchmarks exist.

These can be useful when you know you also have the same generic problem in your domain and can be confident that improvements in the benchmarks translate to better performance in the real world.

Explainability

An emerging phase in the data science process is *explainability*, often abbreviated to *explainable artificial intelligence* (XAI), where you would use techniques to attempt to describe how and why a model made a decision. Some algorithms, like tree-based models, have an inherent ability to provide an explanation whereas others, like standard neural networks, must rely on *black-box* approaches, where you would probe with known inputs and observe what happens at the output.

Explainability is important for several reasons. One is trust; you shouldn't put anything into production that you don't trust. In software engineering this trust is established through tests, but the complexity of models and RL means that the "coverage" is always going to be low. Another reason might be that there is a regulatory or commercial reason to explain decisions like the EU General Data Protection Regulation or Facebook's post recommendation explanation feature. Being able to explain an action or decision can also help with debugging and development.

Explainability is harder in RL than in machine learning because agents select sequential actions whose effects interact and compound over long periods. Some RL algorithms have an intrinsic capability to explain. Hierarchical RL, for example, decomposes policies into subpolicies, which combine to describe high-level behavior. But most research focuses on black-box, post-training explainability, because it is more generally applicable.

The black-box techniques tend to operate on either a global or local level. Global methods attempt to derive and describe the behavior at all states, whereas local methods can only explain particular decisions.

Global methods tend to consist of training other, simpler models that are inherently explainable, like tree or linear models. For example, the *linear model U-trees* model uses a tree structure to represent a value function, with linear approximators at the leaf of each tree to improve generalization. From this model you can extract feature importance and indicative actions for all of the state space.²⁵

Local methods comprise two main ideas. The word "local" is usually meant to denote a global method evaluated in a small region of the state space. For example, Yoon et al. use RL to guide the process that trains the local approximation.²⁶ Other ideas

expand upon the predictive nature of global methods by considering the *counterfactual*, the opposite action or what could have happened. Madumal et al. achieve this by training a model to represent a causal graph that approximates the environment transition dynamics. This sounds promising but only works with discrete states.²⁷

Similar to explaining decisions, it can also be useful to indicate what parts of the state influence a decision. For problems with visual observations, or states that have some notion of position or adjacency, you can use *saliency maps* to highlight regions that are important to action selection.²⁸

Evaluation conclusions

Evaluation is an inherent part of life, from karate belts to food critics. In industrial applications, evaluating your work helps you decide when you are finished. In software engineering, this is usually when acceptance tests have passed. In machine learning acceptable performance is often defined by an evaluation metric. But in RL, because of the strategic nature of the decisions being made, it is very hard to state when enough is enough. You can always make more profit, you can always save more lives.

In general, the performance of an agent is defined by the reward obtained by the policy and how fast it got there. But there are a range of other measures that you should also consider if you want to train and utilize a stable, robust policy in a production situation. This also includes the ability to explain the decisions of a policy, which could be vitally important in high-impact applications.

Deployment

By *deployment*, I refer to the phase where you are productionizing or operationalizing your application. Projects come to a point where everyone agrees that they are both viable and valuable, but to continue being viable and valuable they need to be reliable. This section is about how to take your budding RL proof of concept into the big league so that you can serve real users.

The tricky part is that there isn't much prior work on running RL (RL) applications operationally. So much of this has to be extrapolated from running software and machine learning applications, my experience running Winder Research, and the many luminaries who have penned their own experiences.

Goals

Before I dig into the details of deployment, I think it is important to consider the goals of this phase of the process. This will help you focus on your situation and help you decide what is important to you, because no one idea or architecture is going to suit all situations.

Goals during different phases of development

First, consider how the deployment priorities change depending on what phase of development you are in. There are three phases where you need to be able to deploy an agent: during development, hardening, and production.

During development it is important to have an architecture that allows you to rapidly test new ideas; flexibility and speed trumps all other needs. This is because during early phases of a project it is important to prove the potential viability and value. It is not important to be infinitely scalable or exactly repeatable; that comes later. The largest gains come from rapid and diverse experimentation, so the largest cost at this point in time is the length of that feedback loop.

Once a project has proven to be viable and valuable, then it goes through a phase of hardening. This involves improving the robustness of the processes involved in maintaining a policy. For example, offline policies or ancillary models must be trained, which may require large amounts of computational resource, and the provenance (the origin of) of a codebase or model is vital to ensure you know what will be running in production.

Finally, policies and models are deployed to production and used by real people or processes. Because RL optimizes sequences of actions, it is highly unlikely that you will be able to retrain your policy by batching data once every so often, which is the predominant technique used in production machine learning systems, so care must be taken to preserve the state of the policy. This phase encourages the development and adoption of sophisticated monitoring and feedback mechanisms, because it is so easy for the policy to diverge (or at least not converge).

When deploying an RL solution into a product or service, the value of performing a task changes. Early on, the quickest way to extract value is through rapid experimentation and research; later on reliability, performance, and robustness become more important. This leads to a range of best practices that are important at different times.

Best practices

Machine learning operations (MLOps) is an emerging description of the best practices and appropriate tooling that is required to run production-quality models. The need for this has arisen because machine learning models have different operational requirements compared to traditional software projects, but fundamentally the core needs are exactly the same. I like to summarize these needs into three topics: reliability, scalability, and flexibility.

Reliability lies at the core of modern software engineering. Thousands of books have been written on the topic and these ideas are directly applicable to machine learning and RL, ideas like testing, continuous integration and delivery, architecture, and design. Moving toward machine learning, ideas such as building data pipelines,

provenance, and repeatability all aim to solidify and further guarantee that models will continue to work into the future. RL poses its own challenges and over time I expect new best practices to emerge.

For example, I caught up with an old colleague recently and I was asking about some previous projects that we collaborated on. I had developed some algorithms to perform traffic and congestion monitoring around 2012 and these had been deployed to some major roads in the UK on dedicated hardware. I learned that the hardware had failed twice but my algorithms (in Java!) were still going strong, nearly 10 years later. This fact isn't due to my algorithms, it is due to my colleagues' software engineering expertise and a commitment to reliable software.

Scalability is particularly important to both machine learning and RL, arguably more so than typical software, because they need so much horsepower to do their job. If you consider all software ever written, the vast majority of those applications will probably run on the resources provided by a standard laptop. Only rarely do you need to build something that is as scalable as a global messaging or banking system. But in machine learning and RL, nearly every single product ever produced needs far more than what a laptop can provide. I posit that the average computational requirement for an RL product is several orders of magnitude greater than a software product. I am being intentionally vague, because I don't have any hard facts to support this. But I think my hypothesis is sound and this is why scalability is an important factor when deploying RL.

I think *flexibility* is one of the most underrated best practices. In industry, reliable software was the theme of the first decade in the 21st century, with software architecture, test-driven development, and continuous integration being hot topics at conferences around the world. In the second decade, scalability was the golden child and resulted in important changes like cloud adoption and the subsequent need for orchestration. I think the third decade (I'm ignoring specific themes like artificial intelligence and RL and machine learning) will focus on flexibility, purely because I see industry trending toward hyper-responsibility, where the most valuable engineers are defined by their ability to fulfill all of the roles expected in an enterprise environment, from business development through to operations. The field of DevOps can only ever expand; it can never shrink back to two separate roles.

A similar argument can be made about the pace of change today. Engineers are continuously creating new technologies and it's hard to predict which will withstand the test of time. But I find that when tools attempt to enforce a fixed, inflexible, or proprietary way of working, they quickly become cumbersome to work with and are quickly outdated. No, flexibility is the only way to future-proof systems, processes, and solutions.

Hierarchy of needs

Maslow’s hierarchy of needs was an early psychological model suggesting human necessities depicted as a hierarchical model; in order to have safety you need access to air and water, in order to have friendship you need to be safe, and so on. RL has a similar hierarchy of needs.

To have a viable RL problem then you need an environment in which you can observe, act, and receive a reward. You need the computational ability to be able to decide upon actions given an observation. These things are required and if you don’t have them, then you can’t do RL.

Above that comes things that are valuable, but not strictly necessary. For example, pipelines to transform data in a unified, reliable way will prevent data errors. Being able to specify exactly what is running in production—provenance—is operationally useful.

The layer above this represents an optimal state, one that is probably more conceptual than practical in many cases, because tackling other problems or developing new features might be more valuable in the short term. Here you might consider fully automating the end-to-end process of whatever it is you are doing, or incorporating a sophisticated feedback mechanism to allow your product to automatically improve itself.

Table 10-1 presents some example implementation and deployment challenges that you might face. I want to stress that these have a range of solutions with variations in the amount of automation. But the value of each level of automation depends on your or your stakeholder’s assessment of other pressing needs. Just because there is a more sophisticated solution that doesn’t mean what you have isn’t good enough already. Get the basics right first.

Table 10-1. Example hierarchy of needs for different implementation challenges: the bottom row represents basic requirements, the middle row represents some amount of automation, the top row represents the state-of-the-art

	Infrastructure	State representation	Environment	Learning	Algorithm improvement
Optimal	Self-healing	Continuously learning	Combination	Meta	Automated
Valuable	Automated deployment	Model-driven dimensionality reduction	Real life	Offline	Brute-force selection
Required	Manual deployment	Fixed mapping	Simulation	Online	Manual

Architecture

There hasn't been a huge amount of research into generic RL architectures and I can think of a few reasons to explain this. The first is that RL still isn't universally accepted as a useful tool for industrial problems, yet. Hopefully this book will start to change that. This means that there isn't enough combined experience of designing these kinds of systems and therefore it has not been disseminated to a wider audience. Secondly, researchers, especially academic researchers, tend to be more focused on groundbreaking research, so you will find few papers on the subject. Third, I don't think there is a one-size-fits-all approach. Compare the differences between single-agent and multi-agent RL, for example. And finally, the deployment architecture is mostly a software engineering concern. Of course there are RL-specific needs, but I think that typical software architecture best practices fit well.

With that said, there is some literature available from the domain of contextual bandits. It's not generally applicable, but because industry has been quicker to adopt this technique, many lessons have already been learned. Two of the biggest problems that lead to architectural impacts are the fact that agents can only observe partial feedback and that rewards are often delayed or at worst, sparse:²⁹

- Machine learning is considered to be simpler, because feedback, or ground truth, is complete. The label represents the single correct answer (or answers if multiple are correct). RL only observes the result of a single action; nothing is learned from the unexplored actions (which are possibly infinite).
- Rewards observed by the agent are often delayed, where the delay ranges from millisecond scales in bidding contests to days when people are involved. Even then, the reward may be sparse because the sequence of actions doesn't lead to a real reward for some time, like a purchase.

Some problems have a relationship with prior work in machine learning, but are accentuated by RL. For example, online learning, or continuous learning, is important in some machine learning disciplines, but RL takes this to a whole new level, since decisions have wide reaching consequences and policies are nonstationary. Even if you are incorporating batch RL, it is important to have a low-latency learning loop. Reproducibility becomes a big problem, too, since both the policy and the environment are constantly shifting.

All of these facets lead to the following range of architectural abstractions that might be useful in your problem:

Serving interface

Like a personalized shopping assistant, one abstraction should contain the ability to serve action suggestions and embed exploration. This should implement the

Markov decision process interface and communicate with the wider system or environment. This is the both the entry and exit point of the system and presents a great location to decouple RL from wider deployments.

Log

Like in a message bus queue, it is vitally important to be able to retain state, for several reasons. First is for disaster recovery; you may need to replay interactions to update a policy to the last known state. The second is that it can be used as part of a wider batch or transfer learning effort to improve models. And third, it acts as a queue to buffer potentially slow online updates.

Parameter server

A place for storing the parameters of a model, like a database, crops up in multi-agent RL, but it is useful for situations where you need to replicate single-agent policies, too, to handle a larger load.

Policy servers

Since RL algorithms tend to be customized for a particular problem it makes sense to have an abstraction that encapsulates the implementation. In simpler, more static problems you might be able to bake the parameters of the model into the encapsulation, otherwise you can leverage a parameter server. You can replicate the policy servers to increase capacity.

Replay buffers

Many algorithms leverage replay buffers to increase sample efficiency. This might be an augmentation of the log or a dedicated database.

Actors and critics

You might want to consider splitting the policy server into smaller components.

Learning

Ideally you should use the same policy code to learn or train as you do to serve. This guarantees that there is no divergence between training and serving.

Data augmentation

You might have parts of your algorithm that rely on augmented data; a state representation algorithm, for example. These should exist as traditionally served ML models with their associated continuous integration and learning pipelines.

All or none of these abstractions might appear in your architecture design. Unfortunately they are very domain and application specific, so it doesn't make sense to enforce these abstractions. Also, more abstractions generally increase the maintenance burden; it's easier to work from a single codebase, after all. Instead, provide the tools and technologies to enable these architectural capabilities on a per-application basis.

Ancillary Tooling

I'm hesitant to talk about tooling in detail, because of the fast-pace change in the industry; a component I describe here could be obsolete by the time this book goes to print. Instead, I will talk abstractly about the functionality of groups of tooling and provide a few (historical) examples. Refer back to [“RL frameworks” on page 296](#) for a discussion specific to RL tooling.

Build versus buy

I think the traditional build versus buy recommendations are true in RL, too. If the tool of interest represents a core competency or value proposition then you should build, so that you retain the intellectual property, have much greater depth, and have full control over the customization. Another way of thinking about this is to ask yourself: what are your customers paying you to do?

But if this particular tool does not represent your core competency or if it does not provide a competitive differentiation, then you should consider buying or using off-the-shelf components. A middle ground also exists, where you can leverage an open source project to get you to 80% complete and adapt it or bolt on components to get you the rest of the way.

Monitoring

When your product reaches production, you should invest in monitoring. This, singlehandedly, gives you advance warning of issues that affect trust in an algorithm. And trust is important, because unlike traditional software that simply stops working, machine learning and RL are scrutinized like a racing driver critiques the way their car handles. If users receive an odd or unexpected result, this can knock their confidence. Sometimes it's better to simulate a broken product than it is to serve a bad prediction or recommendation.

I was speaking to a technical lead for a games company once and he said that users react negatively if they play against an automated agent that doesn't conform to their social norms. For example, if the agent were to take advantage of an opportunity in the game that left the user weak, the user would consider that bad sportsmanship and stop playing the game. And often they had to de-tune their agents to intentionally make bad choices to make the user feel like they had a fighting chance. RL can help in this regard, to optimize for what is important, like user retention, rather than raw performance.

[“Evaluation” on page 307](#) talked about a range of RL-specific performance measures that you could implement in a monitoring solution. This is in addition to all the standard machine learning statistics you might monitor and low-level statistical indicators of the underlying data. Of course, software-level monitoring is vitally important.

This forms a hierarchy of metrics that you should consider implementing to gain operational insight and spot issues before your users do.

Logging and tracing

A log is an important debugging tool. It tends to be most important during development, to help you tune and debug your agents, and after a failure, to perform a post-mortem analysis. Bugs in traditional software appear when the state of the program changes in a way that is unexpected. So logging these state changes is a good way of tracking down issues when they do occur. But in RL, every interaction with an agent is a state change, so this can quickly become an overwhelming amount of information. This is why I recommend having a dedicated log for Markov state changes, to free your software of this burden.

One of the most common bugs in RL is due to differences in the runtime and expected shape of data. It is a good idea to log these shapes, maybe in a low-level debug log, so you can quickly find where the discrepancies are when they happen. Some people go to the extent of making these assertions to halt the code as soon as an exception is found. The next most common arise from the data, but I wouldn't save data directly to a text-based log. It's too much data. Summarize it and save it as a metric in the monitoring or dump it to a dedicated directory of offline inspection.

Tracing decisions through a log can be quite tricky because of the potential length of trajectories. A bad recommendation might not have been caused by the previous action. An action a long time ago could have led to this bad state. This problem is very much like the tracing systems used in microservice deployments to track activity across service boundaries. These systems use a correlation identifier (a unique key) that is passed around all calls; this key is used to link activity. A similar approach could be used to associate trajectories.

There can be other sources of state change, too: changes in the policy, user feedback, or changes in the infrastructure, for example. Try to capture all of these to be able to paint a picture when things go wrong. If you do have failures and you don't know why, be sure to add appropriate logging.

Continuous integration and continuous delivery

Continuous integration is a standard software engineering practice of building automated pipelines to deploy your solutions. The pipelines perform tests to maintain the quality of your product. *Continuous delivery* is the idea that if you have enough confidence in your quality gates, then you can release a new version of your software upon every commit.

CI/CD is an important and necessary tool for production deployments, but it only operates on offline, static artifacts like software bundles. It can't provide any guarantees that the data flowing into, through, or out of your agent is valid.

You can tackle the internal part with testing. You can monitor and alert upon the data flowing out. But even though you could monitor the data flowing in, it might still cause a catastrophic failure.

To handle problems with incoming data you need to test it. I call this “unit testing for data.” The idea is that you specify what you expect incoming data to look like in the form of a schema and test that it conforms. If it doesn’t you can either correct it or inform the user there was something wrong with their data. You have to make an architectural decision about where to perform this testing. The simplest place is in the serving agent, but this could bloat the code. You could have a centralized sentry that manages this process, too.

Experiment tracking

During development you will be performing a large number of experiments. Even after this phase, you need to keep track and monitor the performance of your agent or model as you gather new observations. The only way to do this consistently is to track performance metrics.

The underlying goals are similar to monitoring: to maintain quality and improve performance. But monitoring only provides information about the current instantiation of the agent. Experiment tracking intends to provide a longer-term holistic view, with an eye on potential improvements.

The idea is that for every run of an agent, you should log and track the performance. Over time you will begin to see trends in the performance of agents as the data evolves. And you can compare and contrast that to other experiments.

But the main reason why it is important is that training can sometimes take days, so to maintain efficiency you have to run experiments in parallel. For example, imagine you had designed a recommendations agent and had a hypothesis that it was easier to learn policies for older people, because they knew what they wanted. So you segmented the training data by age and started training upon several different datasets. Instead of waiting for each one to finish, you should run all experiments in parallel, and track their results. Yes, your efficiency is down because of the time it takes to get feedback on your hypothesis, but at least you’ll get all the results at the same time. For this reason, you should track these experiments with an external system so that you can spend the time effectively elsewhere—or take the day off, it’s your call.

In effect, you want to track the metrics, the things you are monitoring, and store them in a database. TensorBoard is one common solution and works well for simple experimentation, but it doesn’t scale very well. Even for individuals it can get hard to organize different experiments. It gets even worse when multiple individuals use the same instance. Proprietary systems are available, but many of them are catered toward traditional machine learning, not RL, and because they are proprietary they are hard to improve or augment.

Hyperparameter tuning

The premise of hyperparameter tuning is quite simple: pick values for the hyperparameters, train and evaluate your agent, then repeat for different hyperparameter values. Once again the main problem is the amount of time it takes to do this by brute force, where you exhaustively test each permutation, because of the time it takes to perform a single run.

It is likely that the insight you can provide into your problem will lead to a carefully selected subset of parameters for which brute force might be enough. But there are newer approaches that attempt to build a model of performance versus the hyperparameter values to iterate toward an optimal model faster. They can also cut training runs short if they see that the final performance is going to be suboptimal. I've used **Optuna** for this purpose and it works well when your hyperparameters are continuous and the performance measure is robust. You can run into problems like when hyperparameters slow down learning and the framework thinks the result is suboptimal, but it might lead to better performance in the long run.

Having a good infrastructure setup that autoscales is important to reduce the amount of time it takes to perform hyperparameter experiments.

Deploying multiple agents

The microservice architectural pattern suits machine learning and RL well. Running multiple agents in production is easier than it was because of advancements in tooling that allow you to create advanced deployments. Here is a list of possible deployment strategies:

Canary

Route a certain proportion of users to a new agent, altered over time. Make sure you have the ability to provide *stickiness* so that the user's trajectory is dictated by the same agent. This is usually implemented using the same ideas as tracing.

Shadowing

Running other agents hidden in the background, on real data, but not serving the results to users. This can be tricky to validate, because the hidden agent is not allowed to explore.

A/B testing and RL

Yes, you can use bandits or RL to optimally pick your RL agents. Again, ensure *stickiness*.³⁰

Ensembles

This is more likely to fall under the banner of algorithmic development, but it is possible to use an ensemble of RL agents to make a decision.³¹ The theory suggests that the resulting action should be more robust, if agents are diverse.

Deploying policies

There is significant risk involved in deploying a new policy, because it is hard to guarantee that it will not become suboptimal or diverge when learning online. Monitoring and alerting helps, but another way of viewing this problem is in terms of *deployment efficiency*.

In a way that is similar to measuring sample efficiency, you can count how many times the policy needs to be updated to become optimal, during training. Policies that need to be updated less often are likely to be more robust in production. Matsushima et al. investigate this in their work and propose that policies update only when the divergence between the trajectories of the current and prospective policies become large. This limits the frequency of policy updates and therefore mitigates some of the deployment risk.³²

Considering when you want to update your policy is an interesting exercise. You will find that your agent sits somewhere on a continuum that ranges from continuous learning (updating on every interaction) through to batch learning (updating on a schedule) and ending at never updating, as shown in [Figure 10-9](#). There is no right answer, as it depends on your problem. But I recommend that you stay as close to never updating your policy as possible, since this limits the frequency at which you have to monitor your deployment for errors or divergence.³³

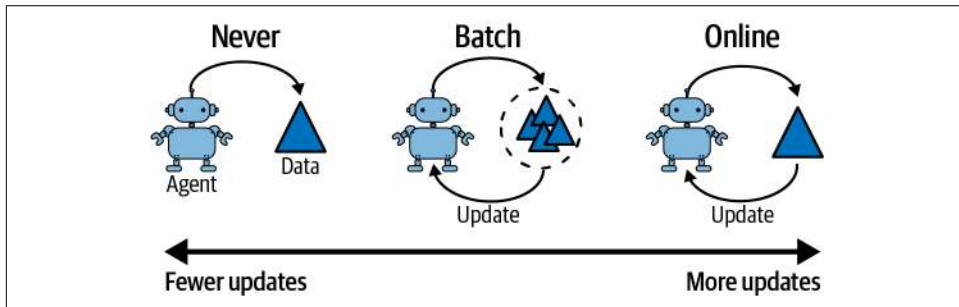


Figure 10-9. How often you update your production policy affects your deployment strategy. It falls on a continuum between never updating (manual deployment) and fully online.

I recommend that you try to make the deployments stateless and immutable, if possible. If you update the parameters of the model this is called a *side effect* of calling a function. These are considered to be bad because side effects can lead to unexpected failures and make it hard to recover from disaster. For example, one way to achieve immutability is by endeavoring to treat policy updates as new deployments and validating them in the same way you would for a code change. This is extreme, but very robust.

The traditional deployment techniques from “[Deploying multiple agents](#)” on page 324 can also help limit user-facing problems by treating each new instantiation of a policy, as defined by the policy parameters, as a separate deployment. Automated A/B testing can ensure that the new parameters do not adversely affect users.

Safety, Security, and Ethics

RL agents can sometimes control hazardous equipment, like robots or cars, which increases the jeopardy of making incorrect choices. A subfield called *safe RL* attempts to deal with this risk. RL agents are also at risk from attack, like any connected system. But RL adds a few new attack vectors over and above traditional software and machine learning exploits. And I think that engineers are the first line of defense against ethical concerns and that they should be considered proactively, so I think it’s important to talk about it here.

Safe RL

The goal of safe RL is to learn a policy that maximizes rewards while operating within predefined safety constraints. These constraints may or may not exist during training and operation. The problem with this definition is that it includes ideas about preventing catastrophic updates, in the reward sense. Algorithms have been designed to solve this problem intrinsically, like trust region methods (see “[Trust Region Methods](#)” on page 163), for example. Another form of research involves extrinsic methods that prevent certain actions from causing physical harm, monetary loss, or any other behavior that leads to damaging outcomes and it is this that I will summarize here. The obvious place to start is with techniques that have already been described in this book.³⁴

If your problem has safety concerns that can be naturally described as a constraint, then you can use any algorithm that performs constrained optimization. TRPO and PPO are good choices, where you can adapt the optimization function to incorporate your problem’s constraints. For example, if you are training an algorithm to deploy replicas of a web service to meet demand, it shouldn’t deploy more than the available capacity of the infrastructure. This information could be provided as a constraint to the optimization function. Ha et al. present a good example of training a robot to prevent falling over using a constrained version of SAC.³⁵

Similarly, you could shape the reward to penalize certain behaviors. For example, a robotic arm will undergo significant stress when fully extended. To prolong the life of the robot you should prevent extensions that might cause metal fatigue. You could incorporate an element that penalizes extension into the reward function.

The problem with these two approaches is that they are hardcoded and likely to be suboptimal in more complex problems. A better approach could be to use imitation or inverse RL to learn optimal constrained actions from an expert.

However, all of the previous approaches do not guarantee safety, especially during learning. Safety is only approximately guaranteed after a sufficient amount of training. Fundamentally, these approaches learn how to act safely by performing unsafe acts, like a child—or an adult—touching a hot cup, despite telling them that the cup is hot.

A more recent set of approaches use an external definition of safety by defining a super-level *safe set* of states. For example, you could define a maximum range in which a robot is allowed to operate or the minimum allowable distance between vehicles in a follow-that-car algorithm. Given this definition, an external filter can evaluate whether actions are safe or not and prevent them.

A crude filter could prevent the action entirely and penalize the agent for taking an action, which is basically the same as reward shaping, except you prevent the dangerous action taking place. But that doesn't incorporate the knowledge of the safe set into the optimization process. With this approach, for example, you would have to jump out of a tree at every height before you know for sure that certain actions are unsafe—may I suggest a safety rope.

A more pragmatic approach is to build models that are able to abort and safely reset agents. You can often design your training to avoid unsafe actions by definition. For example, Eysenbach et al. built a legged robot and trained it to walk by changing the goal to be far away from unsafe states. For example, if it moves too far to the left, then the goal is changed to be on the right. They also learn a reset controller to provide an automated method to get back to a safe state.³⁶

Researchers have looked at a variety of ways to incorporate this information. One suggestion is to use rollouts to predict the probability of an unsafe event and prefer actions that lead to safe trajectories.³⁷ This can be incorporated into value-based RL algorithms by pushing the value approximation parameters toward safe states and into policy gradient methods by directing the gradient steps toward safe states, with a correction that accounts for the biased policy generation.³⁸

Another set of suggestions called *formally constrained RL* attempt to mathematically define and therefore guarantee safety, but these methods tend to rely on perfect, symbolic representations of unsafe states that are only possible in toy examples. One practical implementation from Hunt et al. resorts to using a model-driven action filtering technique.³⁹ Without doubt, this remains an open problem and I anticipate many more improvements to come.

Secure RL

If an attacker could alter the source code of an application they could inject malicious code. This is called a *white box* attack. Software deployments mitigate this risk with strict access control mechanisms on the codebase. But if the developer exposes the software to the public internet, then they run the risk that the attacker might be able

to probe and find a weakness that allows them to alter the running of the application. This external approach is called a *black box* attack.

Machine learning, or any data-driven process, is also vulnerable to attack. These are broadly classified as attacks targeting either the *training phase* or the *prediction phase*. In the training phase, the attacker may be able to alter or manipulate the training data so that the attacker can exploit this data to produce a known result. This is commonly known as *poisoning*. For example, if an attacker could introduce a spam email with an incorrect label into a public spam dataset then the attacker might be able to reuse a similar email to sell pills that apparently enhance your libido, which would be classified as not spam by any model trained upon that data. Prediction attacks provide or augment the input data to produce invalid results. Many methods are white hat techniques and assume knowledge of the underlying model.⁴⁰ But more recently researchers have found black box techniques, like where the attacker can create a second model, similar to the first. Once trained, the attacker can generate white hat attacks for the second model and use them to attack the first. This approach shows that attacks are transferable.⁴¹ Together, these are known as *adversarial attacks*.

RL agents can also be attacked, in all the same ways. Your first step should be to fix the basics. An attacker is not going to go to the trouble of creating a trained replica if they can find an open SSH port on one of your servers or someone's GitHub credentials have been phished and you don't have multifactor authentication enabled. Next, make sure you have robust data integrity testing and monitoring in place; use rate limiting and denial-of-service protection to limit how much of the model a black-box attacker can explore. Ensure your models are simple and robust, as much as practicable. Consider a research spike to investigate how you could improve the countermeasures put in place for your ML models.

RL poses greater risks because of the increased attack surface. ML models generally have a single entry point, the input, and a single response, the output. In RL, attackers could target the environment, the observation of the environment, the rewards, or the actions. Successfully altering any of these could cause the policy to tend toward a different result. If the attacker has access to internal systems then they could also target ancillary services like the experience replay buffer or the parameter server.

Ways to prevent or mitigate these types of attacks are under active research. One branch reminds us that model robustness is at the heart of a security strategy and is strongly related to safe RL. If you have a model or a policy that works well under adverse conditions then it is less likely to succumb to an external attack. Behzadan and Munir suggest perturbing agents during training to teach them to be more robust to prediction attacks, which is a common strategy to harden ML models.⁴²

Another approach proposed by Liu et al. is to try to detect an adversarial attack and stop it before it does any damage. RL is sequential and often correlated over time, so you could build a model to predict the next state given the current

observation. If you calculate the divergence between the prediction and what actually happened, then compare that to “normal” behavior (a model of the typical distribution of divergences), you could build an algorithm to warn you when large divergences are observed.⁴³

You could consider guiding exploration by directing agents’ actions to states that would cause maximum adversarial gain.⁴⁴ Or even mitigate adversarial training examples by considering the problem to be hierarchical; adversarial poisoning is likely to “look” very different from normal interaction, so hierarchical RL should be able to consider this a subpolicy. The subpolicies could then be analyzed to ensure legitimacy and that the poisoned data is removed.⁴⁵

As you can see there are many ideas to tackle the many potential attack vectors. But therein lies the problem: potential. Investing time, energy, and money into mitigating these attacks has to be weighed against the probability of them occurring in the first place. For example, if you have an agent that runs on an internal network and is never exposed to the public internet, then I would consider this level of mitigation to be a poor use of time, given the probability of attack being so low. This is why you have to perform a risk analysis for your particular situation.

Attack mitigation also has diminishing returns, because the knowledge and resources to perform an attack increases exponentially. According to Symantec’s 2019 Internet Security Threat Report, a whopping 65% of all targeted hacks were caused by phishing, 23% by unpatched software, and 5% by bogus software updates. Only when you get to the bottom do you find exploits that directly relate to an exposed RL agent: 2% web server exploits and 1% data storage exploits. These statistics demonstrate that basic security practices are more valuable to a company than RL or possibly even ML attack mitigation techniques. To be clear, I’m not suggesting that RL security is unimportant, just get the basics right first.⁴⁶

Ethical RL

It’s probably not quite the dictionary definition of the term, but I like to think of *ethics* as a professional consensus of individual morality. This is an important distinction, because what you believe is right and wrong is not necessarily what I believe. But I would like to think that there is a compromise that we can both accept. So morals represent a distribution of beliefs and the mean represents the ethical standards. Engineers, in general, don’t have a single unified way or place to define the ethical means, so it is important that you are aware that they exist, on a conceptual level at least, and that your perspective is but one of many.

I say this not to preach philosophy or religion, but to remind you that you are in a position of power. You have the power and the authority to not only automate decisions (machine learning) but also entire strategies (RL). Whenever you enact these techniques, you are effectively removing the opportunity for a human to make

decisions. This could be a good thing, if the decision is trivial or burdensome, but sometimes it is not. Sometimes the decision is so important that it *has* to be made by a human, so that they can take responsibility for that decision, right or wrong. They are free to use the information provided by an algorithm, but this does not remove that responsibility. Other times the decision might not be so important on a global scale, but it might be important on a human scale; a person may feel a sense of loss or disempowerment if they lose the ability to make a decision.

Unfortunately, the stakeholders (both users and owners) of your systems are generally not able or not in a position to independently assess impact. They may lack the authority or opportunity to protest or they are prevented from understanding the technical intricacies. You are in a position to prevent this from happening. You can explain, in simple terms, what the pros and cons of your solution are in a way that relates to them personally. You can listen to their questions and address their concerns. You can explain when your system works and when it doesn't and all the assumptions that you have made along the way.

As for you, you are probably lucky enough to be able to choose what work you do, or at least actively decide not to work on something or for someone that violates your moral principles. This is an unspoken benefit of opportunity, education, and engineering. The fact that you are in a position to read this book is a truly remarkable achievement. You can repay this debt by actively considering what is important to you and how your work affects the world. The following is a list of items that may or may not be important to you. Whenever you are developing an application, ask yourself:

Bias

Does this solution affect certain groups of people more than others? Are there aspects of your data that could accidentally add bias?

Personal impact

How does this affect individuals? What will they feel like when they see/use your solution? If people will be impacted, is there a way to mitigate this? Can you include humans in the decision loop?

Climate

How does your solution affect the climate? Is the application valuable enough to be worth the cost in energy?

Unintended use

How else could your application be used? Would you be uncomfortable if it was used in certain industries? Can you prevent that from happening?

Nefarious use

Is your solution, directly or indirectly, going to be used for an activity that you do not agree with?

Legality

Is it legal? Could it be deemed illegal in certain jurisdictions or countries?

Human rights

Could your solution violate legal or your definition of human rights? For example, could it affect the privacy of individuals?

Justification and explainability

How can you justify the actions of your algorithm? How much explanation do they need to provide to be deemed trustworthy?

Performance misinformation

Have you explained effects of your algorithm or potential performance in the clearest, most independent way possible? Is there a chance that others could misinterpret your results and be unduly penalized?

On a technical front, there are efforts to attempt to incorporate some of these ideas into machine learning and RL directly. For example, **ML-fairness-gym** is a set of components that build simple simulations that assess the long-term impacts of decision systems in social environments. For example, in countries around the world credit scores are used to rate how likely individuals are to repay debt. But the fact that companies use this measure encourages individuals to artificially inflate their credit scores through techniques such as credit utilization and even overstating affordability measures on applications. In turn, this makes it harder for people who don't engage in these tactics, those who don't have the capability or opportunity to access such information, to access credit when they need it.⁴⁷

The long-term consequences are so hard to predict that you might consider not doing it at all. But wait, you have access to a framework that is able to monitor and optimize for long-term sequential rewards. Yes, RL. Researchers have gone so far as to suggest that you can use RL to optimize and make ethical decisions, and even consider using reward shaping as a proxy for *ethics shaping*.^{48,49} Maybe RL will save the world?

Summary

To the best of my knowledge, this chapter is the first to provide cohesive and comprehensive advice on how to implement and deploy RL agents. There are many tidbits scattered throughout the research papers, but engineers, on average, have simply not had enough experience with RL in production. My hope is that this chapter, and indeed this book, will begin to change that.

Of course this also means that I am writing from the perspective of my experience and what I think is important. Feel free to take this advice and adapt, change, or totally ignore it as you see fit. The sheer breadth of applications means that the

circumstances of a project should dictate how these ideas can be applied and the value of doing so.

Despite the lack of production use (compared to machine learning, for example), there are already a wide range of frameworks to help you get started quickly. These range from algorithmic through to operational frameworks and also include specialized ancillary frameworks to implement a specific part of the RL problem. But it is likely, especially if going into production, that you will still need a lot of custom code, since RL is still young. Scaling RL is similarly supported on the training side by frameworks, and traditional software stacks and orchestrators should help you scale into production.

Evaluating agents is crucial, and could have been included in [Chapter 9](#). But I left it to here since it only really becomes crucial when you are looking to deploy your first agent. While you're in the proof-of-concept phase, you can usually get away with ad hoc evaluation with standard metrics and environment-specific debugging.

When you do find yourself wanting to deploy a project into a setting where other people can use it, you should take a step back and consider the operational aspects of running RL. Take the time to consider what you are trying to achieve and build an architecture that supports those goals. Don't be afraid to say that certain aspects are not important to save time, but do so only when considering the risks. To support the deployment you will need to leverage a wide range of software engineering and machine learning tooling. Take advantage of these and adapt them to your RL process.

Safe RL is an important topic for agents that interact with the real world and can directly affect algorithm implementations. Try to catch this requirement as early as you can, since popular algorithms may appear to work well, but could be unsafe. Securing your application is important here as it is in any project and there are RL-specific attack vectors to be aware of. But basic security practices should take priority, since these are the focus of most attacks.

And finally, consider how your application impacts ethical concerns, from an industry perspective, and moral concerns, from a personal perspective. This becomes more important and harder to do the higher you go in the decision stack. Ethical machine learning is hard enough to get right, but since RL is automating whole strategies, there is potential for long-term and wide-reaching consequences.

Further Reading

- Software engineering:
 - Martin, Robert C. 2008. *Clean Code: A Handbook of Agile Software Craftsmanship*. Pearson Education.

- McConnell, Steve. 2004. *Code Complete*. Pearson Education.
- Hunt, Andrew, and David Thomas. 1999. *The Pragmatic Programmer: From Journeyman to Master*. Addison-Wesley Professional.
- Psychology:
 - Kahneman, Daniel. 2012. *Thinking, Fast and Slow*. Penguin UK.
 - Baumeister, Roy F., and John Tierney. 2012. *Willpower: Rediscovering Our Greatest Strength*. Penguin UK.
- Evaluation:
 - Provost, F., and T. Fawcett. 2013. *Data Science for Business: What You Need to Know about Data Mining and Data-Analytic Thinking*. O'Reilly Media.
 - I found this old paper to be a useful resource on traditional RL evaluation methods.⁵⁰
- Explainability:
 - A good review of explainability, although biased toward medicine.⁵¹
 - Check out the [PyTorch Captum](#) and [TensorFlow tf-explain](#) projects for black-box explainability implementations.
 - Recent survey on explainability in RL.⁵²
- Microservices and deployment strategies:
 - Newman, Sam. 2021. *Building Microservices: Designing Fine-Grained Systems*. O'Reilly.
- Continuous integration and deployment:
 - Duvall, Paul M., Steve Matyas, and Andrew Glover. 2007. *Continuous Integration: Improving Software Quality and Reducing Risk*. Pearson Education.
 - Humble, Jez, and David Farley. 2010. *Continuous Delivery: Reliable Software Releases through Build, Test, and Deployment Automation*. Pearson Education.
- Safe RL:
 - A thorough survey, but a bit outdated.⁵³
 - Pfleeger, Charles P., and Shari Lawrence Pfleeger. 2012. *Analyzing Computer Security: A Threat/Vulnerability/Countermeasure Approach*. Prentice Hall Professional.
 - Excellent thesis on the subject by Vahid Behzadan.⁵⁴
- Ethics:
 - Nielsen, Aileen. 2020. *Practical Fairness*. O'Reilly.
 - The [Institute for Ethical AI & ML](#) is pioneering work in this area.

— I was really inspired by this paper on how ML can be used to tackle climate change.⁵⁵

References

- [1] [GitHub Archive Program](#). n.d. GitHub Archive Program. Accessed 26 July 2020.
- [2] Dhariwal, Prafulla, Christopher Hesse, Oleg Klimov, Alex Nichol, Matthias Plappert, Alec Radford, John Schulman, Szymon Sidor, Yuhuai Wu, and Peter Zhokhov. 2017. OpenAI Baselines. GitHub Repository. GitHub.
- [3] Hill, Ashley, Antonin Raffin, Maximilian Ernestus, Adam Gleave, Anssi Kanervisto, Rene Traore, Prafulla Dhariwal, et al. 2018. Stable Baselines. GitHub Repository. GitHub.
- [4] Albin Cassirer, Gabriel Barth-Maron, Thibault Sottiaux, Manuel Kroiss, Eugene Brevdo. 2020. Reverb: An Efficient Data Storage and Transport System for ML Research.
- [5] Espeholt, Lasse, Raphaël Marinier, Piotr Stanczyk, Ke Wang, and Marcin Michalski. 2020. “SEED RL: Scalable and Efficient Deep-RL with Accelerated Central Inference”. ArXiv:1910.06591, February.
- [6] Nair, Arun, Praveen Srinivasan, Sam Blackwell, Cagdas Alcicek, Rory Fearon, Alessandro De Maria, Vedavyas Panneershelvam, et al. 2015. “Massively Parallel Methods for Deep Reinforcement Learning”. ArXiv:1507.04296, July.
- [7] Dean, Jeffrey, Greg Corrado, Rajat Monga, Kai Chen, Matthieu Devin, Mark Mao, Marc’auelio Ranzato, et al. 2012. “Large Scale Distributed Deep Networks”. *Advances in Neural Information Processing Systems* 25, edited by F. Pereira, C. J. C. Burges, L. Bottou, and K. Q. Weinberger, 1223–1231. Curran Associates, Inc.
- [8] Mnih, Volodymyr, Adrià Puigdomènech Badia, Mehdi Mirza, Alex Graves, Timothy P. Lillicrap, Tim Harley, David Silver, and Koray Kavukcuoglu. 2016. “Asynchronous Methods for Deep Reinforcement Learning”. ArXiv:1602.01783, June.
- [9] Niu, Feng, Benjamin Recht, Christopher Re, and Stephen J. Wright. 2011. “HOG-WILD!: A Lock-Free Approach to Parallelizing Stochastic Gradient Descent.” ArXiv: 1106.5730, November.
- [10] Clemente, Alfredo V., Humberto N. Castejón, and Arjun Chandra. 2017. “Efficient Parallel Methods for Deep Reinforcement Learning”. ArXiv:1705.04862, May.
- [11] Horgan, Dan, John Quan, David Budden, Gabriel Barth-Maron, Matteo Hessel, Hado van Hasselt, and David Silver. 2018. “Distributed Prioritized Experience Replay”. ArXiv:1803.00933, March.

- [12] Wijmans, Erik, Abhishek Kadian, Ari Morcos, Stefan Lee, Irfan Essa, Devi Parikh, Manolis Savva, and Dhruv Batra. 2020. “DD-PPO: Learning Near-Perfect Point-Goal Navigators from 2.5 Billion Frames.” ArXiv:1911.00357, January.
- [13] Stooke, Adam, and Pieter Abbeel. 2019. “Accelerated Methods for Deep Reinforcement Learning”. ArXiv:1803.02811, January.
- [14] OpenAI. 2018. “OpenAI Five” 2018.
- [15] Espeholt, Lasse, Hubert Soyer, Remi Munos, Karen Simonyan, Volodymyr Mnih, Tom Ward, Yotam Doron, et al. 2018. “IMPALA: Scalable Distributed Deep-RL with Importance Weighted Actor-Learner Architectures”. ArXiv:1802.01561, June.
- [16] Espeholt, Lasse, Raphaël Marinier, Piotr Stanczyk, Ke Wang, and Marcin Michalski. 2020. “SEED RL: Scalable and Efficient Deep-RL with Accelerated Central Inference” ArXiv:1910.06591, February.
- [17] Hung, Chia-Chun, Timothy Lillicrap, Josh Abramson, Yan Wu, Mehdi Mirza, Federico Carnevale, Arun Ahuja, and Greg Wayne. 2019. “Optimizing Agent Behavior over Long Time Scales by Transporting Value”. *Nature Communications* 10(1): 5223.
- [18] Colas, Cédric, Olivier Sigaud, and Pierre-Yves Oudeyer. 2019. “A Hitchhiker’s Guide to Statistical Comparisons of Reinforcement Learning Algorithms”. ArXiv:1904.06979, April.
- [19] Colas, Cédric, Olivier Sigaud, and Pierre-Yves Oudeyer. 2018. “How Many Random Seeds? Statistical Power Analysis in Deep Reinforcement Learning Experiments”. ArXiv:1806.08295, July.
- [20] Lee, Xian Yeow, Aaron Havens, Girish Chowdhary, and Soumik Sarkar. 2019. “Learning to Cope with Adversarial Attacks”. ArXiv:1906.12061, June.
- [21] Osband, Ian, Yotam Doron, Matteo Hessel, John Aslanides, Eren Sezener, Andre Saraiva, Katrina McKinney, et al. 2020. “Behaviour Suite for Reinforcement Learning”. ArXiv:1908.03568, February.
- [22] Gulcehre, Caglar, Ziyu Wang, Alexander Novikov, Tom Le Paine, Sergio Gomez Colmenarejo, Konrad Zolna, Rishabh Agarwal, et al. 2020. “RL Unplugged: Benchmarks for Offline Reinforcement Learning”. ArXiv:2006.13888, July.
- [23] James, Stephen, Zicong Ma, David Rovick Arrojo, and Andrew J. Davison. 2019. “RLBench: The Robot Learning Benchmark & Learning Environment.” ArXiv:1909.12271, September.
- [24] Leike, Jan, Miljan Martic, Victoria Krakovna, Pedro A. Ortega, Tom Everitt, Andrew Lefrancq, Laurent Orseau, and Shane Legg. 2017. “AI Safety Gridworlds”. ArXiv:1711.09883, November.

- [25] Liu, Guiliang, Oliver Schulte, Wang Zhu, and Qingcan Li. 2018. “Toward Interpretable Deep Reinforcement Learning with Linear Model U-Trees”. ArXiv:1807.05887, July.
- [26] Yoon, Jinsung, Sercan O. Arik, and Tomas Pfister. 2019. “RL-LIM: Reinforcement Learning-Based Locally Interpretable Modeling.” ArXiv:1909.12367, September.
- [27] Madumal, Prashan, Tim Miller, Liz Sonenberg, and Frank Vetere. 2019. “Explainable Reinforcement Learning Through a Causal Lens”. ArXiv:1905.10958, November.
- [28] Atrey, Akanksha, Kaleigh Clary, and David Jensen. 2020. “Exploratory Not Explanatory: Counterfactual Analysis of Saliency Maps for Deep Reinforcement Learning”. ArXiv:1912.05743, February.
- [29] Agarwal, Alekh, Sarah Bird, Markus Cozowicz, Luong Hoang, John Langford, Stephen Lee, Jiaji Li, et al. 2017. “Making Contextual Decisions with Low Technical Debt”. ArXiv:1606.03966, May.
- [30] Merentitis, Andreas, Kashif Rasul, Roland Vollgraf, Abdul-Saboor Sheikh, and Urs Bergmann. 2019. “A Bandit Framework for Optimal Selection of Reinforcement Learning Agents”. ArXiv:1902.03657, February.
- [31] Lee, Kimin, Michael Laskin, Aravind Srinivas, and Pieter Abbeel. 2020. “SUN-RISE: A Simple Unified Framework for Ensemble Learning in Deep Reinforcement Learning”. ArXiv:2007.04938, July.
- [32] Matsushima, Tatsuya, Hiroki Furuta, Yutaka Matsuo, Ofir Nachum, and Shixiang Gu. 2020. “Deployment-Efficient Reinforcement Learning via Model-Based Offline Optimization”. ArXiv:2006.03647, June.
- [33] Tian, Huangshi, Minchen Yu, and Wei Wang. 2018. “Continuum: A Platform for Cost-Aware, Low-Latency Continual Learning.” *Proceedings of the ACM Symposium on Cloud Computing*, 26–40. SoCC ’18. New York, NY, USA: Association for Computing Machinery.
- [34] García, Javier, and Fernando Fernández. 2015. “A Comprehensive Survey on Safe Reinforcement Learning.” *The Journal of Machine Learning Research* 16(1): 1437–1480.
- [35] Ha, Sehoon, Peng Xu, Zhenyu Tan, Sergey Levine, and Jie Tan. 2020. “Learning to Walk in the Real World with Minimal Human Effort”. ArXiv:2002.08550, February.
- [36] Eysenbach, Benjamin, Shixiang Gu, Julian Ibarz, and Sergey Levine. 2017. “Leave No Trace: Learning to Reset for Safe and Autonomous Reinforcement Learning.” ArXiv:1711.06782, November.

- [37] Wabersich, Kim P., Lukas Hewing, Andrea Carron, and Melanie N. Zeilinger. 2019. “Probabilistic Model Predictive Safety Certification for Learning-Based Control.” ArXiv:1906.10417, June.
- [38] Gros, Sebastien, Mario Zanon, and Alberto Bemporad. 2020. “Safe Reinforcement Learning via Projection on a Safe Set: How to Achieve Optimality?”. ArXiv:2004.00915, April.
- [39] Hunt, Nathan, Nathan Fulton, Sara Magliacane, Nghia Hoang, Subhro Das, and Armando Solar-Lezama. 2020. “Verifiably Safe Exploration for End-to-End Reinforcement Learning”. ArXiv:2007.01223, July.
- [40] Goodfellow, Ian J., Jonathon Shlens, and Christian Szegedy. 2015. “Explaining and Harnessing Adversarial Examples.” ArXiv:1412.6572, March.
- [41] Papernot, Nicolas, Patrick McDaniel, Ian Goodfellow, Somesh Jha, Z. Berkay Celik, and Ananthram Swami. 2017. “Practical Black-Box Attacks against Machine Learning”. ArXiv:1602.02697, March.
- [42] Behzadan, Vahid, and Arslan Munir. 2017. “Whatever Does Not Kill Deep Reinforcement Learning, Makes It Stronger”. ArXiv:1712.09344, December.
- [43] Lin, Yen-Chen, Ming-Yu Liu, Min Sun, and Jia-Bin Huang. 2017. “Detecting Adversarial Attacks on Neural Network Policies with Visual Foresight”. ArXiv:1710.00814, October.
- [44] Behzadan, Vahid, and William Hsu. 2019. “Analysis and Improvement of Adversarial Training in DQN Agents With Adversarially-Guided Exploration (AGE)”. ArXiv:1906.01119, June.
- [45] Havens, Aaron J., Zhanhong Jiang, and Soumik Sarkar. 2018. “Online Robust Policy Learning in the Presence of Unknown Adversaries”. ArXiv:1807.06064, July.
- [46] *Internet Security Threat Report* (Volume 24). 2019. Symantec.
- [47] Milli, Smitha, John Miller, Anca D. Dragan, and Moritz Hardt. 2018. “The Social Cost of Strategic Classification.” ArXiv:1808.08460, November.
- [48] Abel, David, James MacGlashan, and Michael L Littman. 2016. “Reinforcement Learning as a Framework for Ethical Decision Making.” *Workshops at the Thirtieth AAAI Conference on Artificial Intelligence*.
- [49] Yu, Han, Zhiqi Shen, Chunyan Miao, Cyril Leung, Victor R. Lesser, and Qiang Yang. 2018. “Building Ethics into Artificial Intelligence”. ArXiv:1812.02953, December.
- [50] Kaelbling, L. P., M. L. Littman, and A. W. Moore. 1996. “Reinforcement Learning: A Survey”. ArXiv:Cs/9605103, April.

- [51] Tjoa, Erico, and Cuntai Guan. 2020. “A Survey on Explainable Artificial Intelligence (XAI): Towards Medical XAI”. ArXiv:1907.07374, June.
- [52] Puiutta, Erika, and Eric MSP Veith. 2020. “Explainable Reinforcement Learning: A Survey”. ArXiv:2005.06247, May.
- [53] García, Javier, and Fernando Fernández. 2015. “A Comprehensive Survey on Safe Reinforcement Learning.” The Journal of Machine Learning Research 16(1): 1437–1480.
- [54] Behzadan, Vahid. 2019. “Security of Deep Reinforcement Learning.” Manhattan, Kansas: Kansas State University.
- [55] Rolnick, David, Priya L. Donti, Lynn H. Kaack, Kelly Kochanski, Alexandre Lacoste, Kris Sankaran, Andrew Slavin Ross, et al. 2019. “Tackling Climate Change with Machine Learning.” ArXiv:1906.05433, November.

Conclusions and the Future

At this point you might be expecting (and be glad) that this is the end of the book. Not quite, because during my time writing this book I collected a smorgasbord of tidbits and ideas for future work. In the first half, I delve into some tips and tricks that I have accumulated that didn't fit into other parts of the book. In the second half I outline the current challenges and provide direction for future research.

Tips and Tricks

I've said this a few times now: RL is hard in real life because of the underlying dependencies on machine learning and software engineering. But the sequential stochasticity adds another dimension by delaying and hiding issues.

Framing the Problem

Often you will have a new industrial problem where the components of the Markov decision process are fuzzy at best. Your goal should be to refine the concept and prove (at least conceptually) the viability using RL for this problem.

To start, try visualizing a random policy acting upon a notional environment. Where does it go? What does it interact with? When does it do the right thing? How quickly does it find the right thing to do? If your random policy never does the right thing, then RL is unlikely to help.

After trying a random policy, try to solve the problem yourself. Imagine *you* are the agent. Could you make the right decisions after exploring every state? Can you use the observations to guide your decisions? What would make your job easier or harder? What features could you generate from these observations? Again, if you can't notionally come up with viable strategies, then RL (or machine learning, for that matter) is unlikely to work.

Use baselines to frame performance. Start with a random agent. Then use a cross-entropy method (which is a simple algorithm that recalls the best previous rollout from random exploration). Then try a simple Q-learning or policy gradient algorithm. Each time you implement a new algorithm, treat it like a scientific experiment: have a theory, test that theory, learn from the experiment, and use it to guide future implementations.

Simplify the task as much as humanly possible, then solve that first. Go for the quick wins. By *simplify*, I mean reducing the state and action space as much as practicable. You can hand engineer features for this or use your domain knowledge to chop out parts of the state that don't make sense. And simplify the goal or the problem definition. For example, rather than trying to pick up an object with a robot, try training it to move to a position first. And simplify the rewards, too. Always try to remove the sparsity by using metrics like the distance to the goal, rather than a positive reward when the agent reaches the goal. Try to make the reward smooth. Remove any outliers or discontinuities; these will make it harder for your model to approximate.

Ask yourself whether you need to remember previous actions to make new ones. For example, if you need a key to open a door, then you need to remember that you have picked up a key. If you want to place an item in a box, you need to pick it up first. These situations have states that must be visited in a specific order. To provide this capability you should incorporate a working memory, and one common implementation is via recurrent neural networks.

Don't confuse this with two actions, though. It's easy to fall into the trap of thinking that you need memory, but in fact you have actions to reach these states directly. For example, you would imagine that before moving forward in a car you need to turn the engine on. This sounds like ordered tasks again, but it is not, because you probably have separate actions to turn the car on and move forward. An agent without memory is fully capable of learning to keep the car turned on in order to move forward. This highlights a way of removing the need for memory, too. If you can find an action that provides a shortcut to this state, then you might be able to do away with the working memory.

Also remember hierarchical policies at this point. Consider if your application has disparate "skills" that could be learned to solve the problem. Consider solving them independently if you can, which is conceptually and practically simpler than using hierarchical RL and the end result may be the same.

Your Data

I think the vast majority of development time is spent retraining models after tweaking some hyperparameter or altering features, for example. For this reason, and this almost goes without saying, you should try to do things right the first time. This only

comes with experience: experience with RL in general, in your domain, and with your problem. But here are a few tips that should provide a bit of a shortcut.

Like in machine learning, scale and correlation is really important. Not all algorithms *need* rescaling (trees, for example), but most do, and most expect normally distributed data. So in general you should always rescale your observations to have a mean of zero and a standard deviation of one. You can do this with something as simple as a rolling mean and standard deviation, or if your input is stationary or bounded, you can use those observed limits. Be aware that the statistics of your data may change over time and make your problem nonstationary.

Also try to decorrelate and clean your observations. Models don't like correlation in time or across features, because they represent complex biases and duplicate information. You can use the z-transform, principal component analysis, or even a simple diff between observations to decorrelate single features. You use your data analysis skills to find and remove correlations between observations.

Monitor all aspects of your data. Look at the reward distributions; make sure there are no outliers. Look at the input data, again, plot distributions or visualizations, and make sure the input data isn't wild. Look at the predictions being made by various parts of the model. Is the value function predicting well? What about the gradients? Are they too big? Look at the entropy of the action space: is it exploring well, or is it stuck on certain actions? Look at the Kullback–Liebler divergence: is it too big or too small?

Think about increasing increasing the amount of discretization, in other words, increase the time between actions. This can make training more stable, because it's less affected by random noise, and speed it up, because there are fewer actions to compute. The de facto frame skipping in the Atari environments is an example of this. A good rule of thumb is to imagine yourself picking actions; at what rate do you need to make decisions to produce a viable policy?

Training

Training can be quite a demoralizing process. You think you have a good idea only to find that it doesn't make any difference and it took you a day to implement and retrain. But at least this is visible. One of the biggest issues in RL right now is that it is easy to overfit the development of an algorithm to your specific environment and therefore produce a policy that doesn't generalize well.

Try to develop your algorithm against a range of environments. It is very easy to overfit your development to the environment you use for ad hoc testing. This takes discipline, but try to maintain a set of representative environments, from simple to realistic, that also have different setups, like larger or smaller, easier or harder, or fully

observable or partially observable. If your algorithm works well across all of them, then that gives you confidence that it is able to generalize.

Use different seeds. I've sometimes thought I was doing well in an environment, but it turned out that one specific seed was massively overfitting and happened to stumble across a viable policy purely by chance. When you use different seeds, this forces your agent to explore in a slightly different way than it did before. Of course, your algorithm should perform similarly irrespective of the seed value, but don't be surprised if it doesn't.

Test the sensitivity of your hyperparameters. Again, you should be aiming for an algorithm that is robust and degrades gracefully. If it works only with a very specific set of hyperparameters then this is an indication of overfitting again.

Play with the learning rate schedules. A fixed value rarely works well because it encourages the policy to change, even when it is doing well. This can have the effect of knocking the agent away from an optimal policy.

Evaluation

Evaluation tends to be one of the least problematic phases because you should already have a good idea of how well things are going from all the monitoring you did during training. But there is still enough fuel left to get burnt.

Be very careful drawing conclusions when comparing algorithms. You can use statistics to quantify improvement (see [“Statistical policy comparisons” on page 310](#)) but I find that I make fewer mistakes when visualizing performance. The basic idea is that if you can see a significant performance increase, where there is clear space between the performance curves over many different seeds, then it's quite likely to be better. If you don't see a gap, then run more trials with more seeds or simply don't trust it. This sounds simple enough but in really complex environments it is not. Often, through no fault of your algorithm, you can end up with a really bad run. This can happen in a complex environment where a bad initial exploration led to states that are really hard to get out of. Take a look at the states that your agent is choosing to explore; you might want to alter the exploration strategy.

No one measure or plot can truly prove performance. I find that a holistic combination of experience with that domain and environment and intuition about how an algorithm *should* behave dictates whether I believe there to be a change in performance. This is why it is important to have baselines to compare yourself against, because it provides another piece of evidence.

Remember that some policies are stochastic by default—many policy gradient algorithms, for example—so evaluate using deterministic versions of these policies if you want a pure performance measure. Similarly, if you have any random exploration in your policy— ϵ -greedy, for example—make sure you disable it. Even though you

might disable it during evaluation, consider retaining the stochasticity in the real environment if there is a chance that the optimal policy could change over time.

Deployment

I think the key to quickly developing successful applications is defined by the ability to combine off-the-shelf components and modules to solve a particular problem. This necessitates a certain amount of composability, which is primarily driven at the software level through good coding practices. The idea is that you look at your problem, pick the components that would help—intrinsic motivation or a replay buffer for example—stick them all together through nicely encapsulated interfaces, and deploy them in unison. At the moment, unfortunately, this dream is a long way away. Any production deployment will involve significant software engineering and DevOps expertise.

Frameworks help here, too, because many of them have the desire to be composable. But that usually means that they are composable within the bounds of their codebase. Frameworks often struggle to interact with external components because of a lack of standards. OpenAI’s Gym is an obvious exception to this.

So think hard about your architecture, because I think this is the key to successful deployment and operation of a real-life product.

Debugging

Despite RL algorithms fueling much of the publicity surrounding artificial intelligence, deep RL methods are “brittle, hard to reproduce, unreliable across runs, and sometimes outperformed by simple baselines.”¹ The main reason for this is that the constituent parts of an algorithm are not well understood: how do the various parts of an implementation impact behavior and performance?

Implementation details are incredibly important. Engstrom et al. find that seemingly small modifications like code-level optimizations can explain the bulk of performance in many popular algorithms. For example, despite the claim that PPO is performant due to update clipping or constraints, these researchers proved that the code-level optimizations improved performance *more* than the choice of algorithm (they compared TRPO to PPO).²

This leads to the obvious conclusion that bugs impact performance. For example, during OpenAI’s random network distillation research (see “[Random embeddings \(random distillation networks\)](#)” on page 280), Cobbe et al. noticed that seemingly small details made the difference between an agent that gets stuck and one that can solve a difficult Atari game. They noticed one issue where the value function approximation, which should be stable after enough training, was oscillating. It turned out that they were accidentally zeroing an array that was used to produce the curiosity

bonus, which meant that the value function was pulled lower at the start of each episode. This explained the oscillation and, once fixed, dramatically improved performance due to the stable value estimates.³

So the question is, how do you prevent these bugs? Well, every single line of code introduces a possibility of a bug, so the only way to eliminate all bugs is to have no code at all. I preach this mantra a lot during my productization projects with Winder Research, which also has the added benefit of reducing the operational burden. Obviously you need code, but the point I'm making is that less is better. If you have an option of using an off-the-shelf library or component, then use it, even if it doesn't quite meet all your requirements. Similarly, if you are writing code or developing a framework, prefer less code over fancy design. This is why I love languages like Go or Python. They actively promote simplicity because they don't have all the so-called sophistication of a language like C++ or Java.

Beyond this, the normal debugging practices of tracing, visualizing your data, testing, and `print` statements will become your day-to-day experience of RL. At its heart, debugging is just a manifestation of the scientific method: you observe something strange (visualizations are really important), you develop a theory to explain the behavior, you develop ways to test that theory (`print` statements), and you evaluate the results. If you want to learn more, pick up a book on debugging software; some recommendations are in [“Further Reading” on page 355](#).

RL is sequential, which means that bugs might not manifest for some time, and RL is stochastic, which means that bugs can be hidden by the noise of the learning process. Combine this with the underlying dependencies of machine learning and software engineering and you end up with a volatile witch's cauldron of bubbling bugs.

If you were to speak to your project manager and estimate how long you would spend developing an algorithm for a new problem, you would estimate something like the top part of [Figure 11-1](#); most of the time is spent on research and implementation, right? No. The vast majority of time is spent debugging, attempting to figure out why your agent can't solve the simplest thing. The bottom part of [Figure 11-1](#) shows something more representative, with the added time that is required to reimplement your code properly, with clean code and tests. The measurements I picked are arbitrary, but they are representative, in my experience.

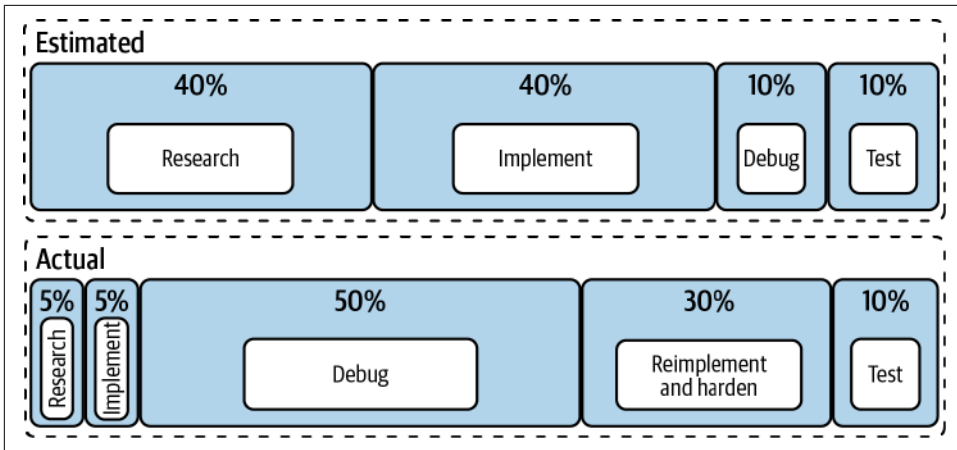


Figure 11-1. A comparison of expected versus actual time spent on activities during algorithm development. Inspired by Matthew Rahtz.⁴

{ALGORITHM_NAME} Can't Solve {ENVIRONMENT}!

Even though these algorithms can achieve amazing feats, with examples that you might even consider intelligent, it is important to remember that they are not magic. They cannot solve every problem and they certainly don't do it automatically.

If you are using RL in industry, you must take a holistic view. It is very likely that you can do something to dramatically simplify the problem. The main reason why RL methods fail is not because the algorithm isn't good enough, but because the problem isn't suited to being solved in this way. For example, the *MountainCar-v0* environment is notoriously difficult for RL algorithms to solve, despite being conceptually simple. This is not a problem with the algorithm; the value function is surprisingly simple. This is an exploration and a credit assignment problem. If you developed a better way to explore or changed the reward to encourage movement up the hill, any of these algorithms can solve the problem in no time.

There is no single solution to solving environments either. Sometimes the problem lies with the observation space, sometimes the exploration, sometimes a hyperparameter. Welcome to RL, where intelligent agents automate complex strategic goals, sometimes. My only recommendation is to use this book to guide you, one experiment at a time, toward a tractable problem. If you forced me to pick one, I'd start with a simplification of the problem or observation space. Get it simple enough so you can visualize what is going on and then solve that simple problem. With that in your bag, you'll probably have enough domain knowledge to know the issues with the more complex version.

Monitoring for Debugging

Monitoring in production, for operational purposes, does not provide enough low-level information to help you debug an RL algorithm. When you are developing your agent you will want to log, plot, and visualize as much as you can. Following is a list of tips to help you when you need more information:

- Log and visualize observations, actions, and rewards. Actions should have high entropy to encourage exploration. The rewards for states should not be the same; if they are the agent can't tell which states are more valuable. Check the observation presented to the agent; it's easy to accidentally throw away important information. For example, check that a grayscale transformation of an image preserves features within that image; green and red traffic lights are clearly different in color, but not so much in grayscale.
- Evaluate the value function for different parts of your state space. If you can, visualize the whole space. If not, pick some representative states and plot them. Is it overfitting?
- Plot the value function over time (for a reasonable number of states). How does it change? Does it get stuck? Does it oscillate? What is the variance during a run? What is the variance over several runs? The estimate should alter over time and then stabilize. Remember that when the policy changes, the perfect value function changes, too. The value function approximation should remain stable enough to cope with the nonstationarity, but flexible enough to react to changes.
- Poor or buggy neural network implementations make RL look bad. Start with something simple, a linear approximator or equivalent single-layer neural network. Make sure everything works as expected before trying more complex networks. Look at a good practical deep learning book for help with this (see [“Further Reading” on page 355](#)).
- Make sure everything works with fixed seeds and single threading, before you scale up to multiple seeds or multithreading/multinode runs.
- Start with hyperparameters that are known to work well from your past experience with other implementations.

The Future of Reinforcement Learning

What is the current opinion of RL right now? This makes me think of Andrew Ng's almost seminal talk on practical deep learning, where he manages to perfectly balance a healthy dose of short-term cynicism with long-term optimism.⁵ This left me appreciating the complexities of the subject, especially when it comes to industrial implementations, but at the same time the optimism made these issues feel like it was just part of the process. The challenges are there, so accept them, deal with them in the

best way you can, and move on. Sometimes deep learning doesn't make sense, and that's OK. But when it does, the confluence of model and data are a beauty to behold.

I think RL is in this position right now. We're on the cusp of where industrial production and operationalization are possible, but tricky. There are still many cases where simpler algorithms outperform RL; for example, if you are able to play your environment offline, then you could just use a Monte Carlo tree search (MCTS) algorithm, like AlphaGo, to evaluate your possible moves. For example, an experiment in 2014 smashed the performance of DQN using MCTS and it took some time before RL methods could catch up.⁶ But this is OK. RL isn't a silver bullet, just like machine learning hasn't had babies and created artificial intelligence yet. It's just a tool. And like a carpenter, you should use the best tool for your job.

RL Market Opportunities

Deep learning has become mainstream, so even my enterprise clients are investing in operationalization. I believe that RL will follow within the next few years. This opens up new markets and opportunities. Unfortunately, like machine learning, RL is a cross-industry technique, not a solution. You will need domain expertise, industry-specific market awareness, and a dollop of inspiration to exploit RL.

There are already signs that RL is especially well suited to specific domains, however. Robotics is one obvious one. But automated bidding and recommendations are others that could overtake robotics in terms of global revenue. There hasn't been any industry-wide research specific to RL yet, but that will come if RL is successful.

The market research firm Gartner reckons that RL is still on the rise of the hype curve and that a third of enterprises are looking to automate strategic decision making (which they call decision intelligence) by 2023.^{7,8} They are suitably vague about market values, especially when it comes to quantifying the impact of specific technologies, but the worldwide value to business is reported to be many trillions of dollars per year.⁹ Interestingly, in some of Gartner's visualizations, they are beginning to separate "agents" and "decision automation" from other forms of more traditional machine learning, which they call "decision support/augmentation," so they can clearly see the emerging value of optimizing sequential decisions.

This promise means that there is value and opportunity in the operational space. Companies won't be able to use the current crop of machine-learning-as-a-service offerings because most are not well suited toward continuous learning and sequential decision making. Some will adapt, of course, but there are opportunities for companies to fill this space.

Open source projects won't be far behind, either, and could even be in front in many cases. There is an argument to be made that only open source projects can be truly successful because, like machine learning, global adoption needs open collaboration.

Otherwise your needs and challenges might conflict with proprietary processes and opinions. If you are considering this, make sure you position your product some distance away from what large vendors might be interested in. And make sure you reflect on how you and the global community can mutually gain from open source development.

This also means that companies will need RL support and expertise for a long time into the future. It could be as much as 10 years before RL is a commodity. So consultants and engineers with demonstrable RL expertise, like those from my company, will be sought after. If you are thinking about specializing in RL as a career, this would not be a bad decision, I don't think, because of the obvious opportunities and projections just discussed. But don't let this lull you into a false sense of security. RL, ML, and software engineering is hard, especially when you try to move these things into production. And like in any knowledge-based industry, the goal posts are constantly moving. You have to accept this and move with them to assure a long-term position in this highly competitive field. To put in this much effort you really have to love it. You have to enjoy the craft and the challenge of adversity, the challenge of being met with a specific industrial situation that nobody, ever, has dealt with before.

Future RL and Research Directions

RL has many challenges, as I have discussed throughout the book. Don't let these prevent you from exploring the potential of RL, because I truly believe that the next level of artificial intelligence is going to be driven by these ideas; they are just waiting for breakthrough applications. But hopefully I have clearly marked which parts of RL can slow you down or cause you to stumble when working on an industrial problem. These problems mostly translate into future research opportunities.

The research directions can be broadly split in two: industrial and academic research. Industrial research is comprised of ways to improve the development and operational efficiency and is driven by business needs. Academic research focuses more on improving or redefining fundamental understanding and is driven by a theoretical notion of novelty or advancement, as defined by their peers. This means that academic researchers are not going to invest their time in improving their code's readability, for example. Likewise, industrial researchers are unlikely to stumble across anything truly groundbreaking. But of course, there are people that sit in between.



I believe that there is a problem in the academic machine learning community at the moment. The primary venues for academic success are conferences and journals. The most popular can afford to pick and choose, so reviewers have a lot of power to direct current and future content. And unfortunately, applications are deemed to be of lesser value than, for example, a novel algorithmic improvement. For example, Hannah Kerner has recently spoken about her experience trying to push application-focused papers to conferences, only to find that “the significance seems limited for the machine-learning community.”¹⁰ Of course, fundamental improvement is the primary goal for academia and scientific discovery in general, but I feel like the scale has tipped too far.

Research in industry

Starting from an industrial perspective, these are the areas in which I think there are research opportunities and momentum:

Operationalization and architecture

RL is being used more and soon there will be a glut of applications moving into production. I don't think that any of the current machine learning frameworks have the capability and the RL frameworks tend to focus on development, not production. Of course, there are a few exceptions, but there is a lot of room for improvement, particularly on the architecture side.

RL design and implementation

There is already a lot of work in this area—see all the RL frameworks for an example—but many of them tend to focus on the wrong problem. Many of them are attempting to implement RL algorithms, but this is not the goal for an industrial problem. The goal is to find, refine, and build an RL solution to a business challenge. This means that things like feature engineering, reward shaping, and everything else listed in the previous two chapters are also important, possibly more so, than the algorithm. The frameworks need to shift their focus away from algorithm development and toward application development.

Environment curation and simulation

Simulators are obviously crucial for initial development. Accurate simulations provide the ability to transfer knowledge to the real world. As more complex industrial problems are attempted, these will benefit from more realistic simulations. This is already the case for 3D simulation engines, which are capable of simulating scarily good representations of the real world. But for all the other domains, decent, realistic simulators are hard to come by. One reason for this is that they contain a lot of proprietary information. The models that drive the simulations are often built from data that the company owns, so they know that this represents a competitive advantage. One way research could help here is to

improve the curation and development process of an environment. This side-steps the data issue but will still help this crucial phase of development.

Monitoring and explainability

The tooling and support surrounding monitoring RL agents is lacking and can be attributed to the lack of operationalization. When in production you need to be certain that your policy is robust and the only way to be sure of that is through adequate oversight. Some applications may even demand explainability, which is quite tricky at the moment, especially in production settings. But more than that, you need tools to help debug the development of applications, tools that give you a view of what is happening inside the agent. The problem with this is that it tends to be algorithm and domain specific, but it is still a worthwhile challenge.

Safety and robustness

This is a hot topic in academic research, too, but the impact of the research is only felt when it is applied to a real-life application. Proving the robustness of your model is important for every application, so I envisage libraries and tools to help solve that problem. Safety is a much bigger concern. I'm less worried about the technical aspects, because I'm confident that engineers can create solutions that are safe by design. I'm concerned about the legal and regulatory impacts, because at this point it becomes more of an ethical problem. Who is liable? How safe does it need to be? How do you test how safe it is?

Applications

And of course, businesses are paying for the application of RL. The research directions discussed here help in delivering that application. There will be a lot of interest and therefore research in applying RL to business challenges. The hard part, like with any technology, is mapping the technology to a business challenge. This is a lot harder than it sounds, because RL is so general; there are many possible applications. So use the suggestions presented in **“Problem Definition: What Is an RL Project?” on page 254** to find a viable RL problem and then prioritize by business value.

Research in academia

From an academic perspective, RL is really exciting, because it feels like we're only a few steps away from something truly revolutionary: an agent that can learn any task as well as a human. This moment is like a bulkhead; once it is breached, this opens up a whole world of possibilities, mostly good, some scary.

Current applications of machine learning and, to a lesser extent, RL, are limited by their inability to utilize the scientific method and retain that knowledge in an intelligent way. If an agent was capable of developing its own theories, testing its own beliefs, and can apply that knowledge to discover new theories, then I think that is the moment where we can generalize RL to solve any sequential problem on any data.

And I feel like we're almost there. But to be specific, the challenges that are preventing this goal, from an application perspective, are:

Offline learning

This is probably the biggest challenge. The primary reason why we need simulators is because the agent must test an action to learn a policy. If we can move away from this regime, training a policy without that interaction, this means we can remove the simulator entirely. Just capture log data and train upon that, just like in machine learning. Researchers have already begun to improve this process; look back at [Chapter 8](#), for example. But there is a lot left to be desired. Intuitively, I don't think it is unreasonable to imagine an algorithm that is capable of understanding the stochasticity of an environment, observing examples of an optimal trajectory, then applying a model to fit a generalized trajectory to the data. This should provide a compromise between finding an optimal policy and retaining enough uncertainty about the states not observed from the log.

Sample efficiency

Improving the sample efficiency of algorithms is desirable because applications generally want an optimal policy in the shortest number of interactions. Consider the shopping cart example: I really don't want to get bags of dog and cat food delivered when I don't have any pets. The faster it can learn this preference, the less upset I'm going to be. Again, there are a range of research directions here, from improving exploration to consolidating knowledge, like ensembles, but these are often driven by the domain. It would be great if there was some universal, theoretical efficiency guarantee that algorithms could work toward.

High-dimensional efficiency

Large state and action spaces are still problematic. The large state spaces are awkward because of the amount of computational energy required to reduce it to something useful. I imagine that for certain domains pretrained transformers or embeddings could be used to simplify the state space without having to train the reduction. A large action space is more problematic because agents have to actively sample all available actions at all times to ensure they have an optimal policy. This is easier when actions are related. For example, a single continuous action is infinite, but it is quite easy to solve because you can model the action with a distribution. But many distinct actions cause more of a problem, mainly because this yields a more difficult exploration problem.

Reward function sensitivity

Supervised machine learning is easy because you know what the goal is. I would *love* to have such a simple way of defining goals in RL. RL rewards are often sparse, have multiple objectives, and are highly multidimensional. To this day defining a reward function is largely an exhaustive manual process, even with techniques such as intrinsic rewards. In the future I still expect some manual

intervention to define the high-level goals, but I expect to do this in a less invasive way; I shouldn't need to shape the reward. This becomes even more important when rewards are delayed.

Combining different types of learning

Is there a way to combine the exploratory benefits of multi-agent RL, the generalization of hierarchical RL, and expert guidance? Could you combine evolutionary or genetic learning with these RL techniques?

Pretrained agents and improving curriculum learning

I once read, I can't remember where, that learning is only possible when the gap between knowing and not knowing is small. I believe this to be true in RL, too. I don't expect a child to be able to drive a car; similarly, you should not expect an untrained agent to either. And overfitting the training to that one task is not the solution, since driving a car is more than just learning how to incorporate complex stereo vision and move a wheel. The agent needs a breadth of knowledge that can be provided by agents pretrained on specified curricula. You could even give them a grade at the end of it if you like.

Increased focus on real-life applications and benchmarks

I think that academia understands that real-life applicability is desirable, but I would like to see more emphasis placed on this. I can see algorithms that are overfitting the Atari benchmarks and that is by design, to be able to claim state-of-the-art performance.

Ethical standards

Ethical or moral machine learning and RL crosses into both industry and academia. There seems to be a melting pot of desire for advancing the discussion, but this is proving difficult. Maybe because it always turns into a philosophical debate or maybe because everyone has a different set of morals or contexts that makes it difficult to agree on what ethical conclusions to make. It's a really sensitive subject that I don't feel qualified to comment on. But that doesn't mean that you shouldn't consider it; you should. In everything you do.

Regarding the future, I don't feel like there has been a flood of support for even attempting to define ethical standards, let alone solving them. The General Data Protection Regulation (GDPR) is one example of a governmental attempt to define the ethical use of someone's data. And although I totally agree with the idea, in practice it is too vague and has too much legal jargon to be useful. For example, article 7, conditions for consent of the GDPR, states:

Consent shall be presented in a manner which is clearly distinguishable from the other matters, in an intelligible and easily accessible form, using clear and plain language.

Fair enough. But in that same exact page, point 4 reads:

When assessing whether consent is freely given, utmost account shall be taken of whether, *inter alia*, the performance of a contract, including the provision of a service, is conditional on consent to the processing of personal data that is not necessary for the performance of that contract.

Clear and plain language? And as evidence of the complete disregard for the letter of this law, the clarifying recital number 32, conditions for consent, states “pre-ticked boxes or inactivity should not therefore constitute consent.” I would hazard a guess at suggesting that nearly all GDPR opt-in forms on a website provide an “accept all” box, which of course, checks all the boxes before you’ve had chance to read what they are for.

This pseudosafety comes at a cost, too. I’m constantly annoyed at having to find a cross to remove the pop-up (what does the close button do, by the way, accept all or reject all?). It’s like being back in the ’90s with GeoCities websites filled to the brim with pop-ups.

So what has the GDPR got to do with machine learning and RL ethical standards? It represents the first and largest government-led attempt to rein in a practice, related to data, which it deemed to be unethical. Has it worked? A bit, maybe. It has curtailed some of the exploits of some large companies or governments that were attempting to steer important conversations about democracy and freedom. But at what cost? Is there a better way? Probably, but I don’t think there is a single right answer.

For more general applications of machine learning and RL, I feel like this will be a slow burner; it won’t become important until it smacks people in the face, a bit like climate change. I think there will be success at a grassroots level, however, a bit like how ethical hacking is forcing companies to invest in digital security. All businesses want to be safe and secure, but it takes the challenge to actually do anything about it. The same will be true for data and the models derived from it. Individuals or groups of loosely affiliated individuals will be passionate enough to attempt to hold businesses or governments to account.

And then there are people like you. The practitioners. The engineers. The people building these solutions. You are first in line to observe the impact of your work, so there is a great responsibility placed upon you to decide whether your algorithm is ethical. This is the burden of knowledge; use it wisely and make good decisions.

Concluding Remarks

I find it ironic that I finish a book about optimizing sequential decision making with a remark asking you to make good decisions. Who knows, maybe in the future we can use RL to make the ethical decisions for us.

Until recently, RL has been primarily a subject of academic interest. The aim of this book was to try to bring some of that knowledge to industry and business, to

continue the journey of improving productivity through automation. RL has the potential to optimize strategic-level decision processes using goals directly derived from business metrics. This means that RL could impact entire business structures, from factory floor workers to CEOs.

This poses unique technical and ethical challenges. Like how decisions can have long-lasting consequences that are not known at the time and how an automatic strategy can impact people. But this shouldn't prevent you from experimenting to see how RL can help you in your situation. It really is just a tool; how you use it is what counts.

Next Steps

You've probably got lots of questions. That's good. Collate those questions and send them to me (see [About the Author](#) for contact details). But in general I recommend that you don't stop here. Try to find a practical problem to solve, ideally at work. If you can persuade your business to support your research, maybe in your 20% time or maybe as a fully fledged time-boxed proof of concept, this gives you the time and the impetus to deal with the real-life challenges that you will face. If you stick to toy examples, I'm afraid that it will be too easy for you. If you can't do this in work time, then find a problem that is interesting to you, maybe related to another hobby. You probably won't get any dedicated time to do this, because life gets in the way, but hopefully it is rewarding and gives you some practical experience.

Beyond practical experience, I recommend you keep reading. Obviously RL is hugely dependent on machine learning, so any books on data science or machine learning are useful. There are some other RL books out there, too, which are very good, and should provide another perspective. And if you want to go full stack, then improve your general software and engineering skills.

To help cement everything you have learned, you need to use it. Experience is one way I've already mentioned, but mentorship, teaching, and general presenting is another great way to learn. If you have to explain something to someone else, especially when the format is formal, like a presentation, it forces you to consolidate your ideas and commit your models and generalizations to memory. Don't fall into the trap of thinking that you are not experienced enough to talk about it. You are; you are an expert in the experiences you have had. It is very likely that some of your experiences can generalize to RL to help apply, explain, or teach these concepts in a way that nobody else could. Give it a go!

Now It's Your Turn

With that, I'll hand it over to you.

As you know, you can find more resources on the [accompanying website](#). But I also want to hear from you. I want to hear about the projects that you are working on,

your challenges, your tips and tricks, and your ideas for the future of RL. I especially want to talk to you if you are thinking about applying RL in industry or you are struggling with a problem. I can make your life easier with solutions, tips, and assessments to reduce effort, reduce risks, and improve performance. My full contact details are printed in [About the Author](#).

Good luck!

Further Reading

- Debugging:
 - Agans, David J. 2002. *Debugging: The 9 Indispensable Rules for Finding Even the Most Elusive Software and Hardware Problems*. AMACOM.
 - Fowler, Martin. 2018. *Refactoring: Improving the Design of Existing Code*. Addison-Wesley Professional.
 - Feathers, Michael C. 2004. *Working Effectively with Legacy Code*. Prentice Hall PTR.
- Deep learning debugging:
 - Goodfellow, Ian, Yoshua Bengio, and Aaron Courville. 2016. *Deep Learning*. MIT Press.
- RL debugging:
 - Excellent Reddit thread, which led to a lot of this information.¹¹
 - John Schulman's fantastic nuts and bolts presentation.¹²
 - Representative experience from Matthew Rahtz.¹³
- Research directions:
 - Many of the ideas were inspired by the paper by Dulac-Arnold et al.¹⁴
 - Offline RL.¹⁵
 - I personally loved Max Tegmark's book on the future for AI.¹⁶

References

[1] Engstrom, Logan, Andrew Ilyas, Shibani Santurkar, Dimitris Tsipras, Firdaus Janoos, Larry Rudolph, and Aleksander Madry. 2020. "[Implementation Matters in Deep Policy Gradients: A Case Study on PPO and TRPO](#)". ArXiv:2005.12729, May.

[2] Ibid.

[3] Cobbe, Karl et al. "[Reinforcement Learning with Prediction-Based Rewards](#)." OpenAI. 31 October 2018.

- [4] “Lessons Learned Reproducing a Deep Reinforcement Learning Paper”. n.d. Accessed 17 August 2020.
- [5] Ng, Andrew. “Nuts and Bolts of Applying Deep Learning”. 2016.
- [6] Guo, Xiaoxiao, Satinder Singh, Honglak Lee, Richard L Lewis, and Xiaoshi Wang. 2014. “Deep Learning for Real-Time Atari Game Play Using Offline Monte-Carlo Tree Search Planning”. *Advances in Neural Information Processing Systems* 27, edited by Z. Ghahramani, M. Welling, C. Cortes, N. D. Lawrence, and K. Q. Weinberger, 3338–3346. Curran Associates, Inc.
- [7] “Hype Cycle for Data Science and Machine Learning, 2019”. n.d. Gartner. Accessed 17 August 2020.
- [8] “Gartner Top 10 Trends in Data and Analytics for 2020”. n.d. Accessed 17 August 2020.
- [9] “Gartner Says AI Augmentation Will Create \$2.9 Trillion of Business Value in 2021”. n.d. Gartner. Accessed 17 August 2020.
- [10] “Too Many AI Researchers Think Real-World Problems Are Not Relevant”. n.d. *MIT Technology Review*. Accessed 20 August 2020.
- [11] “Deep Reinforcement Learning Practical Tips”. R/Reinforcementlearning, n.d., Reddit. Accessed 17 August 2020.
- [12] “Deep RL Bootcamp Lecture 6: Nuts and Bolts of Deep RL Experimentation”. 2017.
- [13] “Lessons Learned Reproducing a Deep Reinforcement Learning Paper”. n.d. Accessed 17 August 2020.
- [14] Dulac-Arnold, Gabriel, Daniel Mankowitz, and Todd Hester. 2019. “Challenges of Real-World Reinforcement Learning”. ArXiv:1904.12901, April.
- [15] Levine, Sergey, Aviral Kumar, George Tucker, and Justin Fu. 2020. “Offline Reinforcement Learning: Tutorial, Review, and Perspectives on Open Problems”. ArXiv: 2005.01643, May.
- [16] Tegmark, Max. 2017. *Life 3.0: Being Human in the Age of Artificial Intelligence*. Penguin UK.

The Gradient of a Logistic Policy for Two Actions

Equation 5-6 is a policy for two actions. To update the policies I need to calculate the gradient of the natural logarithm of the policy (see Equation 5-4). I present this in Equation A-1. You can perform the differentiation in a few different ways depending on how you refactor it, so the result can look different, even though it provides the same result.

Equation A-1. Logistic policy gradient for two actions

$$\nabla \ln \pi(a \mid s, \theta) = \begin{pmatrix} \frac{\delta}{\delta \theta_0} \ln \left(\frac{1}{1 + e^{-\theta_0^\top s}} \right) \\ \frac{\delta}{\delta \theta_1} \ln \left(1 - \frac{1}{1 + e^{-\theta_1^\top s}} \right) \end{pmatrix}$$

I calculate the gradients of each action independently and I find it easier if I refactor the logistic function like in [Equation A-2](#).

Equation A-2. Refactoring the logistic function

$$\begin{aligned}\pi(x) &\doteq \frac{1}{1 + e^{-x}} = \frac{e^x}{e^x(1 + e^{-x})} \\ &= \frac{e^x}{e^x(1 + e^{-x})} \\ &= \frac{e^x}{e^x + e^x e^{-x}} \\ &= \frac{e^x}{e^x + e^{x-x}} \\ &= \frac{e^x}{e^x + e^0} \\ &= \frac{e^x}{e^x + 1} \\ &= \frac{e^x}{1 + e^x}\end{aligned}$$

The derivative of the refactored logistic function, for action 0, is shown in [Equation A-3](#).

Equation A-3. Differentiation of action 0

$$\begin{aligned}
\frac{\delta}{\delta \theta_0} \ln \pi_0(\theta_0^\top s) &= \frac{\delta}{\delta \theta_0} \ln \left(\frac{e^{\theta_0^\top s}}{1 + e^{\theta_0^\top s}} \right) \\
&= \frac{\delta}{\delta \theta_0} \ln e^{\theta_0^\top s} - \frac{\delta}{\delta \theta_0} \ln \left(1 + e^{\theta_0^\top s} \right) \\
&= \frac{\delta}{\delta \theta_0} \theta_0^\top s - \frac{\delta}{\delta \theta_0} \ln \left(1 + e^{\theta_0^\top s} \right) \\
&= s - \frac{\delta}{\delta \theta_0} \ln \left(1 + e^{\theta_0^\top s} \right) \\
&= s - \frac{\delta}{\delta \theta_0} \ln u \quad \text{where } u = 1 + e^{\theta_0^\top s} \\
&= s - \frac{1}{u} \frac{\delta}{\delta \theta_0} u \\
&= s - \frac{1}{1 + e^{\theta_0^\top s}} \frac{\delta}{\delta \theta_0} \left(1 + e^{\theta_0^\top s} \right) \\
&= s - \frac{1}{1 + e^{\theta_0^\top s}} \frac{\delta}{\delta \nu} (e^\nu) \frac{\delta}{\delta \theta_0} \nu \quad \text{where } \nu = \theta_0^\top s \\
&= s - \frac{1}{1 + e^{\theta_0^\top s}} e^{\theta_0^\top s} s \\
&= s - \frac{s e^{\theta_0^\top s}}{1 + e^{\theta_0^\top s}} \\
&= s - s \pi(\theta_0^\top s)
\end{aligned}$$

The method to calculate the derivative of the policy for action 1 is shown in [Equation A-4](#). Note that the derivation towards the end is the same as [Equation A-3](#).

Equation A-4. Differentiation of action 1

$$\begin{aligned}
\frac{\delta}{\delta \theta_1} \ln \pi_1(\theta_1^\top s) &= \frac{\delta}{\delta \theta_1} \ln \left(1 - \frac{e^{\theta_1^\top s}}{1 + e^{\theta_1^\top s}} \right) \\
&= \frac{\delta}{\delta \theta_1} \ln \left(\frac{1 + e^{\theta_1^\top s}}{1 + e^{\theta_1^\top s}} - \frac{e^{\theta_1^\top s}}{1 + e^{\theta_1^\top s}} \right) \\
&= \frac{\delta}{\delta \theta_1} \ln \left(\frac{1 + e^{\theta_1^\top s} - e^{\theta_1^\top s}}{1 + e^{\theta_1^\top s}} \right) \\
&= \frac{\delta}{\delta \theta_1} \ln \left(\frac{1}{1 + e^{\theta_1^\top s}} \right) \\
&= \frac{\delta}{\delta \theta_1} \ln 1 - \frac{\delta}{\delta \theta_1} \ln \left(1 + e^{\theta_1^\top s} \right) \\
&= - \frac{\delta}{\delta \theta_1} \ln \left(1 + e^{\theta_1^\top s} \right) \\
&= \dots \text{ same as the derivative of } \pi_0 \\
&= -s\pi(\theta_1^\top s)
\end{aligned}$$

These result in the gradient shown in [Equation A-5](#).

Equation A-5. Logistic policy

$$\nabla \ln \pi(a \mid s, \theta) = \begin{pmatrix} s - s\pi(\theta_0^\top s) \\ -s\pi(\theta_1^\top s) \end{pmatrix}$$

The Gradient of a Softmax Policy

The derivation of the gradient of a softmax policy is shown in [Equation B-1](#). Note how this is a similar form to the logistic gradients in [Appendix A](#).

Equation B-1. Gradient of a softmax policy

$$\begin{aligned}
 \nabla_{\theta} \ln \pi(\theta^{\top} s) &= \nabla_{\theta} \ln \frac{e^{\theta^{\top} s}}{\sum_a e^{\theta^{\top} s}} \\
 &= \nabla_{\theta} \ln e^{\theta^{\top} s} - \nabla_{\theta} \ln \sum_a e^{\theta^{\top} s} \\
 &= \nabla_{\theta} \theta^{\top} s - \nabla_{\theta} \ln \sum_a e^{\theta^{\top} s} \\
 &= s - \nabla_{\theta} \ln \sum_a e^{\theta^{\top} s} \\
 &= s - \nabla_{\theta} \ln \sum_a e^{\theta^{\top} s} \\
 &= s - \frac{\nabla_{\theta} \sum_a e^{\theta^{\top} s}}{\sum_a e^{\theta^{\top} s}} \\
 &= s - \frac{\sum_a s e^{\theta^{\top} s}}{\sum_a e^{\theta^{\top} s}} \\
 &= s - \sum_a s \pi(\theta^{\top} s)
 \end{aligned}$$

Acronyms and Common Terms

ACER	Actor-critic with experience replay	BDQN	Bootstrapped deep Q-network
ACKTR	Actor-critic using Kronecker-factored trust region	CAQL	Continuous action Q-learning
AGE	Adversarially guided exploration	CDQ	Clipped double Q-learning
Agent	An instantiation of an RL algorithm that interacts with the environment	CI/CD	Continuous integration/continuous delivery
AGI	Artificial general intelligence	CNN	Convolutional neural network
AI	Artificial intelligence	CORA	Conditioned reflex analog (an early robot)
ANN	Artificial neural network	CPU	Central processing unit
API	Application programming interface	CRISP-DM	Cross-industry process for data mining
AWS	Amazon Web Services	CTR	Click-through rate
BAIL	Best-action imitation learning	CVAE	Convolutional variational autoencoder
BCQ	Batch-constrained deep Q-learning	DAG	Directed acyclic graph
		DBN	Deep belief network

DDPG

DDPG

Deep deterministic policy gradient

Dec-HDRQN

Hysteretic Deep Recurrent Q-networks

DEC-MDP

Decentralized Markov decision process

Deep

A method that uses multilayer neural networks

Deterministic policy

Maps each state to a single specific action

DIAYN

Diversity is all you need

DL

Deep learning

DP

Dynamic programming

DPG

Deterministic policy gradient

DQN

Deep Q-network

DRL

Deep reinforcement learning

EFG

Extensive-form game

Environment

Abstraction of real life, often simulated

ESN

Echo state network

FIFO

First in, first out

Fully observable

Agents can observe the complete state of the environment

GAIL

Generative adversarial imitation learning

GAN

Generative adversarial network

GDPR

General data protection regulation

GQ and Greedy-GQ

Generalized gradient algorithms for temporal-difference prediction

GPI

Generalized policy iteration

GPT-3

Generative Pre-trained Transformer 3

GPU

Graphics processing unit

GRU

Gated recurrent unit

GTD

Gradient temporal difference

HAAR

HRL with advantage-based rewards

HIRO

Hierarchies with intrinsic rewards

HRL

Hierarchical reinforcement learning

IID

Independent and identically distributed

IMPALA

Importance weighted actor-learner architecture

IRIS

Implicit reinforcement without interaction at scale

IRL

Inverse reinforcement learning

KPI

Key performance indicator

KL

Kullback–Leibler (divergence)

LSTM

Long short-term memory

MADDPG	Multi-agent deep deterministic policy gradient	Offline	In the ML sense, training without inferencing
MARL	Multi-agent reinforcement learning	On-policy	Algorithms that improve a policy that is being used to make decisions
MC	Monte Carlo	Online	In the ML sense, training at the same time as inferencing
MCTS	Monte Carlo tree search	PAAC	Parallel advantage actor-critic
MDP	Markov decision process	PAC	Probably approximately correct
MG	Markov game	Partially observable	Agents can observe a limited view of the environment
ML	Machine learning	PCL	Path consistency learning
MLP	Multilayer perceptron	PG	Policy gradient
Model	Understand phenomena through a mathematical representation	Policy	Rules that define how to act
Model-based	Literal, based on a model	POMDP	Partially observable Markov decision process
Model-free	Literal, no model assumed	PPO	Proximal policy optimization
MSE	Mean squared error	QMIX	Monotonic value function factorization for deep multi-agent reinforcement learning
NN	Neural network	RAM	Random access memory
Nonstationary	Not stable, in a statistical sense; chaotic, not predictable	RBM	Restricted Boltzmann machine
NPG	Natural policy gradient	REINFORCE	A Monte Carlo-inspired policy gradient algorithm
Observation	Externally observed features of an environment		
Off-policy	Algorithms that learn an optimal policy while another policy makes decisions		

Reward

Reward

A numeric value given to agents to signify success or failure

RL

Reinforcement learning

RNN

Recurrent neural network

ROBEL

Robotics Benchmarks for Learning

RTB

Real-time bidding

SAC

Soft actor-critic

SARSA

Exhaustive version of Q-learning

SAS

Stochastic action set

SEED

Scalable and efficient deep RL

State

The fully observable internal state of an environment, often a misnomer for observation

Stationary

Stable, in a statistical sense; implies predictability

Stochastic policy

Maps states to a probability distribution over actions

Strategy

Long-term sequential decision making; see policy

SQL

Soft Q imitation learning

SQL

Soft Q-learning

TD

Temporal difference

TRPO

Trust region policy optimization

UAV

Unmanned aerial vehicle

UCB

Upper confidence bound

VAE

Variational autoencoder

VIME

Variational information maximizing exploration

XAI

Explainable artificial intelligence

Symbols and Notation

$\|$

Divergence between two distributions, typically KL divergence

α

Typically a learning rate parameter

β

The behavior policy, also a second learning rate parameter in some algorithms

γ

Discount factor

∇

Gradient operator

π

A policy, or a target policy, often used in a subscript to denote following a policy

θ

Parameters of a model or policy

ϕ

A function that maps from one dimensionality to another

$\mathcal{A}$

All possible actions, a set

a

A specific state, a member of $\mathcal{A}$

$\mathbb{E}$	Expectation operator	$\mathcal{R}$	All possible rewards, a set
G	The return (sum of all rewards between two points on a trajectory)	r	A specific reward, a member of $\mathcal{R}$
$\mathcal{H}$	Entropy operator	$\mathcal{S}$	All possible states, a set
J	The objective function for a policy gradient algorithm	s	A specific state, a member of $\mathcal{S}$
Q	Action-value function, also known as the Q-function, the expected return for a given state and action	V	The state-value function, the expected return for a given state
		Z	Distributional action-value function

A

- A/B testing, 324
- academic pseudocode, xxi
- acronyms and common terms, 363
- action-value function, 47
- actions
 - bonuses added to, 34
 - discrete or continuous, 10
 - mapping policies to action spaces, 273-277
 - in Markov decision process, 35
 - role in reinforcement learning, 4
 - stochastic action set (SAS), 219
- activation functions, 88
- actor-critic algorithms
 - actor-critic using Kronecker-factored trust regions (ACKTR), 183
 - actor-critic with experience replay (ACER), 182
 - advantage actor-critic (A2C), 129-134, 301
 - asynchronous advantage actor-critic (A3C), 301
 - development of, 18
 - eligibility traces actor-critic, 134
 - n-step actor-critic, 129-134
 - on- and off-policy, 151
 - parallel advantage actor critic (PAAC), 302
 - soft actor-critic (SAC), 193-196, 198
- advantage function, 102
- adversarial attacks, 328
- advertising, real-time bidding in, 70-73
- agents
 - deploying multiple, 324
 - effect of policies on, 40
 - how agents use and update strategy, 9
 - improving learning in
 - expert guidance, 235-239
 - further reading on, 242
 - hierarchical RL, 220-225
 - multi-agent RL, 225-235
 - other paradigms, 240
 - overview of, 215
 - rethinking MDP, 216-220
 - in Markov decision process, 35
 - on- and off-policy agents, 9, 64-65, 145
 - pretrained, 352
 - role in reinforcement learning, 5
 - tracing trajectory of, 38
- algorithms
 - actor-critic, 18, 151
 - advantage actor-critic (A2C), 129-134, 301
 - Ape-X, 303
 - asynchronous advantage actor-critic (A3C), 301
 - comparison of basic policy gradient algorithms, 135
 - debugging, 343-346, 355
 - eligibility traces actor-critic, 134
 - experience replay buffers, 77
 - first RL algorithm, 12
 - generalizing, 45
 - gradient-temporal-difference (GTD), 151
 - imitation-based, 11
 - improving sample efficiency of, 351
 - maximizing rewards, 57
 - model-based, 8
 - model-free, 9
 - Monte Carlo (MC), 50-52, 58
 - multi-agent DDPG (MADDPG), 231

- n-step actor-critic, 129-134
- n-step algorithms, 76-79, 85
- off-policy, 145-152
- parallel advantage actor critic (PAAC), 302
- performance measures, 313
- policy-based, 11
- Q-learning, 62-64
- REINFORCE, 122-127
- Retrace(λ), 182
- sampling efficiency of, 56
- selecting, 185
- single-agent, 229
- soft actor-critic (SAC), 193-196, 198
- value-based, 11
- e-greedy, 33
- Ape-X algorithm, 303
- application containers, scaling, 68-70
- arbitrary policies, 122
- artificial intelligence (AI), 3
- artificial neural networks (ANNs), 88
- artificial neurons, 16
- asynchronous advantage actor-critic (A3C), 301
- asynchronous methods, 185
- attack mitigation, 329
- autoencoders, 89, 267
- autonomous vehicles, using POMDPs in, 218
- average-reward model, 308

B

- backup diagrams, 53
- batch RL, 109-110, 258
- batch-constrained deep Q-learning (BCQ), 110
- behavior cloning, 236
- behavior policy, 148
- behavior suite, 313
- Bellman optimality equation, 49
- Bellman, Richard, 18
- best practices
 - flexibility, 317
 - reliability, 316
 - scalability, 317
- bidding, real-time in advertising, 70-73
- binary actions, 10, 274
- binning, 272
- black box attacks, 328
- black-box approaches, 314
- bootstrap confidence interval test, 311
- bootstrapping, 59

C

- canary deployment strategy, 324
- CartPole environment, 94-98
- case studies (see also industrial examples)
 - automated traffic management, 195
 - POMDPs in autonomous vehicles, 218
 - recommendations using reviews, 161
 - reducing energy usage in buildings, 98
 - scaling application containers, 68-70
 - single-agent decentralized learning in UAVs, 230
- catastrophic forgetting, 197
- centralized learning, 228, 231
- chess programming, 2
- clipped double Q-learning (CDQ), 158
- CNTK, 90
- code examples, obtaining and using, xvi, xix, xxi
- cold start problem, 278
- comments and questions, xxii
- concurrent learning, 260
- conditional variational autoencoders, 110
- constraints, 169, 267
- contact information, xxii
- contextual MDP (CMDP), 219
- continuous actions, 274
- continuous features, 270
- continuous integration/continuous delivery (CI/CD), 322, 333
- continuous learning, 219
- continuous states, 270-272
- control, 15
- convolutional NNs (CNNs), 89
- counterfactual actions, 97
- counterfactual reasoning, 183, 315
- curiosity, 279
- curriculum learning, 238, 352

D

- data science, 3
- debugging, 343-346, 355
- decay rates, 133
- decentralized distributed proximal policy optimization (DD-PPO), 303
- decentralized learning, 228, 232
- decisions, 36
- deep artificial neural networks, 88
- deep belief networks (DBNs), 89

- deep deterministic policy gradients (DDPG), 154-157
- deep learning, 16
- deep learning architectures
 - common neural network architectures, 89
 - deep learning frameworks, 90
 - deep reinforcement learning, 91
 - fundamentals, 88
- deep Q-learning (see deep Q-networks (DQNs))
- deep Q-learning from demonstrations (DQfD), 236
- deep Q-networks (DQNs)
 - benefits of, 87
 - case study: reducing energy usage in buildings, 98
 - deep learning architectures, 88-91
 - design of DQNs, 92
 - DQN on CartPole environment, 94-98
 - experience replay, 92
 - further reading on, 112
 - implementing DQN, 93
 - neural network architecture, 93
 - other DQN improvements, 107-111
 - Q-network clones, 92
 - rainbow DQN on Atari games, 103-107
 - rainbow DQNs, 99-103
- delayed policy updates (DPU), 158
- delayed Q-learning, 74
- delayed rewards, 17
- deployment
 - ancillary tooling
 - build versus buy, 321
 - continuous integration/continuous delivery (CI/CD), 322
 - deploying multiple agents, 324
 - deploying policies, 325
 - experiment tracking, 323
 - hyperparameter tuning, 324
 - logging and tracing, 322
 - monitoring, 321
 - architecture, 319-320
 - deployment efficiency, 325
 - goals
 - best practices, 316
 - by development phase, 316
 - hierarchy of needs, 318
 - implementation and deployment challenges, 318
- safety, security, and ethics
 - ethical RL, 329-331
 - safe RL, 326-327
 - secure RL, 327-329
 - tips and tricks, 343
- deterministic policy gradients (DPG)
 - benefits of, 152-154
 - case study: recommendations using reviews, 161
 - deep deterministic policy gradients, 154-157
 - improvements to, 163
 - twin delayed deep deterministic policy gradients (TD3), 158-161
- dimensionality reduction, 267
- directed acyclic graphs (DAGs), 90, 215
- discount factor, 43
- discount rate, 43
- discounted rewards, 42
- discrete features, 269
- discrete state spaces, 269, 272
- distance to novelty, 281
- distributed replay, 303
- distributed training, 300
- distribution of rewards, 101
- distributional RL, 100
- diversity is all you need (DIAYN), 223
- Donkey Car, 205-211
- dot product, 14
- double Q-learning
 - benefits of, 74
 - clipped double Q-learning (CDQ), 158
 - optimistic exploration using, 196
- drones, using single-agent learning in, 229
- dueling networks, 102
- dynamic programming
 - development of, 18
 - further reading on, 58
 - implementing value iteration, 54-55
 - results of value iteration, 56
 - value iteration with, 52-53

E

- echo state networks (ESNs), 90
- eligibility traces, 80-85
- eligibility traces actor-critic algorithm, 134
- emphatic methods, 183
- energy, reducing use in buildings, 98
- engineering and refinement
 - environment engineering, 263-266

- exploration strategies, 277-283
- mapping policies to action spaces, 273-277
- policy engineering, 268-273
- reward engineering, 283-287
- state engineering, 266-268
- workflow, 262
- ensembles, 324
- entropy methods
 - comparing SAC to PPO, 198
 - concept of entropy, 191
 - extensions to maximum entropy methods
 - experience replay, 197
 - optimistic exploration using double Q-learning, 196
 - other ways of measuring entropy, 196
 - path consistency learning, 198
 - soft policy gradient, 197
 - soft Q-learning, 197
 - how entropy encourages exploration, 200-204
 - industrial example: remote control cars, 205-211
 - maximum entropy reinforcement learning, 192
 - soft actor-critic (SAC), 193-196
- environment
 - CartPole environment, 94-98
 - considering different environments, 45
 - defined, 263
 - Gridworld environment, 65
 - in Markov decision process, 35
 - observation of, 42
 - role of in reinforcement learning, 5
 - searching through state, 57
 - simulating the environment, 31, 264
- environment engineering
 - implementation, 264
 - interacting with real life, 265
 - role of environment in RL, 263
 - simulation, 264
- episodic curiosity, 281
- episodic learning, 42
- epochs, defined, 45
- ε-greedy algorithm, 33
- error clipping, 73
- ethical RL, 329-331, 352
- ethics shaping, 331
- evaluation
 - algorithm performance measures, 313
 - evaluation conclusions, 315
 - explainability, 314
 - further reading on, 333
 - policy performance measures, 308-310
 - problem-specific performance measures, 314
 - statistical policy comparisons, 310-313
 - tips and tricks, 342
- expected return, 43
- expected values, 60
- experience replay buffers, 77, 92, 197
- experiment tracking, 323
- expert guidance
 - behavior cloning, 236
 - curriculum learning, 238, 352
 - imitation RL, 236
 - inverse RL, 237
- explainability, 314, 333
- explainable artificial intelligence (XAI), 314
- exploration strategies
 - comparison of, 35
 - exploration versus exploitation, 29
 - improving, 277-283

F

- feature engineering, 17, 266
- feed-forward layers, 89
- feedback
 - in Markov decision process, 35
 - rewards and, 16
- finite horizon problems, 308
- formally constrained RL, 327
- forward models, 267
- Fourier expansions, 271
- frameworks
 - implementing other frameworks, 298
 - implementing RL frameworks, 296-298
- framing problems, 339
- fully connected layers, 89
- fuzzy learning, 84

G

- gates, 90
- General Data Protection Regulation (GDPR), 352
- general reinforcement learning architecture (Gorila), 300
- generalization, 45

- generalized off-policy actor-critic (Geoff-PAC), 183
- generalized policy iteration (GPI), 53
- generative adversarial imitation learning (GAIL), 236
- Gluon, 90
- gradient of a logistic policy for two actions, 357
- gradient of a softmax policy, 361
- gradient temporal-difference learning, 149
- gradient variance reduction, 127
- gradient-temporal-difference (GTD) algorithms, 151
- greedy-GQ algorithm, 150
- Grey Walter, William, 19
- grid environments, 65, 79
- guiding principles, xvi

H

- hashing, 273
- heads, 103
- hidden layers, 88
- hierarchical reinforcement learning (HRL)
 - challenges of, 221
 - high-low hierarchies with intrinsic rewards (HIRO), 222
 - HRL conclusions, 225
 - idea behind, 220
 - learning skills and unsupervised RL, 223
 - naive HRL, 221
 - potential for, 241
 - using skills in HRL, 224
- hierarchy of expectations, 13
- high-dimensional efficiency, 351
- high-low hierarchies with intrinsic rewards (HIRO), 222
- Hoff, Ted, 19
- Hogwild technique, 301
- HRL with advantage-based rewards (HAAR), 224
- hybrid action spaces, 275
- hyperparameter tuning, 178, 324

I

- Ignite, 90
- imitation reinforcement learning, 236
- imitation-based algorithms, 11
- implementation
 - frameworks, 296-298
 - scaling RL

- distributed replay, 303
- distributed training, 300
- improving utilization, 304
- need for, 299
- scaling conclusions, 306
- single-machine training, 301
- synchronous distribution, 303
- importance sampling, 146
- importance weighted actor-learner architectures (IMPALA), 304
- industrial examples (see also case studies)
 - automatically purchasing products for customers, 136-142
 - learning to drive with remote control cars, 205-211
 - real-time bidding in advertising, 70
 - using servos for robotic control, 174-181
- infinite-horizon problems, 308
- information gain, 279
- input layer, 88
- intrinsic motivation, 278
- inventory control problem
 - applying Markov decision process to, 36-40
 - inventory control simulation, 40-42
- inverse reinforcement learning (IRL), 237

K

- k-nearest neighbors, 111
- Keras, 90
- KL-divergence-based trust method (Trust-PCL), 198
- Klopf, Harry, 19
- Kronecker-factored approximation (K-FAC), 183
- Kullback-Leibler (KL) divergence, 165

L

- labels, 3
- latent information, 89
- law of large numbers, 15
- learning forward models, 267
- learning types
 - concurrent learning, 260
 - offline or batch, 258
 - online, 258
 - reset-free learning, 261
- lifelong learning, 219
- Lighthill report, 19
- linear approximations, 271

- linear model U-trees model, 314
- linear models, 13
- linear policies, 120-122
- logging and tracing, 322
- logistic policy, 120
- logistic policy gradient for two actions, 357
- lower bound, 196

M

- machine learning (ML)
 - introduction to, 3
 - versus reinforcement learning, 15-16
- machine learning operations (MLOps), 316
- Manchester computer, 2
- market opportunities, 347
- Markov chain, 36
- Markov decision process (MDP)
 - agents, environments, actions, and rewards in, 35
 - contextual MDP (CMDP), 219
 - development of, 18
 - dynamics of, 36
 - further reading on, 58
 - interface and feedback loop in, 35
 - inventory control problem, 36-40
 - inventory control simulation, 40-42
 - key elements of, 57
 - key to, 36
 - MDPs with changing actions, 219
 - partially observable MDPs (POMDPs), 41, 216-219
 - purpose of, 5
 - regularized MDPs, 211, 220
 - semi-MDPs, 84
 - Thomas and Okal's MDP Notation, xx
 - transition models in, 36
- Markov process, 36
- mate-in-2 problems, 2
- mathematical notation, xx
- maximum entropy reinforcement learning, 192
- maximum return, 49
- MDP-inspired autoencoders, 268
- mean squared error (MSE) loss function, 94
- meta-learning, 240
- mixed state spaces, 273
- model-based algorithms, 8
- model-free algorithms, 9
- modeling, 3
- models

- average-reward model, 308
- increasing complexity of, 271
- learning forward models, 267
- linear model U-trees model, 314
- linear models, 13
- transition models, 36
- world models, 268
- monitoring, 321
- Monte Carlo (MC) algorithm, 50-52, 58
- Monte Carlo (REINFORCE), 122
- multi-agent DDPG (MADDPG) algorithm, 231
- multi-agent reinforcement learning (MARL)
 - application types, 226
 - case study: using single-agent decentralized learning in UAVs, 230
 - centralized learning, decentralized execution, 231
 - centralized or decentralized learning, 228
 - decentralized learning, 232
 - MARL challenges, 234
 - MARL conclusions, 235
 - MARL frameworks, 226
 - other combinations, 233
 - potential for, 241
 - problems addressed by, 225
 - single-agent algorithms, 229
- multi-arm bandit testing
 - further reading on, 57
 - improving the ϵ -greedy algorithm, 33
 - limitations of, 57
 - policy evaluation, 26
 - policy improvement, 29
 - problem definition, 25
 - reward engineering, 26
 - running the experiment, 31
 - simulating the environment, 31
- multilayer perceptrons (MLPs), 89
- multimodal rewards, 100
- MXNet, 90

N

- n-step actor-critic algorithm, 129-134
- n-step algorithms, 76-79, 85
- naive HRL, 221
- natural policy gradients (NPG), 167, 186
- neural networks, 16
- neurons, 88
- no free lunch theorem, 134, 212
- noisy nets, 102

- noisy states, 280
- nonlinear approximations, 271
- nonparametric approximations, 271
- numerical optimization, 186

O

- observation function, 216
- observational data, 3
- observations, 35
- off-policy agents, 9, 64, 145
- off-policy algorithms
 - behavior and target policies, 148
 - drawbacks of, 163
 - gradient temporal-difference learning, 149
 - greedy-GQ, 150
 - importance sampling, 146
 - off-policy actor-critics, 151
 - off-policy Q-learning, 149
 - versus on-policy algorithms, 145
- offline learning, 109-111, 258, 351
- on-policy agents, 9, 65, 145
- online learning, 258
- online networks, 92
- ONNX, 90
- OODA (observe, orient, decide, and act) loop, 262
- operational reinforcement learning
 - deployment, 315-331
 - ancillary tooling, 321
 - architecture, 319
 - goals, 315
 - safety, security, and ethics, 326
 - further reading on, 332
 - implementation, 296-315
 - evaluation, 307-315
 - frameworks, 296
 - scaling RL, 299
- opposition learning, 75
- optimal control, 18
- optimal policies
 - Bellman optimality equation, 49
 - defined, 48, 57
 - factors relating to, 41
 - finding directly, 115
 - maximum return and, 49
- optimization methods, 11
- output layer, 88
- overfitting, 45

P

- parallel advantage actor critic (PAAC), 302
- parameters, 13
- partially observable MDPs (POMDPs), 41, 216-219
- path consistency learning (PCL), 198
- Pavlov, Ivan, 12
- permutation test, 311
- pessimistic underexploration, 196
- poisoning, 328
- policies
 - arbitrary policies, 122
 - behavior and target policies, 148
 - deploying, 325
 - distinct tasks of, 268
 - expert guidance on, 235-239
 - linear policies
 - logistic policy, 120
 - softmax policy, 121
 - mapping policies to action spaces, 273-277
 - in Markov decision process, 40
 - Monte Carlo policy generation, 50-52
 - optimal policies, 41, 48, 57, 115
 - purpose of, 43, 268
 - value functions and, 42-47
- policy decay rates, 133
- policy engineering, 268-273
 - continuous states, 270
 - converting to discrete states, 272
 - discrete states, 269
 - distinct tasks of policies, 268
 - goal of, 268
- policy evaluation
 - defined, 53
 - process of, 11
 - value function, 26
- policy gradient (PG) algorithms
 - actor-critic using Kronecker-factored trust regions (ACKTR), 183
 - actor-critic with experience replay (ACER), 182
 - emphatic methods, 183
 - extensions to, 184
 - further reading on, 186
 - quantile regression in, 184
 - Retrace(λ), 182
 - selecting, 185
- policy gradient methods
 - basic implementations

- algorithm comparison, 135
 - eligibility traces actor-critic, 134
 - gradient variance reduction, 127
 - Monte Carlo (REINFORCE), 122
 - n-step actor-critic and advantage actor-critic, 129-134
 - REINFORCE with baseline, 124-127
 - benefits of direct learning, 115
 - compared to soft Q-learning, 211
 - further reading on, 143
 - industrial example: automatically purchasing products, 136-142
 - policy functions, 119-122
 - policy gradient calculation, 116
 - policy gradient theorem, 117-119
 - policy improvement
 - choosing the best action, 29
 - defined, 53
 - process of, 11
 - policy performance measures, 308-310
 - policy-based algorithms, 11
 - polynomial expansion, 271
 - Popplewell, Cicely, 2
 - population mean, 146
 - practical reinforcement learning
 - characteristics of RL projects, 254-262
 - further reading on, 288
 - RL engineering and refinement
 - environment engineering, 263-266
 - exploration, 277-283
 - mapping policies to action spaces, 273-277
 - policy engineering, 268-273
 - process, 262
 - reward engineering, 283-287
 - state engineering, 266-268
 - RL project life cycle, 249-254
 - prediction error, 14
 - prediction problem, 15
 - Prinz, Dietrich, 2
 - prioritized experience replay, 102
 - priors, 267
 - probably approximately correct (PAC) learning, 74
 - problem design, 25, 233, 256-257, 339
 - projects
 - characteristics of
 - learning types, 258-262
 - low-level RL indicators, 256
 - sequentiality, 254
 - strategic value, 255
 - life cycle of, 249-254
 - data science life cycle, 252
 - project velocity, 250
 - reinforcement learning life cycle, 252
 - rollercoaster of feelings involved, 250
 - small development cycles, 251
 - starting simple, 250
 - proximal policy optimization (PPO), 169-174, 198
 - punishment, 19
 - PyTorch, 90
- ## Q
- ### Q-learning
- benefits of, 62-64
 - clipped double Q-learning (CDQ), 158
 - compared to policy gradient methods, 211
 - creation of, 19
 - extensions to
 - comparing types of, 75
 - delayed Q-learning, 74
 - double Q-learning, 74
 - opposition learning, 75
 - off-policy Q-learning, 149
 - versus SARSA, 65-67
 - speedy Q-learning, 84
 - quantile regression, 184
- ## R
- radial basis functions, 272
 - rainbow DQNs
 - creation of, 99
 - distributional RL, 100
 - dueling networks, 102
 - noisy nets, 102
 - on Atari games, 103-107
 - prioritized experience replay, 102
 - random distillation networks, 280
 - random seed, 312
 - ranked t-test, 311
 - Reacher problem, 174
 - real-time bidding (RTB), 70-73
 - recommendation systems, 161
 - recurrent NNs (RNNs), 90
 - regret, 309
 - regularization, 211
 - regularized MDPs, 211, 220

- REINFORCE algorithm, 122-127
- reinforcement learning (RL) (see also operational reinforcement learning; practical reinforcement learning)
 - basics of
 - acronyms and common terms, 363
 - approach to learning, xvi
 - industrial applications for, xv
 - mathematical notation, xx, 366
 - prerequisites to learning, xvii
 - scope of book, xviii
 - supplementary materials, xix
 - support and advice on, xxii
 - challenges of real-life application, 233
 - foundation of, 25
 - future of, 346-355
 - ethical standards, 352
 - market opportunities, 347
 - next steps, 354
 - research directions, 348-352
 - RL potential, 353
 - introduction to, 1-20
 - components of RL, 4
 - discrete or continuous actions, 10
 - factors enabling RL, 2
 - fundamental concepts in RL, 12
 - further reading on, 20
 - how agents use and update strategy, 9
 - human learning, 1
 - machine learning, 3
 - optimization methods, 11
 - policy evaluation and improvement, 11
 - RL applications, 7
 - RL as a discipline, 18
 - RL versus ML, 15
 - taxonomy of RL approaches, 8
 - trial and error learning, 4
 - when to use, 5
 - learning from offline data, 109-110, 258
 - types of learning, 258-262
- reparameterization trick, 194
- Rescorla, Robert, 12
- research directions, 348-353
 - in academia, 350
 - in industry, 349
- reset-free learning, 261
- residual RL, 239
- Retrace(λ) algorithm, 182
- returns
 - expected returns, 43
 - maximum returns, 49
- reward engineering
 - common rewards, 286
 - defined, 17
 - guidelines for, 284
 - purpose of, 26
 - reward conclusions, 286
 - reward shaping, 17, 285
- reward function sensitivity, 351
- rewards
 - delayed rewards, 17
 - discounted rewards, 42
 - distribution of rewards, 101
 - early use of term, 19
 - feedback and, 16
 - improving rewards, 109
 - in Markov decision process, 35
 - maximizing, 57
 - multimodal rewards, 100
 - predicting with action-value function, 47
 - predicting with state-value function, 43-47
 - role in reinforcement learning, 5
- ROBEL platform, 174
- robotic control (real-life reacher), 174-181
 - experiment setup, 175
 - hyperparameter tuning in simulation, 178
 - increasing algorithm complexity, 177
 - resulting policies, 180
 - RL algorithm implementation, 175
- rollouts, 62
- Rosenblatt, Frank, 19

S

- safety, security, and ethics
 - ethical RL, 329-331
 - future of ethical standards, 352
 - safe RL, 326-327
 - secure RL, 327-329
- saliency maps, 315
- sample efficiency, 351
- sample mean, 146
- sampling efficiency, 56, 279
- SARSA (state-action-reward, state-action), 64-67
- scalable and efficient deep RL with accelerated central inference (SEED), 305
- scaling
 - application containers, 68-70

- distributed replay, 303
 - distributed training, 300
 - improving utilization, 304
 - need for, 299
 - scaling conclusions, 306
 - single-machine training, 301
 - synchronous distribution, 303
 - self-play, 239
 - self-reflection, 279
 - semi-MDPs, 84
 - sequentiality, 254
 - shadowing deployment strategy, 324
 - Shannon entropy, 191
 - (see also entropy methods)
 - shape, 285
 - side effects, 325
 - single-agent algorithms, 229
 - single-machine training, 301
 - Skorch, 90
 - soft actor-critic (SAC) algorithm, 193-196, 198
 - soft policy gradient, 197
 - soft Q imitation learning (SQIL), 236
 - soft Q-learning, 197, 211
 - softmax function, 34, 89
 - softmax policy, 121
 - softmax policy gradient, 361
 - speed of convergence to near-optimality, 309
 - speedy Q-learning, 84
 - state
 - continuous states, 270-272
 - converting to discrete states, 272
 - defined, 35
 - discrete states, 269
 - mixed state spaces, 273
 - noisy states, 280
 - observation of, 42
 - safe set of states, 327
 - searching through an environment's, 57
 - terminal state, 42
 - state engineering
 - constraints, 267
 - domain expertise and, 266
 - goal of, 266
 - learning forward models, 267
 - transformations, 267
 - state prediction, 279
 - state representation learning, 266
 - state-value function
 - backup diagram for, 53
 - predicting rewards with, 43-47
 - temporal-difference learning, 60
 - state-value learning decay rates, 133
 - statistical policy comparisons, 310-313
 - Stein variational gradient descent, 194
 - steps, 35
 - stickiness, 324
 - stochastic action set (SAS), 219
 - strategy
 - how agents use and update, 9
 - strategic value in RL problems, 255
 - student's t-test, 311
 - subscription box model, 136
 - supervised learning, 3
 - supervised methods, 273
 - supplementary materials, xix
 - symbols and notation, 366
 - synchronous distribution, 303
- ## T
- tabular methods, 64
 - target networks, 92
 - target policy, 148
 - target policy smoothing (TPS), 159
 - temperature parameter, 203-204
 - temporal-difference (TD) learning
 - approaches combined in, 59
 - emphatic methods, 183
 - fields united in, 19
 - formulation of
 - case study: scaling containers, 68-70
 - Q-learning, 62-64
 - Q-learning versus SARSA, 65-67
 - SARSA, 64
 - state-value function, 60
 - further reading on, 85
 - gradient temporal-difference learning, 149
 - industrial example: real-time bidding, 70-73
 - TensorFlow, 90
 - terminal state, 42
 - Theano, 90
 - Thomas and Okal's Markov Decision Process Notation, xx
 - tile coding, 273
 - tips and tricks
 - data, 340
 - deployment, 343
 - evaluation, 342
 - framing problems, 339

- training, 341
- tooling
 - build versus buy, 321
 - continuous integration/continuous delivery (CI/CD), 322
 - deploying multiple agents, 324
 - deploying policies, 325
 - experiment tracking, 323
 - hyperparameter tuning, 324
 - logging and tracing, 322
 - monitoring, 321
- tracers, 80-85
- traffic management, reducing queuing, 195
- transfer learning, 141, 240
- transformations, 267
- transition graphs, 38
- transition matrix, 39
- transition models, 36
- transition tables, 38
- trust region methods, 163-174
 - Kullback–Leibler (KL) divergence, 165
 - natural policy gradients and trust region policy optimization, 167
 - proximal policy optimization (PPO), 169, 198
- trust region policy optimization (TRPO), 167, 186
- tuning tips, 179
- Turing, Alan, 2, 18, 88
- twin delayed deep deterministic policy gradients (TD3), 158-161

U

- unmanned aerial vehicles (UAVs), 229
- unrolling, 76
- unsupervised learning, 3
- unsupervised methods, 273
- unsupervised reinforcement learning, 223

- update rules, 15
- upper bound, 196
- upper-confidence-bound (UCB) methods, 34

V

- value estimation, 13
- value functions
 - discounted rewards, 42
 - policy evaluation and, 26
 - predicting rewards with action-value function, 47
 - predicting rewards with state-value function, 43
- value iteration
 - with dynamic programming, 52-53
 - implementing, 54-55
 - results of, 56
- value-based algorithms, 11
- variable actions, 10
- variational information maximizing exploration (VIME), 279
- visitation counts, 279
- VISSIM traffic simulation tool, 195

W

- Wagner, Allan, 12
- Watkins's $Q(\lambda)$, 83
- weights
 - role in deep learning, 88
 - role in reinforcement learning, 13
 - weight update rules, 15
- Welch's t-test, 311
- white box attacks, 327
- Widrow, Bernard, 19
- Wilcoxon Mann-Whitney rank sum test, 311
- workflow, 262
- world models, 268

About the Author

Dr. Phil Winder is a multidisciplinary software engineer, data scientist, and CEO of **Winder Research**, a cloud native data science consultancy. He helps startups and enterprises improve their data-based processes, platforms, and products. Phil specializes in implementing production-grade cloud native machine learning and was an early champion of the MLOps movement.

He has thrilled thousands of engineers with his data science training courses in public, private, and on the O'Reilly online learning platform. Phil's courses focus on using data science in industry and cover a wide range of hot yet practical topics, from cleaning data to deep reinforcement learning. He is a regular speaker and is active in the data science community.

Phil holds a Ph.D. and M.Eng. in electronic engineering from the University of Hull and lives in Yorkshire, UK, with his brewing equipment and family.

Contact Details

For any support or advice, please get in touch with Phil using the following details:

- Website: <https://WinderResearch.com>
- Email: phil@winderresearch.com
- LinkedIn: [DrPhilWinder](#)
- Twitter: [@DrPhilWinder](#)

Colophon

The animal on the cover of *Reinforcement Learning* is an African penguin (*Spheniscus demersus*). The four species in the *Spheniscus* genus are known as banded penguins due to the band of black that runs around their bodies. These penguins also have black dorsal coloring, black beaks with a small vertical white band, spots on their bellies, and a patch of unfeathered or thinly feathered skin around their eyes.

The penguins live in colonies along the southwestern coast of Africa, mainly across 24 islands, with only a few colonies on the mainland.

At the beginning of the nineteenth century there were roughly 4 million African penguins. Today, the population has declined by 95%. Some estimate that the penguins will be extinct by the end of the decade. The penguins face many threats, including declining food sources because the fish they eat are sensitive to rising water temperatures.

The African penguin's conservation status is endangered. Many of the animals on O'Reilly covers are endangered; all of them are important to the world.

The cover illustration is by Karen Montgomery, based on a black and white engraving from *Lydekker's Royal Natural History*. The cover fonts are Gilroy Semibold and Guardian Sans. The text font is Adobe Minion Pro; the heading font is Adobe Myriad Condensed; and the code font is Dalton Maag's Ubuntu Mono.

The O'Reilly logo is displayed in white, bold, sans-serif capital letters. The background of the entire advertisement is a vibrant red-to-orange gradient, overlaid with several large, semi-transparent, overlapping circles in varying shades of red and orange, creating a dynamic, abstract pattern.

O'REILLY®

There's much more where this came from.

Experience books, videos, live online training courses, and more from O'Reilly and our 200+ partners—all in one place.

Learn more at oreilly.com/online-learning